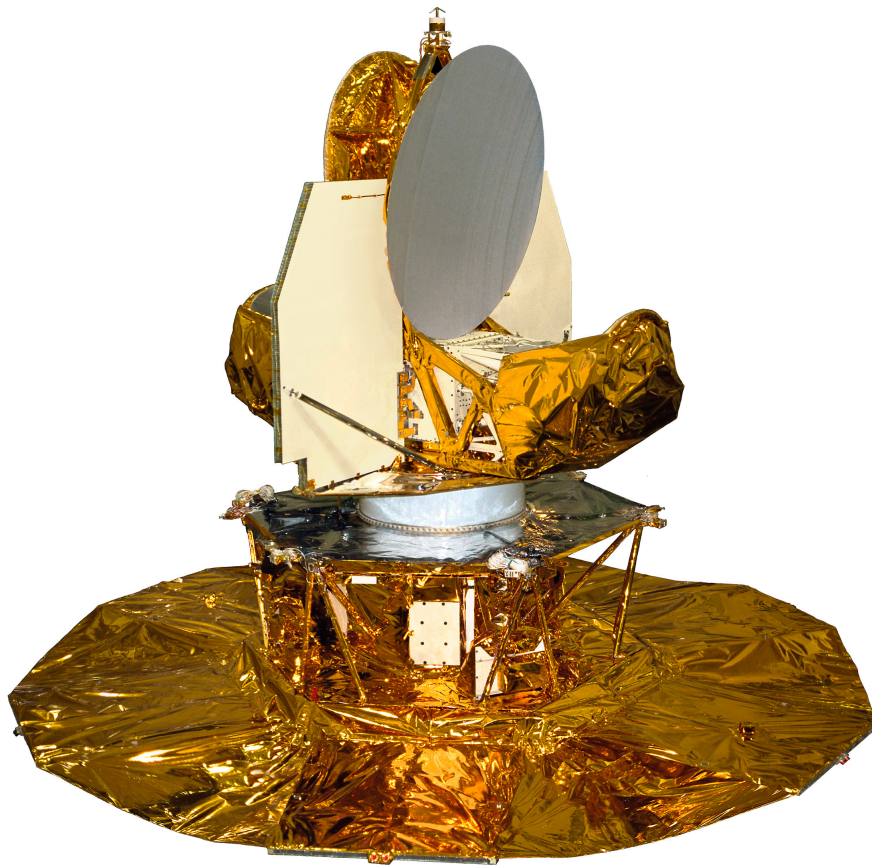


SATELLITE DYNAMICS AND ATTITUDE CONTROL

Mike Timmerman
12 June 2024



REVISION HISTORY

VERSION	REVISION NOTES
PS1	- Created document - Added PS1 material
PS2	- Added PS2 material - Removed all Pset questions - Attached all code in appendices - Added geometric data in appendix - Quantified pointing accuracy
PS3	- Added PS3 material - Reformat code in appendix
PS4	- Added PS4 material - PS2/P2 Referred to the principle moments of inertia earlier mention in PS1 - PS2/P3 Referred to the 3D view of discretized spacecraft earlier given in PS1 - PS2/P8 Added a sentence mentioning that the Polhodes visually match with the parameters earlier given in PS2/P3
PS5	- Added PS5 material
PS6	- Added PS6 material
PS7	- Added PS7 material
PS8	- Added PS8 material
PS9	- Added PS9 material - Added complete Simulink model is Appendix
PS10	- Added PS10 material - Added conclusion - Added introduction

Table 1: Summary of project revisions.

TABLE OF CONTENTS

1	INTRODUCTION	6
2	Satellite Mission Definition and Requirements	7
2.1	PROBLEM 1	7
2.2	PROBLEM 2	7
2.3	PROBLEM 3	9
2.4	PROBLEM 4	9
2.5	PROBLEM 5	13
2.6	PROBLEM 6	14
2.7	PROBLEM 7	15
3	Satellite Dynamics	16
3.1	PROBLEM 1	16
3.2	PROBLEM 2	17
3.3	PROBLEM 3	17
3.4	PROBLEM 4	19
3.5	PROBLEM 5	19
3.6	PROBLEM 6	20
3.7	PROBLEM 7	21
3.8	PROBLEM 8	22
3.9	PROBLEM 9	23
4	Satellite Kinematics	26
4.1	PROBLEM 1	26
4.2	PROBLEM 2	26
4.3	PROBLEM 3	27
4.4	PROBLEM 4	28
4.5	PROBLEM 5	28
4.6	PROBLEM 6	29
4.7	PROBLEM 7	30
5	Single- and Dual-spin Stability–	33
5.1	PROBLEM 1	33
5.2	PROBLEM 2	34
5.3	PROBLEM 3	37
5.4	PROBLEM 4	42
6	Modeling of External Perturbations	48
6.1	PROBLEM 1	48
6.2	PROBLEM 2	52
6.3	PROBLEM 3	56
7	Attitude Determination	58
7.1	PROBLEM 1	58
7.2	PROBLEM 2	59
7.3	PROBLEM 3	60
7.4	PROBLEM 4	61

7.5	PROBLEM 5	61
7.6	PROBLEM 6	62
8	Extended Kalman Filter - predict step	63
8.1	PROBLEM 1	63
8.2	PROBLEM 2	63
8.3	PROBLEM 3	64
8.4	PROBLEM 4	65
8.5	PROBLEM 5	66
8.6	PROBLEM 6	69
9	Extended Kalman Filter - update step	71
9.1	PROBLEM 1	71
9.2	PROBLEM 2	71
9.3	PROBLEM 3	71
9.4	PROBLEM 4	72
9.5	PROBLEM 5	73
9.6	PROBLEM 6	73
9.7	PROBLEM 7	75
9.8	PROBLEM 8	76
10	Actuator Sizing and Modeling	77
10.1	PROBLEM 1	77
10.2	PROBLEM 2	78
10.3	PROBLEM 3	81
10.4	PROBLEM 4	83
11	Attitude Control Algorithm	87
11.1	PROBLEM 1/2	87
11.2	PROBLEM 3	88
11.3	PROBLEM 4	89
12	CONCLUSION	98
13	Appendices	101
13.1	Spacecraft Geometric Data	101
13.2	Problem Set 1 Main File	102
13.3	Problem Set 2 Main File	102
13.4	Problem Set 3 Main File	105
13.5	Problem Set 4 Main File	113
13.6	Problem Set 5 Main File	125
13.7	Problem Set 6 Main File	133
13.8	Problem Set 7 Main File	137
13.9	Problem Set 8 Main File	143
13.10	Problem Set 9 Main File	152
13.11	Problem Set 10 Main File	157
13.12	Utility Function Code	161
13.12.1	Spacecraft Properties	161
13.12.2	Orbit Properties	162
13.12.3	Center of Gravity Calculation	163

13.12.4 Inertia Tensor Calculation	163
13.12.5 Spacecraft Discretization	165
13.12.6 Orbit Dynamics	167
13.12.7 Attitude Dynamics	167
13.12.8 Equations of Motion	168
13.12.9 Earth's Equations of Motion	169
13.12.10 Sensor measurement models	170
13.12.11 External Torque Calculation	170
13.12.12 Gravity Gradient Torque	170
13.12.13 Magnetic Field Torque	170
13.12.14 Atmospheric Drag Torque	171
13.12.15 Solar Radiation Torque	171
13.12.16 Determine equilibrium configuration for dual spin satellite	172
13.13 Discrete-time linear state propagation	172
13.13.1 Convert Euler Angles to Rotation Matrix	173
13.13.2 Convert Euler Angles to Quaternion	173
13.13.3 Convert Quaternion to Euler Angles	173
13.14 Simulink	174
13.14.1 Integrated Satellite ADCS Model	174
13.14.2 Equations of Motion	174
13.14.3 Sensor Model	175
13.14.4 Actuator Model	175
13.14.5 On-board Computer	178
13.14.6 Attitude Determination	178
13.14.7 Attitude Control	184

1 INTRODUCTION

The Attitude Determination and Control Subsystem (ADCS) is a critical component of satellite engineering, ensuring the precise orientation and stabilization required for optimal satellite performance. The ADCS is responsible for determining the satellite's orientation in space using a combination of sensors that provide accurate and real-time data. Additionally, it includes control algorithms and actuators to adjust and maintain the satellite's orientation to meet mission requirements. Proper functioning of the ADCS is vital for various satellite operations, ensuring that the satellite's instruments are correctly pointed towards their targets.

This design report delves into the comprehensive design of the ADCS for our satellite mission, exploring the methodologies employed for attitude determination, the control mechanisms utilized for maintaining and adjusting orientation, and the integration of various sensors and actuators. By examining these elements, the report aims to provide a thorough understanding of how the ADCS contributes to the overall mission objectives, ensuring reliability and accuracy in the satellite's operations amidst the dynamic conditions of space.

Throughout the report, models of the satellite's internal dynamics and external effects, such as the magnetic field and solar radiation, are implemented and tested, culminating in an integrated ADCS system. Several aspects are investigated, including the stability characteristics under gravity gradient and dual spin stabilization. The end result is a simulation integrating the satellite, sensors, attitude determination, control algorithms, and actuators, which simulates the satellite carrying out a full sequence of maintaining nominal attitude and performing any necessary attitude changes.

2 Satellite Mission Definition and Requirements

2.1 PROBLEM 1

For my project, I decided to conduct an Earth observation mission for which the satellite will orbit in a low Earth orbit. The objective of the satellite is to image the Earth surface, hence the satellite will be Earth pointing. The spacecraft attitude will be represented using quaternions. For attitude determination, gyroscopes, star sensors, Sun sensors and magnetometer will be employed. Lastly, attitude is controlled through reaction wheels, magnetic torquers and a thruster. The sensor and actuator suites are summarized in Table 2

Table 2: Attitude Determination and Control Subsystem

Sensor Suite	Actuator Suite
Gyroscope	Reaction wheel
Star sensor	Magnetic torquer
Magnetometer	Thruster
Sun sensor	

2.2 PROBLEM 2

An overview of the satellites considered in the survey is given in Table 3. More detailed characteristics for each spacecraft are provided below the table. Characteristics which are relevant are the orbit type, altitude and inclination. Specifications of the sensors and actuators in the ADCS as well as performance requirements of the ADCS. Lastly, some specification on spacecraft size, shape and weight.

Table 3: Survey of Earth observation satellites in LEO

Satellite	Launch Mass [kg]	Operator	Year
Orbview-3	360	NASA	2003
Eros A	250	ImageSat	2000
Ikonos 2	725	Carterra	1999
Quickbird 2	951	DigitalGlobe	2001
Aeolus	1500	ESA	2018
SPOT 7	712	French Government	2014
Landsat 7	2200	NASA	1999
EOS 7	156	Indian Space Research Organization	2023
ESSA 9	145	Environmental Science Service Administration	1969

Orbview-3 [9] has the objective to provide high-resolution imagery of Earth. It was put in a Sun synchronous orbit at an altitude of 470km and an inclination of 97.25° . The spacecraft body is a cylinder of 1.2m in diameter and 1.9m in length. A solar panel is mounted to the top side of the cylinder. Orbview-3 is three-axis stabilised with a capability to perform body-pointing of -50° to 50° in the in-track and cross-track directions. The attitude determination and control subsystem (ADCS) employed a gyroscope, star sensors, Sun sensors and a magnetometer for attitude sensing and four reaction wheels and thrusters for attitude control. The ADCS achieves a pointing accuracy of up to 100arcsec .

EROS A [10] provides commercial images with an optical resolution of $1.8m$. The satellite orbits Earth in a Sun synchronous orbit at an altitude of $480km$ and inclination of 97.32° . Its dimensions are $2.3m$ in height and $1.2m$ in diameter.

Ikonos 2 [17] is a high-resolution satellite operated by GeoEye. The satellite orbits Earth in a Sun synchronous orbit at an altitude of $681km$ and inclination of 98.1° . It has a hexagonal shape with dimensions $1.83m \times 1.57m$.

Quickbird 2 [18] is a high-resolution imaging satellite. The satellite orbits Earth in a Sun synchronous orbit at an altitude of $482km$. It employs star trackers, reaction wheels and gyroscopes for 3-axis stabilization.

Aeolus [15] was an Earth observation mission with the objective to acquire profiles of Earth's wind on global scale. The satellite orbits Earth in a sun-synchronous orbit at an altitude of $320km$ and inclination of 96.97° . The AOCS is equipped with 2 magnetometers, 1 sun sensor, 2 GPS receivers, 2 star trackers, 1 rate measurement unit and 1 inertial reference unit. For actuators, it uses 4 reaction wheels, 3 magnetic torquers and 4 thrusters.

SPOT 7 [19] was a commercial Earth-imaging satellite. It orbited Earth in a Sun-synchronous orbit at an altitude of $694km$ at an inclination of 98.2° . The spacecraft is equipped with 4 reaction wheels for fine-pointing and 3 magnetic torquers for off-loading. The attitude determination is performed by star trackers, providing a pointing accuracy of up to $500\mu rad$ and pointing knowledge of up to $30\mu rad$. The spacecraft has a body size of $1.55m \times 1.75m \times 2.7m$.

Landsat 7 [8] mission objective is to provide high-quality, visible-, infrared-, and thermal-wavelength data of all land and near-coastal areas on Earth. The satellite orbits Earth in a Sun synchronous orbit with an inclination of 98.2° at an altitude of $705km$. The ADCS employs gyroscopes and star trackers for attitude determination. The ADCS has a precision mode which achieves a 0.015° accuracy of Earth pointing.

EOS 7 [16] is an earth observing satellite developed by ISRO as a test payload for the second SSLV launch. It orbits Earth at an altitude of $450km$. The satellite's AOCS is equipped with star sensor, sun sensor, inertia reference unit and magnetometer as well as thrusters, reaction wheels and magnetic torquers. The AOCS achieves a pointing accuracy of 0.1° .

ESSA 9 [14] was a sun-synchronous meteorological satellite. It orbited Earth at an altitude of $1470km$ and an inclination of 101.79° . The satellite spin rate was controlled by a Magnetic Attitude Spin Coil (MASC), with the spin axis maintained normal to the orbital plane to within $\pm 1^\circ$.

From this survey, several aspects can be derived. Firstly, all Earth imaging satellites orbit Earth in a Sun synchronous orbit. This is due to the consistent lighting being a useful characteristic for satellites that image the Earth's surface in visible or infrared wavelengths. Secondly, orbit inclinations are around 98° with an altitude between $450km$ and $700km$ which is typical for a Sun synchronous orbit. Lastly, pointing accuracies range from 1° to $100arcsec$, where the ADCS employs gyroscopes, star trackers and magnetometers for attitude determination and reaction wheels, magnetic torquers and thrusters for attitude control.

2.3 PROBLEM 3

I decided to base my project on the Aeolus satellite since, from the surveyed satellites, it is the most recently launched and I was able to find the most amount of public information about it. The Aeolus satellite carries one large payload, namely ALADIN, a Doppler wind lidar.

2.4 PROBLEM 4

Mission Description

Aeolus is equipped with a payload that gathers wind profiles of Earth on a worldwide scale. Its one instrument, the Atmospheric Laser Doppler Instrument (ALADIN), precisely measures the winds circulating Earth. By delivering precise wind profiles in the troposphere and lower stratosphere, Aeolus addresses a gap in the Global Observing System and advances the understanding of Earth's global energy balance.

Following mission phases are described in the Aeolus documentation [6]:

1. Early Orbit Phase:
 - Launch and separation of the satellite from the launcher
 - Acquisition phase, including:
 - attitude rate reduction, attitude acquisition
 - appendages deployment
 - delivery of power from the solar panels
 - Orbit correction phase: manoeuvres from injection orbit to nominal orbit
2. Commissioning Phase:
 - Satellite basic functions and health are verified
 - ALADIN switch on and functional checkout to verify health
 - Various performance and parameters of ALADIN and satellite subsystems are determined
 - Ground segment data acquisition and final commissioning
3. Operational Phase:
 - Nominal operation
 - End-to-end calibration
4. End of Life Phase:
 - Satellite de-orbiting to allow a controlled re-entry in the atmosphere

Requirements

The Aeolus system requirements are documented in a publicly available document [6]. I included the relevant operational requirements in Table 4 and attitude determination and control subsystem requirements in Table 5.

Table 4: Operational requirements

Early Operations Phase	
OPS-1	Satellite shall be capable to run an automatic sequence without ground intervention which will undertake all deployments, initiate and complete attitude acquisition.
Operational Phase	
OPS-2	Aeolus shall have the capability to operate in a End-to-End Calibration Mode where the instrument line-of-sight is pointing near Nadir.
OPS-3	Aeolus shall be capable to transit from nominal operation mode to the End-to-End Calibration Mode or back in less than 15 min
End of Life Phase	
OPS-4	Aeolus shall allow controlled de-orbiting to avoid that harmful debris could reach populated areas.
Safe Mode	
OPS-5	The Safe Mode shall maintain a safe attitude within constraints allowing a continuous supply of power and maintaining a stable thermal environment.

Table 5: Attitude Determination and Control System (ADCS) requirements

ADCS General Requirements	
ADCS-1	The spacecraft shall provide an Attitude Determination and Control Subsystem (ADCS) that shall be able to: • provide attitude control around three orthogonal axes during all mission phases • correct for worst case launcher injection errors • supply sensor data to allow on-ground attitude and orbit determination in the case of full fuel tank as well as in the case of partially filled fuel tanks.
ADCS Functional Requirements	
ADCS-2	After separation from the launch vehicle, the ADCS shall: • damp out the residual angular rates • bring the spacecraft into a power safe pointing attitude • maintain the spacecraft in this pointing attitude ready to receive ground commands
ADCS-3	The ADCS shall include all on-board hardware and software to provide an autonomous capability to determine and to maintain the required spacecraft attitude pointing during all the mission phases.
ADCS-4	The ADCS shall acquire and maintain a three-axis stabilised Earth pointing control throughout its orbital life in the presence of perturbation torques.
ADCS-5	All ADCS functions shall be maintained with full performance after a single failure.
ADCS Performance Requirements	
ADCS-6	After separation, the ADCS shall be able to acquire nominal Earth pointing attitude and rates from any initial attitude and, as a minimum, 1.5 times the worst case rates at separation.
ADCS-7	In Nominal Operation Mode and End-to-End Calibration Mode, the ADCS performance shall achieve a pointing accuracy of up to $100arcsec$.
ADCS Safe Mode Requirements	
ADCS-8	The satellite shall provide an autonomous Safe Mode that shall: • bring the spacecraft into a stable pointing mode which minimises altitude loss. • use actuators and sensors different from those used in operational modes.

The satellite is flown with ALADIN instrument pointing towards Earth in a plane quasi-perpendicular to the flight path and 35° offset from nadir in anti-sun direction. The line-of-sight is oriented such that the relative velocity at the intersection with the Earth is zero (yaw steering). This nominal measurement geometry and coverage are shown in Figure 1.

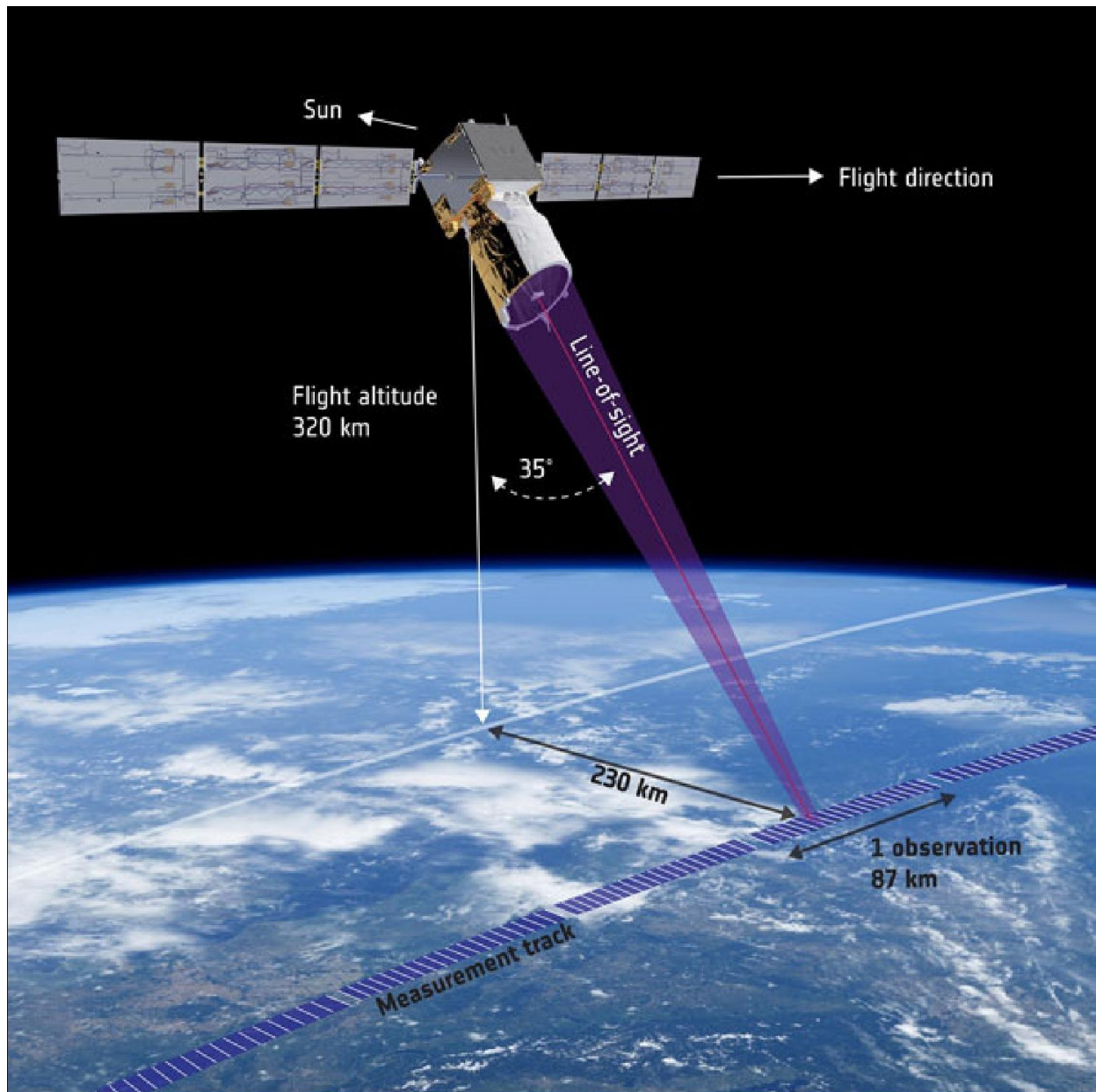


Figure 1: Nominal measurement geometry and coverage of the Aeolus mission

ADCS Sensors and Actuators

Aeolus' ADCS is equipped with thrusters, magnetorquers and reactions wheels as actuators, and magnetometers, coarse Earth and sun sensors, inertial measurement units, rate measurement units, autonomous star tracker and a GPS receiver. These sensors and actuators are given in Table 6 along with their key properties.

Table 6: Aeolus sensors and actuators with key properties.

Type	Equipment	Number	Main Characteristics	Technology
Sensors	Magnetometer	2	Three-axis	Magneto-resistor
	Coarse Earth sun sensor	1	4 pi steradian FOV	Thermal
	GPS receiver	2		RF
	Star tracker	2	Large FOV	
	Rate Measurement Unit	1	Three-axis	Solid-state rate sensor
	Inertial reference unit	1	Three-axis	Fiber Optic Gyro
Actuators	Reaction wheels	4	10Nm/s/0.1Nm	Ball bearings
	Magnetic torquers	3	100Am ²	Windings
	Thrusters	4	5N	Hydrazine

Mechanical Layout

The spacecraft consists of four main components, namely the payload, the spacecraft bus, the solar arrays and the propellant. The spacecraft bus is box-shaped upon which the payload module is mounted via three isostatic bipods. The bus also has the two deployable solar arrays attached to it.

The spacecraft bus has a size of $1.74m \times 1.9m \times 2.0m$. The solar arrays have a total surface area of $14.5m^2$ with a span of $13m$. The payload is cylindrical shaped with a diameter of $1.5m$ and a length of $2.0m$. Lastly, the propellant tank is mounted inside the bus at the opposite side of the payload and has a cylindrical shape. [21, sec. 18.5.1] provides tank sizing relations. Based on the propellant mass and density of hydrazine, with a recommended 20% margin on tank volume, the hydrazine tank has a volume of $0.3468m^3$. Hence, the height is estimated to be $1m$ with a diameter of $0.66m$. A schematic of the spacecraft layout is shown in Figure 2.

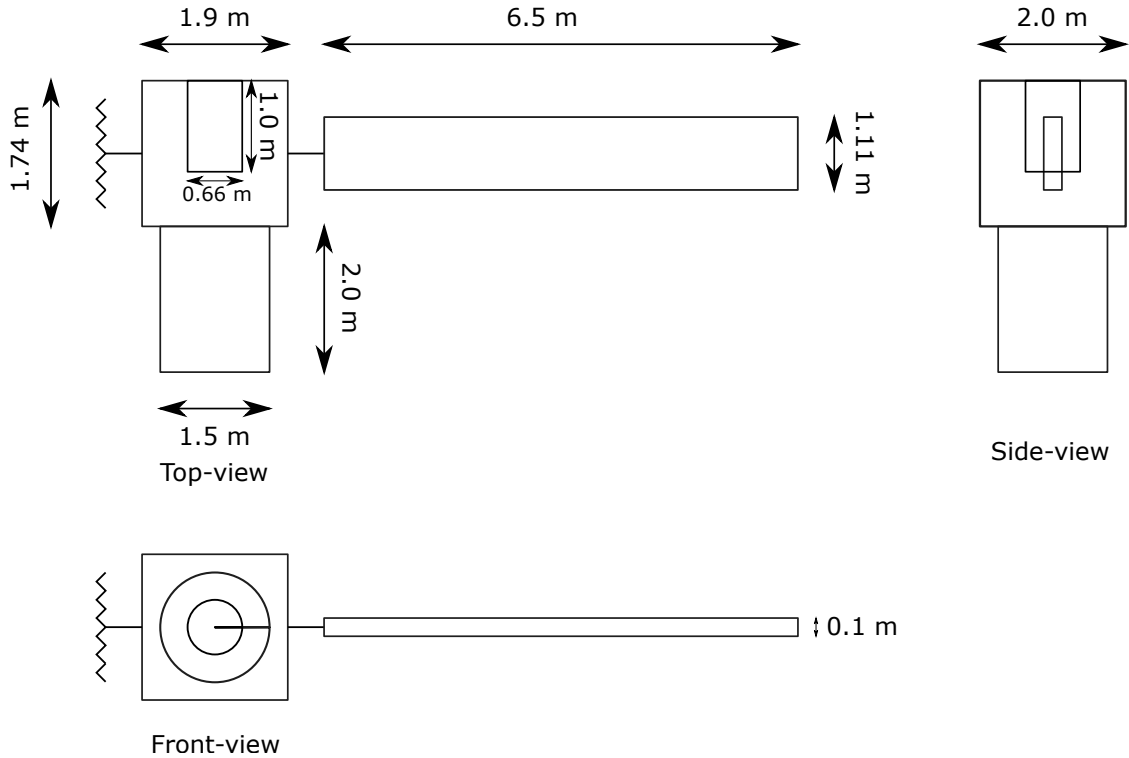


Figure 2: Spacecraft 3-view layout schematic

Mass Budget

The spacecraft wet mass at launch was 1360 kg. From this, the mass budget of the different spacecraft subsystems can be determined using data for preliminary sizing of a spacecraft. [21, Table 14-18] contains a mass breakdown per subsystem as percentage of the spacecraft dry mass for a spacecraft in low-earth orbit with propulsion subsystem. The propellant is typically 27% of the dry mass. Hence propellant mass and dry mass are derived from the wet mass as follows:

$$\begin{aligned} 1.27M_{dry} &= M_{wet} \\ 0.27M_{dry} &= M_{prop} \end{aligned}$$

The spacecraft dry mass is 1071kg, while the propellant mass is 289kg. Furthermore, Aeolus documentation states that payload mass is 266kg [15]. For the solar wing mass, [21, eq 10-13] gives a relation between solar array mass and provided power. From documentation, it is known that the solar arrays provide 2.4kW of power. The relation states that $M_{array} = 0.04P$, assuming a specific power of 25W/kg. Thus the solar array mass is estimated to be 96kg. The mass budget breakdown of the relevant spacecraft components is given in Table 7.

Table 7: Estimated mass budget breakdown of the Aeolus spacecraft.

Spacecraft Component	Mass Breakdown [kg]
Wet mass	1360
Propellant	↪ 289
Dry mass	↪ 1071
Payload	↪ 266
Solar array	↪ 96
Spacecraft bus	↪ 709

2.5 PROBLEM 5

The simplified spacecraft geometry is provided in Figure 2. The spacecraft geometry is simplified in that the mechanisms which attach the solar arrays and the payload to the spacecraft bus are neglected. Furthermore, the solar arrays and payload are assumed to have uniform mass distribution. The spacecraft bus is assumed to have some small variations in mass distributions which is reflected in the off-diagonal terms in the inertia tensor.

The body axes frame is defined such that the origin is located at the center of mass of the spacecraft and z-axis pointing parallel to the axis of the cylinder representing the payload. The y-axis points along the solar array and the x-axis points towards the front.

To determine the spacecraft inertia tensor, first the center of mass is calculated with respect to a specified reference point, chosen to be the center of the bus. Since the spacecraft has two axes of symmetry, only the displacement of the center of mass with respect to this reference point along the z-axis needs to be calculated:

$$\Delta z_{cg} = \frac{M_{pl}\Delta z_{pl} + M_{bus}\Delta z_{bus} + M_{sw}\Delta z_{sw} + M_p\Delta z_p}{M_{pl} + M_{bus} + M_{sw} + M_p} = 0.287m$$

This calculation is also provided in code in Listing 13.12.3. The body axes frame, reference point and axes of symmetry are shown in Figure 3. It should be noted that this center of mass is valid for any configuration of the solar arrays as long as the symmetry is maintained about the z-axis.

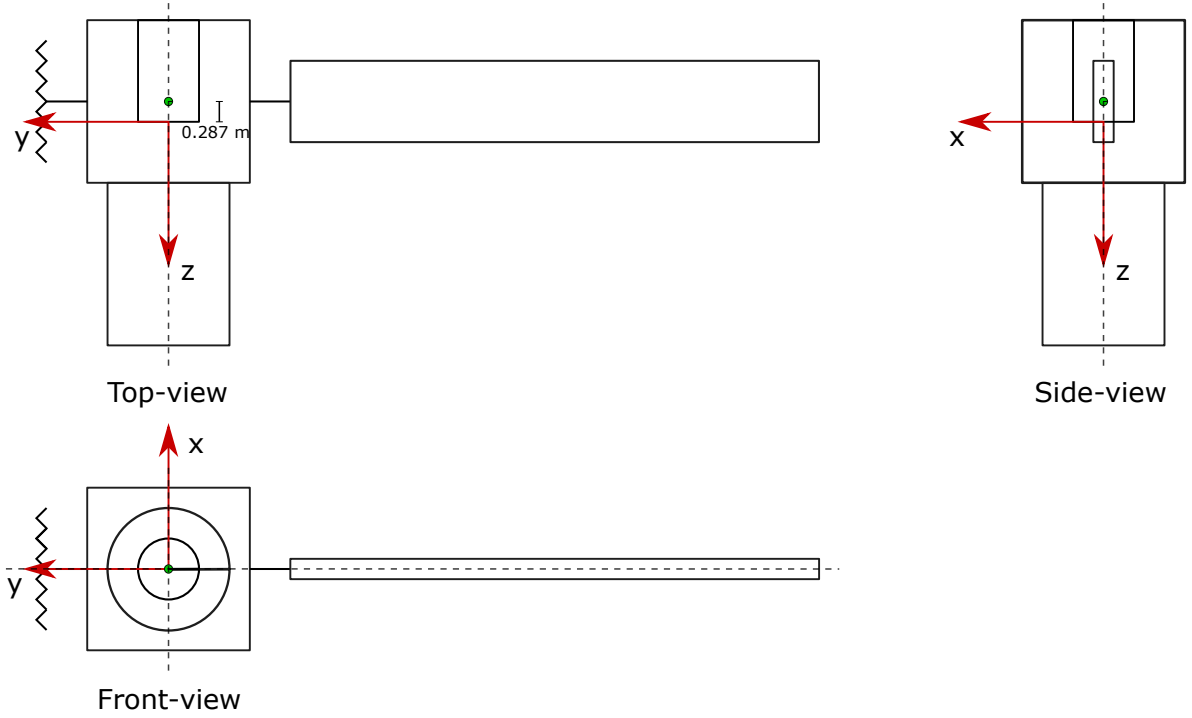


Figure 3: Satellite layout with body axes with respect to point of reference and axes of symmetry.

The inertia tensor with respect to the body axes can now be calculated. Again, the four main mass components of the spacecraft are considered: bus, solar arrays, propellant, and payload. The calculations are provided in the Matlab script Listing 13.12.4. This function might be extended with the functionality to consider the rotation of the solar arrays as well as inertia tensor calculations for the satellite in early-orbit phase in case this proves to be valuable. The full inertia tensor w.r.t. the body frame and principal frame are given by:

$$I_b = \begin{bmatrix} 2769.7 & -15 & 19 \\ -15 & 1440.8 & 25 \\ 19 & 25 & 1892.3 \end{bmatrix} \quad I_p = \begin{bmatrix} 1439.2 & 0 & 0 \\ 0 & 1893.3 & 0 \\ 0 & 0 & 2770.2 \end{bmatrix} \quad (1)$$

2.6 PROBLEM 6

For the discretization of the outer geometry of the spacecraft, I used Matlab's *triangulation* functionality. This function takes as input the Cartesian coordinates and a connectivity list and returns the discretized geometry along with different properties such as the unit vectors normal to each surface and the coordinates of the center of each triangulation element. Additionally, the surface area of each geometry element is computed.

The Matlab function, given in Listing 13.12.5, takes as input all sizing properties of the spacecraft, the coordinates of the center of mass in the inertial reference frame and the solar array rotation. The spacecraft is discretized in three steps. First, the spacecraft bus is discretized using a simple rectangular prism as approximation. Secondly, the same geometrical approximation is made for the solar arrays, where the geometry is split into two pieces. Lastly, the payload geometry is approximated using an N -gonal prisms, where N is a parameter which specifies the fidelity of the approximated cylindrical geometry. The surface area of each triangular element is added to the vector S and returned along with the triangulation object which captures all geometric information. This geometric data is summarized in Table 20.

2.7 PROBLEM 7

A 3D model of the spacecraft geometry is represented in Figure 4. The body axes and principal axes are aligned at the spacecraft center of mass. The geometry can be transformed into an arbitrary inertial frame using an affine transformation of the coordinates defining the geometry. In the figure, the satellite is pitched up 45° and the solar arrays are rotated 60° about the y -axis w.r.t. the body frame. The satellite has also been translated 320 units in the z -direction. The figure also includes two coordinate systems which are fixed with the origin to the spacecraft center of mass. The red, blue green axes represent the body coordinate system and the yellow axes represents the principal coordinate system. The direction cosine matrix from the body axes to principal axes is given by:

$$R_b^p = \begin{bmatrix} 0.012 & -0.021 & -0.999 \\ 0.998 & 0.056 & 0.0109 \\ -0.056 & 0.998 & -0.021 \end{bmatrix} \quad (2)$$

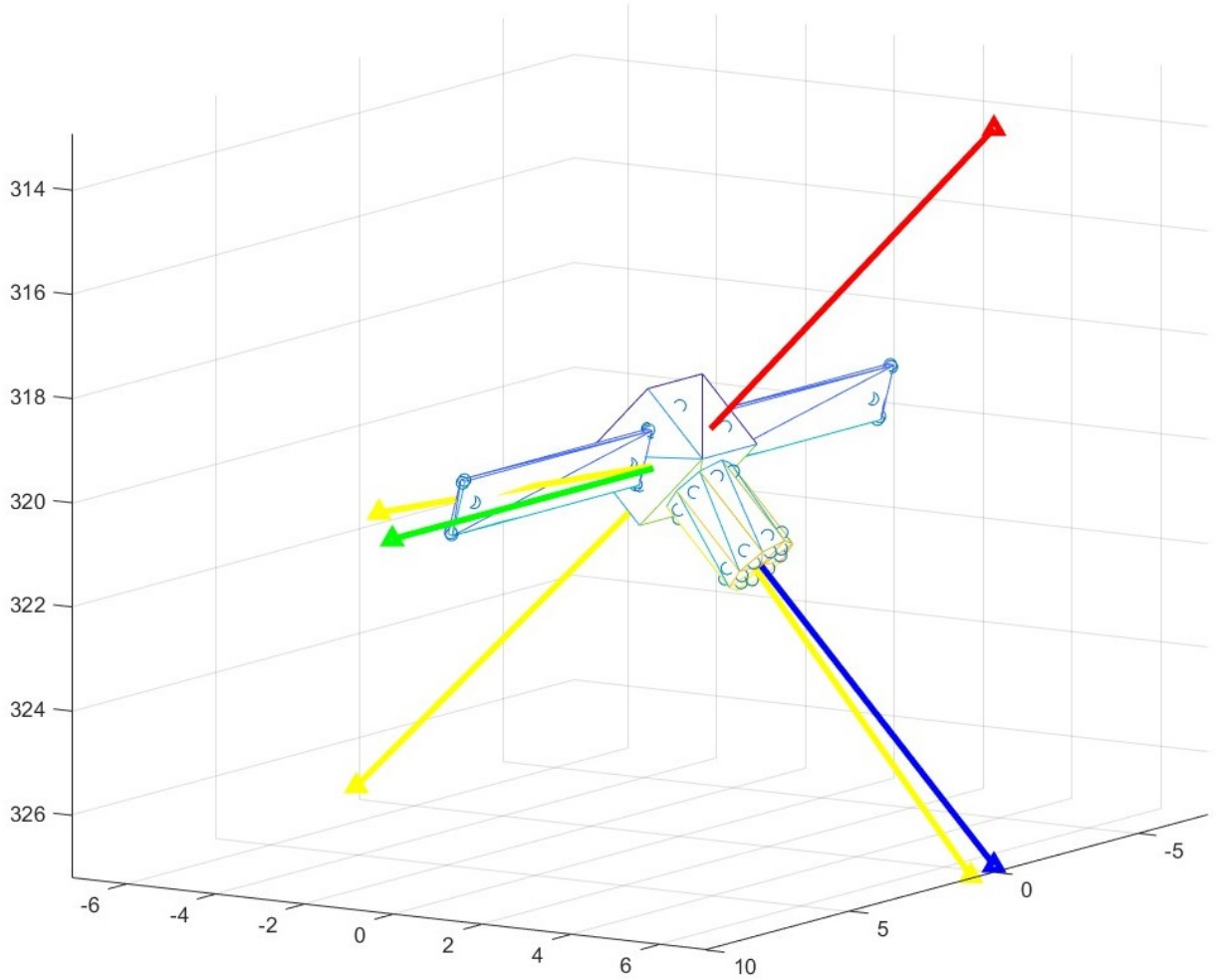


Figure 4: A 3D model of the simplified satellite geometry. The body axes is indicated by the red (x -axis), green (y -axis), blue (z -axis) axes. The principal axes is indicated by the yellow axes. Both are located at the spacecraft center of mass.

3 Satellite Dynamics

3.1 PROBLEM 1

The initial orbital conditions are stated as the six Keplerian elements and are obtained from [20]. The initial orbit conditions are as follows:

$$\begin{aligned} e &= 0.006[-] & \Omega &= 214.9[deg] \\ a &= 6691 \times 10^3[m] & \omega &= 264.2[deg] \\ i &= 96.7[deg] & \nu &= 95.4[deg] \end{aligned}$$

where e is eccentricity, a is semi-major axis, i is inclination, Ω is longitude of the ascending node, ω is argument of periapsis and ν is true anomaly. These parameters are converted to the position and velocity vectors in the geocentric equatorial coordinate system, which is assumed to be the inertial reference frame, using a build-in *Matlab* command.

The initial orbit conditions can be propagated by numerically integrating the equations of motion, derived from $F = ma$. For now, only the gravitational force from Earth is considered. The equations are integrated over four orbital periods using *Matlab*'s *ode113*, which is shown in Figure 5. The code containing the trajectory generation and equations of motion are given in Listing 13.3 and Listing 13.12.6, respectively.

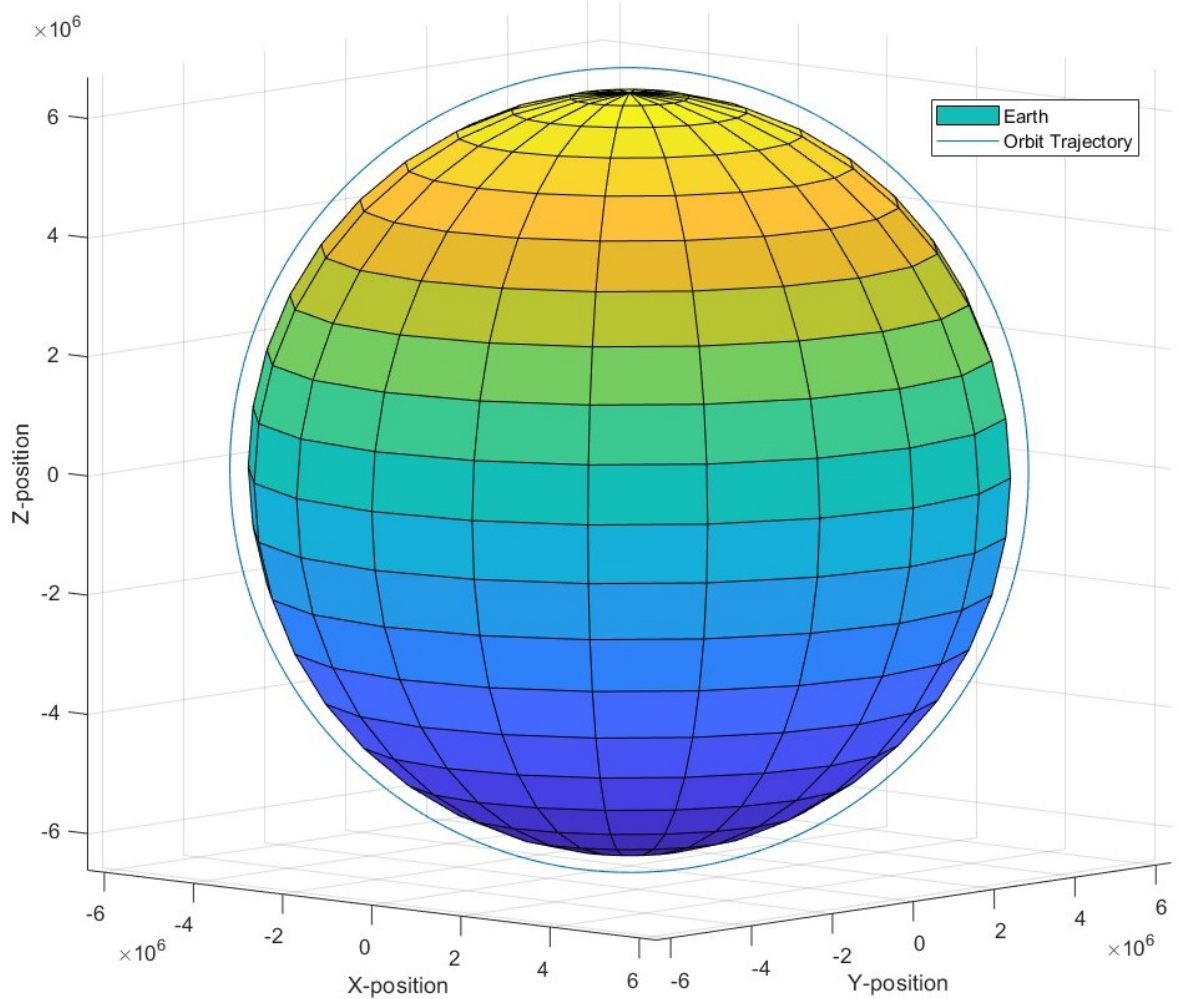


Figure 5: Orbital trajectory of satellite around Earth.

3.2 PROBLEM 2

The rotation matrix from body to principal axes can be found through the following eigenvector/eigenvalue problem:

$$I_b R_b^p = R_b^p I_p \quad (3)$$

where I_b represents the inertia tensor in the body axes frame, I_p represents a diagonal matrix, hence is defined as the inertia tensor in the principal axes frame. Then, the rotation matrix from body axes to principal axes frame is R_b^p . Solving the eigenvector/eigenvalue problem, the eigenvalues give the inertia values along the principal axes and the eigenvectors give the desired rotation matrix. For the considered satellite, the principle moments of inertia are given in Equation 1 as well as the inertia tensor in body axes. The resulting eigenvectors, hence rotation matrix is given by:

$$R_b^p = \begin{bmatrix} 0.012 & -0.021 & -0.999 \\ 0.998 & 0.056 & 0.0109 \\ -0.056 & 0.998 & -0.021 \end{bmatrix} \quad (4)$$

3.3 PROBLEM 3

A 3D model of the discretized spacecraft geometry was obtained in subsection 2.6, while a full 3D-view is given in Figure 4. The origin of the body axes and principal axes are aligned at the spacecraft center of mass. Figure 6, Figure 7, and Figure 8 show the satellite model from a 2d side-view along the xz -plane, yz -plane and xy -plane, respectively. Each figure also includes two coordinate frames with the origin fixed to the spacecraft center of mass. The red, blue green axes represent the body coordinate system and the yellow axes represents the principal coordinate system. The direction cosine matrix from the body axes to principal axes is given in Equation 4.

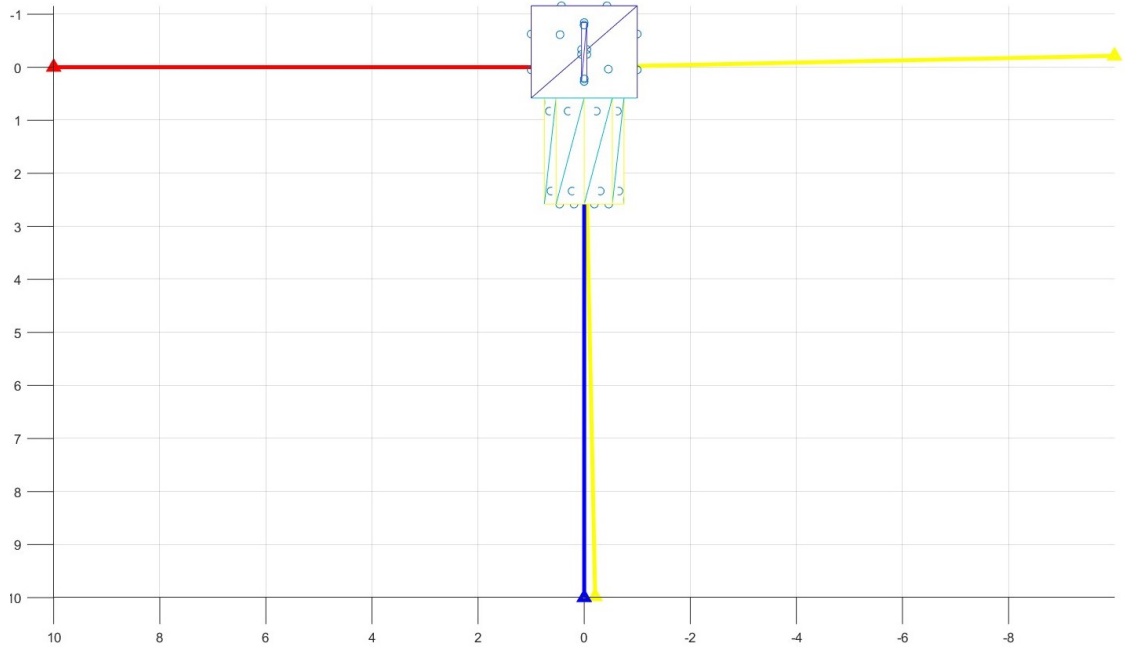


Figure 6: xz -view of the spacecraft geometry model. The body axes is indicated by the red (x -axis), green (y -axis), blue (z -axis) axes. The principal axes is indicated by the yellow axes. Both are located at the spacecraft center of mass.

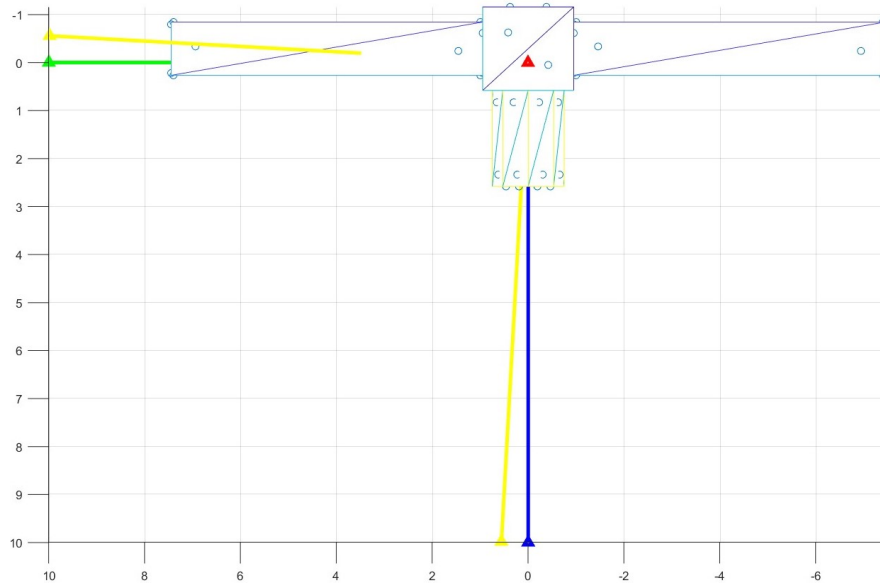


Figure 7: yz -view of the spacecraft geometry model. The body axes is indicated by the red (x -axis), green (y -axis), blue (z -axis) axes. The principal axes is indicated by the yellow axes. Both are located at the spacecraft center of mass.

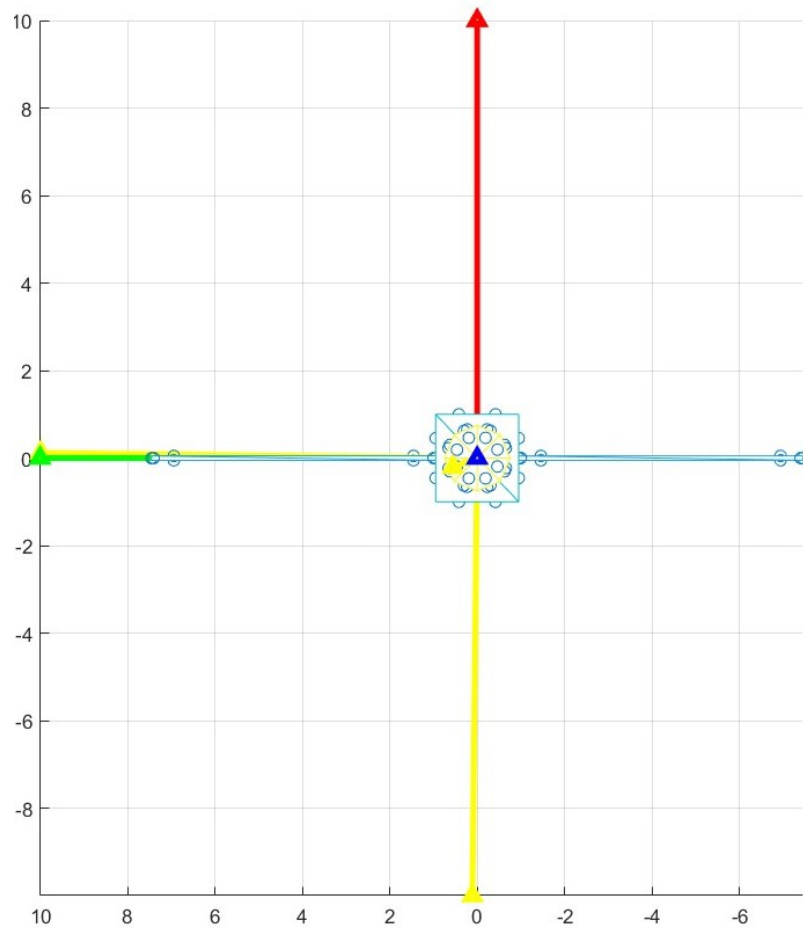


Figure 8: xy -view of the spacecraft geometry model. The body axes is indicated by the red (x -axis), green (y -axis), blue (z -axis) axes. The principal axes is indicated by the yellow axes. Both are located at the spacecraft center of mass.

3.4 PROBLEM 4

The Euler equations in the principal axes frame are given by:

$$\begin{aligned} I_x \dot{\omega}_x + (I_z - I_y) \omega_y \omega_z &= M_x \\ I_y \dot{\omega}_y + (I_x - I_z) \omega_z \omega_x &= M_y \\ I_z \dot{\omega}_z + (I_y - I_x) \omega_x \omega_y &= M_z \end{aligned} \quad (5)$$

Given no external torques, i.e. constant angular momentum vector $\vec{L} = 0$, the time derivative of the angular velocity vector becomes:

$$\begin{aligned} \dot{\omega}_x &= \frac{(I_y - I_z)}{I_x} \omega_y \omega_z \\ \dot{\omega}_y &= \frac{(I_z - I_x)}{I_y} \omega_z \omega_x \\ \dot{\omega}_z &= \frac{(I_x - I_y)}{I_z} \omega_x \omega_y \end{aligned} \quad (6)$$

The attitude kinematics, where attitude is represented by the Euler angles, are given by:

$$\begin{bmatrix} \dot{\phi} \\ \dot{\theta} \\ \dot{\psi} \end{bmatrix} = \begin{bmatrix} 1 & \sin\phi \tan\theta & \cos\phi \tan\theta \\ 0 & \cos\phi & -\sin\phi \\ 0 & \sin\phi \sec\theta & \cos\phi \sec\theta \end{bmatrix} \begin{bmatrix} \omega_x \\ \omega_y \\ \omega_z \end{bmatrix} \quad (7)$$

where ϕ , θ and ψ are roll, pitch and yaw angles, respectively. The implementation of the attitude dynamics in Matlab is given in Listing 13.12.7.

3.5 PROBLEM 5

The attitude dynamics from previous section are numerically integrated using the *Runge–Kutta–Fehlberg* integration scheme, provided in Listing 13.3. The initial conditions are chosen to be $\eta_0 = [0 \ 0 \ 0]^T [deg]$ for attitude and $\omega_0 = [2 \ 3 \ 1]^T [deg/s]$ for angular velocity. The results of integrating the attitude dynamics given these initial conditions are given in Figure 19 and Figure 10, showing angular velocity trajectories and attitude angle trajectories, respectively.

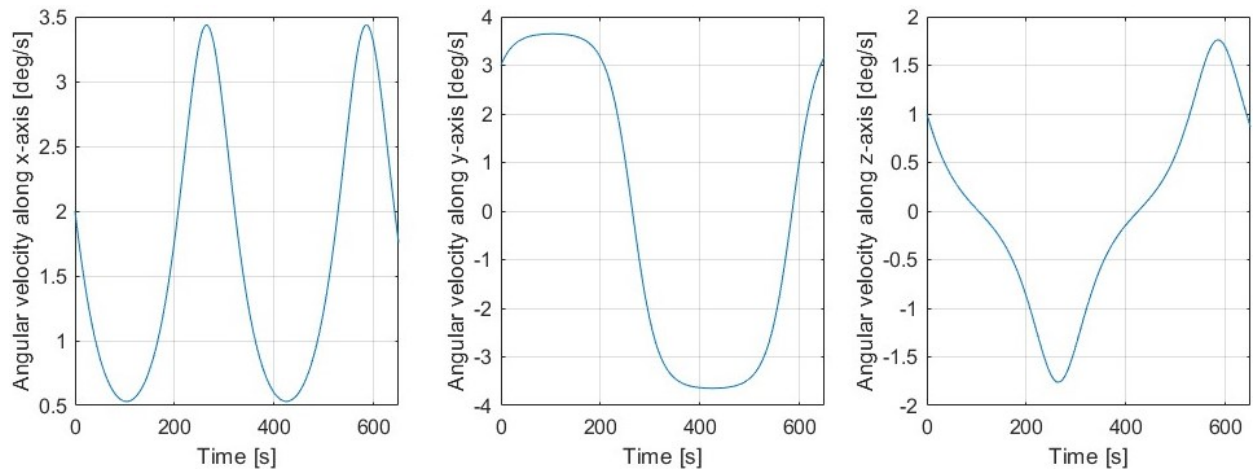


Figure 9: Angular velocity trajectories, decomposed in the roll rate, pitch rate and yaw rate.

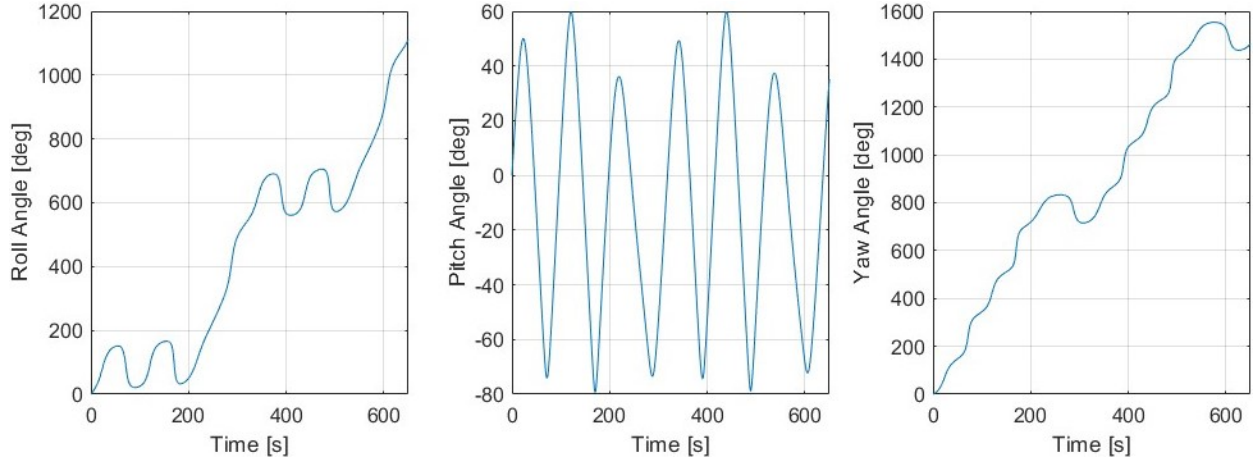


Figure 10: Attitude trajectories, decomposed in roll angle, pitch angle and yaw angle.

In order to verify correctness of the implementation, the angular momentum is calculated over time. Since no external torques apply, the angular momentum should be constant, which is the case as can be seen from Figure 11.

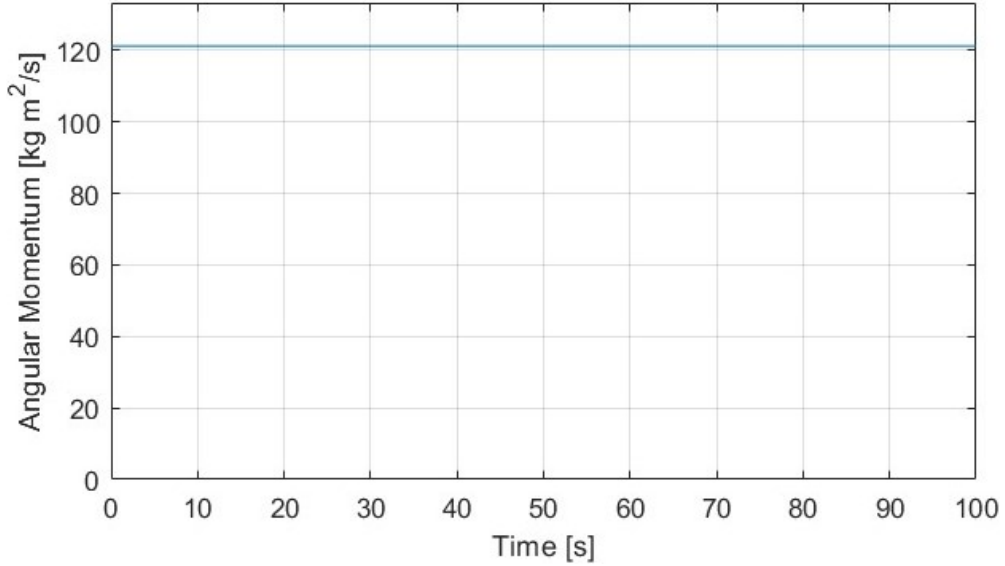


Figure 11: Conservation of angular momentum under no external torques.

3.6 PROBLEM 6

The equations for the rotational kinetic energy and momentum ellipsoids is given by:

$$\begin{aligned} \text{Energy ellipsoid: } & \frac{\omega_x^2}{2T/I_x} + \frac{\omega_y^2}{2T/I_y} + \frac{\omega_z^2}{2T/I_z} = 1 \\ \text{Momentum ellipsoid: } & \frac{\omega_x^2}{L^2/I_x^2} + \frac{\omega_y^2}{L^2/I_y^2} + \frac{\omega_z^2}{L^2/I_z^2} = 1 \end{aligned} \quad (8)$$

where T is the rotational kinetic energy and L is the angular momentum, given by following equations:

$$\begin{aligned} L^2 &= \omega_x^2 I_x^2 + \omega_y^2 I_y^2 + \omega_z^2 I_z^2 \\ 2T &= \omega_x^2 I_x + \omega_y^2 I_y + \omega_z^2 I_z \end{aligned} \quad (9)$$

Hence, given no external torques, their values are completely specified by the initial angular velocity vector and inertia tensor in principal axes frame, given in Equation 1. For the initial conditions given in previous section, the ellipsoid parameters are given in Table 8. These parameters correspond to the length of the three semi-major axes of the ellipsoids. The ellipsoids are plotted in Figure 12.

Table 8: Ellipsoid parameters corresponding to the energy and momentum ellipsoids.

Ellipsoid	Parameters		
	a	b	c
Energy	0.0736	0.0641	0.0530
Momentum	0.0842	0.0640	0.0437

3.7 PROBLEM 7

The polhode corresponds to the angular velocity trajectory computed earlier through numerically integrating the attitude dynamics. Plotting the polhode together with the energy and momentum ellipsoids results in Figure 12. As can be seen, the polhode (green) is the intersection between the energy ellipsoid (red) and the momentum ellipsoid (blue).

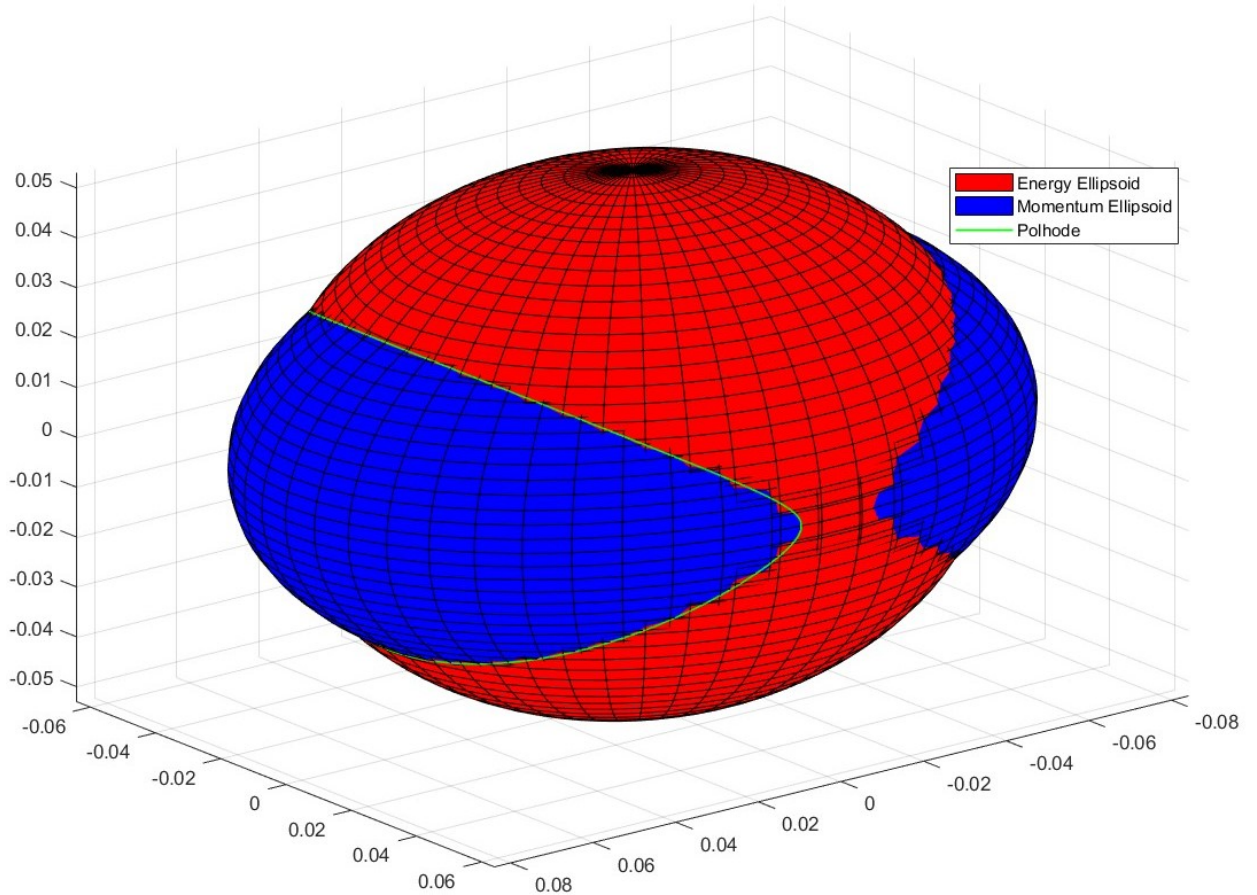


Figure 12: 3D plot of the polhode (green), the energy ellipsoid (red) and the momentum ellipsoid (blue). The polhode is the intersection between the energy and momentum ellipsoid.

3.8 PROBLEM 8

The polhode is plotted in the yz -plane, xz -plane and the xy -plane in Figure 13, Figure 14 and Figure 15, respectively. These conic section views correspond to theory since this corresponds to the stable situation in which the angular velocity vector rotates about the minimum inertia axis. The semi-major axes of both ellipsoids match with the values given in Table 8.

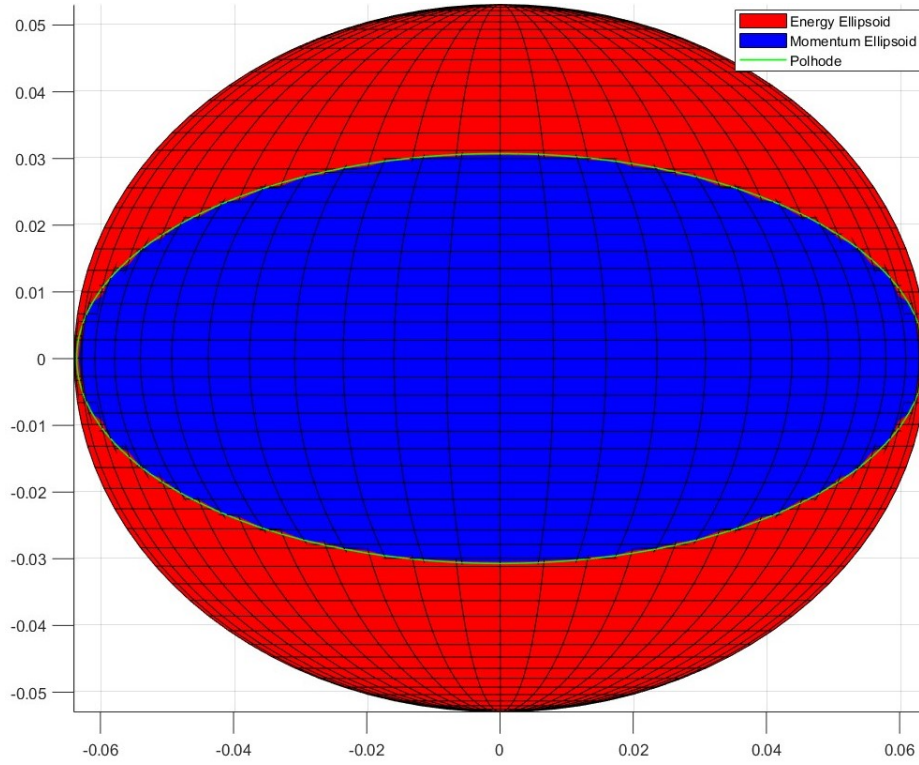


Figure 13: Energy and momentum ellipsoid and polhode as seen from first principal axis, which corresponds to minimum inertia axis.

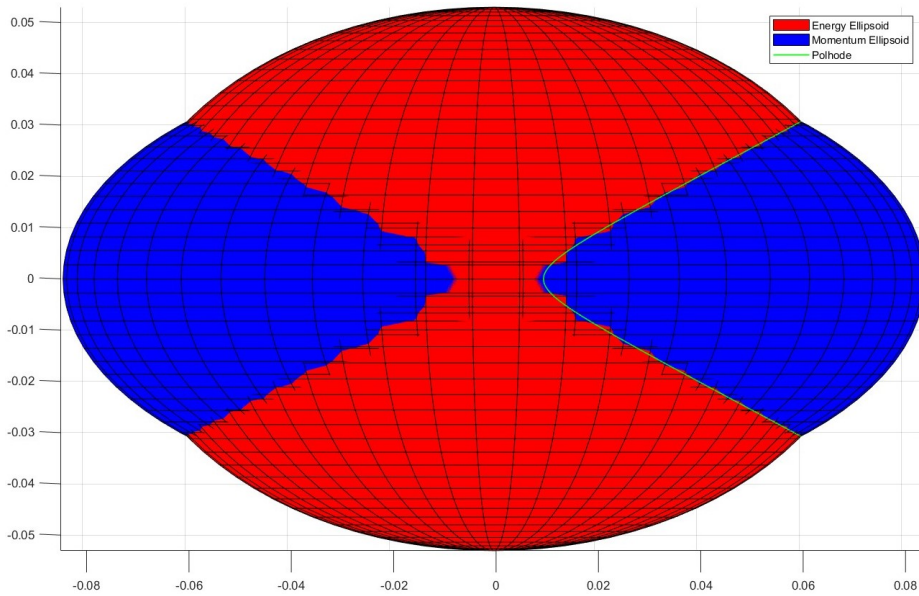


Figure 14: Energy and momentum ellipsoid and polhode as seen from second principal axis, which corresponds to intermediate inertia axis.

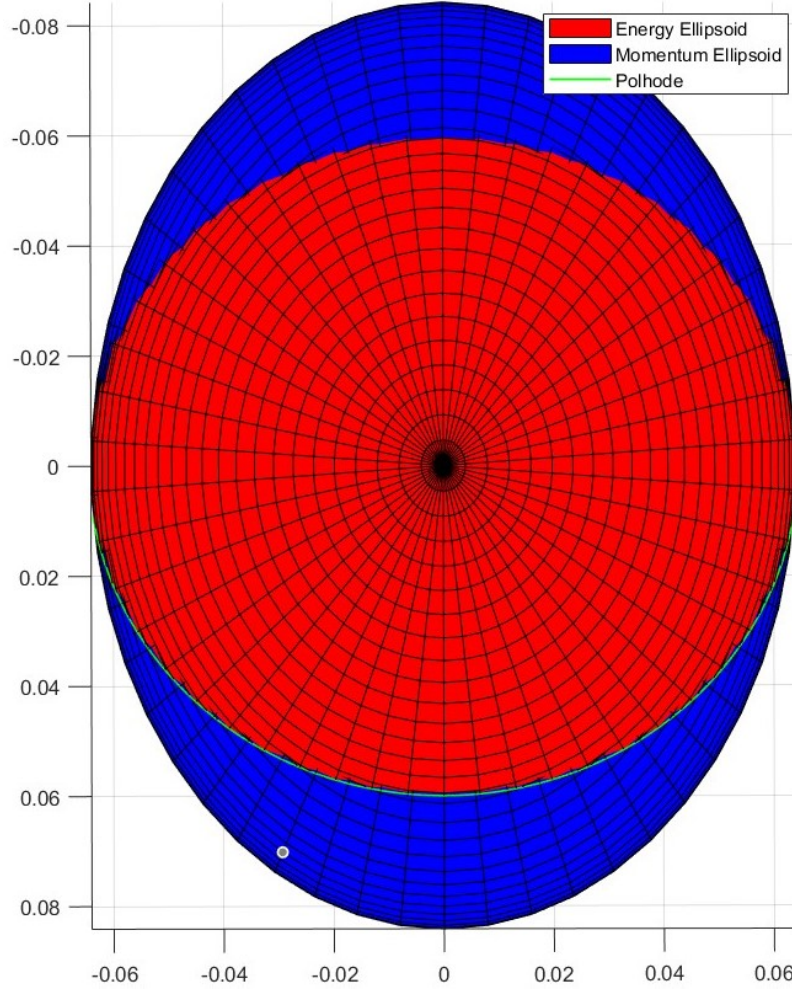


Figure 15: Energy and momentum ellipsoid and polhode as seen from first principal axis, which corresponds to maximum inertia axis.

3.9 PROBLEM 9

Repeating above experiment for the the initial condition of $\omega_0 = [1 \ 2 \ 4]^T [deg]$, the ellipsoid parameters are given in Table 9. The polhode is plotted in the yz -plane, xz -plane and the xy -plane in Figure 16, Figure 17 and Figure 18, respectively. These conic section views correspond to the stable situation in which the angular velocity vector rotates about the maximum inertia axis.

Table 9: Ellipsoid parameters corresponding to the energy and momentum ellipsoids.

Ellipsoid	Parameters		
	a	b	c
Energy	0.1062	0.0926	0.0766
Momentum	0.1431	0.1088	0.0743

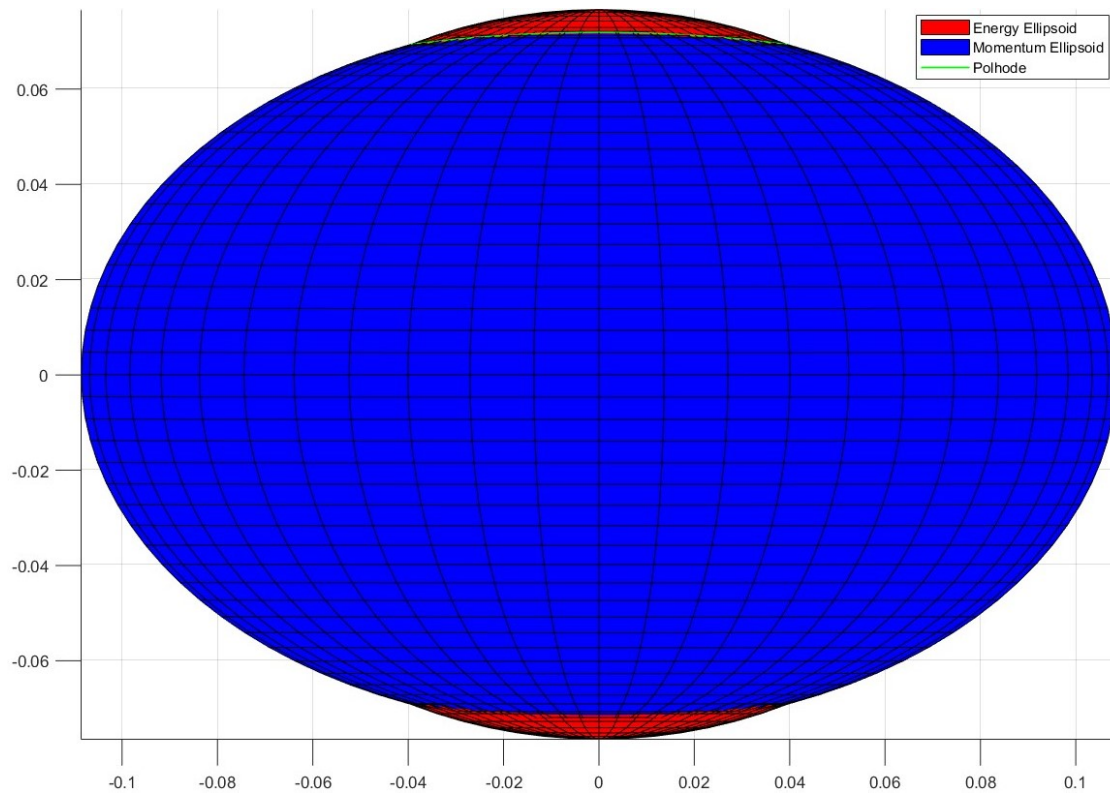


Figure 16: Energy and momentum ellipsoid and polhode as seen from first principal axis, which corresponds to minimum inertia axis.

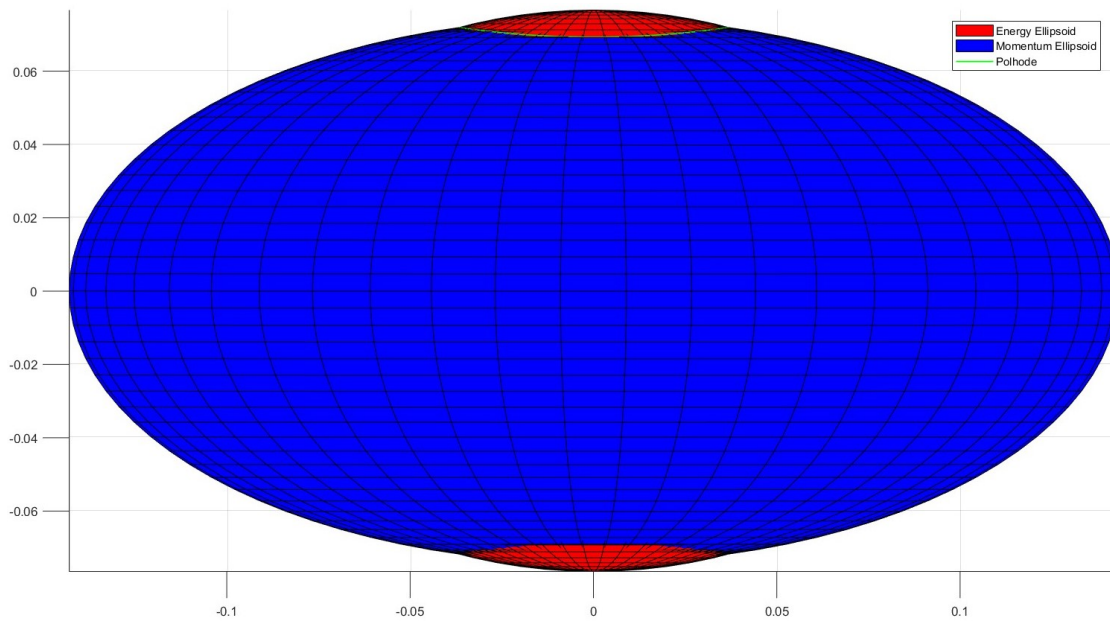


Figure 17: Energy and momentum ellipsoid and polhode as seen from second principal axis, which corresponds to intermediate inertia axis.

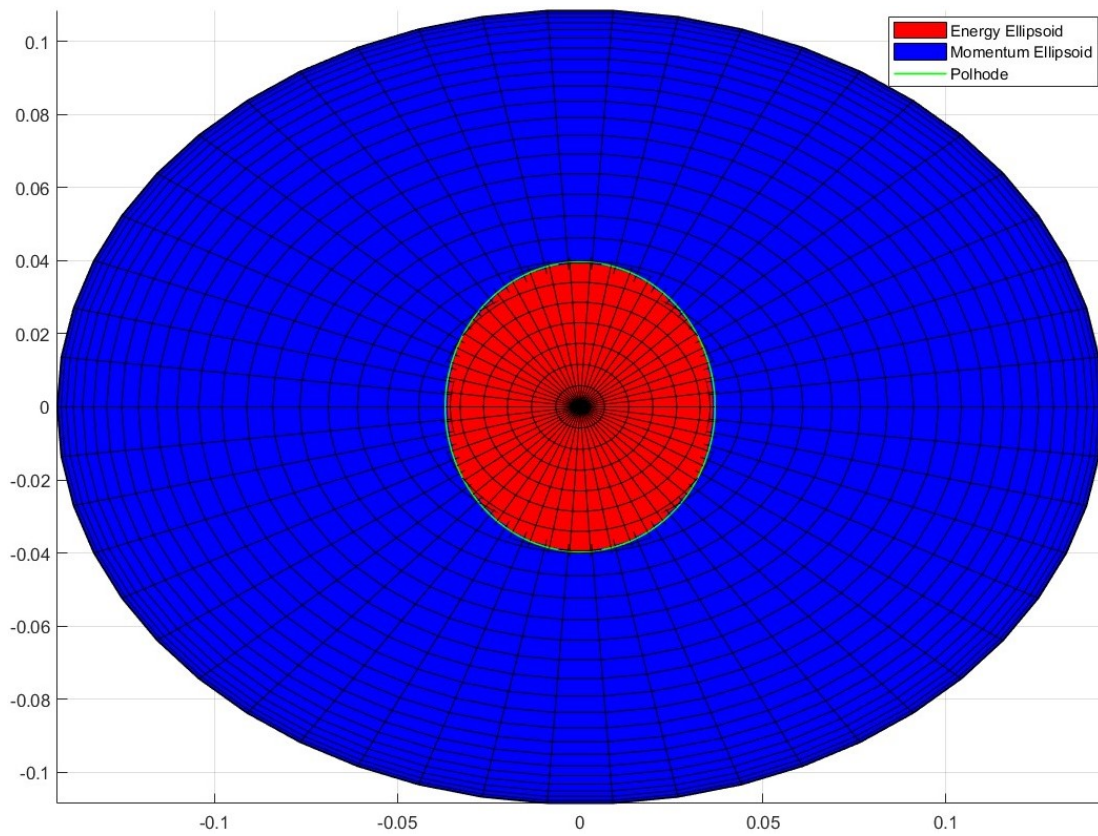


Figure 18: Energy and momentum ellipsoid and polhode as seen from first principal axis, which corresponds to maximum inertia axis.

4 Satellite Kinematics

4.1 PROBLEM 1

The attitude dynamics for an axial-symmetric satellite ($I_x = I_y \neq I_z$) are numerically integrated using the *Runge–Kutta–Fehlberg* integration scheme. The initial conditions are chosen to be $\omega_0 = [2 \ 3 \ 1]^T$ [deg/s] for angular velocity. The results of integrating the attitude dynamics given these initial conditions are given in Figure 19, showing angular velocity trajectories.

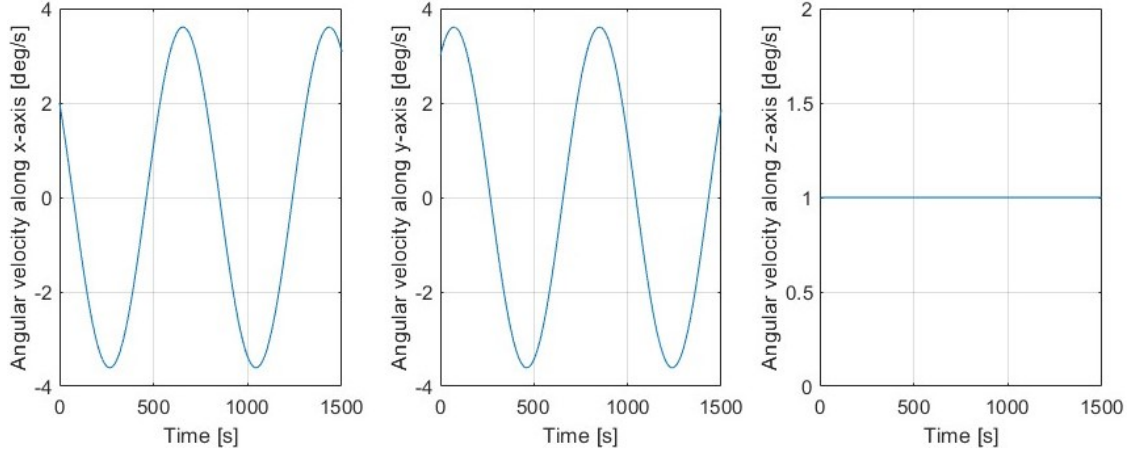


Figure 19: Angular velocity trajectories, decomposed in the roll rate, pitch rate and yaw rate, for an axial-symmetric satellite ($I_x = I_y \neq I_z$).

4.2 PROBLEM 2

The analytical solution to the Euler equations for the axial-symmetric satellite are given by:

$$\omega = \begin{bmatrix} \omega_x \\ \omega_y \\ \omega_z \end{bmatrix} = \begin{bmatrix} \omega_{x_0} \cos(\lambda t) - \omega_{y_0} \sin(\lambda t) \\ \omega_{y_0} \cos(\lambda t) + \omega_{x_0} \sin(\lambda t) \\ \omega_{z_0} \end{bmatrix} \quad (10)$$

where $\lambda = \frac{I_z - I_x}{I_x} \omega_{z_0}$. The time trajectories of the analytical solution angular velocities are given in Figure 20.

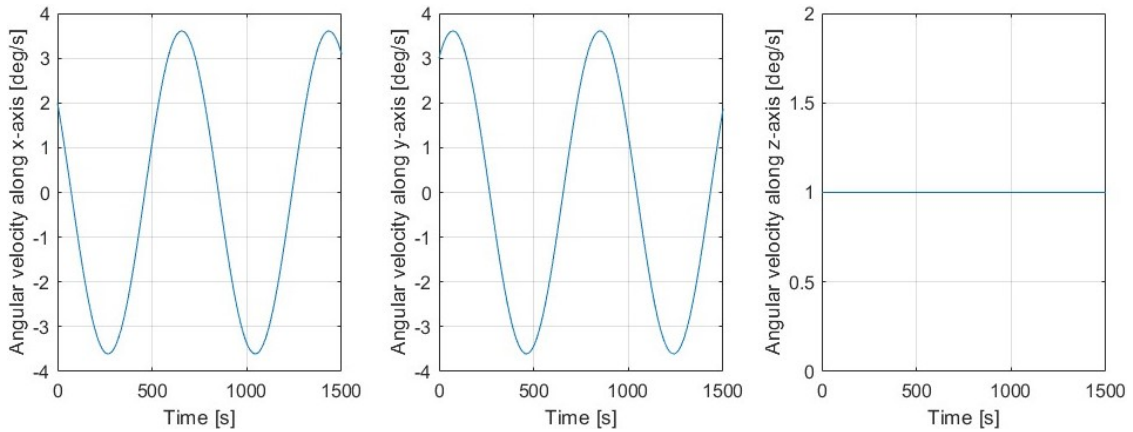


Figure 20: Angular velocity trajectories, decomposed in the roll rate, pitch rate and yaw rate, for an axial-symmetric satellite ($I_x = I_y \neq I_z$) obtained by analytically solving Euler's equations.

4.3 PROBLEM 3

The comparison between the analytically and numerically obtained solution to the Euler equation for the angular velocity is shown in Figure 21. The error between the solutions is down to floating point precision which indicates that the numerical solution agrees with the analytical solution.

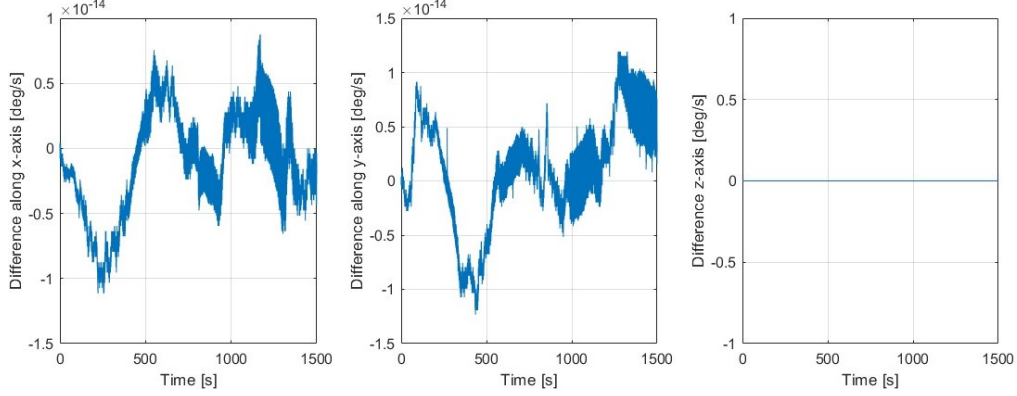


Figure 21: Angular velocity trajectories, decomposed in the roll rate, pitch rate and yaw rate, for an axial-symmetric satellite ($I_x = I_y \neq I_z$) obtained by analytically solving Euler's equations.

The angular velocity components along the axes with equal inertia represent harmonic motion, while the remaining component retains its initial condition. Furthermore, for both angular velocity and angular momentum, the magnitude of the equal inertia components, the third component and overall vector magnitude are constant, as shown in Figure 22 and Figure 23.

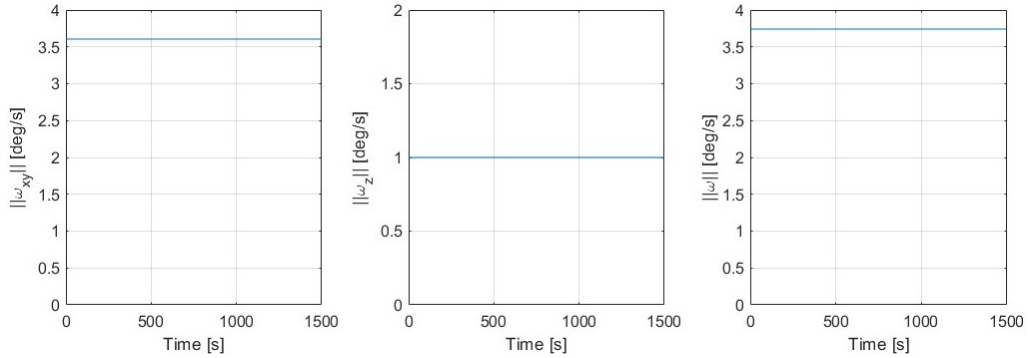


Figure 22: Magnitude plots of ω_{xy} , ω_z and ω .

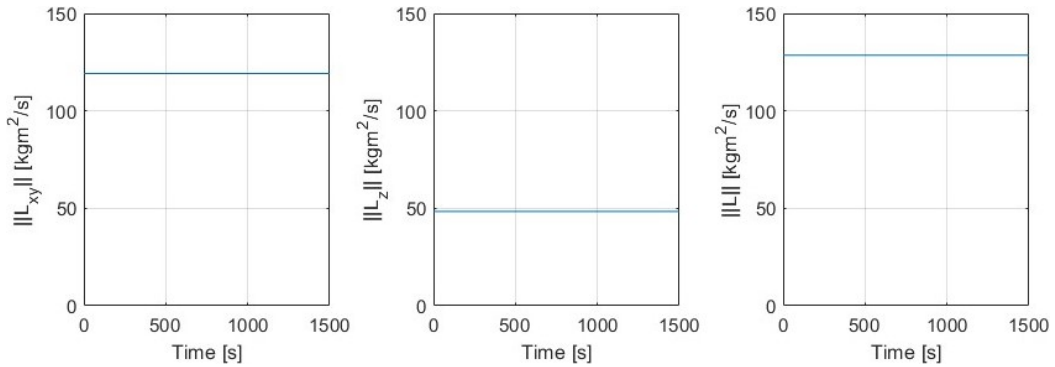


Figure 23: Magnitude plots of L_{xy} , L_z and L .

4.4 PROBLEM 4

The nominal choice of attitude representation for the principal axes frame is the quaternion parameterization. The quaternion has 4 parameters $\mathbf{q} = [q_0 \ q_1 \ q_2 \ q_3]$. The quaternion follows the following convention:

$$\mathbf{q} = \begin{bmatrix} \cos(\phi/2) \\ e_1 \sin(\phi/2) \\ e_2 \sin(\phi/2) \\ e_3 \sin(\phi/2) \end{bmatrix} = \cos(\phi/2) + \sin(\phi/2) (e_1 \mathbf{i} + e_2 \mathbf{j} + e_3 \mathbf{k}) \quad (11)$$

Then the rotation matrix from the inertial frame to the principal axes frame, using above convention, is given by:

$$R_i^p = \begin{bmatrix} q_0^2 + q_1^2 - q_2^2 - q_3^2 & 2q_1q_2 + 2q_0q_3 & 2q_1q_3 - 2q_0q_2 \\ 2q_1q_2 - 2q_0q_3 & q_0^2 - q_1^2 + q_2^2 - q_3^2 & 2q_2q_3 + 2q_0q_1 \\ 2q_1q_3 + 2q_0q_2 & 2q_2q_3 - 2q_0q_1 & q_0^2 - q_1^2 - q_2^2 + q_3^2 \end{bmatrix} \quad (12)$$

The kinematic equations of motion corresponding to the quaternion is as follows:

$$\dot{\mathbf{q}} = \frac{1}{2} W(\mathbf{q})^T \boldsymbol{\omega} \quad (13)$$

where $\boldsymbol{\omega}$ is the angular velocity in the principal axes frame and

$$W(\mathbf{q}) = \begin{bmatrix} -q_1 & q_0 & q_3 & -q_2 \\ -q_2 & -q_3 & q_0 & q_1 \\ -q_3 & q_2 & -q_1 & q_0 \end{bmatrix} \quad (14)$$

The implementation of these attitude kinematics are provided in Listing 13.12.7.

4.5 PROBLEM 5

As a second attitude parameterization for the principal axes frame, the Euler angles are chosen. These are a set of three angles (ϕ, θ, ψ) which represent elemental rotations to go from one frame to another. The convention for the sequence of rotations is chosen to be 1 – 2 – 3 to go from the inertial frame to the principal axes frame. Hence, vectors can be expressed in either frame as follows:

$$\begin{aligned} \mathbf{v}^b &= R_i^b \mathbf{v}^i = R_1(\phi) R_2(\theta) R_3(\psi) \mathbf{v}^i \\ \mathbf{v}^i &= R_b^i \mathbf{v}^b = R_3(-\psi) R_2(-\theta) R_1(-\phi) \mathbf{v}^b \end{aligned} \quad (15)$$

where $\mathbf{v}^b$ and $\mathbf{v}^i$ are vectors in the principal axes and inertial frame, respectively and $A_i(\alpha)$ represents an elemental rotation about axis i through angle α . Then the rotation matrix, using above convention, is given by:

$$R_i^p = \begin{bmatrix} c(\theta)c(\psi) & c(\theta)s(\psi) & -s(\theta) \\ s(\phi)s(\theta)c(\psi) - c(\phi)s(\psi) & s(\phi)s(\theta)s(\psi) + c(\phi)c(\psi) & c(\theta)s(\phi) \\ c(\phi)s(\theta)c(\psi) + s(\phi)s(\psi) & c(\phi)s(\theta)s(\psi) - s(\phi)c(\psi) & c(\theta)c(\phi) \end{bmatrix} \quad (16)$$

The kinematics equation of motion corresponding to the Euler angles, as defined above, is given by the following relation:

$$\begin{bmatrix} \dot{\phi} \\ \dot{\theta} \\ \dot{\psi} \end{bmatrix} = \begin{bmatrix} 1 & \sin(\phi)\tan(\theta) & \cos(\phi)\tan(\theta) \\ 0 & \cos(\phi) & -\sin(\phi) \\ 0 & \sin(\phi)\sec(\theta) & \cos(\phi)\sec(\theta) \end{bmatrix} \begin{bmatrix} \omega_x \\ \omega_y \\ \omega_z \end{bmatrix} \quad (17)$$

where the angular velocity vector is w.r.t. the principal axes frame. The implementation of these attitude kinematics are provided in Listing 13.12.7.

4.6 PROBLEM 6

The Euler equations and kinematic equations of motion are numerically integrated using the *Runge–Kutta–Fehlberg* integration scheme. The initial conditions for the angular velocity in principal axes is chosen to be $\omega_0 = [2 \ 3 \ 1]^T [deg/s]$. Furthermore, the initial conditions for both attitude representations is chosen to be $\eta_0^{euler} = [0 \ 0 \ 0]^T [deg]$ for the Euler angle parameterization and $\eta_0^{quat} = [1 \ 0 \ 0 \ 0]^T [deg]$ for the quaternion parameterization. These initial conditions for attitude correspond to the principal axes frame having the same orientation as the inertial frame. The resulting angular velocity trajectories are given in Figure 24.

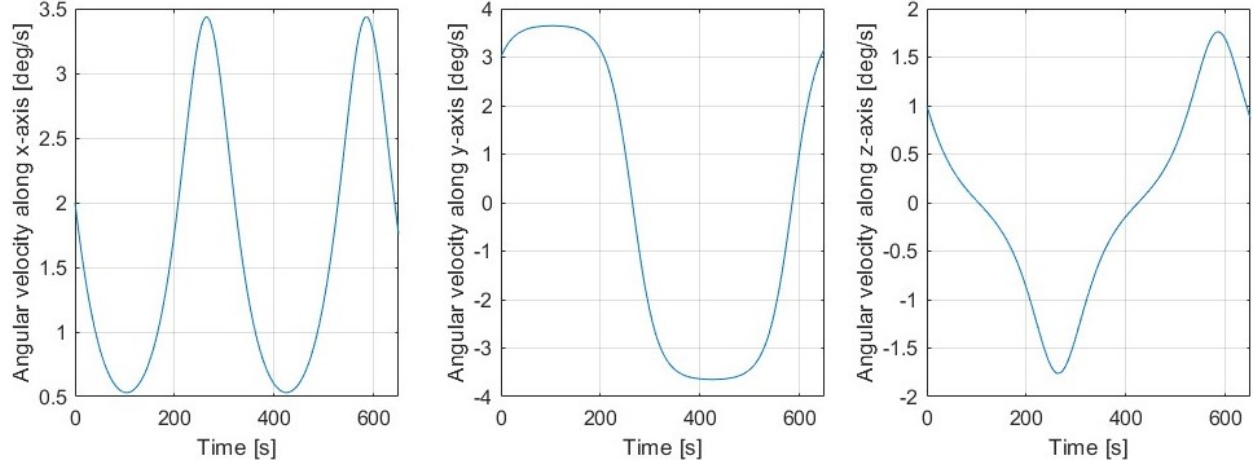


Figure 24: Angular velocity trajectories, decomposed in the roll rate, pitch rate and yaw rate.

Figure 25 shows attitude trajectories for the quaternion parameterization. The figure shows the quaternion trajectories obtained from the kinematics equations and the Euler angle trajectories converted to quaternion using the relation in Equation 18, of which the implementation in code is given in Listing 13.13.2. It can be seen that both trajectories overlap completely.

$$\mathbf{q}_{euler}(\phi, \theta, \psi) = \begin{bmatrix} c(\phi/2)c(\theta/2)c(\psi/2) + s(\phi/2)s(\theta/2)s(\psi/2) \\ -c(\phi/2)s(\theta/2)s(\psi/2) + c(\theta/2)c(\psi/2)s(\phi/2) \\ c(\phi/2)c(\psi/2)s(\theta/2) + s(\phi/2)c(\theta/2)s(\psi/2) \\ c(\phi/2)c(\theta/2)s(\psi/2) - s(\phi/2)c(\psi/2)s(\theta/2) \end{bmatrix} \quad (18)$$

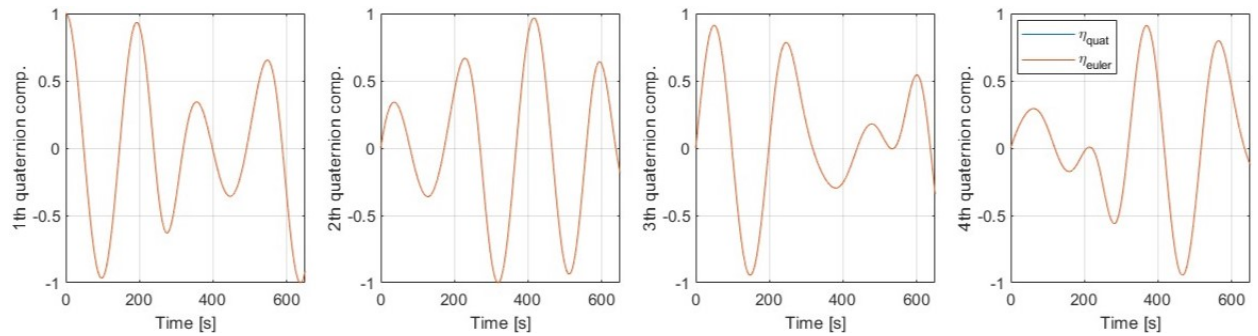


Figure 25: Attitude trajectories using the quaternion parameterization.

Figure 26 shows attitude trajectories for the Euler parameterization. The figure shows the roll, pitch and yaw trajectories obtained from the kinematics equations and the quaternion trajectories converted to Euler angles using the relation in Equation 19, of which the implementation in code is given in Listing 13.13.3. From the figure it can be seen that both attitude representations match up again. In this case, the Euler angles were made to wrap around to be contained within $[-180^\circ, +180^\circ]$.

$$\begin{bmatrix} \phi \\ \theta \\ \psi \end{bmatrix}_{quat} = \begin{bmatrix} \text{atan2}(2q_2q_3 + 2q_0q_1, q_3^2 - q_2^2 - q_1^2 + q_0^2) \\ -\text{asin}(2q_1q_3 - 2q_0q_2) \\ \text{atan2}(2q_1q_2 + 2q_0q_3, q_1^2 + q_0^2 - q_3^2 - q_2^2) \end{bmatrix} \quad (19)$$

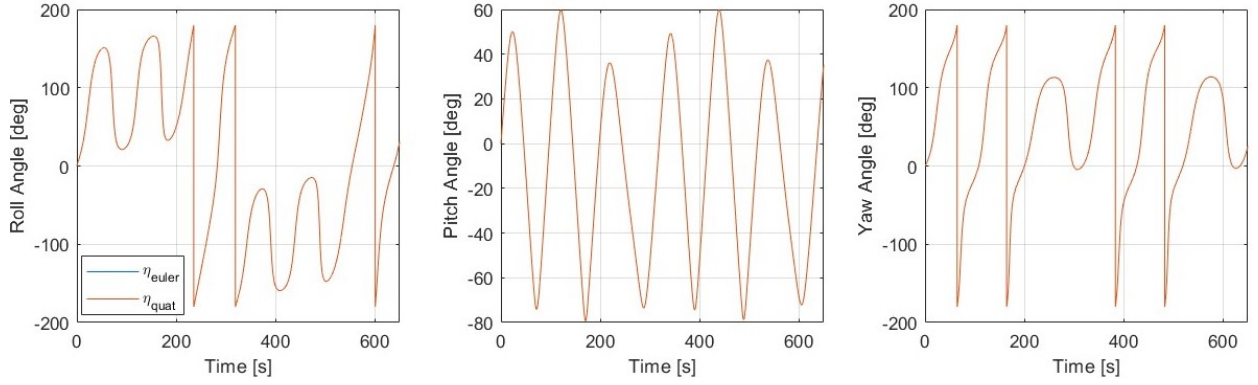


Figure 26: Attitude trajectories using the Euler angle parameterization.

4.7 PROBLEM 7

The angular momentum and angular velocity vectors can be expressed in the inertial frame using the attitude representations defined above. The rotation matrix can be extracted from the Euler angles, which can be used to express the angular momentum vector in the inertial frame as follows:

$$\mathbf{L}^i = \mathbf{R}_p^i \mathbf{L}^p \quad (20)$$

Figure 27 shows the components of the angular momentum vector expressed in the inertial axes frame. Since there are no external torques applied, all components of the angular momentum vector expressed in the inertial frame remain constant.

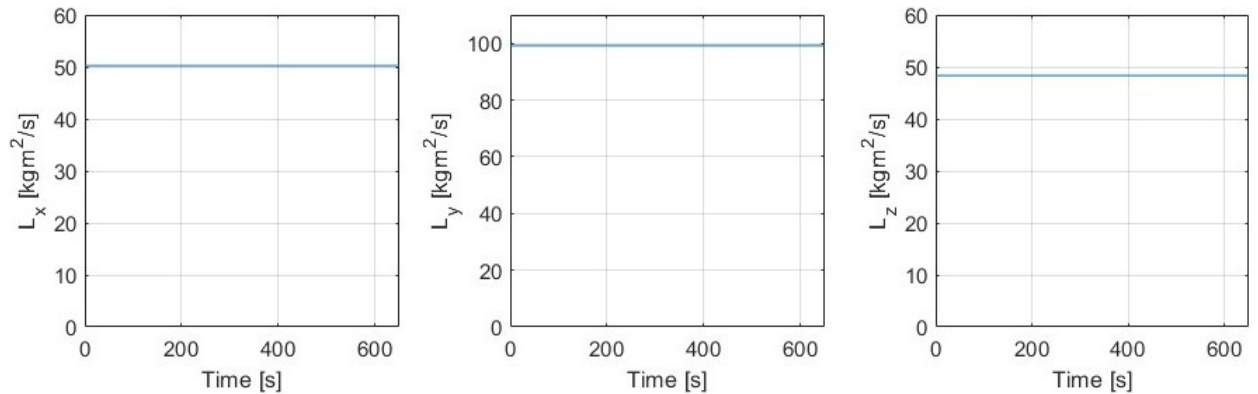


Figure 27: Components of the angular velocity vector expressed in inertial reference frame.

The angular velocity vector expressed in inertial frame follows a complex motion in a plane perpendicular to the angular momentum vector. Figure 28 shows the side-view of the angular velocity motion, which is perpendicular to the angular momentum vector. Figure 29 also has the plane which is perpendicular to the angular momentum vector and contains the herpolhode.

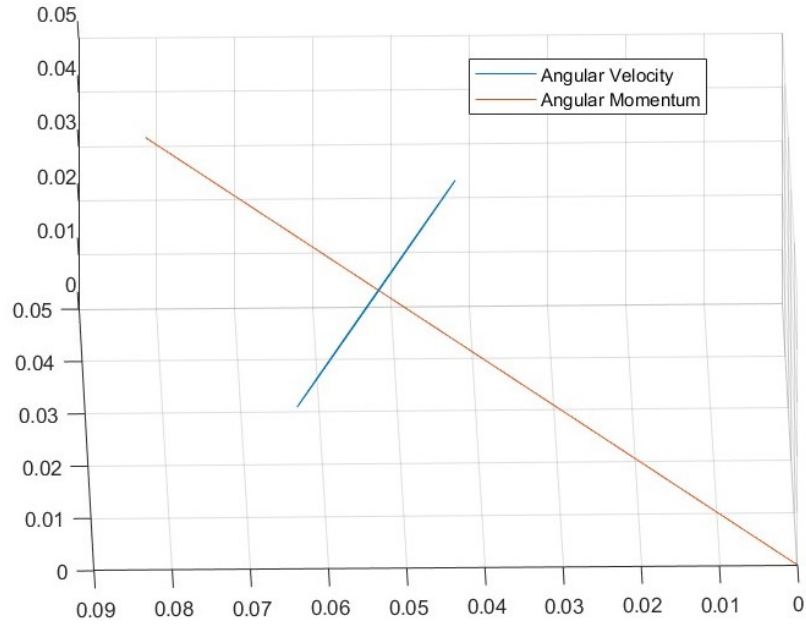


Figure 28: Side-view of herpolhode and angular momentum vector.

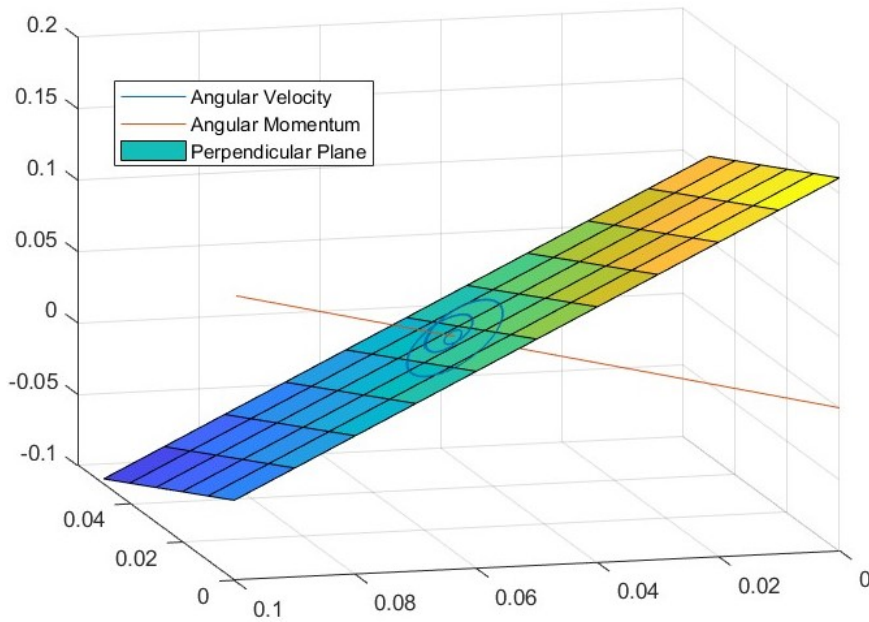


Figure 29: Hepolhode, angular momentum vector and plane which contains the herpolhode and is perpendicular to the angular momentum vector

The unit vectors of the orbital frame, body axes frame, and principal axes frame plotted in 3D as a function of time in inertial coordinates is given as an animated image below. The time span is one full angular velocity period.

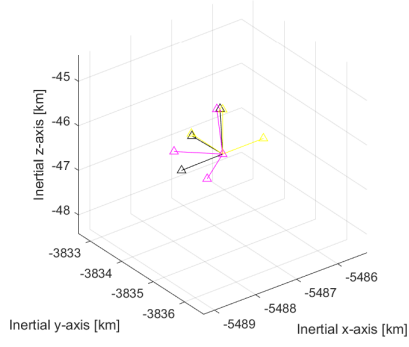


Figure 30: $t = 0[s]$

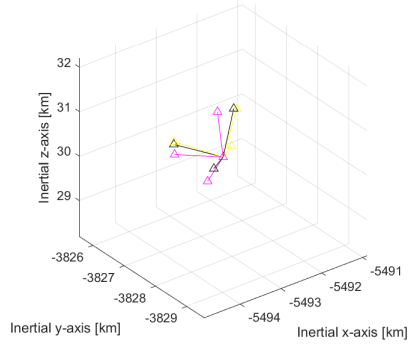


Figure 31: $t = 10[s]$

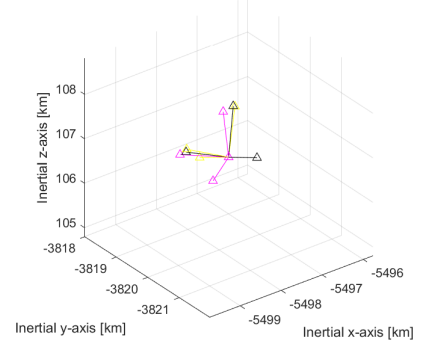


Figure 32: $t = 20[s]$

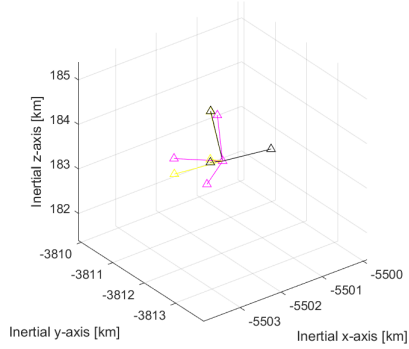


Figure 33: $t = 30[s]$

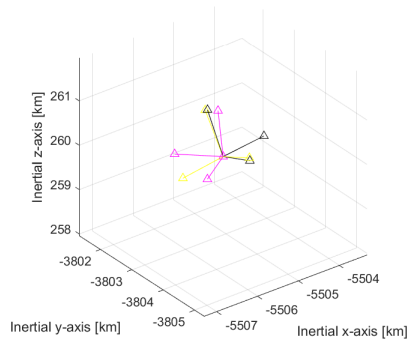


Figure 34: $t = 40[s]$

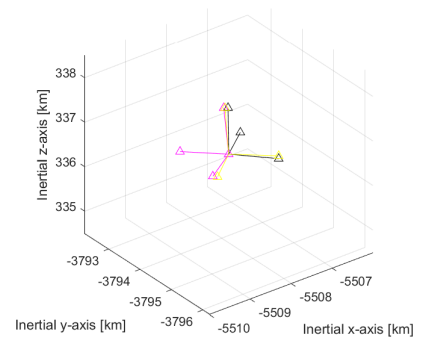


Figure 35: $t = 50[s]$

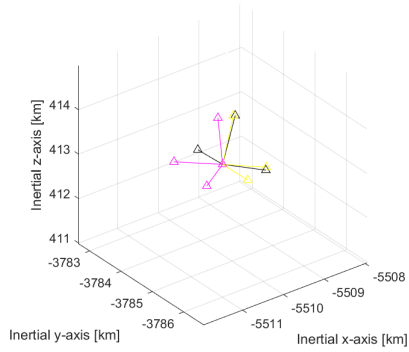


Figure 36: $t = 60[s]$

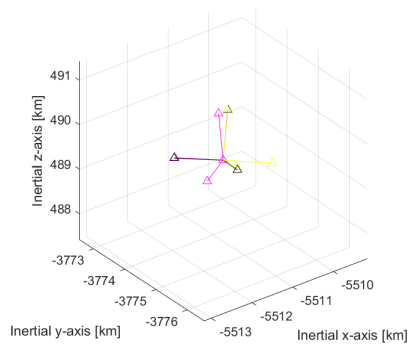


Figure 37: $t = 70[s]$

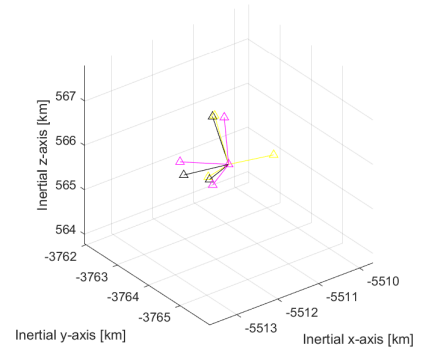


Figure 38: $t = 80[s]$

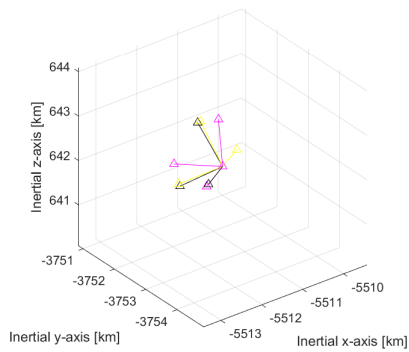


Figure 39: $t = 90[s]$

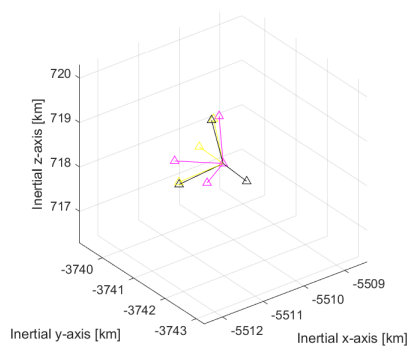


Figure 40: $t = 100[s]$

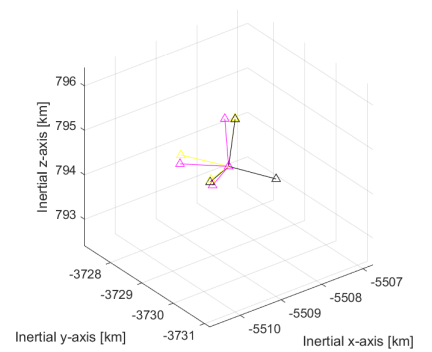


Figure 41: $t = 110[s]$

5 Single- and Dual-spin Stability–

5.1 PROBLEM 1

The equilibrium of the spacecraft under no external torques can be observed by setting the spacecraft to spin about one of its principal axes. In this case, the initial angular velocity is chosen to be $\omega_0 = [0 \ 0 \ 1] [deg/s]$, hence pure rotation about the z -axis. From Figure 42 it can be seen that the angular velocity remains constant over its components. Additionally, Figure 43 shows the Euler angle attitude trajectories, where $\eta_0 = [0 \ 0 \ 0] [deg]$. Here, the angles are not wrapped to be within $[-180^\circ, +180^\circ]$ to better observe the linear increase in the yaw angle.

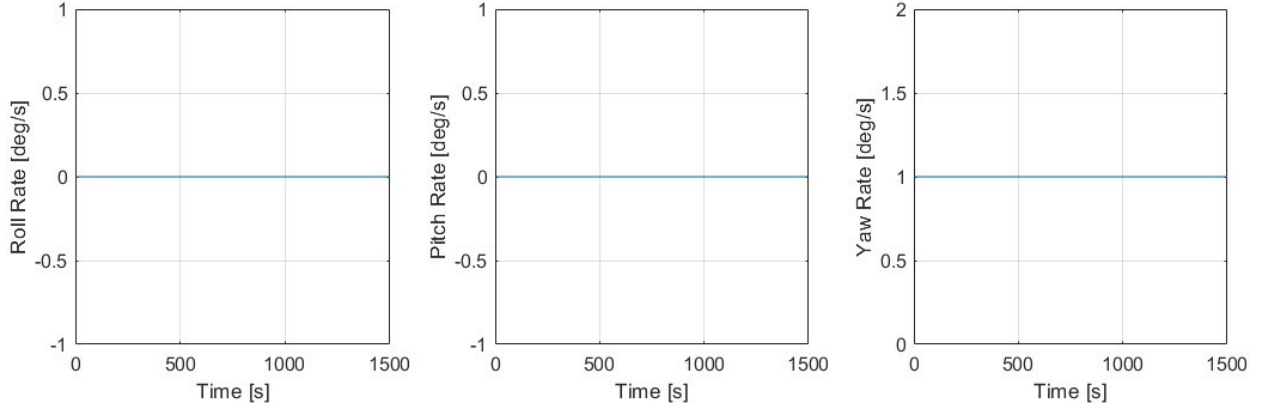


Figure 42: Angular velocity trajectories for $\omega_0 = [0 \ 0 \ 1] [deg/s]$.

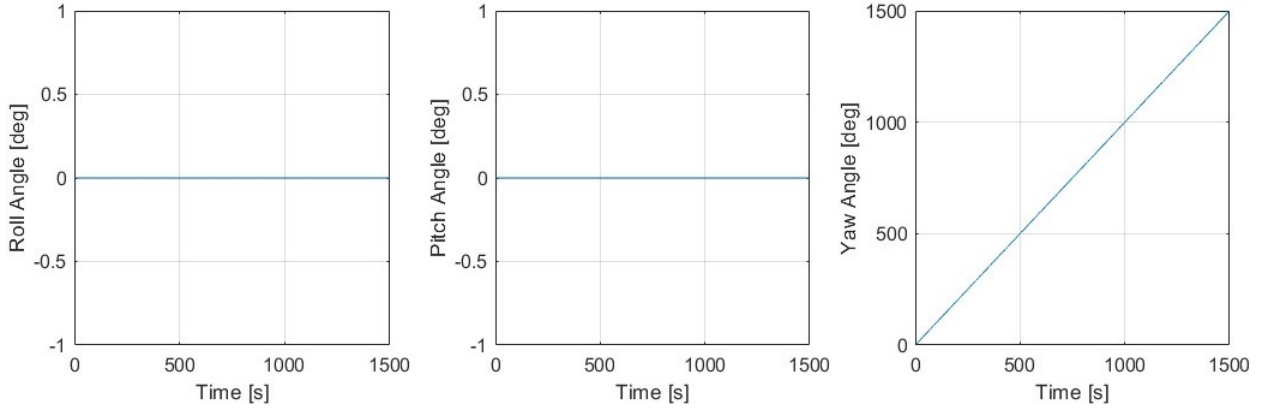


Figure 43: Angular velocity trajectories for $\omega_0 = [0 \ 0 \ 1] [deg/s]$.

Instead of aligning the spacecraft principal axes with the inertial frame, for this experiment the principal axes are aligned with the orbital RTN -frame with a pure rotation about the normal axis. Since the normal axis is aligned with the principal z -axis, this condition also corresponds to an equilibrium condition in which the angular velocity remains constant, as shown in Figure 44. Furthermore, when expressing the orientation of the principal axes w.r.t. the RTN -frame, it is observed that only a linear increase in yaw angle occurs, as shown in Figure 45. This is consistent with the fact that only a pure rotation about the normal axis occurs, hence the other two attitude angles with respect to the orbital frame remain zero. On the other hand, the attitude angles describing the orientation of the principal axes w.r.t. the inertial frame do change with time.

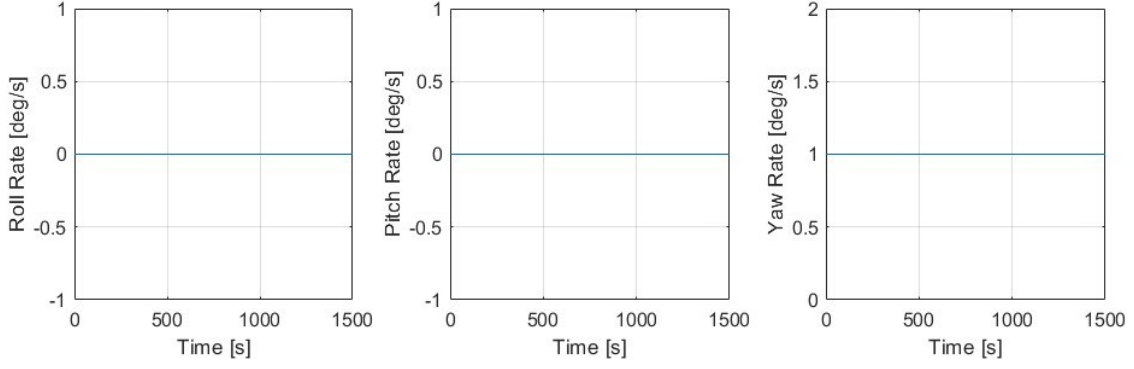


Figure 44: Caption

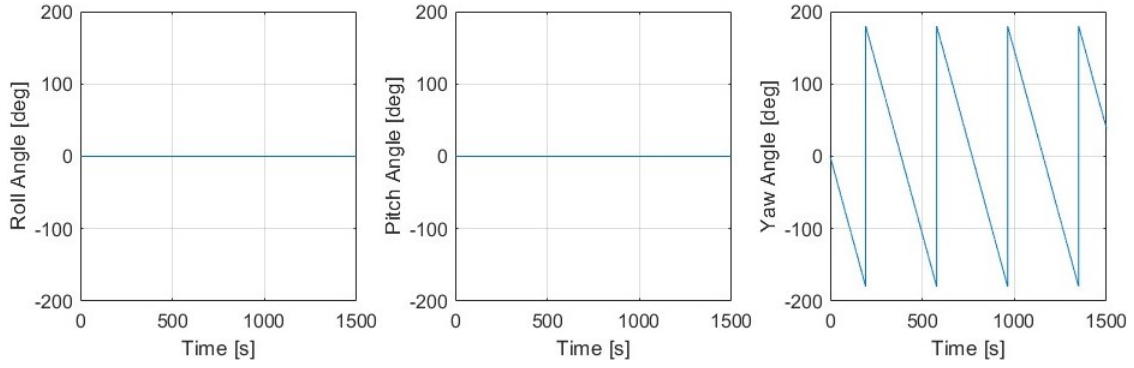


Figure 45: Caption

5.2 PROBLEM 2

The attitude dynamics are simulated starting the the three equilibrium configurations, each time with a small perturbation added. For each case, the equilibrium is perturbed by $\Delta\omega_x = 1 \times 10^{-4}[\text{rad/s}]$, $\Delta\omega_y = 1 \times 10^{-4}[\text{rad/s}]$ and $\Delta\omega_z = 1 \times 10^{-4}[\text{rad/s}]$. The results corresponding to the equilibrium configuration $\omega_0 = [1 \ 0 \ 0] [\text{deg/s}]$ are shown in Figure 46 and Figure 47. This configuration is a rotation about the principal axis of minimum inertia. As can be seen, the attitude is periodically stable in the angular velocity since there is a small oscillation in the pitch and yaw rate, while the roll rate maintains its initial condition. The attitude angles are unstable since the roll angle linearly increase with time.

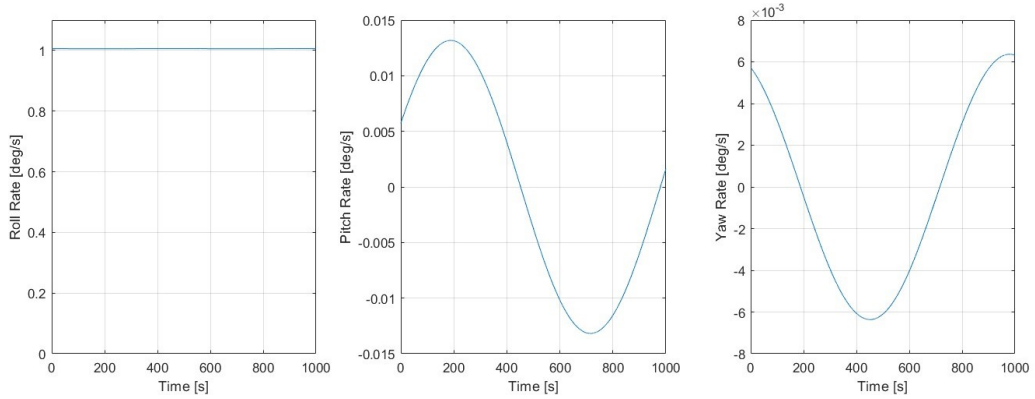


Figure 46: Angular velocity trajectories for a perturbed initial rotation about the principal axis of minimum inertia.

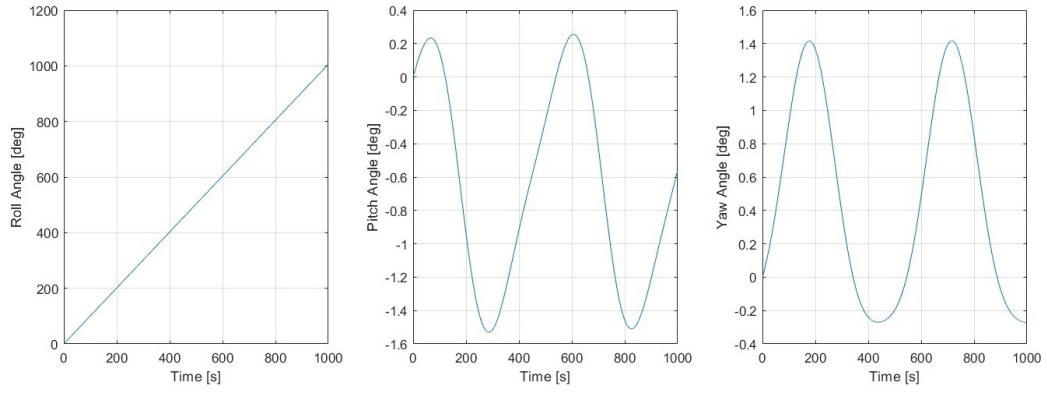


Figure 47: Attitude trajectories for a perturbed initial rotation about the principal axis of minimum inertia.

The results corresponding to the equilibrium configuration $\omega_0 = [0 \ 1 \ 0] [deg/s]$ are shown in Figure 48 and Figure 49. This configuration is a rotation about the intermediate inertia axes. As can be seen, the attitude is unstable in the angular velocity since each component deviates exponentially from its initial condition. The attitude angles are also unstable since the roll and yaw angle grow unbounded.

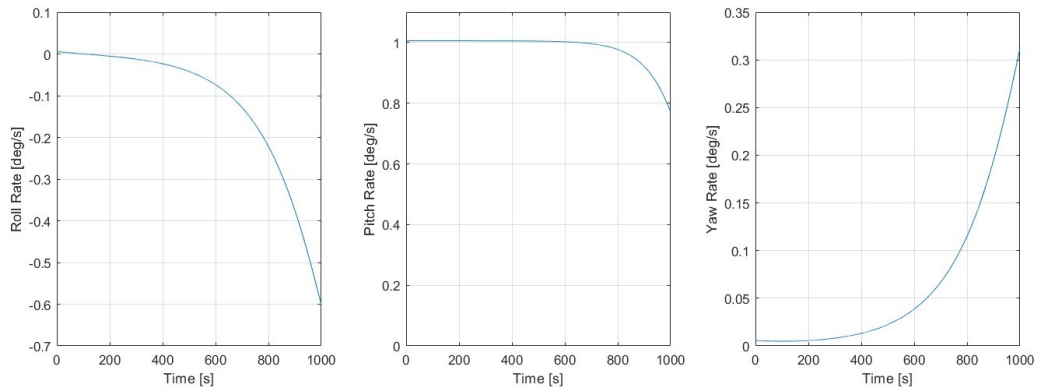


Figure 48: Angular velocity trajectories for a perturbed initial rotation about the principal axis of intermediate inertia.

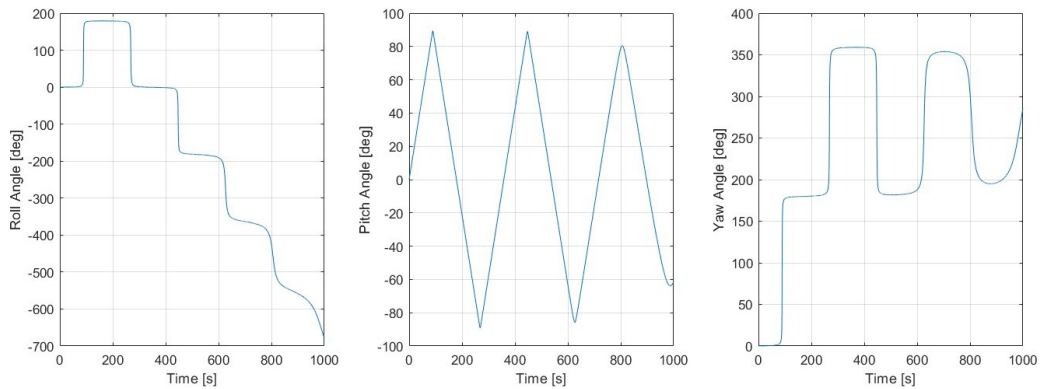


Figure 49: Attitude trajectories for a perturbed initial rotation about the principal axis of intermediate inertia.

The results corresponding to the equilibrium configuration $\omega_0 = [0 \ 0 \ 1] [deg/s]$ are shown in Figure 50 and Figure 51. This configuration is a rotation about the principal axis of maximum inertia. As can be seen, the attitude is periodically stable in the angular velocity since there is a small oscillation in the roll and pitch rate, while the yaw rate maintains its initial condition. The attitude angles are unstable since the yaw angle linearly increase with time.

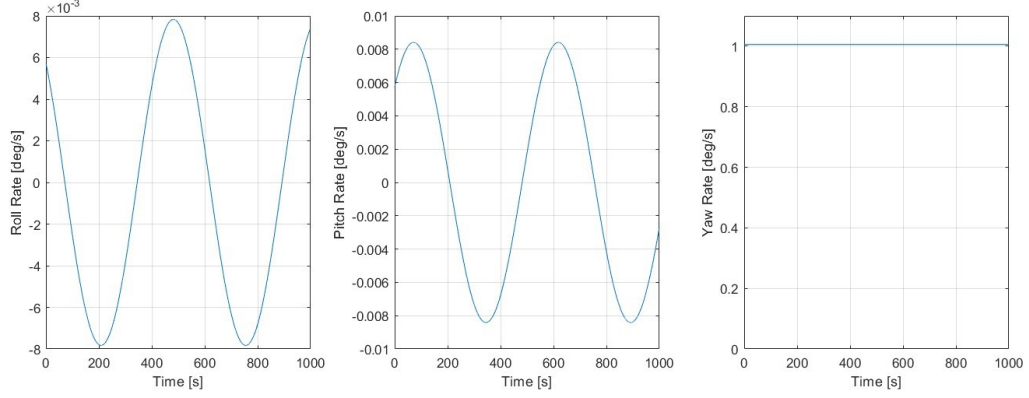


Figure 50: Angular velocity trajectories for a perturbed initial rotation about the principal axis of maximum inertia.

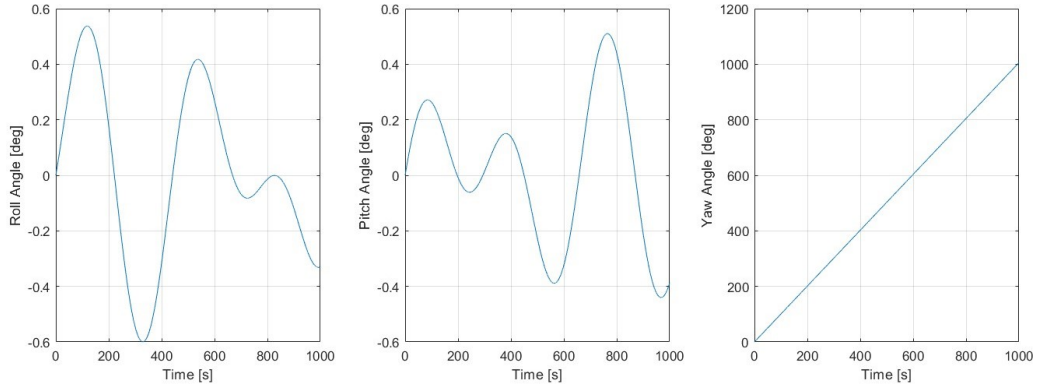


Figure 51: Attitude trajectories for a perturbed initial rotation about the principal axis of maximum inertia.

5.3 PROBLEM 3

The Euler equations Listing 13.12.7 are extended to include a momentum wheel. The specifications of the momentum wheel are taken from the *RSI 45 Momentum Wheel* by Rockwell Collins [7]. This reaction wheel is capable of achieving an angular momentum of $45[Nms]$. The correct implementation is verified through the conservation of momentum, namely the components of the angular momentum in the inertial reference frame are constant, as shown in Figure 52. These are simulation results based on an initial condition of $\omega_0 = [3 \ 1 \ 2] [deg/s]$.

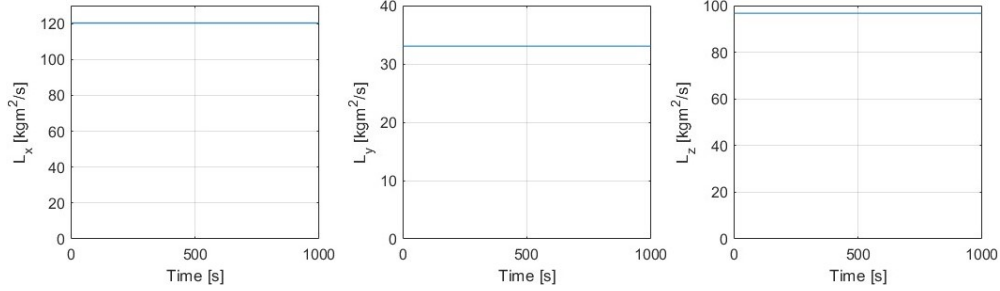


Figure 52: Conservation of angular momentum for satellite with momentum wheel.

In the presence of a momentum wheel, equilibrium occurs in the case of rotation about a principal axis, with the momentum wheel aligned with this axis of rotation. One such configuration is for $\omega_0 = [0 \ 0 \ 1] [deg/s]$ and $\hat{r}_{wheel} = [0 \ 0 \ 1]$. The corresponding angular velocity trajectories are shown in Figure 54. From these results it can be seen that the angular velocity maintains its initial conditions. The two other trivial equilibrium configurations corresponding to $\omega_0 = [0 \ 1 \ 0] [deg/s]$ and $\hat{r}_{wheel} = [0 \ 1 \ 0]$, and $\omega_0 = [1 \ 0 \ 0] [deg/s]$ and $\hat{r}_{wheel} = [1 \ 0 \ 0]$ have also been confirmed in simulation.

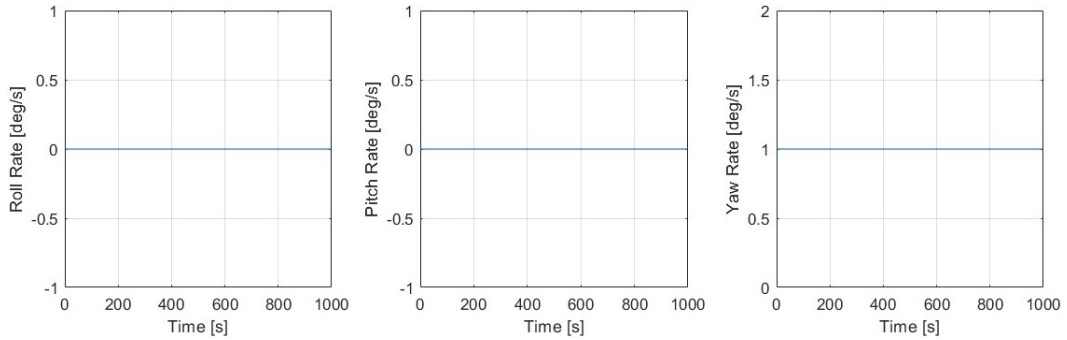


Figure 53: Angular velocity trajectories for the equilibrium configuration in the presence of a momentum wheel.

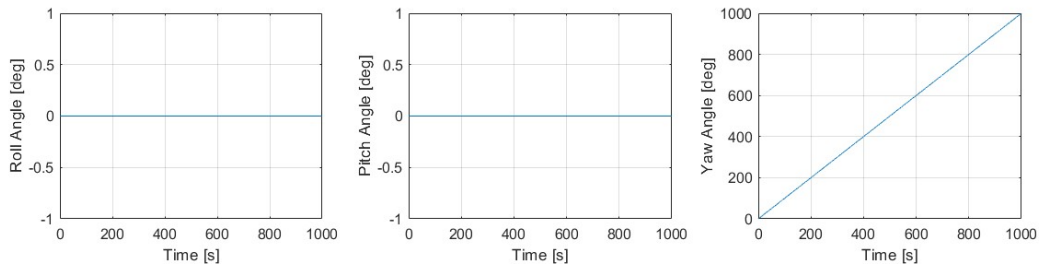


Figure 54: Attitude angle trajectories for the equilibrium configuration in the presence of a momentum wheel.

In order to study the stability of this configuration we consider again a perturbation to the angular velocity from the equilibrium. The first equilibrium configuration corresponds to rotation about the principal axis of minimum inertia and the momentum wheel aligned to this axis, $\omega_0 = [1 \ 0 \ 0] [deg/s]$ and $\hat{r}_{wheel} = [1 \ 0 \ 0]$. The angular velocity and attitude trajectories are shown in Figure 55 and Figure 56. As can be seen, the attitude is periodically stable in the angular velocity since there is a small oscillation in the pitch and yaw rate, while the roll rate maintains its initial condition. The attitude angles are unstable since the roll angle linearly increase with time.

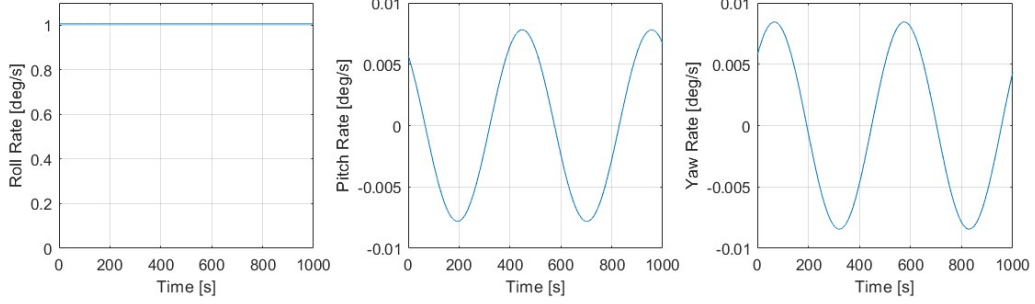


Figure 55: Angular velocity trajectories for a perturbed initial rotation about the principal axis of minimum inertia, for the dual spin satellite.

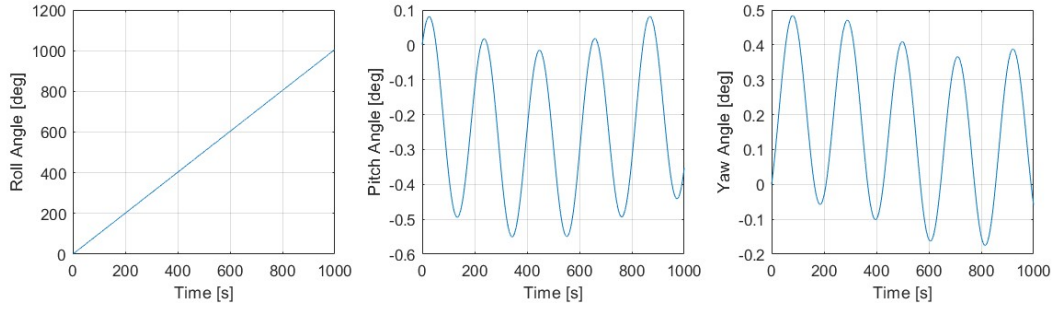


Figure 56: Attitude trajectories for a perturbed initial rotation about the principal axis of minimum inertia, for the dual spin satellite.

The second equilibrium configuration corresponds to rotation about the principal axis of intermediate inertia and the momentum wheel aligned to this axis, $\omega_0 = [0 \ 1 \ 0] [deg/s]$ and $\hat{r}_{wheel} = [0 \ 1 \ 0]$. The angular velocity and attitude trajectories of the perturbed equilibrium and shown in Figure 57 and Figure 58. Again, angular velocity is periodically stable, while the attitude angles are unstable.

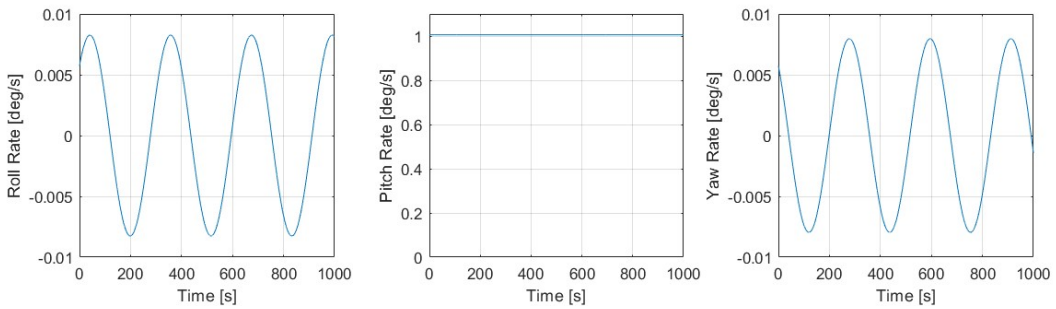


Figure 57: Angular velocity trajectories for a perturbed initial rotation about the principal axis of intermediate inertia, for the dual spin satellite.

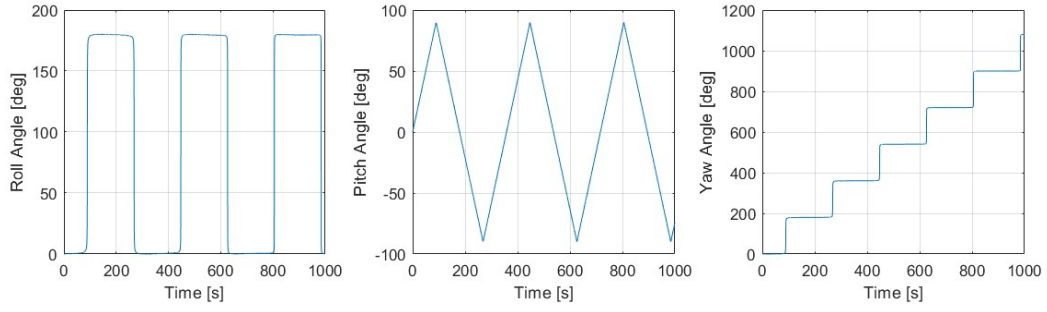


Figure 58: Attitude trajectories for a perturbed initial rotation about the principal axis of intermediate inertia, for the dual spin satellite.

Lastly, the equilibrium configuration corresponding to rotation about the principal axis of maximum inertia and the momentum wheel aligned to this axis, $\omega_0 = [0 \ 0 \ 1] [deg/s]$ and $\hat{r}_{wheel} = [0 \ 0 \ 1]$. The angular velocity and attitude trajectories of the perturbed equilibrium and shown in Figure 59 and Figure 60. For this configuration too is the angular velocity periodically stable, while the attitude angles are unstable.

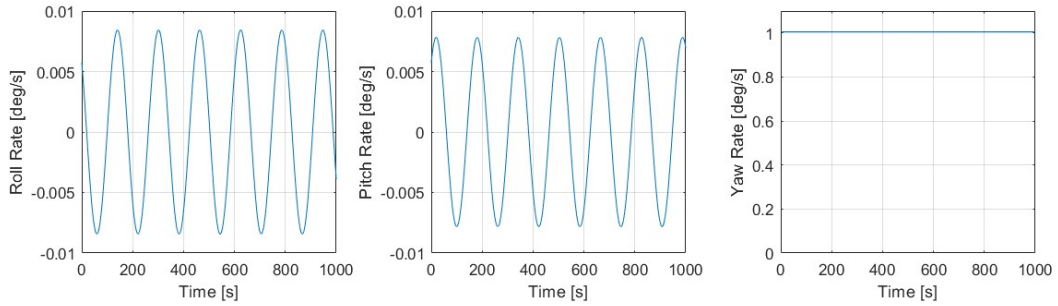


Figure 59: Angular velocity trajectories for a perturbed initial rotation about the principal axis of maximum inertia, for the dual spin satellite.

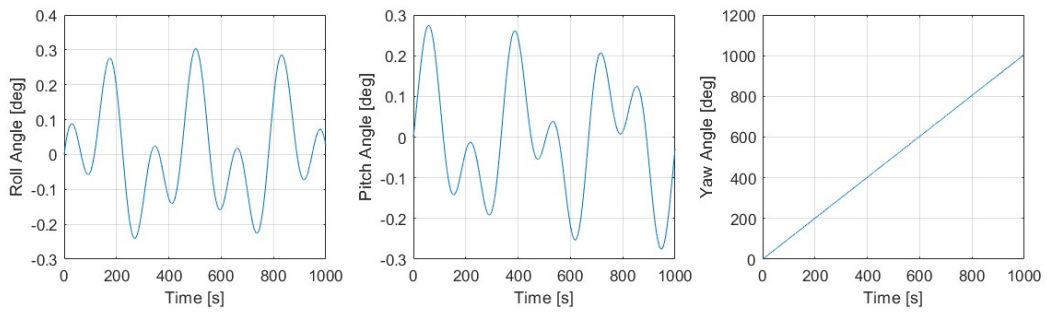


Figure 60: Attitude trajectories for a perturbed initial rotation about the principal axis of maximum inertia, for the dual spin satellite.

While for the dual spin satellite, stability for attitude rotation about the intermediate moment of inertia was shown, a further analysis is carried out to determine the conditions under which this motion remains stable. For this, the following conditions need to be satisfied:

$$I_r \bar{\omega}_r > (I_z - I_y) \bar{\omega}_y$$

$$I_r \bar{\omega}_r > (I_x - I_y) \bar{\omega}_y$$

These inequalities hold for the current sizing of the momentum wheel and chosen initial conditions. However, Figure 61 shows the complete operational region for which rotation about the principal axis of intermediate inertia is stable. For the current configuration of the momentum wheel, rotation of up to $\bar{\omega}_y = 2.94[deg/s]$ is stable. As a check, the stability results for the initial condition $\omega_0 = [0 \ 3 \ 0][deg/s]$ are shown in Figure 62 and Figure 63, from which it can be seen that the angular velocity and attitude angles are unstable.

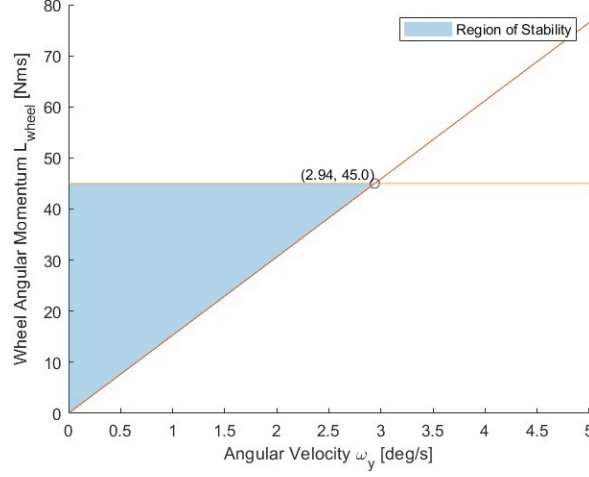


Figure 61: Region of stable rotation about the principal axis of intermediate inertia for the dual spin satellite.

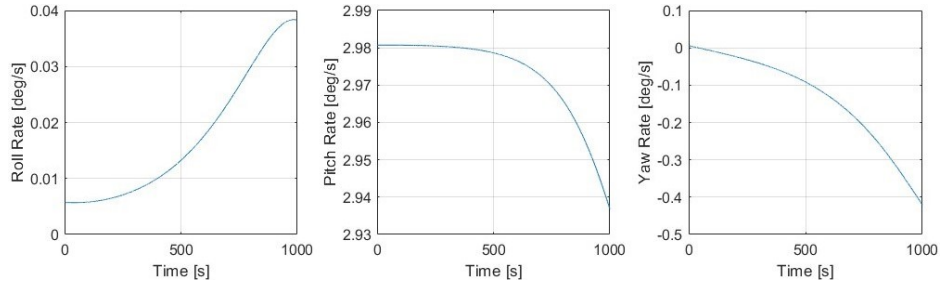


Figure 62: Angular velocity for the unstable configuration of the dual spin satellite with $\omega_0 = [0 \ 3 \ 0][deg/s]$.

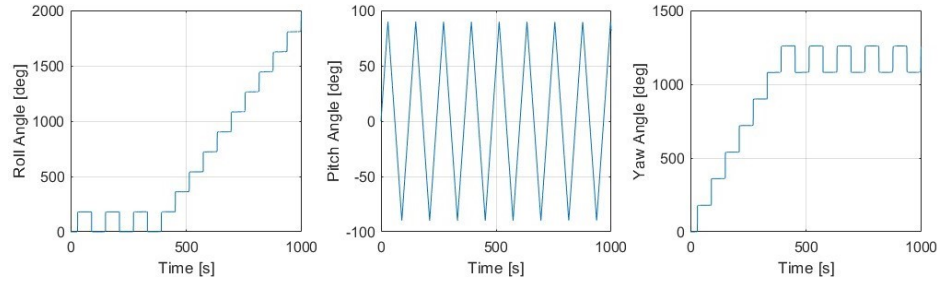


Figure 63: Attitude angles for the unstable configuration of the dual spin satellite with $\omega_0 = [0 \ 3 \ 0][deg/s]$.

The dual spin satellite can achieve stable rotation about an arbitrary axis by specifying the axis of the momentum wheel and its angular momentum. For a given initial angular velocity $\omega_0 = [\bar{\omega}_x \ \bar{\omega}_y \ \bar{\omega}_z] [deg/s]$ the following nonlinear equations can be solved:

$$\begin{aligned} (I_{p_z} - I_{p_y})\bar{\omega}_y\bar{\omega}_z + L_{wheel}(\bar{\omega}_y\hat{r}_z - \bar{\omega}_z\hat{r}_y) &= 0 \\ (I_{p_x} - I_{p_z})\bar{\omega}_z\bar{\omega}_x + L_{wheel}(\bar{\omega}_z\hat{r}_x - \bar{\omega}_x\hat{r}_z) &= 0 \\ (I_{p_y} - I_{p_x})\bar{\omega}_x\bar{\omega}_y + L_{wheel}(\bar{\omega}_x\hat{r}_y - \bar{\omega}_y\hat{r}_x) &= 0 \\ \|\hat{r}\|_2 - 1 &= 0 \end{aligned} \quad (21)$$

where L_{wheel} is the angular momentum of the momentum wheel and $\hat{r}$ is the axis of rotation w.r.t. the principal axes. Solving these equation for a certain angular velocity yields the required angular momentum and axis of rotation of the momentum wheel to achieve a stable rotation. Figure 64 shows the angular velocity trajectories for a stable rotation of $\omega_0 = [1 \ 1 \ 1] [deg/s]$. The implementation is given in Listing 13.12.16.

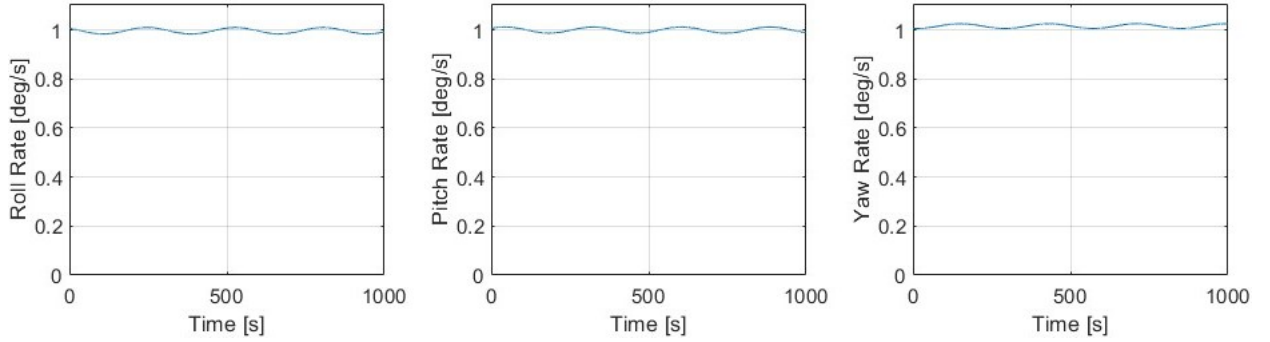


Figure 64: Dual spin satellite stable rotation about arbitrary axis with $\omega_0 = [1 \ 1 \ 1] [deg/s]$.

Solving the equations in Equation 21 leads to an equilibrium, however it does not guarantee rotational stability. Specifically, since the first three equations are not linearly independent, the fourth equation is used to constraint the unit vector $\hat{r}$, while the momentum wheel angular momentum L_{wheel} is set to the initial value given. In other words, these equations can be solved for any given angular momentum L_{wheel} . It is this angular momentum which dictates for which angular velocity the rotational motion remains stable. Carrying out above experiment again, this time with $\omega_0 = [2 \ 2 \ 2] [deg/s]$, the results are shown in Figure 65. It can be seen that the angular velocity motion is unstable. By increasing the momentum wheel angular momentum, stable rotation can be achieved as shown in Figure 66.

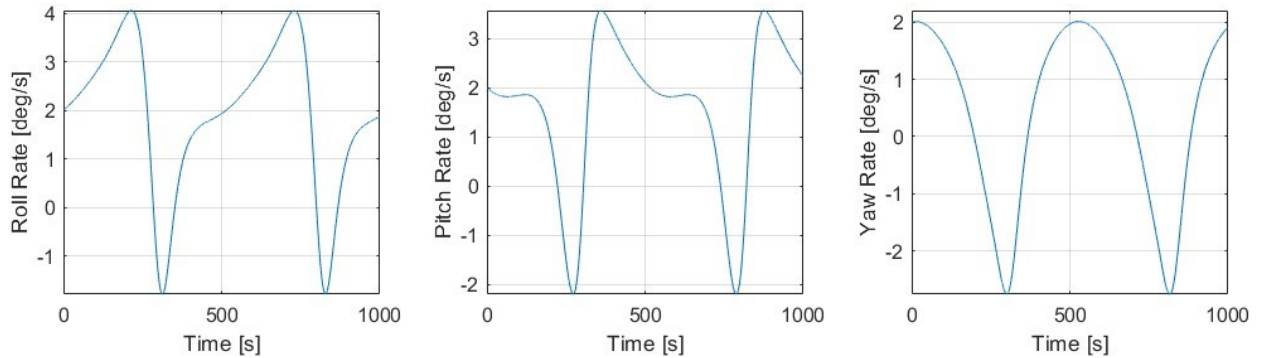


Figure 65: Dual spin satellite unstable rotation about arbitrary axis with $\omega_0 = [2 \ 2 \ 2] [deg/s]$.

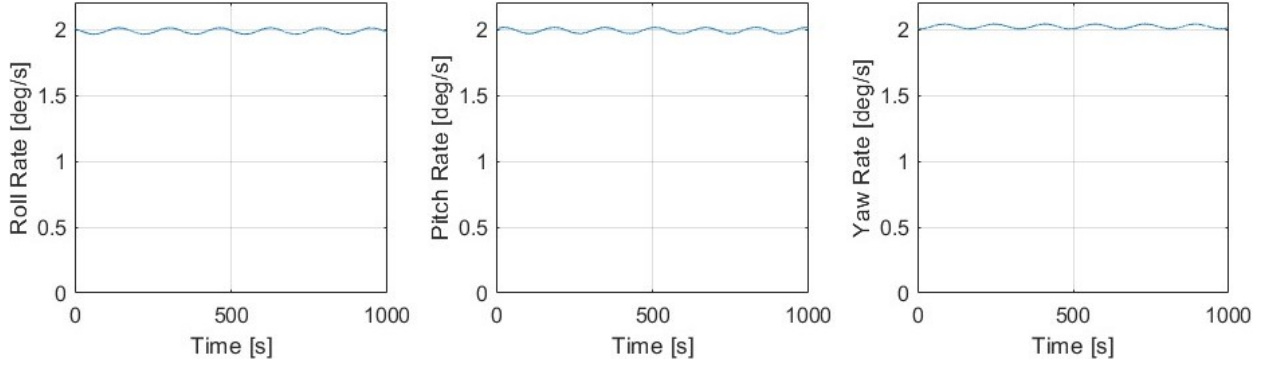


Figure 66: Dual spin satellite stable rotation about arbitrary axis with $\omega_0 = [2 \ 2 \ 2] [deg/s]$.

5.4 PROBLEM 4

The gravity gradient acts as an external torque on the satellite. In order to model its effects, we make simplifying assumptions among which a circular orbit, leading to following simplified expression:

$$\mathbf{M}_{gg} = \frac{3\mu}{R^3} \begin{bmatrix} (I_z - I_y)c_y c_z \\ (I_x - I_z)c_z c_x \\ (I_y - I_x)c_x c_y \end{bmatrix} \quad (22)$$

where μ is the gravitational parameter of Earth, R is the radius of the circular orbit and the vector $\mathbf{c}$ is the unit vector of the position vector of the center of mass of the satellite in principal axes:

$$\mathbf{c} = \begin{bmatrix} c_x \\ c_y \\ c_z \end{bmatrix} = R_i^p \frac{\mathbf{p}^i}{\|\mathbf{p}^i\|_2} \quad (23)$$

where R_i^p is the rotation matrix from inertial to principal axes. The implementation of the gravity gradient torque in *Matlab* is given in Listing 13.12.11. The implementation of the gravity gradient torque can be verified through testing for several relations and conditions.

Firstly, the relation to the principal moments of inertia. This is tested by varying the inertia of each principal axis, given the initial conditions of the orbit and the principal axes not aligned with the inertial frame. For these conditions, we have $c_x > 0$, $c_y > 0$ and $c_z < 0$. Considering an increase in I_x , constant M_x , a decrease in M_y and decrease in M_z should be observed. Then, an increase in I_y should lead to a linear increase in M_x , constant M_y and linear increase in M_z . Lastly, for an increase in I_z , a linear decrease in M_x , linear increase in M_y and constant M_z should be observed. This is summarized in Table 10. These relations have been verified in simulation, as shown in Figure 67, Figure 68 and Figure 69.

Table 10: Summary of relations between principal moments of inertia and the gravity gradient torque.

Principal Moment of Inertia	M_x	M_y	M_z
$I_x \uparrow$	=	$\downarrow$	$\downarrow$
$I_y \uparrow$	$\uparrow$	=	$\uparrow$
$I_z \uparrow$	$\downarrow$	$\uparrow$	=

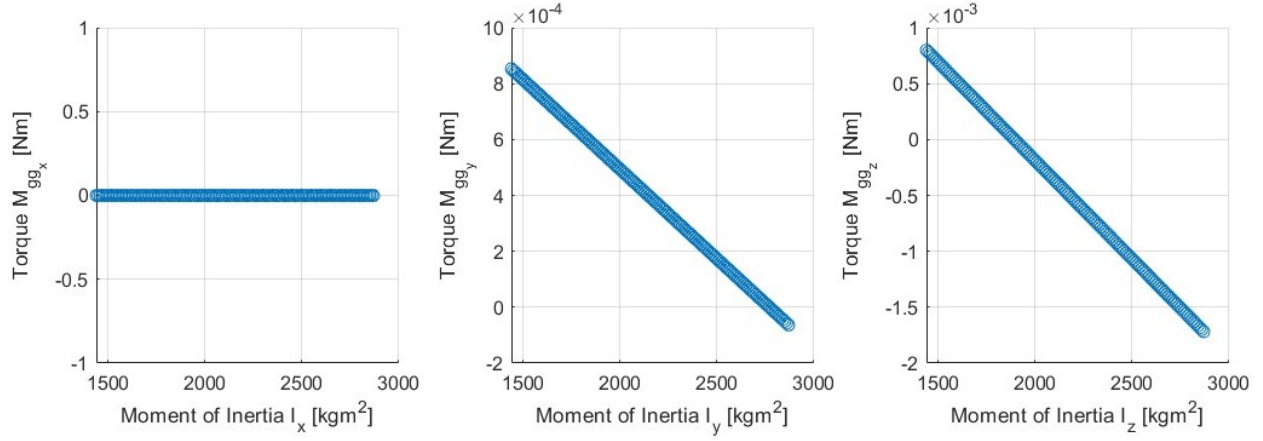


Figure 67: Relation between gravity gradient torque and principal moment of inertia I_x .

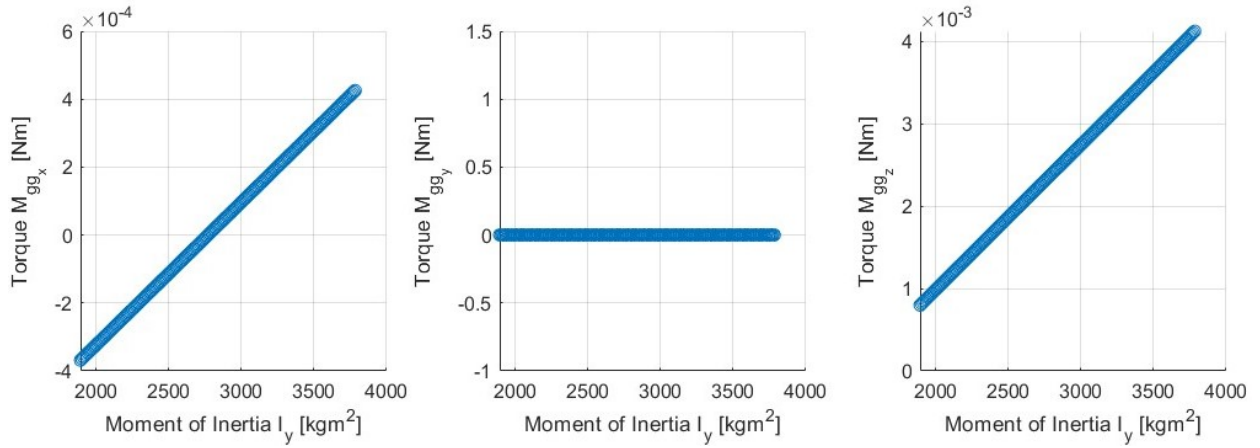


Figure 68: Relation between gravity gradient torque and principal moment of inertia I_y .

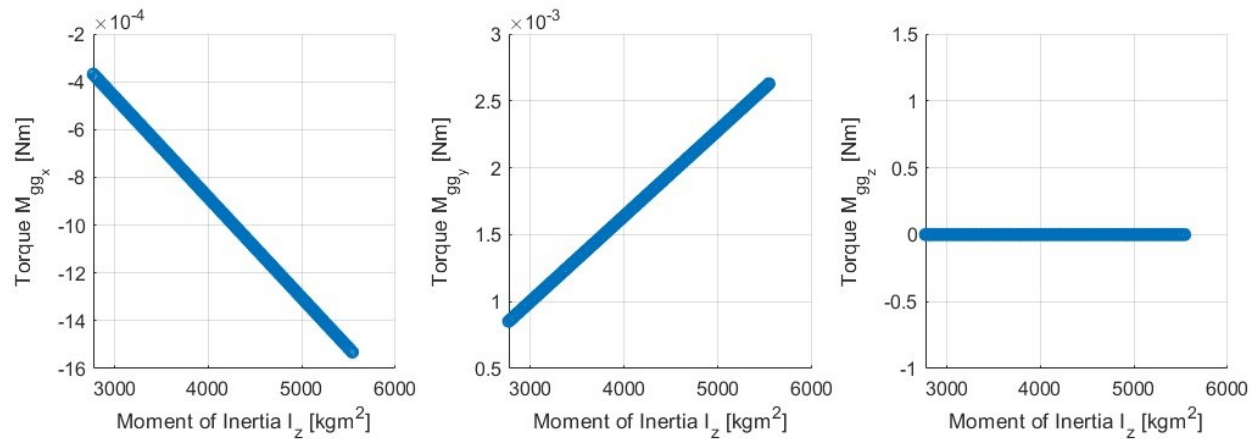


Figure 69: Relation between gravity gradient torque and principal moment of inertia I_z .

As a second verification, the relation of the gravity gradient torque magnitude to the radius of the orbit is verified. As the radius increases, the magnitude of the gravity gradient torque should decrease as $\|\mathbf{M}_{gg}\|_2 \sim R^{-1/3}$. This relation is verified and shown in Figure 70.

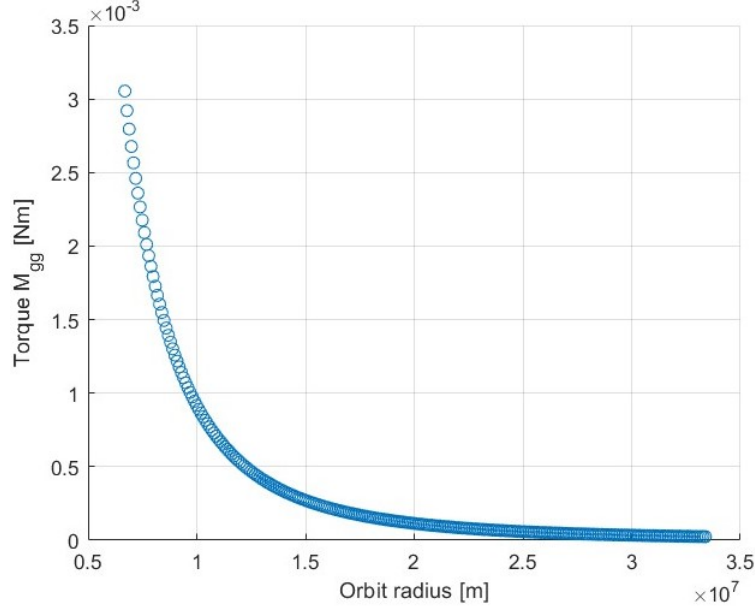


Figure 70: Relation between magnitude of the gravity gradient torque and radius of the orbit R .

From the physical principal of the gravity gradient, its magnitude in radial direction is expected to be zero. Equation 22 expresses the gravity gradient torque in principal axes such that it can be integrated in the numerical integration of the Euler equations. However, it can be expressed in the orbit RTN -frame through $\mathbf{M}_{gg}^o = \mathbf{R}_i^o \mathbf{R}_p^i \mathbf{M}_{gg}^p$. Figure 71 shows the result of numerically integrating the Euler equation, including the gravity gradient perturbation, from initial conditions $\eta_0 = [5 \ 0 \ 0] [deg]$ and $\omega_0 = [3 \ 1 \ 2] [deg/s]$, after which the gravity gradient torque is expressed in the orbit frame. As expected, the gravity gradient torque in radial direction is zero.

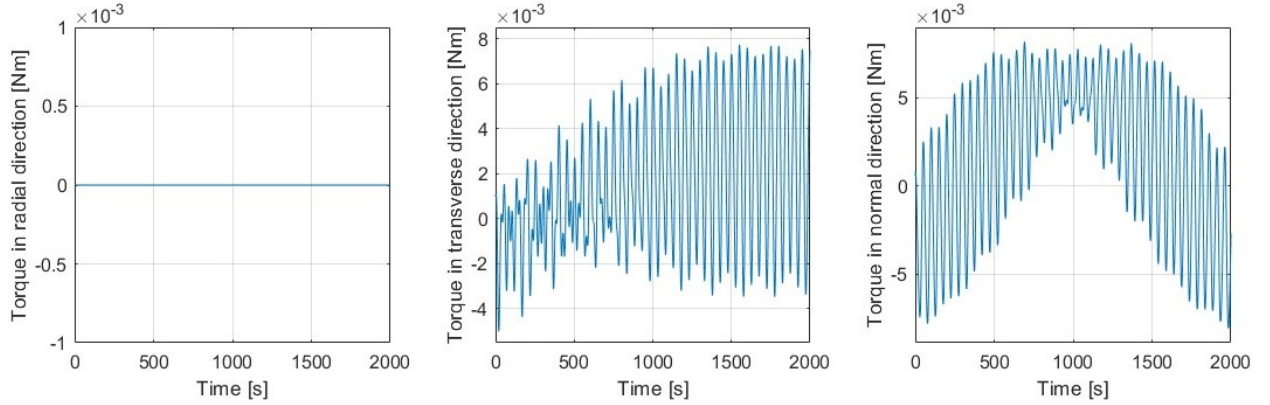


Figure 71: Gravity gradient torque expressed in orbit RTN -frame.

An equilibrium point for the Euler equations including gravity gradient exists when the principal axes are aligned with the orbit RTN -frame and the angular velocity has initial condition $\omega_0 = [0 \ 0 \ n] [rad/s]$, where $n = \sqrt{\frac{\mu}{a^3}}$ is the mean motion of the satellite. Under these initial conditions, the angular velocity is in an equilibrium points, hence remains constant, and the gravity gradient torque is zero. This is verified in simulation as shown in Figure 72, in which the angular velocity remains constant, and Figure 73, in which the gravity gradient torque is zero up to floating point precision.

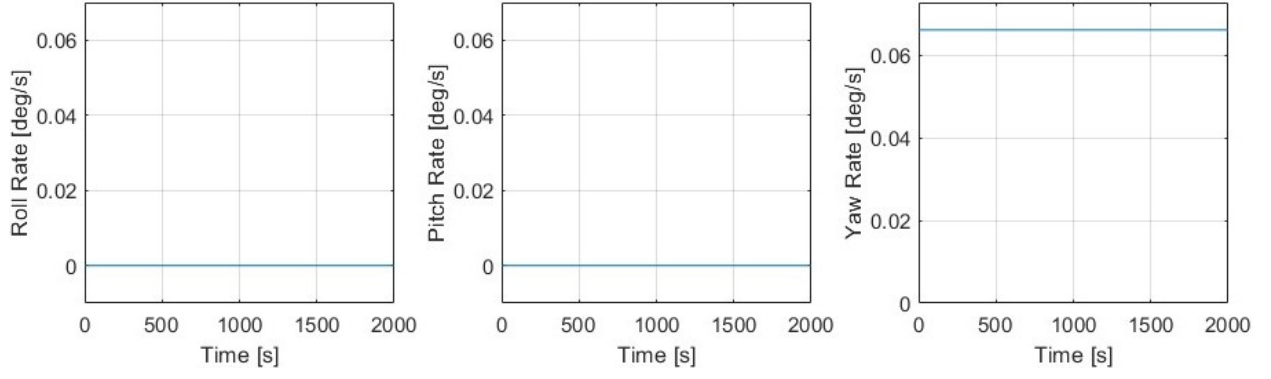


Figure 72: Equilibrium point for angular velocity under the initial condition of principal axes aligned with RTN -frame and $\omega_0 = [0 \ 0 \ n] [rad/s]$, where n is the mean motion of the satellite.

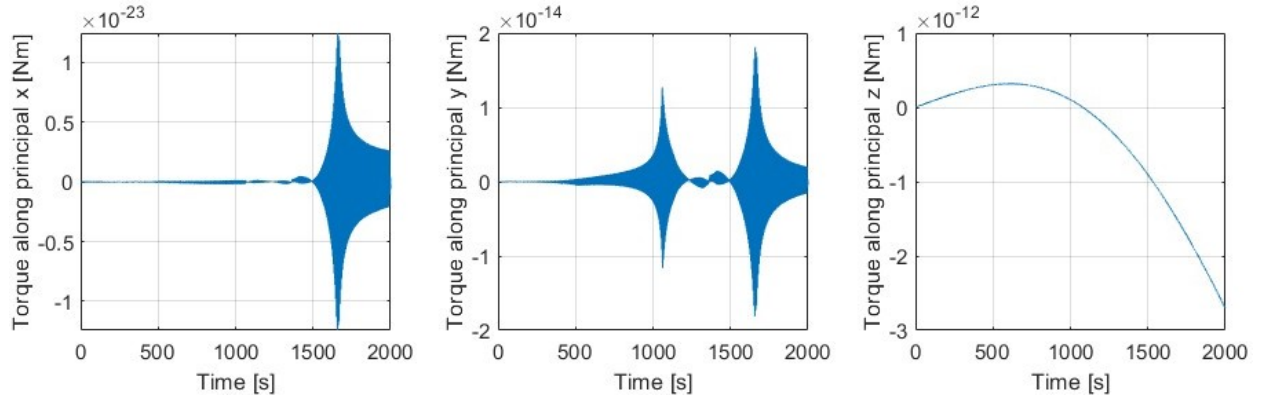


Figure 73: Zero gravity gradient torque under the initial condition of principal axes aligned with RTN -frame and $\omega_0 = [0 \ 0 \ n] [rad/s]$, where n is the mean motion of the satellite.

After having verified the implementation of the gravity gradient torque, its effects on the satellite dynamics can be simulated under its intended orbit and initial conditions. Relevant to the mission, as shown in Figure 1, if the body z -axis is aligned with the radial direction, body x -axis is aligned with normal direction and body y -axis aligned with transverse direction, then in this configuration, the satellite is rotated by 35° around the transverse direction for its intended orientation. This orientation is depicted in Figure 97, which also contains the body axes, orbital axes and principal axes frames.

The equations of motion are integrated over a time span of three orbit revolutions with the initial conditions as specified by the mission. The gravity gradient torque experience by the satellite is shown in Figure 75. The gravity gradient torque has a magnitude in the order of mNm . Additionally, Figure 76 and Figure 77 show the angular velocity and attitude angles, respectively. The magnitude of the angular velocity remains relatively small, but nonzero. This can be seen in the attitude angle trajectories which drift significantly from the initial attitude. This indicates that active control is required to maintain the satellite in its desired orientation.

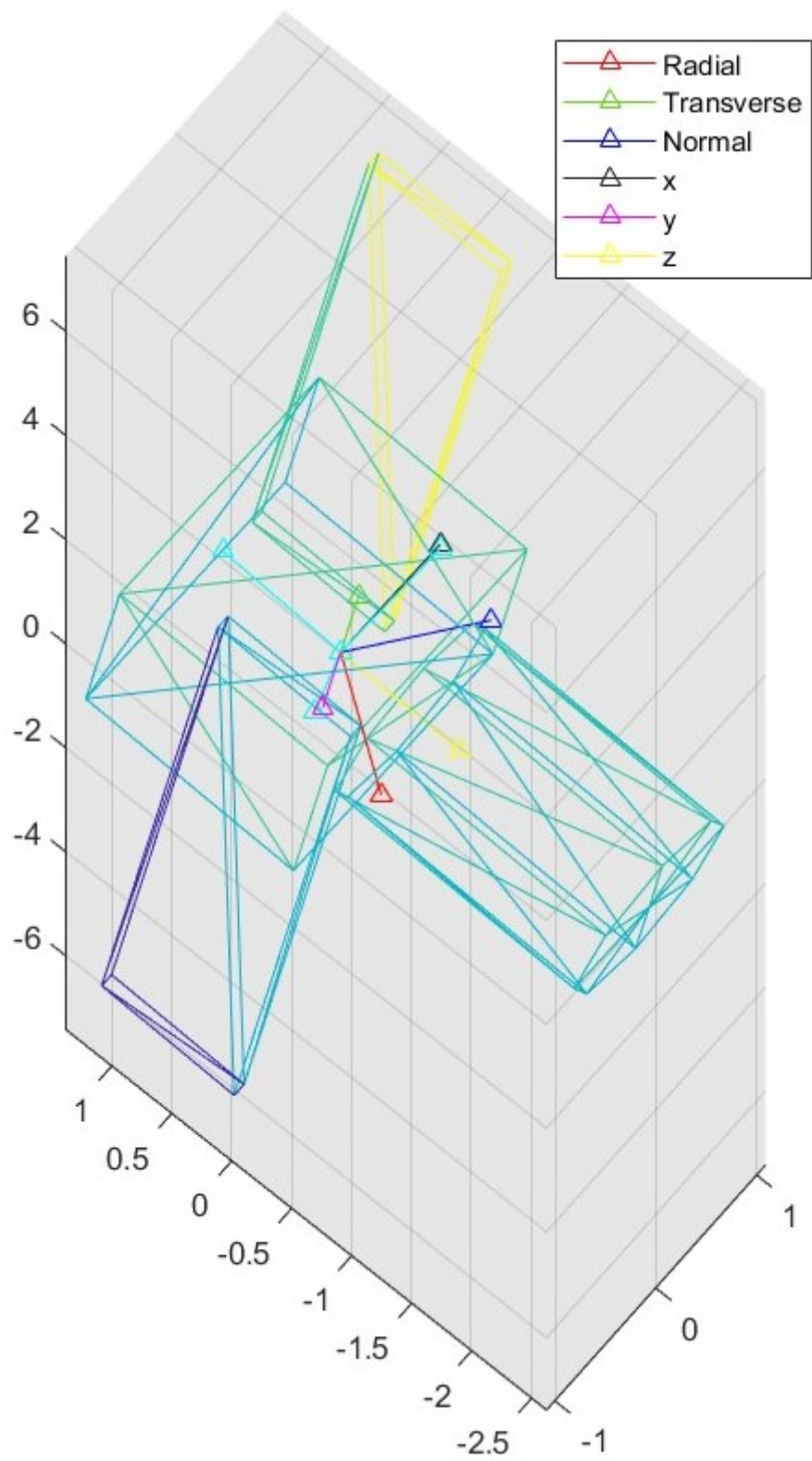


Figure 74: Satellite orientation corresponding to the mission specifications. The graph depicts the body axes and orbit axes.

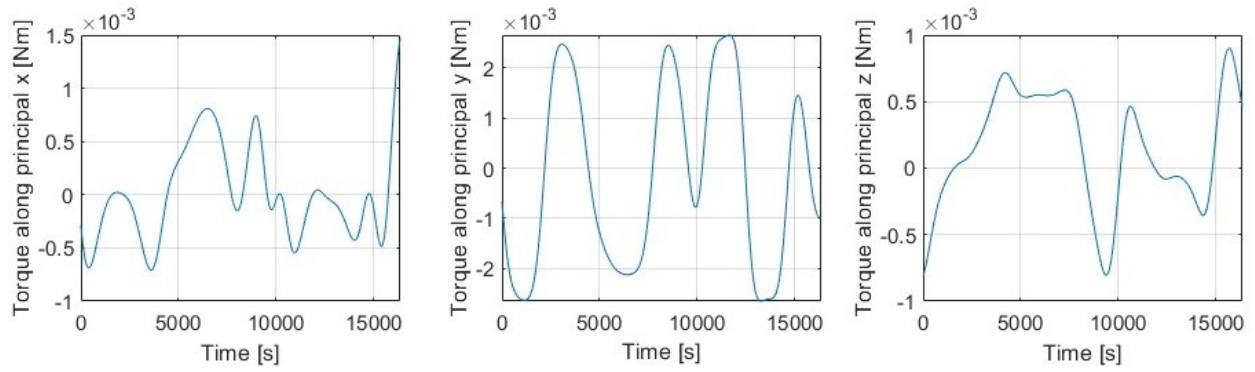


Figure 75: Gravity gradient torque trajectories for initial conditions consistent with mission specifications.

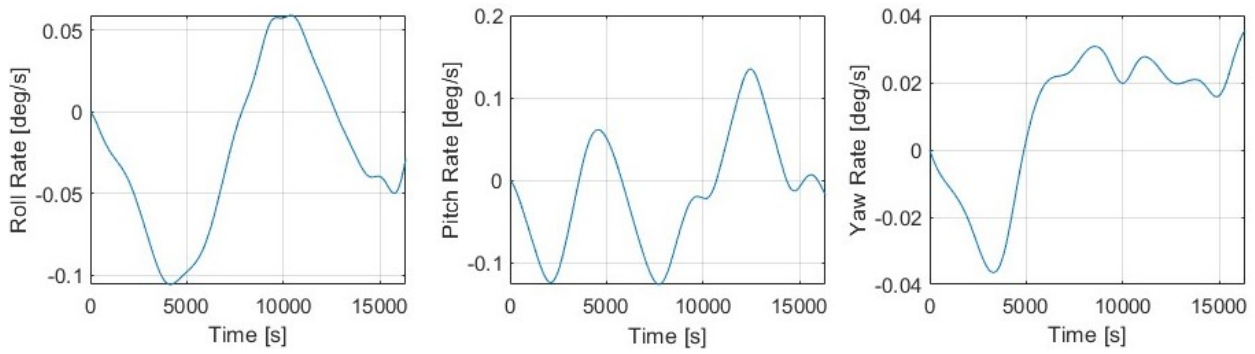


Figure 76: Angular velocity trajectories for initial conditions consistent with mission specifications.

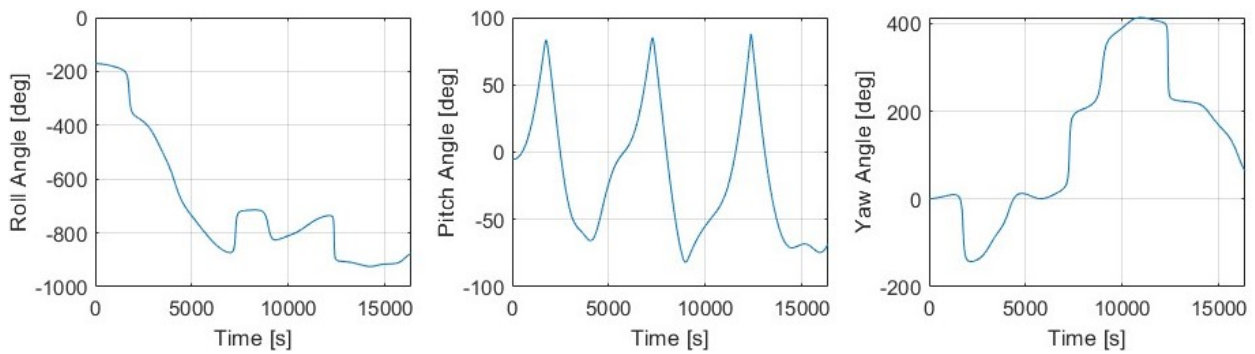


Figure 77: Attitude angle trajectories for initial conditions consistent with mission specifications.

6 Modeling of External Perturbations

6.1 PROBLEM 1

The coefficients of inertia which drive the stability under gravity gradient for the considered satellite are as follows:

$$k_N = \frac{I_y - I_x}{I_z} = 0.1639, \quad k_T = \frac{I_z - I_x}{I_y} = 0.7030, \quad k_R = \frac{I_z - I_y}{I_x} = 0.6093$$

Then the stability conditions for the equilibrium condition in which the principal axes are aligned with the RTN -frame and a pure rotation about the N -axis are:

$$k_T > k_R, \quad k_R k_T > 0, \quad 1 + 3k_T + k_R k_T > 4\sqrt{k_R k_T}$$

These conditions are graphically shown in Figure 78 as well as under which condition the satellite falls. From the figure it can be seen that the satellite is expected to have stable roll, pitch and yaw motion under the gravity gradient.

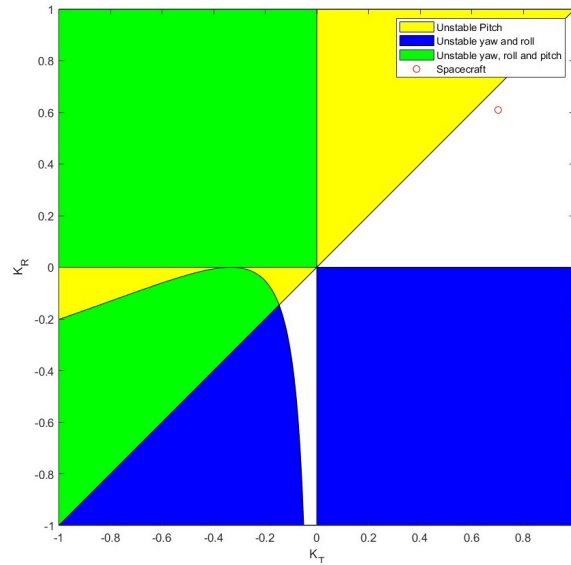


Figure 78: Regions of stable and unstable motion under gravity gradient.

The satellite is simulated over five orbits from the above equilibrium condition with a pure rotation about the N -axis corresponding to the mean motion n of the orbit. The satellite is perturbed from this equilibrium by rotating the principal axes by a small angle from each axis $\alpha_{perturb} = [\alpha_x \ \alpha_y \ \alpha_z]^T$. Then the perturbed initial conditions are:

$$\omega^p = A\omega^{RTN} = [-\alpha_y n \ \alpha_x n \ n]^T$$

$$\eta^p = AR_i^p$$

where A is the rotation matrix for small angles:

$$A = \begin{bmatrix} 1 & \alpha_z & -\alpha_y \\ -\alpha_z & 1 & \alpha_x \\ \alpha_y & -\alpha_x & 1 \end{bmatrix}$$

Given the above configuration results in Figure 79 and Figure 80 showing, respectively, angular momentum of principal axes and attitude angles of principal axes with respect to RTN -frame. As can be seen, the attitude motion is stable as predicted.

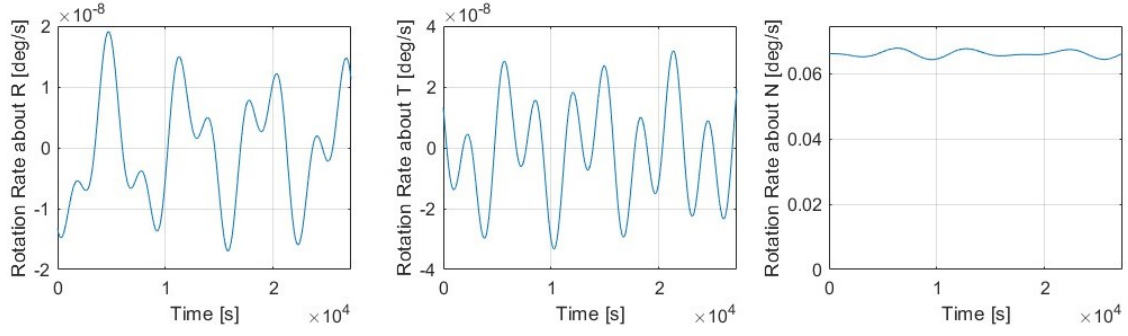


Figure 79: Angular velocity of principal axes under perturbed equilibrium condition.

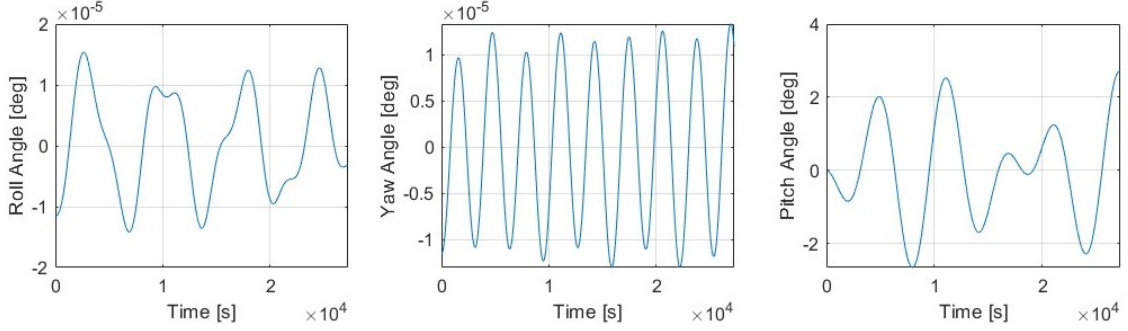


Figure 80: Euler angles of principal axes with respect to RTN -frame under perturbed equilibrium condition.

While the satellite is stable for its current mass distribution, different mass distributions can lead to unstable behaviour. For example, considering the following principle mass moments of inertia

$$I'_p = \text{diag}([1295.3 \quad 1136.0 \quad 1662.1]),$$

leads to a pitch unstable satellite as shown in Figure 81.

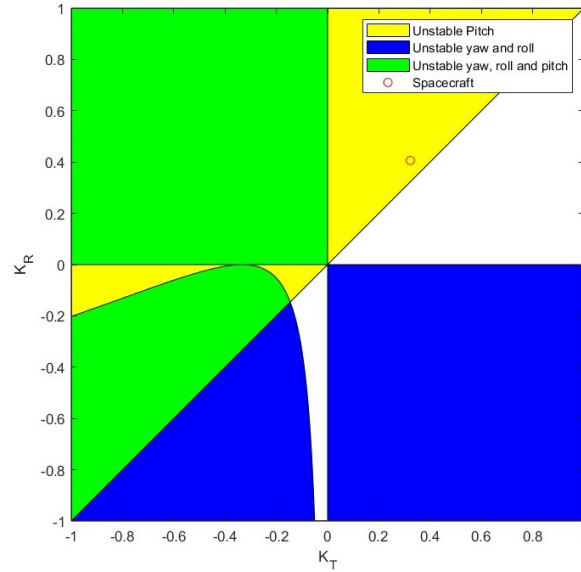


Figure 81: Gravity gradient stability for satellite mass distribution I'_p .

The simulation results are given in Figure 82 and Figure 83 showing, respectively, angular momentum and attitude angles of principal axes with respect to RTN -frame. The pitch motion is unstable as the pitch angle drifts away, while roll and yaw angles remain roughly zero.

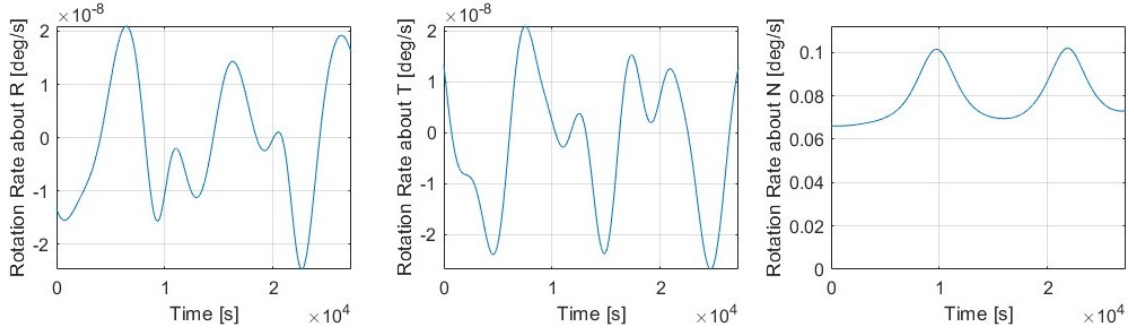


Figure 82: Angular velocity for pitch unstable configuration of I'_p .

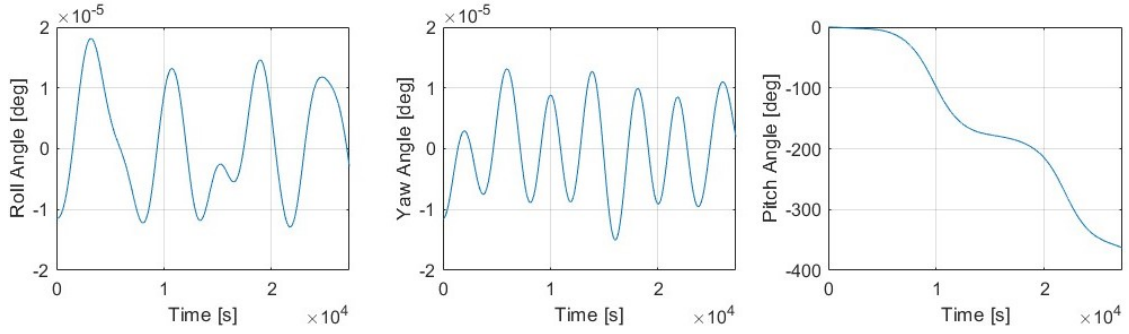


Figure 83: Euler angles for pitch unstable configuration of I'_p .

Lastly, a mass distributions for which all angles are unstable is considered. This configuration is achieved by following principle mass moments of inertia,

$$I''_p = \text{diag}([1295.3 \quad 1136.0 \quad 1662.1]),$$

leading to an unstable yaw, roll and pitch as shown in Figure 84.

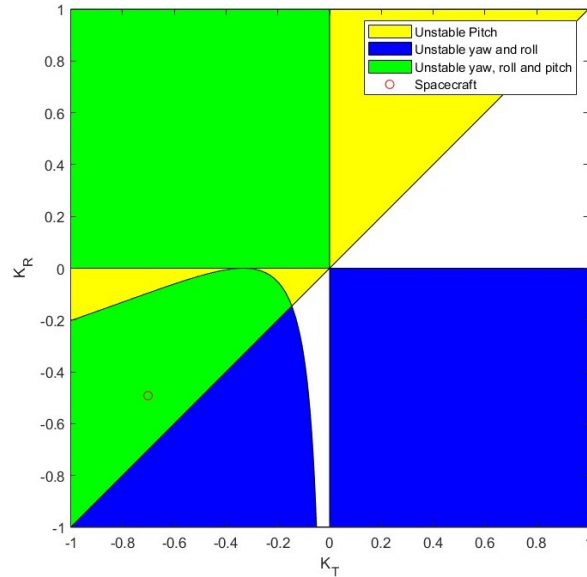


Figure 84: Gravity gradient stability for satellite mass distribution I''_p .

The simulation results are given in Figure 85 and Figure 86. Clearly, all angles are unstable, deviating majorly from the RTN -frame. This unstable motion is also reflected in the angular velocity oscillations.

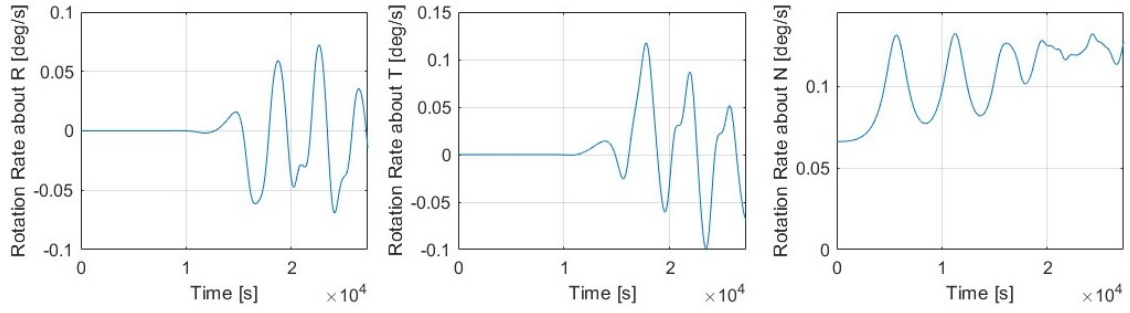


Figure 85: Angular velocity for unstable yaw, roll and pitch configuration of I_p'' .

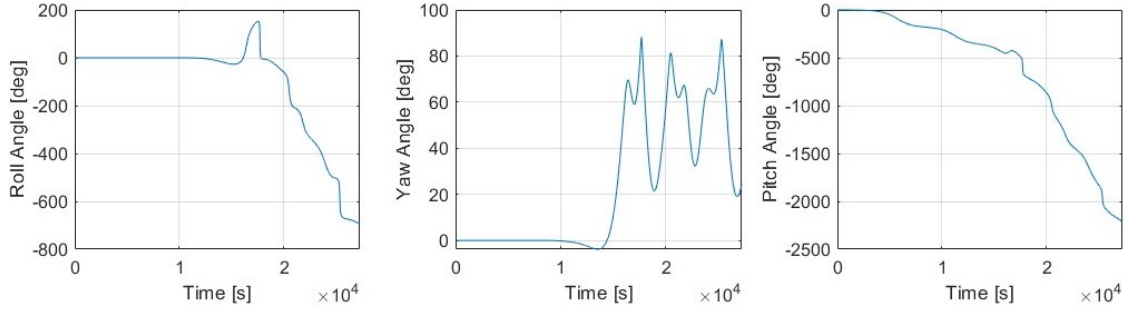


Figure 86: Euler angles for unstable yaw, roll and pitch configuration of I_p'' .

Given that the current configuration of the satellite is already stable under the gravity gradient, no changes are necessary to be considered.

6.2 PROBLEM 2

Magnetic Field

Given that a satellite has some residual magnetic moment, if the axis is not aligned with the local magnetic field, then the satellite experiences a magnetic torque. As a first-order estimate, the maximum possible value of the magnetic torque for the given altitude can be calculated as [21]:

$$M_{m_{max}} = mB = m \left(\frac{R_E^3 B_0}{R^3} \lambda \right) \quad (24)$$

where m is the satellite's residual dipole moment, $R_E^3 B_0 = 7.943 \times 10^{15} [Wb \cdot m]$ is Earth's dipole strength, R is the distance between the satellite and Earth's center and λ is a unitless function that ranges from 1 at magnetic equator to 2 at magnetic poles.

For the considered satellite, its residual dipole, given its bigger size, is estimated to be $10Am^2$. Furthermore, since the satellite is in a near polar orbit and to remain conservative, λ is taken to be 2. Then the maximum expected magnetic torque experienced by the satellite is $M_{m_{max}} = 5.3868 \times 10^{-4} [Nm]$.

While Earth's magnetic field is complex, it can be approximated to be roughly dipolar in nature. This results in the following expression for the magnetic field of Earth [1, App. H]:

$$\mathbf{B} = -\frac{R_E^3 B_0}{R^3} [3(\hat{m} \cdot \hat{r})\hat{r} - \hat{m}] \quad (25)$$

where $\hat{m}$ is the unit vector of the dipole and $\hat{r}$ the unit vector of the position vector of the satellite. Expressing both unit vectors in the Earth centered inertial frame results in a magnetic field strength vector in the inertial frame. The torque due to magnetic field is now calculated as:

$$\mathbf{M}_m = \mathbf{m} \times \mathbf{B} \quad (26)$$

where it is assumed that the satellite's dipole moment points along the z -axis of the body reference frame since this coincides with the payload centerline. The implementation is given in Listing 13.12.13. This torque is plotted in Figure 87 over one orbit with initial conditions as specified by the mission. The figure shows the three components as well as the magnitude. The magnitude plot also shows the conservative maximum expected magnetic torque that encountered in orbit given in Equation 24.

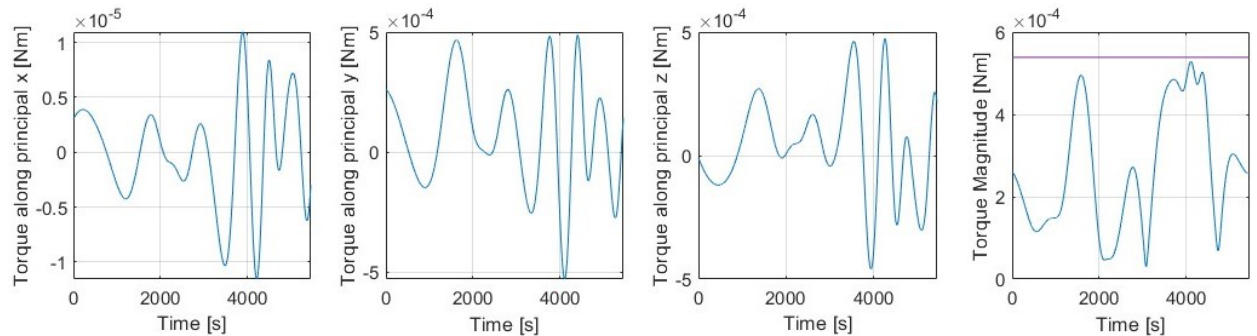


Figure 87: Torque due to Earth's magnetic field decomposed in the principal axes frame and its magnitude. The magnitude plot also contains the conservative maximum theoretical torque from Equation 24.

More sophisticated models of Earth's magnetic field exist, namely the spherical harmonic model, however for the purpose of modelling disturbance torques, the fidelity of the dipole model suffices for now.

Solar Radiation Pressure

Since sunlight has momentum, it exerts pressure on the surfaces it strikes. This phenomena results in a solar radiation pressure torque. Again, a first, conservative, approximation of solar radiation pressure torque can be calculated as [21]:

$$M_{s_{max}} = PA_s(1 + q)|r_p - r_m|\cos(\phi) \quad (27)$$

where P is the mean momentum flux on a surface normal to Sun's radiation, A_s is the sunlit surface area, q is the reflectance factor ranging from 0 to 1, ϕ is the angle of incidence of the Sun, and $|r_p - r_m|$ is the distance between centers of pressure and mass.

In order to get a conservative estimate, the scenario is considered in which the satellite's surfaces are in direct sunlight, i.e. $\cos(\phi) = 1$, and x -axis pointing towards the sun with the solar panels not deflected, resulting in maximum sunlit surface of $A_s = 20.81[m^2]$. Furthermore, since the satellite is symmetric about the center of mass, a small distance between centers of pressure and mass is considered of $|r_p - r_m| = 0.15[m]$. Lastly a reflectance factor of 0.5 is used as a median between perfect absorption and perfect reflection. This results in a maximum expected solar radiation torque of $M_{s_{max}} = 2.119 \times 10^{-5}[Nm]$.

A more detailed vector approximation can be made by discretizing the satellite geometry and determining the solar radiation force for each element. The contribution of these individual force elements is summed to the total torque generation about the center of mass. The solar radiational force is made up of incident radiation being either absorbed, reflected specularly or reflected diffusely. Then the total differential solar radiational force is [1]:

$$d\mathbf{f}_s = -P \left[(1 - C_s)\hat{s} + 2(C_s\cos(\theta) + \frac{1}{3}C_d)\hat{n} \right] \cos(\theta)dA \quad (28)$$

where P is the momentum flux incident on a surface elemental area dA with outward unit normal $\hat{n}$. C_s and C_d are the coefficient of specular reflection and diffuse reflection. Together they make up the reflectance factor $q = C_s + C_d$, which was previously set to be 0.5. These are properties of the satellite's material, since no detailed information is available on this, these are simply set to 0.25 for all surface elements. θ is the angle between $\hat{s}$ and $\hat{n}$, hence $\cos(\theta) = \hat{s} \cdot \hat{n} \in [0, 1]$. If $\cos(\theta)$ is outside this range, the surface is not affected by radiation pressure. Lastly, $\hat{s}$ is the unit vector in the direction from the satellite to the Sun. Hence, this requires knowledge of where the Sun is with respect to the satellite. The implementation is given is given in Listing 13.12.15.

A coarse model of the Sun is implemented to provide the relative direction from the satellite to the Sun. For this, the Earth's orbit about the sun is numerically integrated, where the Sun is the inertial reference frame. Earth's orbit properties are obtained from [12]. Furthermore, the Earth's angular velocity is assumed to be constant about its axis of rotation, which is 23.5° tilted from the plane of its orbit around the Sun. The implementation is given in Listing 13.12.9. This model provides the position vector of the Earth in the Sun inertial frame as well as the rotation matrix to convert between this inertial frame and the principal axes of Earth. It is further assumed that the principal axes of Earth and the Earth centered inertial frame used for the satellite orbit propagation coincide. Hence, this provides a method to obtain the position vector of the Sun in the Earth centered inertial frame.

The validate proper behaviour of the implementation, Figure 88 and Figure 89 show the Earth's positional trajectory and velocity magnitude over a time span of one year. As can be seen, a full orbit around the Sun is completed and the velocity magnitude also completes one period. Both have magnitudes corresponding to theory. Furthermore, the attitude and angular velocity plots over a time span of one day are given in Figure 90 and Figure 91. As can be seen, the attitude motion represents a rotation about a tilted axis with period of one day and tilt of 23.5° .

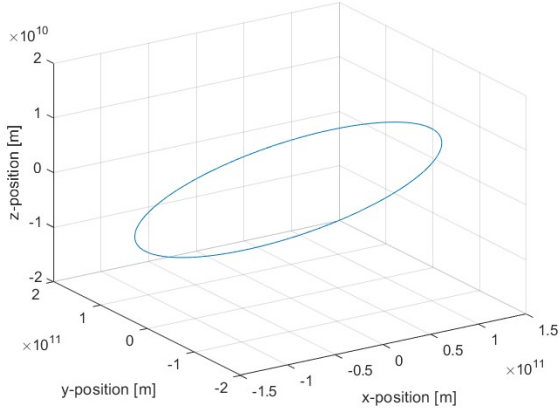


Figure 88: Orbital position trajectory of Earth around the Sun.

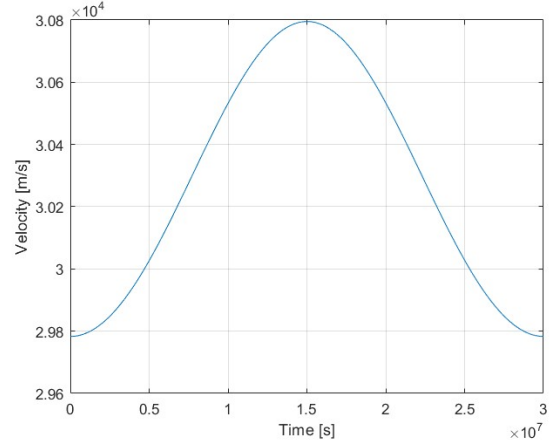


Figure 89: Velocity magnitude of Earth over one period around the Sun.

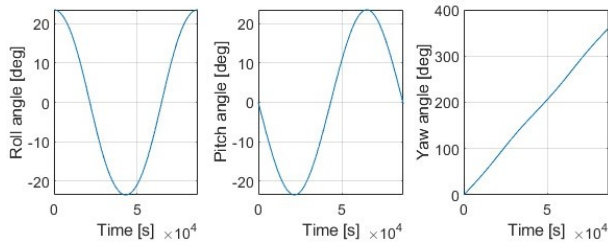


Figure 90: Attitude trajectories of Earth throughout one day.

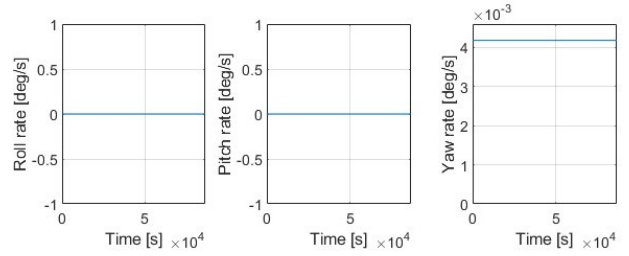


Figure 91: Angular velocity trajectories throughout one day.

Given the implemented model of the Sun, the position vector from the satellite to the Sun can be obtained in the Earth centered inertial frame through $p_{sat,sun}^i = p_{sat}^i - p_{sun}^i$, where $p_{sat,sun}^i$ is the relative vector from satellite to Sun in inertial frame and p_{sat}^i, p_{sun}^i are position vectors of the satellite and the Sun in Earth centered inertial frame. Then the desired unit vector is obtained as $\hat{s} = R_i^p \frac{p_{sat,sun}^i}{\|p_{sat,sun}^i\|}$, expressed in principal axes frame. The torque is plotted in Figure 92 over one orbit with initial conditions as specified by the mission. The figure shows the three components as well as the magnitude. The magnitude plot also shows the conservative maximum expected magnetic torque that encountered in orbit given in Equation 24.

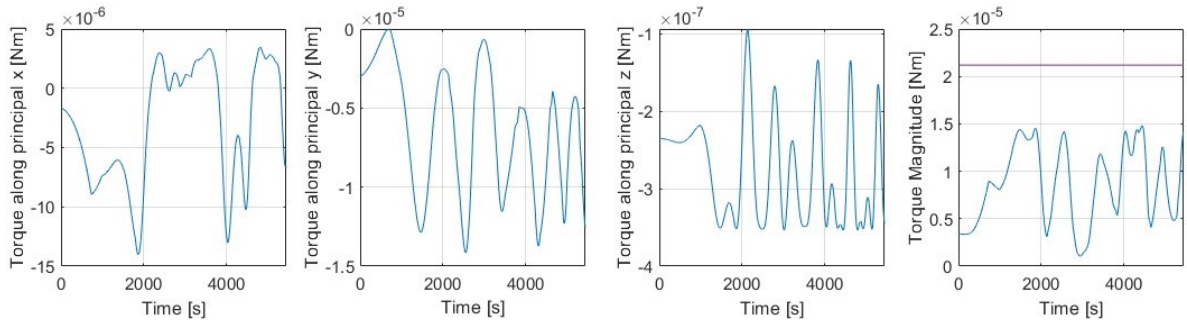


Figure 92: Torque due to solar radiation decomposed in the principal axes frame and its magnitude. The magnitude plot also contains the conservative maximum theoretical torque from Equation 27.

This is a simple model for the solar radiation torque, where effects such as Earth's albedo are neglected. Furthermore, this model also does not account for shadowing of one part of the satellite by another, nor does it account for eclipse. Some of these will be addressed later.

Atmospheric Drag

Interaction of the atmosphere with the satellite's surface produces a torque, especially for satellites in low earth orbit. An upper limit on this torque can be obtained as follows [21]:

$$M_{d_{max}} = \frac{1}{2}\rho V^2 A C_d |r_p - r_m| \quad (29)$$

where ρ is the atmospheric density, V is the velocity magnitude of the satellite, A is the surface area, C_d is the drag coefficient and $|r_p - r_m|$ is the distance between centers of pressure and mass.

For the considered satellite, the altitude above Earth's surface is approximately constant at $320[km]$. From [1, App. L], the atmospheric density at this altitude is approximately $\rho = 2.418 \times 10^{-11}$. The satellite's orbital speed is calculated as $V = \sqrt{\frac{\mu}{a}}$, where μ is the gravitational parameter of Earth and a is the semi-major axis of the orbit, hence $V = 7.7183 \times 10^3[m/s]$. The total surface area of the satellite is $A = 59.312[m^2]$. Lastly, for practical applications, [1, Chap. 17] recommends setting C_d to 2.0 in the absence of additional data. This results in a maximum expected torque due to atmospheric drag of $M_{d_{max}} = 0.0130[Nm]$.

[1, Chap. 17] also provides a method to obtain the torque vector through considering a discretization of the satellite, where the aerodynamic force on each surface element is determined and its contribution to the torque about the center of mass is summed to the contributions of the other surface elements. The aerodynamic force on an individual element is obtained as follows:

$$d\mathbf{f}_{aero} = -\frac{1}{2}\rho V^2 C_d (\hat{n} \cdot \hat{v}) \hat{v} dA \quad (30)$$

where $\hat{n}$ is the unit normal vector of the surface element, $\hat{v}$ is the unit vector in direction of the translational velocity of the surface element relative to the incident stream and dA is the area of the surface element. The translational velocity of element dA is $\mathbf{V} = \mathbf{V}_0 + \boldsymbol{\omega} \times \mathbf{r}$, where $\mathbf{V}_0$ is the velocity of the center of mass, $\boldsymbol{\omega}$ is the angular velocity and $\mathbf{r}$ is the position vector from the center of mass to element dA . The aerodynamic torque acting on the satellite is then obtained as:

$$\mathbf{M}_d = \sum_A \mathbf{r} \times d\mathbf{f}_{aero} \quad (31)$$

The implementation is given in Listing 13.12.14. This torque is plotted in Figure 93 over one orbit with initial conditions as specified by the mission. It shows the three components and the magnitude. The magnitude plot also shows the maximum expected torque that is encountered in orbit given in Equation 24. This is the most dominant torque, which is a result of the low Earth orbit.

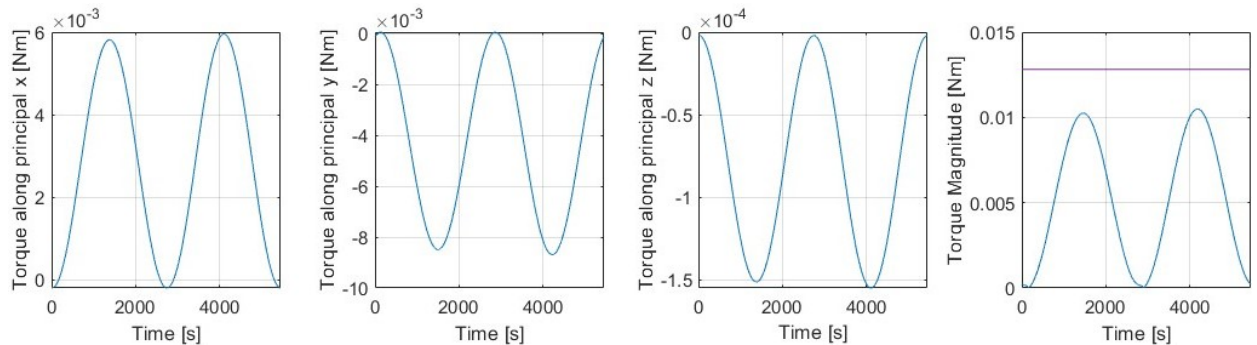


Figure 93: Torque due to aerodynamic drag decomposed in the principal axes frame and its magnitude. The magnitude plot also contains the maximum theoretical torque from Equation 29.

Gravity Gradient

For completeness, the upper limit on the gravity gradient torque is given by [21]:

$$M_{gg_{max}} = \frac{3}{2} \frac{\mu}{R^3} |I_{max} - I_{min}| \quad (32)$$

where I_{max} and I_{min} are the maximum and minimum principal moments of inertia. μ is the gravitational parameter of Earth and R is the distance from Earth's center to the satellite. This results in a maximum theoretical gravity gradient torque of $M_{gg_{max}} = 2.7 \times 10^{-3} [Nm]$.

The gravity gradient torque is plotted in Figure 94 over one orbit with initial conditions as specified by the mission. The figure shows the three components and the magnitude. The magnitude plot also shows the maximum expected torque that is encountered in orbit given in Equation 32.

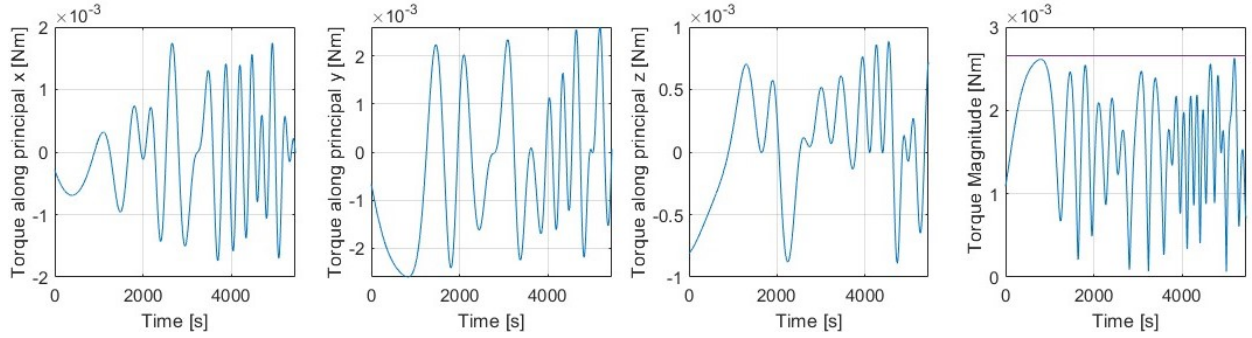


Figure 94: Torque due to gravity gradient decomposed in the principal axes frame and its magnitude. The magnitude plot also contains the conservative maximum theoretical torque from Equation 32.

6.3 PROBLEM 3

The integration of all torques into a resultant torque is given in Listing 13.12.11. A comparison of all disturbance torques is provided in Figure 95 and Figure 96, which show a comparison of their components and magnitude, respectively. Clearly, the atmospheric drag and gravity gradient torques are the most dominant. Furthermore, the magnetic field torque is higher than the solar radiation torque, which is smallest in magnitude of all disturbance torques.

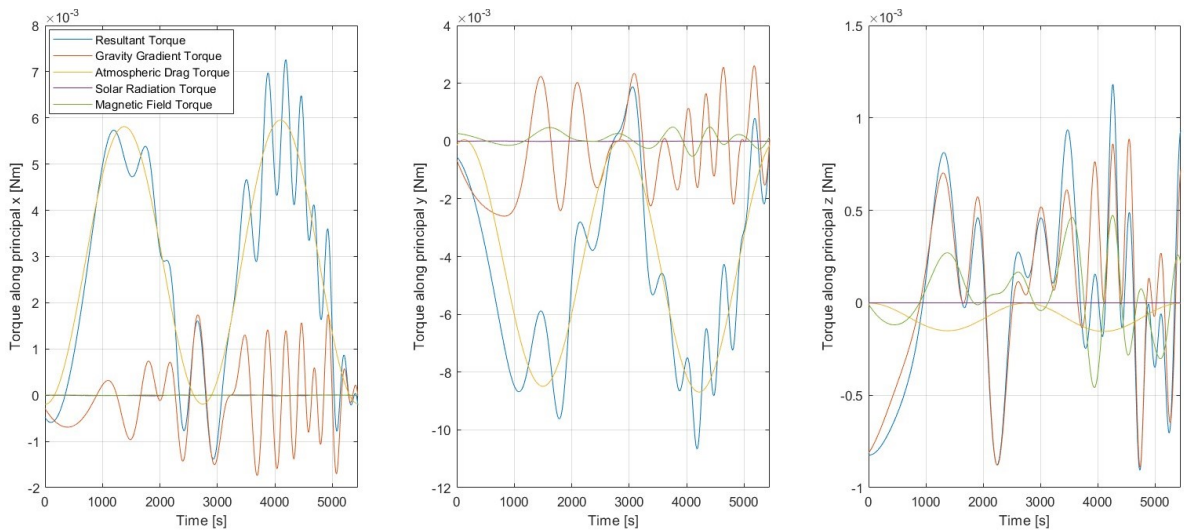


Figure 95: Comparison of the components of the disturbance torques

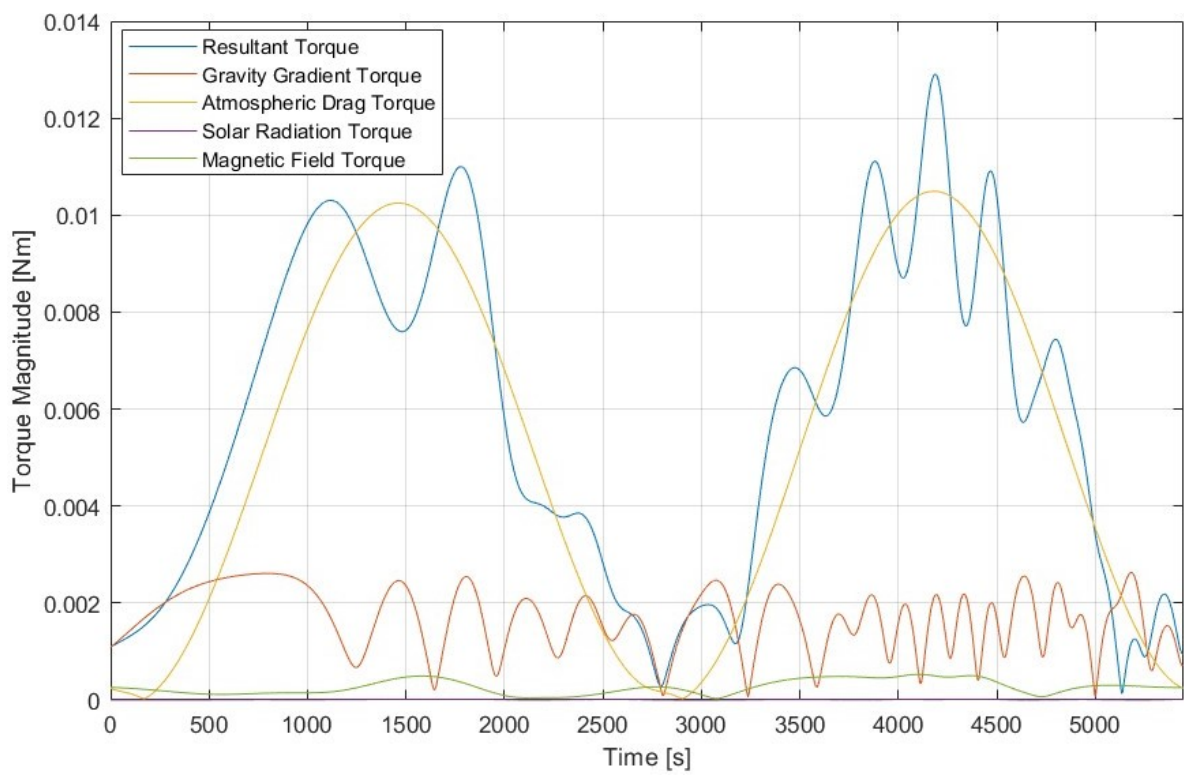


Figure 96: Comparison of the magnitude of the disturbance torques

7 Attitude Determination

7.1 PROBLEM 1

The perturbation modelling is extended by including eclipses for the solar radiation torque. Since the satellite does not experience any solar radiation while in the shadow of Earth, it is required to know when the satellite find itself in eclipse.

While an eclipse is unlikely to have major effects on the dynamics of the satellite since it is the smallest disturbance torque in magnitude out of the modelled torques. Furthermore, a satellite generally spends less time in eclipse, in this case 40% of its orbit, than in sunlight. However, eclipses also have another important impact on the satellite and its mission. Namely, during eclipse, the satellite is not able to receive solar power, it is not able to use the Sun as a reference measurement for attitude determination and it is also not able to take any scientific measurements. Therefore, it is deemed crucial to model when the satellite experiences an eclipse.

To determine the conditions under which eclipses occur, first the length, C , is determined and angular radius, P_c of the shadow cone for any of the planets or natural satellites. Then, let $R_{sun,earth}$ be the distance from Earth to the Sun, R_E be the radius of the Earth, R_S be the radius of the of the Sun. Then, following condition must apply for the satellite to be in eclipse [1]:

$$C = \frac{R_E R_{sun,earth}}{(R_S - R_E)}$$

$$\rho_c = \arcsin\left(\frac{R_S - R_E}{R_{sun,earth}}\right)$$

Then let D_S be the vector from the spacecraft to the Sun and let D_E be the vector from the spacecraft to the center of Earth. Furthermore, define:

$$\rho_s = \arcsin(R_S/D_S)$$

$$\rho_p = \arcsin(R_p/D_E)$$

$$\theta = \arccos(\hat{D}_S \cdot \hat{D}_E)$$

Then the condition for total ecplise is:

$$R_{sun,earth} < D_S < R_{sun,earth} + C \text{ and } \rho_p - \rho_s > \theta$$

7.2 PROBLEM 2

The nominal attitude of the satellite is specified by the mission objective. As shown in Figure 1, the satellite is flown with the payload pointing towards Earth in a plane quasi-perpendicular to the flight path and 35° offset from nadir. Using the earlier defined body axes frame, if the body z-axis is aligned with the radial direction, body x-axis is aligned with normal direction and body y-axis aligned with transverse direction, then in this configuration, the satellite is rotated by 35° around the transverse direction to reach its nominal orientation. This orientation is depicted in Figure 97, which also contains the body axes, orbital axes and principal axes frames.

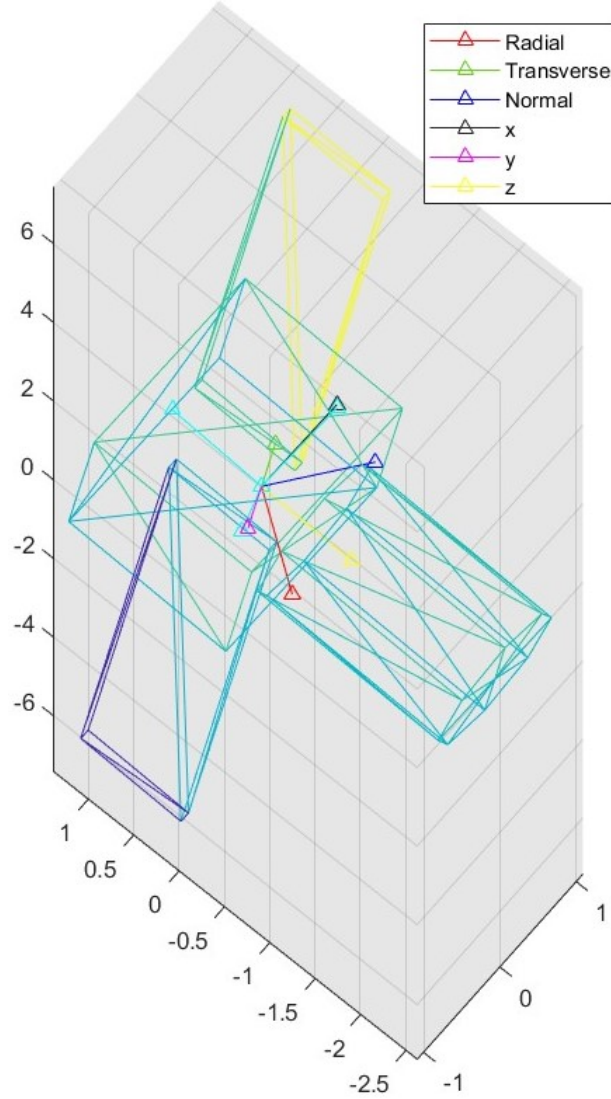


Figure 97: Satellite orientation corresponding to the mission specifications. The graph depicts the body axes and orbit axes.

The desired orientation is defined relative to the RTN -frame, where Euler angles of the body frame with respect to the RTN -frame should be maintained at $\eta_d^{bo} = [0 \ -55.0 \ -180.0] [deg]$. In terms of the principal axes, the desired attitude then becomes $\eta_d^{po} = [94.0 \ 34.3 \ 178.6] [deg]$. The Euler angles of the desired attitude of principal axes with respect to inertial frame and the actual attitude of principal axes with respect to inertial frame are given in Figure 98 over one orbit. Additionally, the control error, which is the Euler angles of the desired attitude of principal axes with respect to actual attitude of principal axes, is given in Figure 99. This error indicates the rotations required to go from the satellite's current attitude to the desired attitude.

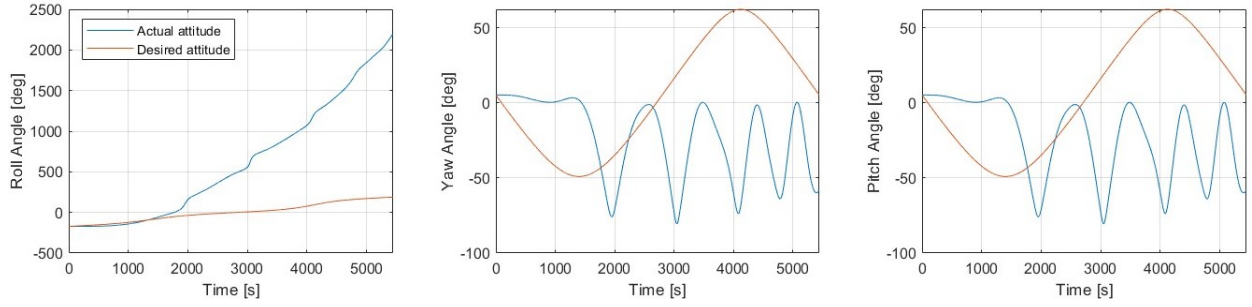


Figure 98: Actual and desired attitude angles of the principal axes with respect to inertial frame.

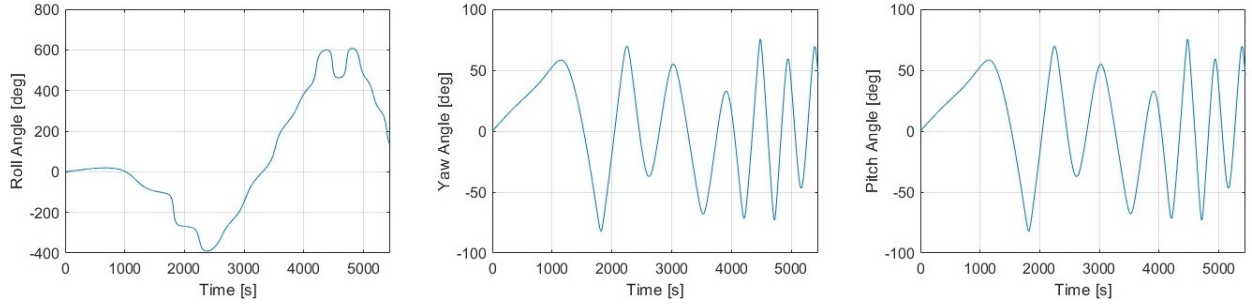


Figure 99: Control error of principal axes in Euler angle representation.

7.3 PROBLEM 3

Given that the control error represents a rotation matrix between the desired attitude and the actual attitude, any attitude parameterization can be used to visualize the control error. In previous section, the Euler angles were used to show the control error, however Euler angles are not unique and sometimes hard to interpret. One single quantity which is able to quantify the control error is by using the axis-angle representation. In this case, the angle represent the angle between the desired state and actual state around a given axis. Hence, it gives a single metric of how big the control error is. This angle is given in Figure 100.

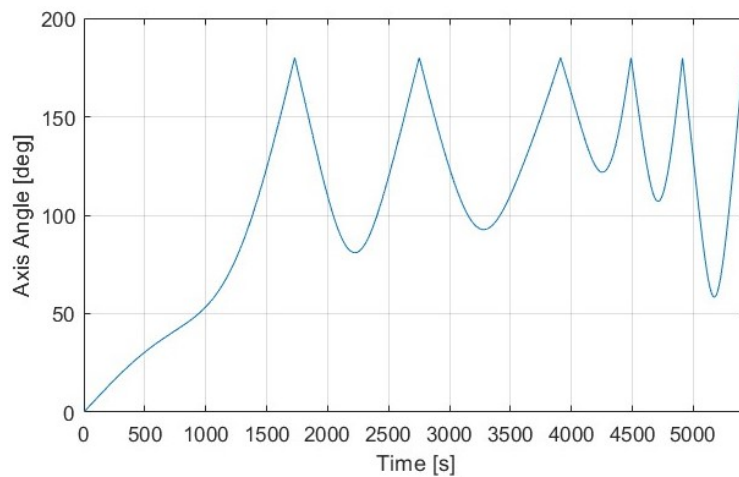


Figure 100: Control error of principal axes in axis-angle representation.

The control error in both rotation representations is sensible and consistent with the applied disturbances since it largely follows the period of oscillation of the atmospheric drag torque, the most dominant disturbance torque.

7.4 PROBLEM 4

The satellite relies on sensors to gain knowledge about its position and orientation in space, relative to an inertial reference frame. The satellite is equipped with a star sensor, earth sensor, magnetometer, inertial measurement unit (IMU) and GPS sensor. These sensors collectively provide the information given in Table 14. These sensor models are implemented in *Simulink* and are given under subsection 13.14.3. These models do not include any noise or bias yet.

Table 11: Sensor suite and information about satellite state.

Sensor	Measurement	Reference Frame
IMU	Angular velocity and acceleration	Principal axes relative to inertial frame
GPS	Position	Inertial frame
Sun sensor	Unit vector pointing in direction of Sun	Body frame
Earth sensor	Unit vector pointing in direction of Earth	Body frame
Magnetometer	Unit vector pointing along Earth's magnetic field	Body frame

7.5 PROBLEM 5

The satellite is equipped with three sensors providing attitude measurements directly, namely a magnetometer, Earth sensor and Sun sensor. These sensors respectively provide a unit vector in direction of Earth's magnetic field, in direction of the Earth and in direction of the Sun. While only two independent measurements are required, the third measurement provides redundancy in situations such as eclipse where the Sun sensor cannot provide a measurement or the situation in which the Earth and Sun are collinear. The full attitude determination is implemented in *Simulink* and given under subsection 13.14.6.

The first attitude estimation method is using the deterministic attitude determination algorithm. The rotation matrix can be obtained as follows:

$$R_i^b = [\mathbf{m}_1 \quad \mathbf{m}_2 \quad \mathbf{m}_3] [\mathbf{v}_1 \quad \mathbf{v}_2 \quad \mathbf{v}_3]^{-1} = MV^{-1} \quad (33)$$

where M contains the measured unit vectors, which are in body frame and V contains those unit vectors in inertial frame. These unit vectors are obtained from a model based on the satellite's position in orbit. The attitude angles of the principal axes are then extracted from the rotation matrix $R_i^p = R_b^p R_i^b$. This method can be extended to using fictitious measurements by combining the actual measured unit vector as follows:

$$\tilde{\mathbf{m}}_1 = \frac{\mathbf{m}_1 + \mathbf{m}_2}{2} \quad \tilde{\mathbf{m}}_2 = \frac{\mathbf{m}_1 - \mathbf{m}_2}{2} \quad \tilde{\mathbf{m}}_3 = \mathbf{m}_3$$

Since the Sun sensor is expected to have the highest accuracy measurement, it is used as $\mathbf{m}_3$. The implementation is given in Figure 184

A second method is the statistical attitude determination algorithm (q-method). This method is based on a weighted least-squares and results in an eigenvalue/eigenvector problem to find the estimated attitude quaternion. Since no noise or bias is modelled yet, there is no real basis on which to choose these weights, hence they are set to 1 and will be later adjusted accordingly. The same fictitious measurements as defined above may be used for this method. The implementation is given in Figure 185

A third method is to simply integrate the angular velocity measurements of the IMU using the attitude kinematics. The implementation given in Figure 186

7.6 PROBLEM 6

The satellite is simulated for over one orbital period and its attitude is estimated using the three above described methods. The results of each estimation method along with the ground truth attitude in Euler angles is shown in Figure 101, Figure 102 and Figure 103 for the deterministic, statistical and integration method, respectively. Clearly, the estimations exactly match the ground truth for all three methods in the absence of sensor noise and bias. Note, the Euler angles are wrapped to be within $[-180, +180][deg]$

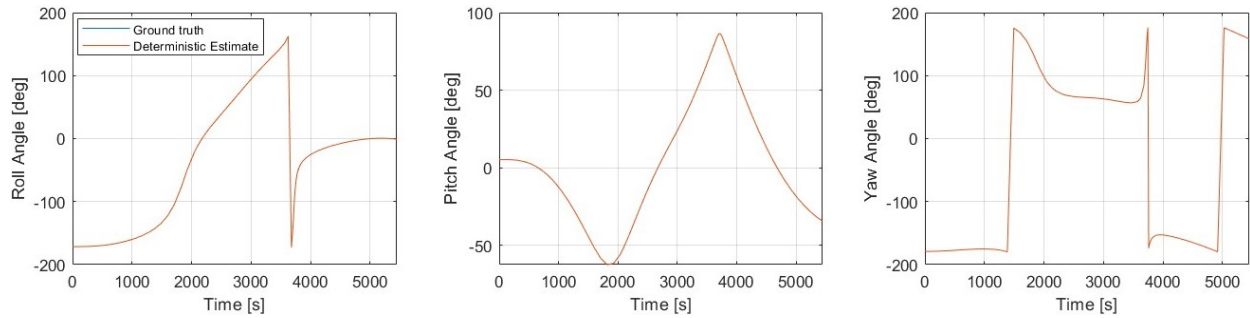


Figure 101: Ground truth attitude along with the estimated attitude using the deterministic estimation method.

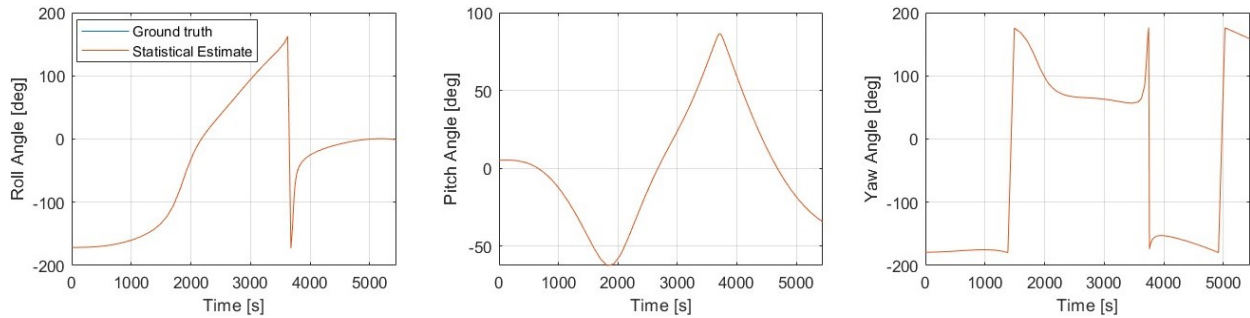


Figure 102: Ground truth attitude along with the estimated attitude using the statistical estimation method.

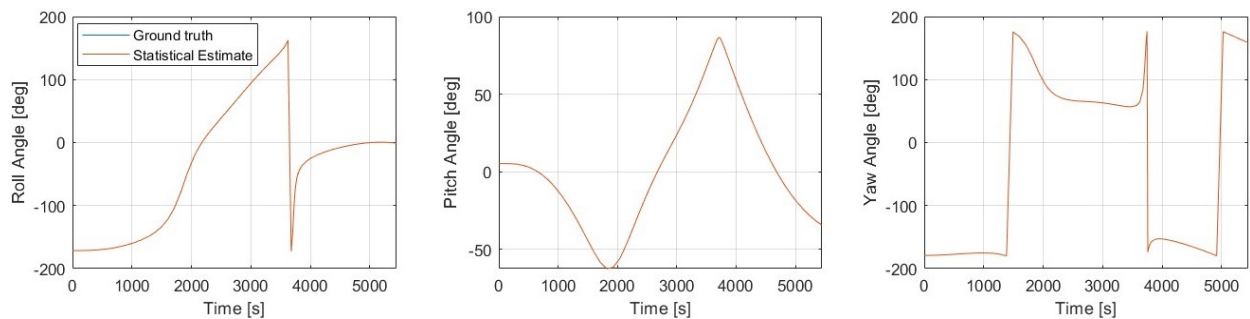


Figure 103: Ground truth attitude along with the estimated attitude using the integrated angular velocity method.

8 Extended Kalman Filter - predict step

8.1 PROBLEM 1

Sensor measurements contain noise due to external interference or intrinsic sensor imperfections. It is important to include these inherent imperfections in the sensor models. The noise of a sensor can be expressed as the accuracy and bias around the true measurement, where the sensor noise is modelled as Gaussian.

Specifically for attitude sensors, these measure a direction hence the measurement noise can be expressed as the accuracy in the measured angle. To model the noise, a random unit vector is drawn from a uniform distribution for the direction of the measurement error and the angle is drawn from a normal distribution. This results in a rotation vector representation for the measurement error which can be converted to an error rotation matrix. For the attitude sensors, i.e. Earth sensor, Sun sensor and magnetometer, *CubeSpace's GEN2* product series [5][4][3] is used as a reference. The measurement noise properties for each attitude sensor is given in Table 12, where the accuracy indicates the variance and bias indicates the mean. It is assumed that the sensors are well-calibrated to avoid major biases.

Table 12: Sensor Noise Properties for Attitude Sensors

Sensor	Bias [deg]	Accuracy [deg/s]
Earth Sensor	0.01	0.33
Sun Sensor	0.01	0.1
Magnetometer	0.1	3.0

For the inertial measurement unit, the *HG2801* Fiber Optic Gyro Inertial Measurement Unit by *Honeywell* [13] is used as reference. The gyroscope has a reported noise density of $0.05[deg/\sqrt{hr}]$ and bias of $0.05[deg/hr]$, while the accelerometer has bias of $50\mu g$. Given a sampling time of $0.1[sec]$, the sensor noise properties are given in Table 13.

Table 13: Sensor Noise Properties for Attitude Sensors

Sensor	Bias	Accuracy
Gyroscope	$8.33 \times 10^{-4} [deg]$	$2.63 \times 10^{-3} [deg/s]$
Accelerometer	$4.905 \times 10^{-4} [m/s^2]$	$4.905 \times 10^{-4} [m/s^2]$

8.2 PROBLEM 2

The satellite is simulated for part of its orbital period and its attitude is estimated using the deterministic, statistical and angular velocity integration methods. An initial angular velocity of $\omega_0 = [0 \ 1 \ 1] [deg/s]$ is used. The results of each estimation method along with the ground truth attitude in Euler angles is shown in Figure 104, Figure 105 and Figure 106 for the deterministic, statistical and integration method, respectively. It can be seen that the estimations still match the ground truth well for each method, however, the estimated attitude is now noisy. It can be observed that the quality of the estimation improves from deterministic to statistical to integrated. This is likely due to the high-quality of the gyroscope. It is, however, expected that the integrated estimate will drift off over multiple orbits.

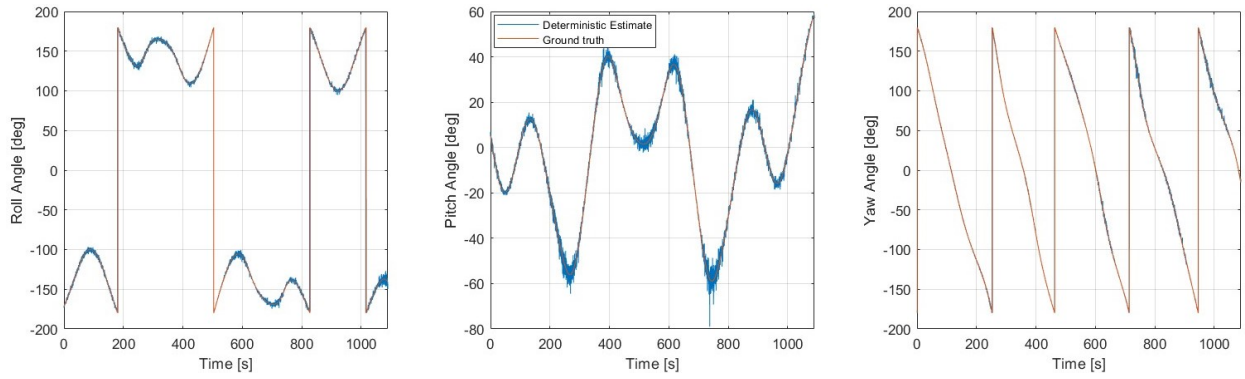


Figure 104: Ground truth attitude along with the estimated attitude using the deterministic estimation method.

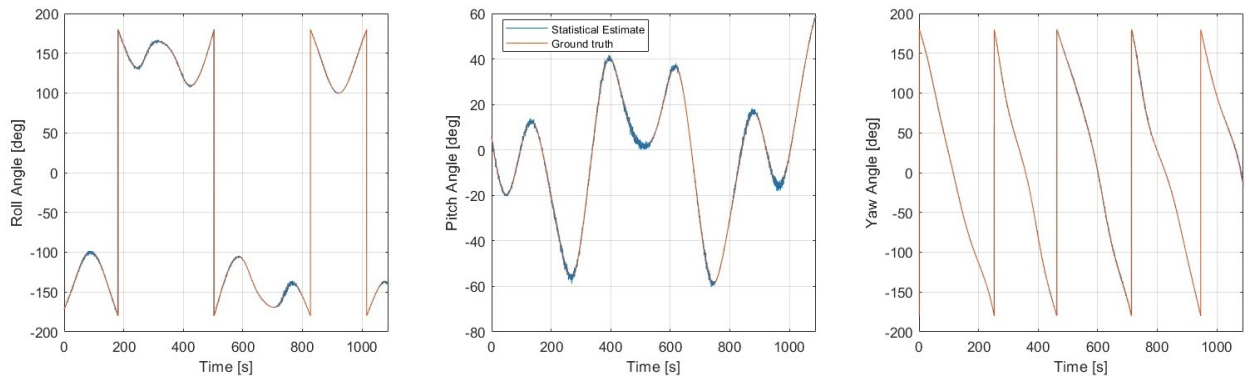


Figure 105: Ground truth attitude along with the estimated attitude using the statistical estimation method.

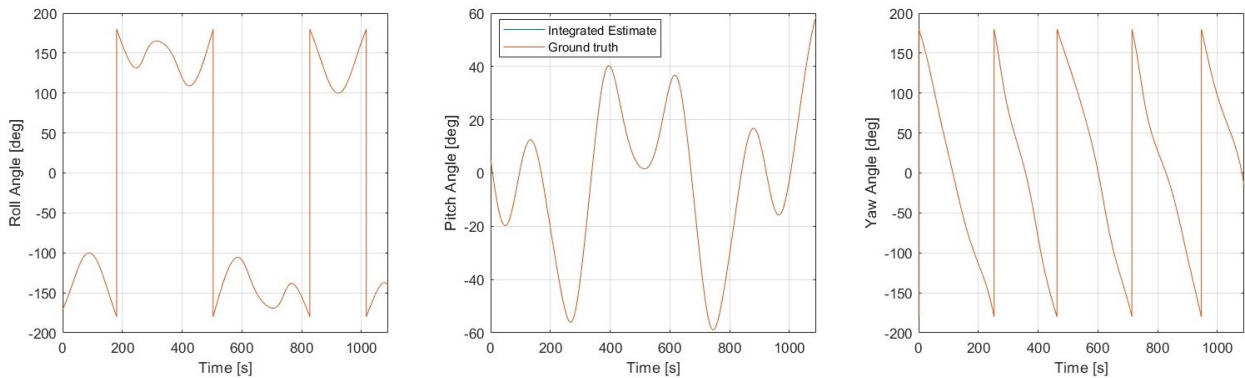


Figure 106: Ground truth attitude along with the estimated attitude using the integrated angular velocity method.

8.3 PROBLEM 3

For small sensor errors, the error rotation matrix corresponds to a small rotation. Therefore, it can be represented using the small angle approximation of the rotation matrix. From previous section, it is observed that the estimate with largest error is the deterministic estimation method. The estimation errors are given in Figure 107. From the figure, it can be seen that the majority of the errors lie within $10[deg]$. Specifically, 99.92% lies within $10[deg]$, 99.56% lies within a $5[deg]$ error and 87.46% lies within $1[deg]$ error. This shows that the sensor errors remain small, hence the small angle approximation of the rotation matrix can be used to represent the attitude estimation error.

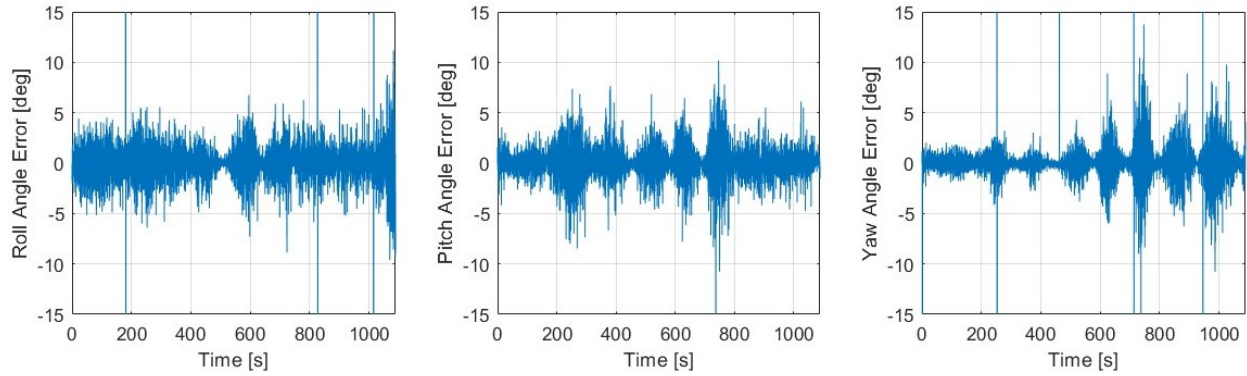


Figure 107: Attitude estimation errors for the deterministic estimation error

8.4 PROBLEM 4

The satellite relies on sensors to gain knowledge about its orientation in space, relative to an inertial reference frame. The satellite is equipped with a star sensor, earth sensor, magnetometer, inertial measurement unit (IMU). These sensors collectively provide the information given in Table 14. Since the satellite relies on on-board models of the magnetometer as well as knowledge of the position of the Earth and the Sun in inertial frame, it is assumed that the satellite has a GPS where the position measurement is the ground-truth position of the satellite in inertial frame.

For the sensor models, it is assumed that the Sun sensor, Earth sensor and Magnetometers provide unit vector measurements directly. Specially, the Sun sensor provides the direction of the Sun in the body frame, the Earth sensor provides the direction of Earth in body frame, lastly the magnetometer provides the direction of the magnetic field in body frame. For simplicity, it is assumed that all sensors are mounted such that their local reference system coincides with the body reference frame. In terms of modelling sensor errors and noise, the noise models from above section are used, which are based on custom of the shelf hardware. These sensor models are implemented in *Simulink* and are given under subsection 13.14.3.

Table 14: Sensor suite and information about satellite state.

Sensor	Measurement	Reference Frame
IMU	Angular velocity and acceleration	Principal axes relative to inertial frame
Sun sensor	Unit vector pointing in direction of Sun	Body frame
Earth sensor	Unit vector pointing in direction of Earth	Body frame
Magnetometer	Unit vector pointing along Earth's magnetic field	Body frame

8.5 PROBLEM 5

In order to estimate the attitude of the satellite, an extended Kalman Filter can be used to integrate the sensor measurements with the state propagation using the dynamics equations. Since the problem consists of estimating attitude, the state vector is defined to contain the attitude and angular velocity $x = [\phi \ \theta \ \psi \ p \ q \ r]^T$. The inputs vector is the control torque $u = [M_x \ M_y \ M_z]^T$. The dynamics equation for this state vector are given as follows:

$$\dot{x} = f(x, u) = \begin{bmatrix} p + \sin(\phi)\tan(\theta)q + \cos(\phi)\tan(\theta)r \\ \cos(\phi)q - \sin(\phi)r \\ \sin(\phi)\sec(\theta)q + \cos(\phi)\sec(\theta)r \\ \frac{I_y - I_z}{I_x}qr + \frac{M_x}{I_x} \\ \frac{I_z - I_x}{I_y}rp + \frac{M_y}{I_y} \\ \frac{I_x - I_y}{I_z}pq + \frac{M_z}{I_z} \end{bmatrix}$$

where I_x , I_y and I_z are the principal moments of inertia. Furthermore, the angular velocity components p , q and r are expressed in principal axes. Since these equations are nonlinear, they need to be linearized to be used in the Kalman Filter update equations. However, it is noted that the Euler angle kinematics are linear in the angular velocity, hence the linear terms in the Euler angles are neglected assuming that changes in these angles will remain small over one timestep. Similarly, these equations are linear in the input u . Then linearizing the remainder of the dynamics equations results in:

$$\dot{x} \approx \begin{bmatrix} 0 & 0 & 0 & 1 & \sin(\phi)\tan(\theta) & \cos(\phi)\tan(\theta) \\ 0 & 0 & 0 & 0 & \cos(\phi) & -\sin(\phi) \\ 0 & 0 & 0 & 0 & \sin(\phi)\sec(\theta) & \cos(\phi)\sec(\theta) \\ 0 & 0 & 0 & 0 & \frac{I_y - I_z}{I_x}r & \frac{I_y - I_z}{I_x}q \\ 0 & 0 & 0 & \frac{I_z - I_x}{I_y}r & 0 & \frac{I_z - I_x}{I_y}p \\ 0 & 0 & 0 & \frac{I_x - I_y}{I_z}q & \frac{I_x - I_y}{I_z}p & 0 \end{bmatrix} x + \begin{bmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \\ \frac{1}{I_x} & 0 & 0 \\ 0 & \frac{1}{I_y} & 0 \\ 0 & 0 & \frac{1}{I_z} \end{bmatrix} u$$

It is assumed that the linearization point is an equilibrium point, i.e. $f(x_0, u_0) = 0$. The linearized dynamics can be discretized using a simple Euler integration scheme $x_{k+1} = x_k + dt\dot{x}$, where dt is the timestep. This results in the following time update equation:

$$x_{k+1} = \Phi x_k + Bu$$

$$= \begin{bmatrix} 1 & 0 & 0 & dt & dt\sin(\phi)\tan(\theta) & dt\cos(\phi)\tan(\theta) \\ 0 & 1 & 0 & 0 & dt\cos(\phi) & -dt\sin(\phi) \\ 0 & 0 & 1 & 0 & dt\sin(\phi)\sec(\theta) & dt\cos(\phi)\sec(\theta) \\ 0 & 0 & 0 & 1 & dt\frac{I_y - I_z}{I_x}r & dt\frac{I_y - I_z}{I_x}q \\ 0 & 0 & 0 & dt\frac{I_z - I_x}{I_y}r & 1 & dt\frac{I_z - I_x}{I_y}p \\ 0 & 0 & 0 & dt\frac{I_x - I_y}{I_z}q & dt\frac{I_x - I_y}{I_z}p & 1 \end{bmatrix} x_k + \begin{bmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \\ \frac{dt}{I_x} & 0 & 0 \\ 0 & \frac{dt}{I_y} & 0 \\ 0 & 0 & \frac{dt}{I_z} \end{bmatrix} u_k$$

where Φ is the state transition matrix and B is the control input matrix. These matrices allow propagating the state from k to $k + 1$, including the known control input torques. The output of the propagation step can be verified by comparing it to the numerically integrated attitude. The implementation of the discrete-time linearized state propagation is given in Listing 13.13

Firstly, the satellite is simulated from initial attitude corresponding to mission description and a constant angular velocity of $1.0[deg/s]$ about the z -axis of the principal axes. Given no external torques, this is an equilibrium of the angular velocity. However, the numerically integrated attitude does include the disturbance torques. Since the state is only propagated over small timesteps, the satellite is simulated for $2[sec]$. The result is given in Figure 108 and Figure 109, for attitude and angular velocity, respectively. It can be seen that the attitude propagation is accurate, while the propagated angular velocity slightly deviates since it does not include the disturbance forces.

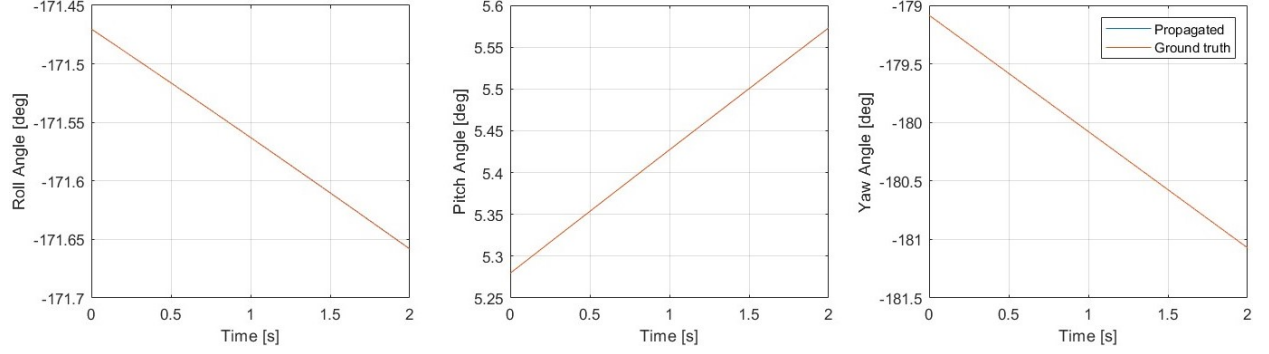


Figure 108: Attitude angles propagation for initial rotation about the z -principal axis.

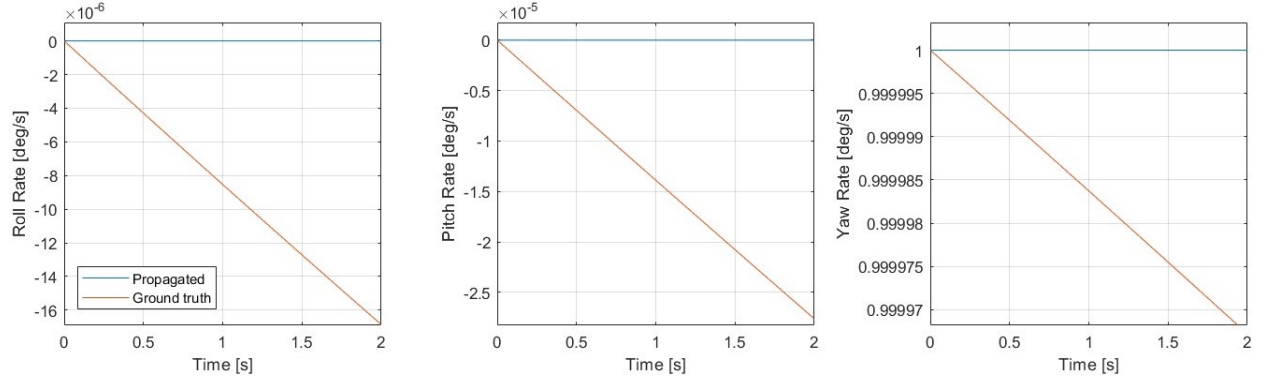


Figure 109: Angular velocity propagation for initial rotation about the z -principal axis.

Secondly, the satellite is simulated for an initial angular velocity vector $\omega_0 = [0 \ 1 \ 1][deg/s]$. This scenario does not lead to an equilibrium condition for angular velocity. The result is given in Figure 110 and Figure 111, for attitude and angular velocity, respectively. It can be seen that the attitude propagation deviates slightly in the roll angle after 1 second. This is due to the propagated angular velocity deviating from the numerically integrated angular velocity.

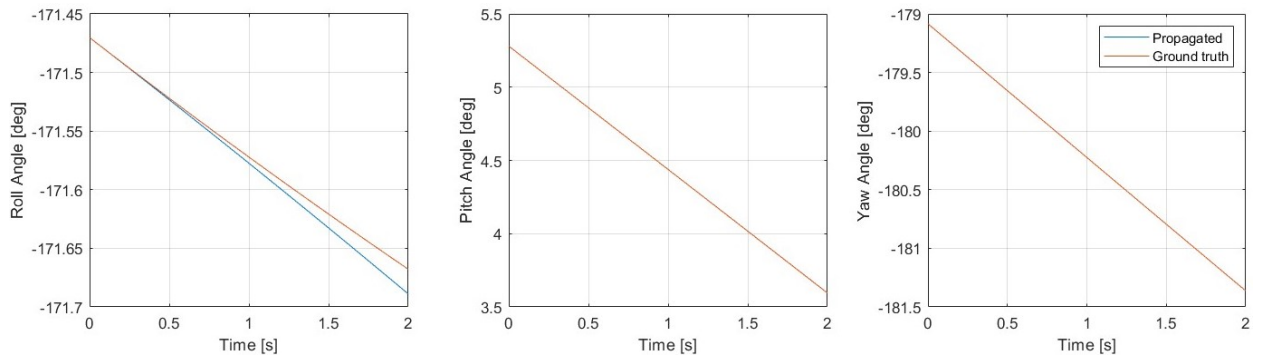


Figure 110: Attitude angles propagation for initial angular velocity vector $\omega_0 = [0 \ 1 \ 1][deg/s]$.

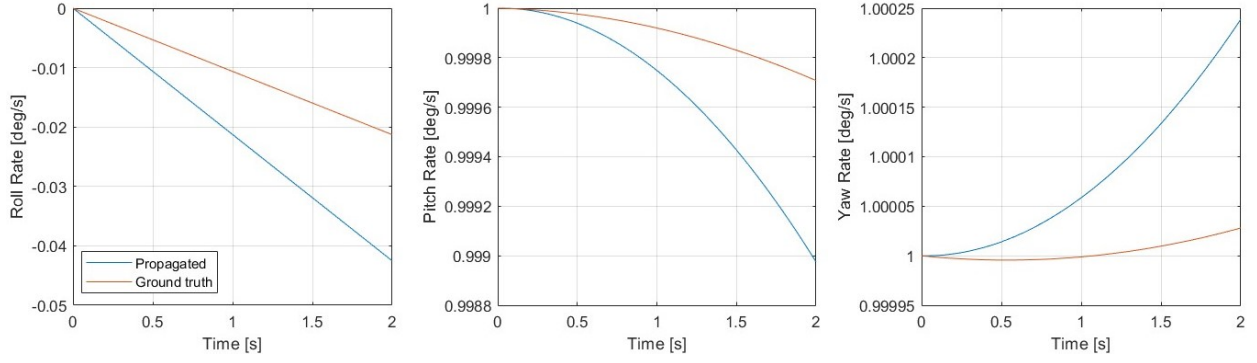


Figure 111: Angular velocity propagation for initial angular velocity $\omega_0 = [0 \ 1 \ 1] [deg/s]$.

The attitude error between propagated and numerically integrated attitude can also be represented through a rotation matrix which specifies the rotation required to go from the propagated attitude to the numerically integrated attitude. This error rotation matrix can be converted to the axis-angle representation where the angle is a single number quantification of the propagation error. For above simulation conditions, this propagation error is given in Figure 112. From the figure, it can be seen that the attitude error evolves quadratically with time and reaches an error of $0.02[deg]$ after 2 seconds.

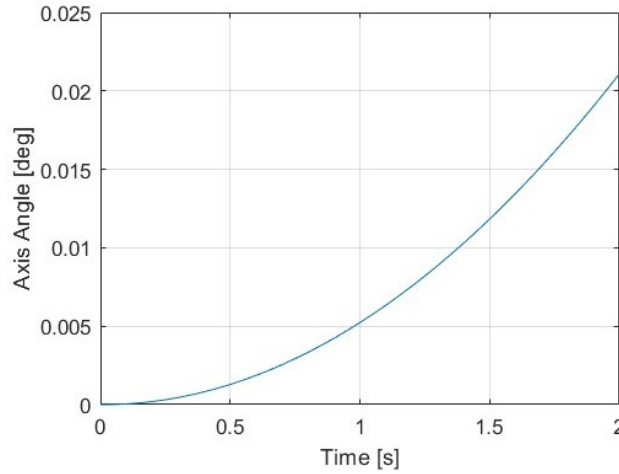


Figure 112: Attitude propagation error.

To complete the time update step of the Kalman Filter, it is required to define an initial state error covariance matrix P which quantifies the uncertainty of the initial state. Furthermore, a process noise Q can be defined which accounts for the uncertainty in the update step. The initial attitude of the satellite is assumed to be the ground-truth value for now. This results in relatively small values for the state error covariance. Furthermore, it is assumed that the initial state error covariance has a diagonal structure where the diagonal values represent the initial variance σ^2 in the initial states. To start, a standard deviation of $1 \times 10^{-2}[deg]$ in the attitude angles and a standard deviation of $1 \times 10^{-2}[deg/s]$ is taken. For the process noise, a matrix with a factor of 10 smaller than the state error covariance is taken. This results in following state error covariance and process noise matrices:

$$P = 1 \times 10^{-2} \frac{\pi}{180} \mathbb{I}_6 \quad Q = 1 \times 10^{-5} \frac{\pi}{180} \mathbb{I}_6$$

where $\mathbb{I}_n$ indicates the identity matrix with dimensions $n \times n$. The factor $\frac{\pi}{180}$ is included to convert from degrees to radians.

8.6 PROBLEM 6

To verify the proper functioning of the time update step of the Kalman Filter algorithm, the algorithm is implemented with the output matrix set to zero and the initial state is set to the true initial state. The rest of the parameters, i.e. state transition matrix Ψ , input matrix B , state error covariance matrix P and the process noise matrix Q are defined as above. Lastly, since the satellite does not yet have a controller implemented, the control input is zero. The extended Kalman Filter is implemented in *Simulink* and provided under subsubsection 13.14.6.

The results for a simulation over a timespan of 10 seconds with initial angular velocity of $\omega_0 = [1 \ 1 \ 1] [deg/s]$ are given in Figure 113 and Figure 114. From the figure, it can be seen that the state error covariance diverges with time. This is expected since the estimation of the attitude is performed only using the time update step, hence without introducing any sensor information. Moreover, the formal covariance contains the true estimate, however the bounds are quite loose.

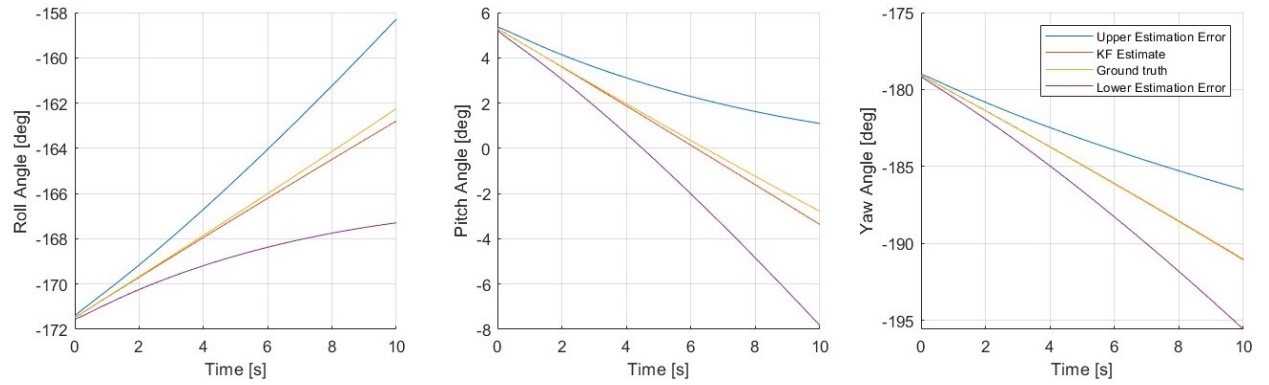


Figure 113: Attitude angles estimation using Kalman filter time update step. The upper and lower estimation limits are $1 - \sigma$ as obtained from the state error covariance.

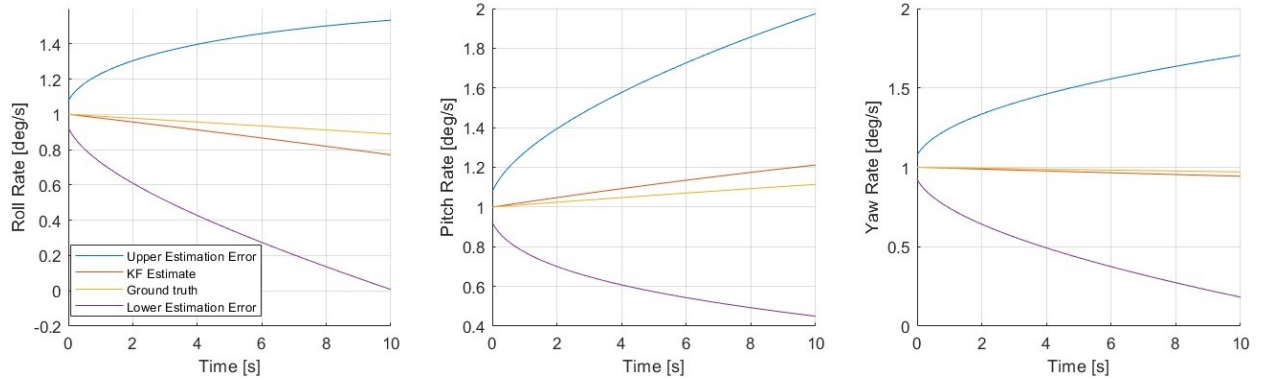


Figure 114: Angular velocity estimation using Kalman filter time update step. The upper and lower estimation limits are $1 - \sigma$ as obtained from the state error covariance.

The error between the true attitude and the estimated attitude along with the standard deviation obtained from the state error covariance matrix are given in Figure 115 and Figure 116 for the attitude angles and angular velocity, respectively. It can be seen that the true error is contained within one standard deviation of the state error covariance.

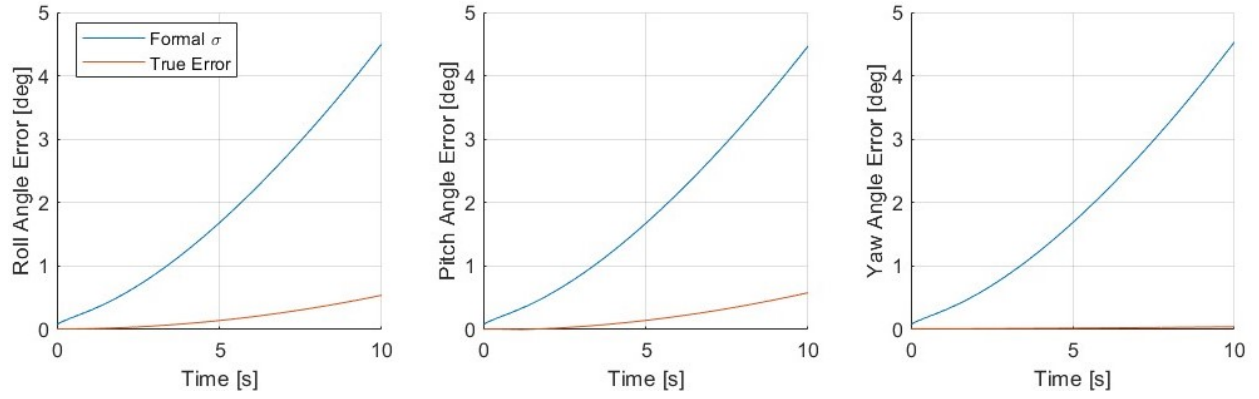


Figure 115: Attitude angles estimation using Kalman filter time update step. The upper and lower estimation limits are $1 - \sigma$ as obtained from the state error covariance.

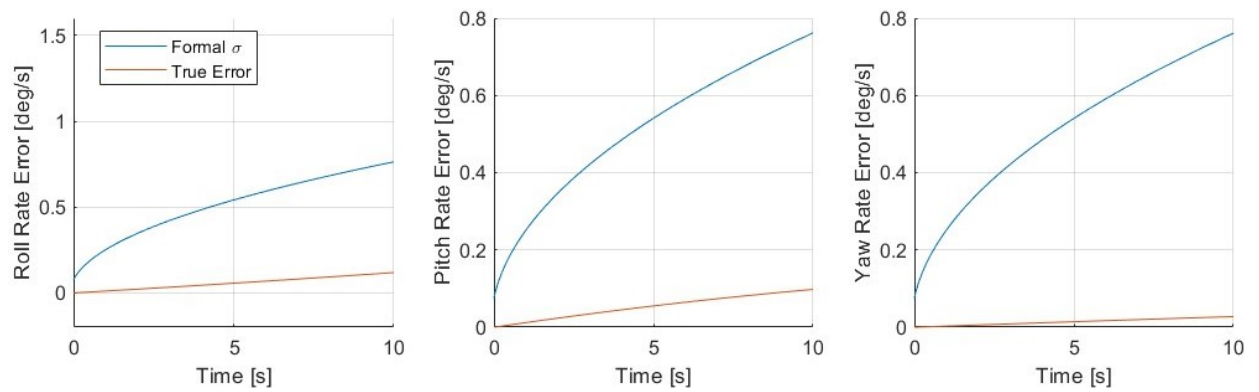


Figure 116: Angular velocity estimation using Kalman filter time update step. The upper and lower estimation limits are $1 - \sigma$ as obtained from the state error covariance.

9 Extended Kalman Filter - update step

9.1 PROBLEM 1

In order to include measurement information into the state estimation, a measurement model is required which maps the state of interest to the available measurements. The satellite received four measurements, angular velocity, Earth direction, Sun direction and magnetic field direction, as given in Table 14. Consider that the state that is being estimated is given by $\hat{x} = [\hat{\eta} \ \hat{\omega}]^T$, where $\hat{\eta}$ are the estimated Euler angles and $\hat{\omega}$ is the estimated angular velocity. The measurement model for the angular velocity is given as follows:

$$\hat{y}_{\omega} = [\mathbf{0}_{3 \times 3} \ \mathbb{I}_3] \hat{x}$$

The remaining measurement models are given as follows:

$$\begin{aligned}\hat{y}_{Earth} &= R_{bp} R(\hat{\eta}) v_{Earth} \\ \hat{y}_{Sun} &= R_{bp} R(\hat{\eta}) v_{Sun} \\ \hat{y}_{magnetic} &= R_{bp} R(\hat{\eta}) v_{magnetic}\end{aligned}$$

where v_{Earth} , v_{Sun} and $v_{magnetic}$ are based on models which depend on the position of the satellite in orbit. These models give these directions in the inertial frame. $R(\hat{\eta})$ is the rotation matrix parameterization based on the current estimate of the attitude angles. R_{bp} is the rotation matrix from principal axes frame to body frame. These equations give the predicted measurements from the current state estimate.

The sensitivity matrix of these measurement equations can be derived by taking the partial derivative w.r.t. the state vector. This is performed numerically by the Extended Kalman Filter implementation in *Simulink*.

9.2 PROBLEM 2

The measurement error covariance matrices are defined based on the accuracy of the sensor given in Table 13 and Table 12. This results in following covariance matrices:

$$\begin{aligned}R_{\omega} &= 2.63 \times 10^{-3} \frac{\pi}{180} \mathbb{I}_3 \\ R_{Earth} &= 3.3 \times 10^{-1} \frac{\pi}{180} \mathbb{I}_3 \\ R_{Sun} &= 1.0 \times 10^{-1} \frac{\pi}{180} \mathbb{I}_3 \\ R_{magnetic} &= 3.0 \frac{\pi}{180} \mathbb{I}_3\end{aligned}$$

The measurement function configuration as well as the measurement error covariance matrix configuration is given in Figure 188.

9.3 PROBLEM 3

The modelled measurements $\hat{y}$ are computed using the non-linear measurement function defined above. This provides a better performance since it is not guaranteed that the state about which is linearized is an equilibrium point. Hence, $\hat{y} = g(\hat{x})$, where g is the measurement function defined above for each sensor.

9.4 PROBLEM 4

The pre-fit residuals z are computed by differencing the modelled and actual measurements as follows, $z = y - \hat{y}$. These pre-fit residuals are calculated before including any sensor information into the state estimation yet. The pre-fit residuals are given in Figure 117, Figure 118, Figure 119 and Figure 120. Clearly, these residuals are relatively high considering that the measured directions are captured through unit vectors which have magnitude 1. The *Simulink* implementation to determine the residuals is given in Figure 189.

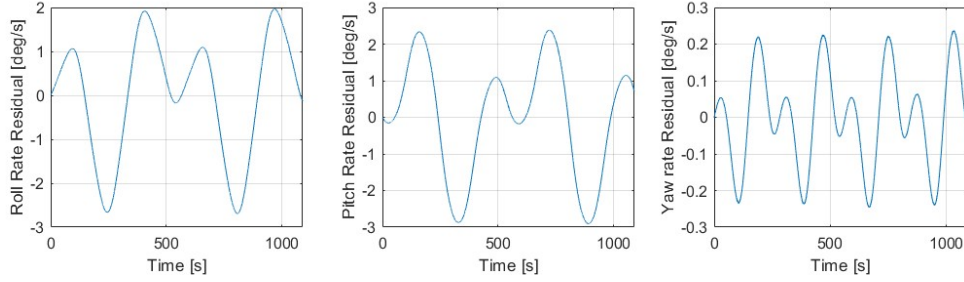


Figure 117: Pre-fit residual of angular velocity $y_\omega - \hat{y}_\omega$.

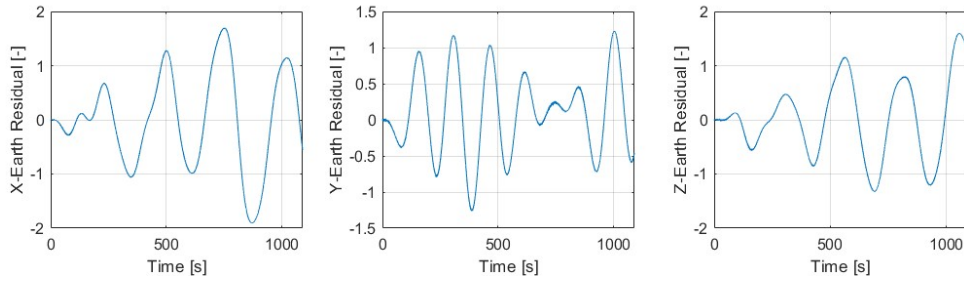


Figure 118: Pre-fit residual of Earth direction $y_{earth} - \hat{y}_{earth}$.

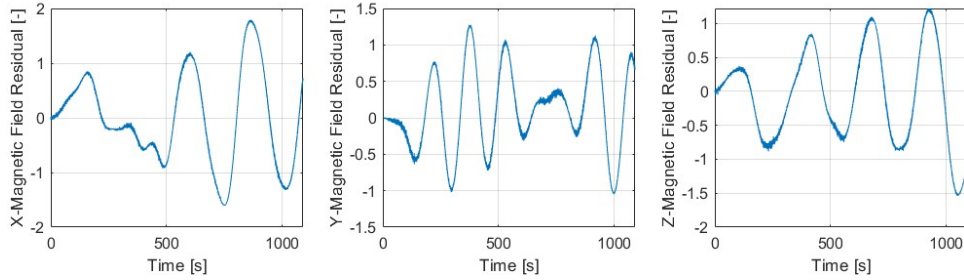


Figure 119: Pre-fit residual of magnetic field direction $y_{magnetic} - \hat{y}_{magnetic}$.

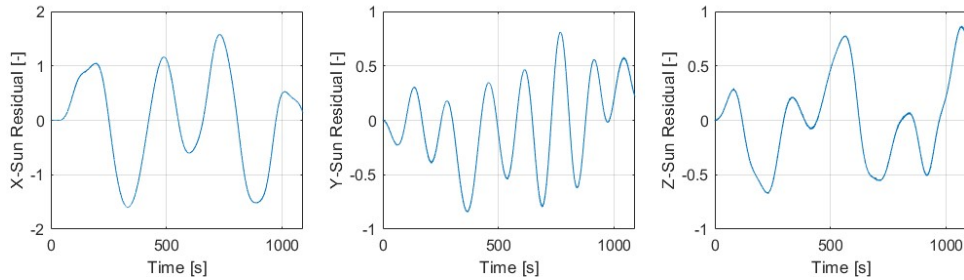


Figure 120: Pre-fit residual of Sun direction $y_{sun} - \hat{y}_{sun}$.

9.5 PROBLEM 5

Given the above information, the measurement step for the EKF can be performed. Let P be the state error covariance matrix. For each measurement, the following computation is performed:

$$\begin{aligned} K &:= PH^T(HPH^T + R)^{-1} \\ \hat{x} &:= \hat{x} + Kz \\ P &:= P - KHP \end{aligned}$$

where K is the Kalman gain, and $\hat{x}$ is the state estimate and P is its associated covariance matrix. This measurement update can be performed sequentially for each measurement independently and at a different rate whenever a new measurement becomes available. The complete Extended Kalman Filter implementation is given under subsubsection 13.14.6

9.6 PROBLEM 6

The post-fit residuals z are computed by differencing the modelled and actual measurements as follows, $z = y - \hat{y}$. These post-fit residuals are calculated after including all sensor information into the state estimation. The post-fit residuals are given in Figure 121, Figure 122, Figure 123 and Figure 124. Additionally, three standard deviations are indicated as defined in the measurement error covariance matrices above. It can be seen that these residuals capture the standard deviation of the measurements.

Initially, the residuals for the angular velocity measurements were biased and drifted. This is due to the process noise being defined as given in Figure 8.5. However, since the disturbance torques are not included in the state estimation, this assigned process noise is too low. Therefore, the process noise matrix is changed to $Q = 1 \times 10^{-1} \frac{pi}{180} \mathbb{I}_6$, which results in better performance.

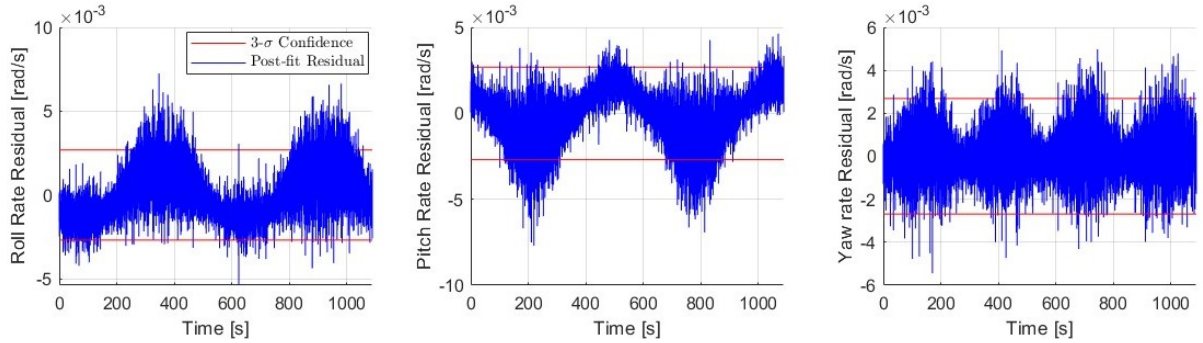


Figure 121: Post-fit residual of angular velocity $y_\omega - \hat{y}_\omega$.

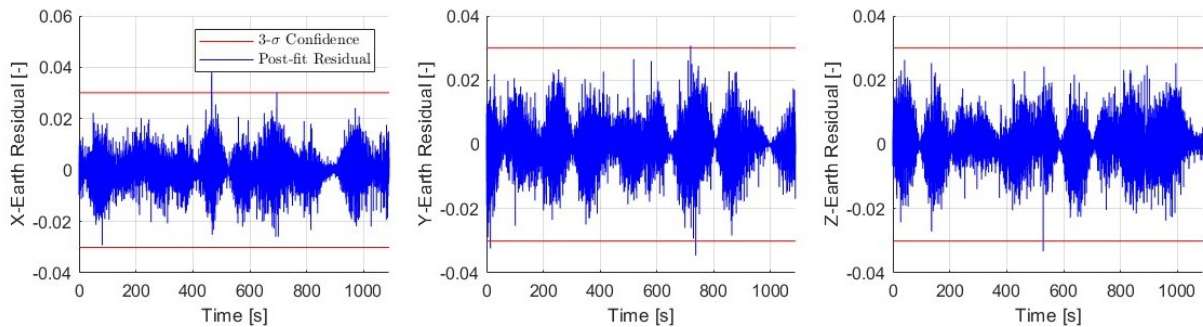


Figure 122: Post-fit residual of Earth direction $y_{earth} - \hat{y}_{earth}$.

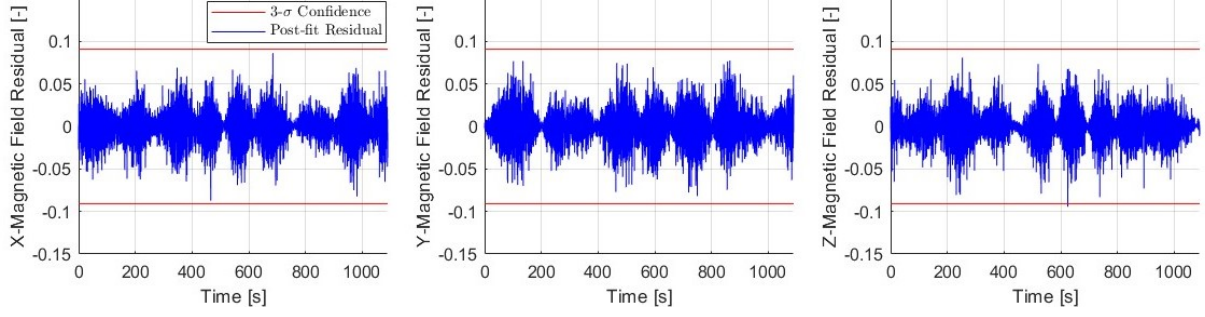


Figure 123: Post-fit residual of magnetic field direction $y_{\text{magnetic}} - \hat{y}_{\text{magnetic}}$.

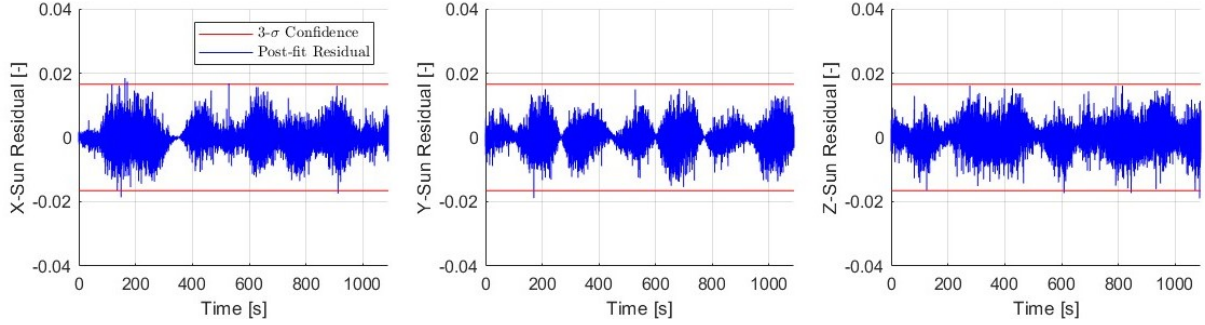


Figure 124: Post-fit residual of Sun direction $y_{\text{sun}} - \hat{y}_{\text{sun}}$.

The mean and standard deviation for these residuals are given in Table 15. From this table, it can be seen that the residuals are unbiased and that the defined measurement error covariance explains the variance in the residuals well.

Table 15: Summary of statistics of the post- fit residuals.

Measurement	Mean	Standard Deviation	
		Statistic	Measurement Covariance
Angular velocity	1.0519×10^{-6}	2.6248×10^{-5}	8.9507×10^{-4}
	-7.6252×10^{-7}	2.7131×10^{-5}	8.9507×10^{-4}
	1.8013×10^{-6}	1.8662×10^{-5}	8.9507×10^{-4}
Earth direction	-4.9842×10^{-5}	0.0054	0.0100
	-5.6158×10^{-5}	0.0057	0.0100
	1.7239×10^{-4}	0.0055	0.0100
Magnetic field direction	-2.2958×10^{-4}	0.0144	0.0302
	-1.0681×10^{-4}	0.0149	0.0302
	2.0384×10^{-5}	0.0137	0.0302
Sun direction	-8.9107×10^{-5}	0.0034	0.0055
	5.4664×10^{-5}	0.0032	0.0055
	-5.7129×10^{-6}	0.0037	0.0055

9.7 PROBLEM 7

With the measurement update step of the Kalman Filter algorithm implemented, its full functionality is now tested where the relevant parameters are defined in previous sections. Since the satellite does not yet have a controller implemented, the control input is zero.

The results for a simulation over a time span of about 1000 seconds with initial angular velocity of $\omega_0 = [1 \ 1 \ 1] \text{ [deg/s]}$ are given in Figure 125 and Figure 126. From the figure, it can be seen that the state error covariance remains bounded around the estimate and true attitude states over the complete time span. Furthermore, the estimated attitude closely approximates the true attitude.

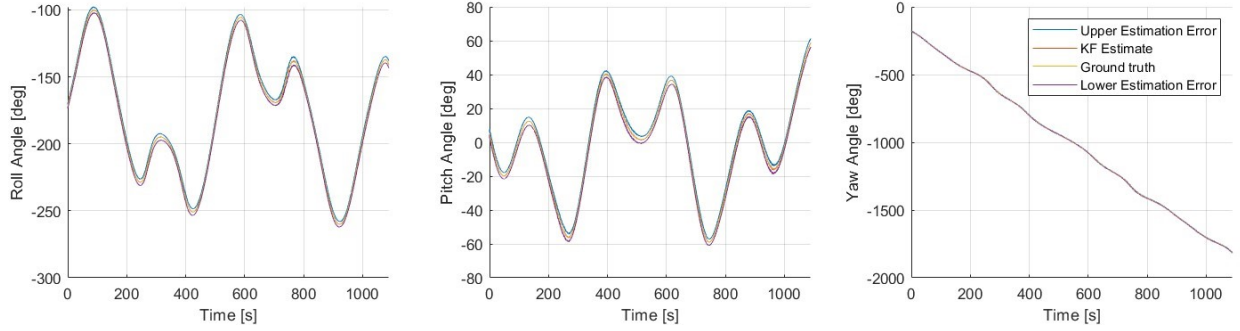


Figure 125: Attitude angles estimation using Kalman filter time update step. The upper and lower estimation limits are $1 - \sigma$ as obtained from the state error covariance.

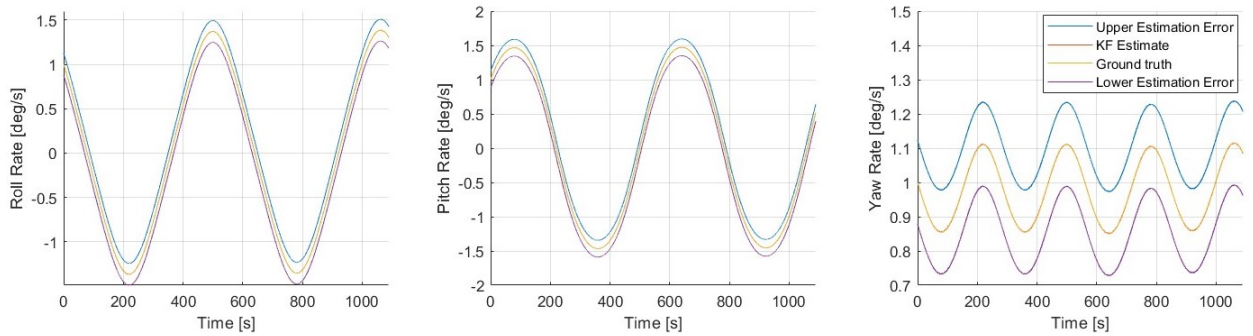


Figure 126: Angular velocity estimation using Kalman filter time update step. The upper and lower estimation limits are $1 - \sigma$ as obtained from the state error covariance.

The absolute value of the error between the true attitude and the estimated attitude along with the standard deviation obtained from the state error covariance matrix are given in Figure 127 and Figure 128 for the attitude angles and angular velocity, respectively. It can be seen that the estimated attitude angle estimation errors closely bound the true attitude angles estimation error. For the angular velocity, the estimated errors also bound the true errors, however rather conservatively.

The state estimation statistics are given in Table 16. From the table it can be seen that the mean error is close to zero for all estimated states. However, it is noticeable that the roll and yaw angles have a mean of two order of magnitude higher. Furthermore, the true standard deviation of the estimation error is an order of magnitude lower than the predicted one for the attitude angles, while it is two orders of magnitude lower for the angular velocity. So in general, to conclude, the Extended Kalman filter behaves as expected and is able to estimate the attitude of the satellite accurately.

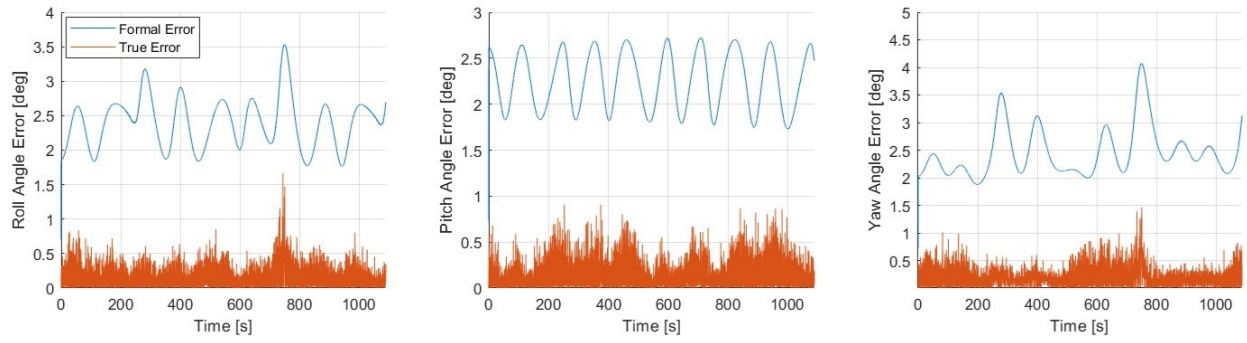


Figure 127: Attitude angles estimation using Kalman filter time update step. The upper and lower estimation limits are $1 - \sigma$ as obtained from the state error covariance.

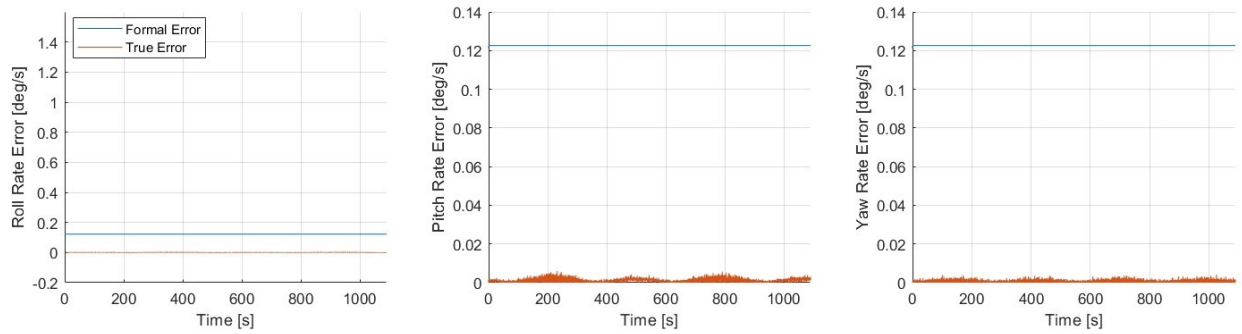


Figure 128: Angular velocity estimation using Kalman filter time update step. The upper and lower estimation limits are $1 - \sigma$ as obtained from the state error covariance.

Table 16: Summary of error statistics of the estimated attitude state.

Measurement	Mean	Standard Deviation	
		Statistic	Measurement Covariance
Attitude Angles [deg]	-0.0124	0.1953	2.4007
	-4.8634×10^{-4}	0.1861	2.2624
	0.1604	0.1817	2.4585
Angular Velocity [deg/s]	-6.0901×10^{-5}	0.0013	0.1226
	3.2944×10^{-5}	0.0014	0.1226
	-8.7616×10^{-5}	7.6768×10^{-4}	0.1226

9.8 PROBLEM 8

Since the Extended Kalman filter already has a good performance, for the final project, I would like to implement a more realistic measurement update rule in which certain sensor don't provide a measurement, such as the Sun sensor during eclipse, as well as consider different update rates depending on the sensor.

10 Actuator Sizing and Modeling

10.1 PROBLEM 1

According to mission specification, the actuators have two main objectives, namely to reject disturbances as to maintain a target attitude and perform a slew maneuver between nominal operational attitude and the calibration attitude. In order to size the actuators, the torque required for both these objectives is considered.

Firstly, the torque required for disturbance rejection is determined. For this, the worst-case anticipated disturbance torque M_d is calculated based on the maximum disturbance torques in subsection 6.2:

$$\begin{aligned} M_d &= M_{mag} + M_{sol} + M_{atm} + M_{grav} \\ &= 5.387 \times 10^{-4} + 2.119 \times 10^{-5} + 1.28 \times 10^{-2} + 2.7 \times 10^{-3} [Nm] \\ &= 1.606 \times 10^{-2} [Nm] \end{aligned}$$

Taking a margin of 50%, the torque requirement for disturbance rejection is $M_c = 2.4 \times 10^{-2} [Nm]$. Therefore, the actuators should be capable of delivering this torque.

Secondly, considering the torque requirement for the slew maneuver. Based on Figure 1, the satellite needs to slew $\theta = 35^\circ$ from its nominal line-of-sight to a nadir pointing attitude. The rotation is performed about its y -axis which has moments of inertia roughly $I_y = 1.5 \times 10^3 [kgm^2]$. Lastly, the slew maneuver may be performed over a time of 5 minutes, hence this results in a required torque of:

$$M_c = \frac{4I_y\theta}{t^2} = 4.07 \times 10^{-2} [Nm]$$

From this analysis, it results that the actuators need to be able to provide a torque of at least $M_c = 4.07 \times 10^{-2} [Nm]$. Additionally, the required momentum storage capabilities of the actuators need to be determined. For this, it is assumed that the worst-case disturbance torque accumulates over 1/4 orbit, after which momentum needs to be dumped. Then:

$$L = M_d P \frac{\sqrt{2}}{8} = 15.5 [NMs]$$

where $P = 5.447 \times 10^3 [s]$ is the orbital period. This concludes the full specification of the actuator requirements. Since the actuators are used for absorbing cyclic torques and temporarily storing momentum from the satellite during slew maneuvers, the choice of actuators is reaction wheels. Based on the requirements, the *RSI 45 Reaction Wheel* by Rockwell Collins [7] is selected. This reaction wheel is rated at an angular momentum at nominal speed of $45 [Nms]$ and is capable of delivering a torque of $7.5 \times 10^{-2} [Nm]$. The operational speed range is $\pm 6 \times 10^3 [rpm]$. The reaction wheel is $310 [mm]$ in diameter, $160 [mm]$ in height and has a mass of $7.7 [kg]$. The reaction wheel model implementation is given in Figure 179.

In addition to the reaction wheels, actuators are necessary which can desaturate the reaction wheels. Since the satellite requires thrusters for orbit maintenance and insertion after separation, these can also be used to desaturate the reaction wheels. Assuming a minimum moment arm of $b = 1m$ and a required momentum dumping time of 10 seconds, this results in a required thrust of:

$$F = \frac{L}{bt} = 1.55 [N]$$

Based on this requirement, the $5 [N]$ monopropellant hydrazine thruster by *Rafael Ltd.* [11] is selected. The thruster model implementation is given in Figure 180.

For the actuator configuration, three reaction wheels are necessary for 3-axis control and a fourth wheel is added for redundancy. One reaction wheels is placed along each body axes and the fourth is aligned with the trisctrix. This provides full control along the body axes with a redundant fourth wheel in case on of the primary ones break. For the thrusters a minimum of three is necessary to provide torque about each axes. However, additional thrusters may be required for orbit maintenance. Each thruster is assumed to provide a torque about a body axis with a moment arm of $1m$. The actuator model implementation is given in Figure 178.

10.2 PROBLEM 2

In order to simulate the control of the satellite, it is necessary to model the actuators. Starting with the reaction wheels, their angular momentum vector changes according to the following relation:

$$\dot{L}_w = I_w \dot{\omega}_w = M_w$$

where I_w is the moments of inertia about spin axis and $\dot{\omega}$ is the angular acceleration about spin axis. The applied torque is M_w . The torque is provided by an AC two-phase induction motor, resulting in a change of angular momentum of the reaction wheel $\dot{L}_w$. The collection of reactions wheels results an angular momentum vector L_w , which is given as follows:

$$L_w = A \begin{bmatrix} L_{w^1} \\ L_{w^2} \\ L_{w^3} \\ L_{w^4} \end{bmatrix}$$

where A contains the configuration of the reactions wheels and a coordinate change from body axes to prinicpal axes. Then the control torque for the reactions wheels is defined as follows:

$$\dot{L}_w = A \begin{bmatrix} I_w \dot{\omega}_{w^1} \\ I_w \dot{\omega}_{w^2} \\ I_w \dot{\omega}_{w^3} \\ I_w \dot{\omega}_{w^4} \end{bmatrix} = M_c$$

From a desired control torque M_c , the torque which needs to be supplied to each reaction wheel can be determine as follows:

$$\begin{bmatrix} M_{w^1} \\ M_{w^2} \\ M_{w^3} \\ M_{w^4} \end{bmatrix} = A^* M_c$$

The resulting effect on the satellite of supplying these torques through motors to the reactions wheels is derived from the Euler equations:

$$\dot{L}_t = \dot{L}_s + \dot{L}_w = I\dot{\omega} + \omega \times I\omega + \dot{L}_w = M_{dist}$$

where M_{dist} is the disturbance torque. This model allows for incorporating systematic and random errors into the torques supplied to the reaction wheels, as well as modelling the saturation, which is when the reaction wheels achieve a certain angular speed or maximum torque. The saturation is modelled by simply setting the torque supplied to the reaction wheel to zero when it hits the saturation limit. For a simple error model, random noise is considered on the torque applied to the reaction wheel. The noise is Gaussian with bias of zero and standard deviation of $1 \times 10^{-4}[Nm]$.

Figure 129 shows the commanded and actual control torque. As can be seen, the actual control torque follows the commanded control torque, but is noisy. Figure 130 shows the corresponding angular velocities of the four reaction wheels. As can be seen, they are far below saturation limit.

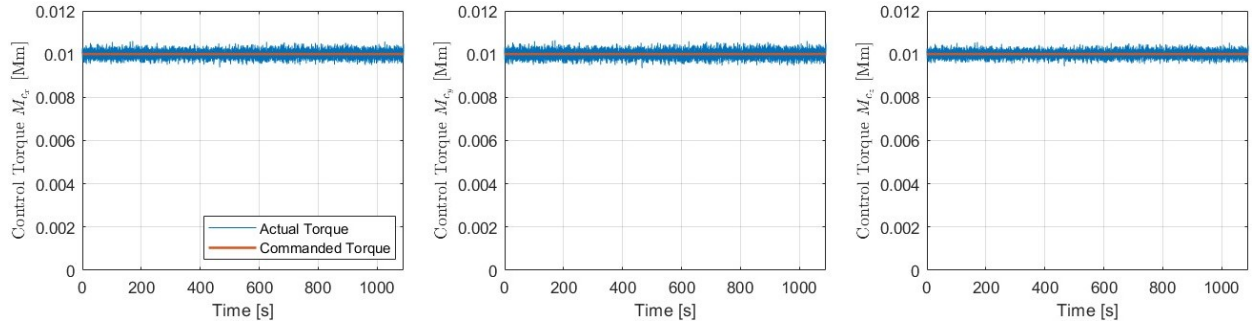


Figure 129: Commanded control torque (orange) and actual achieved torque by the reaction wheels (blue).

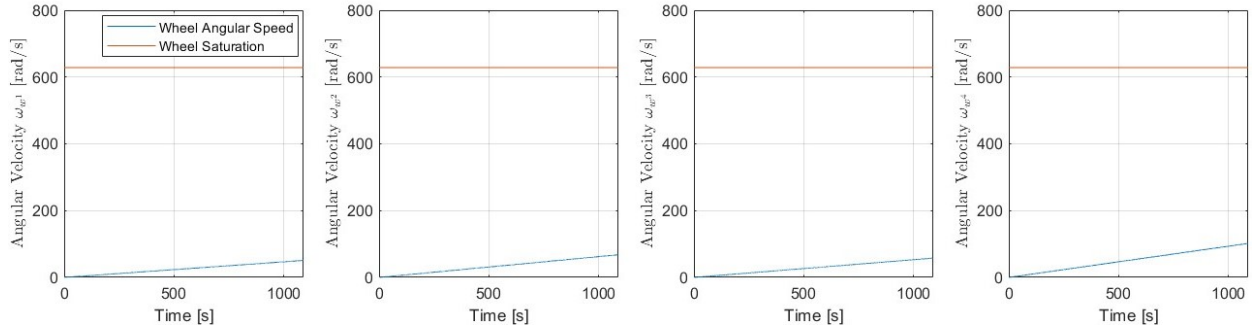


Figure 130: Angular speed of the four reaction wheels (blue) and maximum saturation angular speed (orange).

Another example of the reaction wheel model is given in Figure 131. In this case the commanded control torque is $M_c = [0.1 \ 0.1 \ 0.1]^T [Nm]$ (a torque which is expected to never be encountered in orbit). Figure 132 and Figure 133 show the angular velocity and motor torque for each reaction wheel, respectively. It is clear that the actual torque supplied by the reaction wheels does not match the commanded torque, regardless of noise. This is caused by the two saturation limits of the reaction wheels. Firstly, Figure 133 shows that the fourth reaction wheel cannot supply more torque which results in the offset at the beginning. Towards the end, the torque supplied by the reaction wheels drops due to saturated angular momentum of the reaction wheels, which can be seen in Figure 132. The last three reaction wheels reach their maximum angular speed.

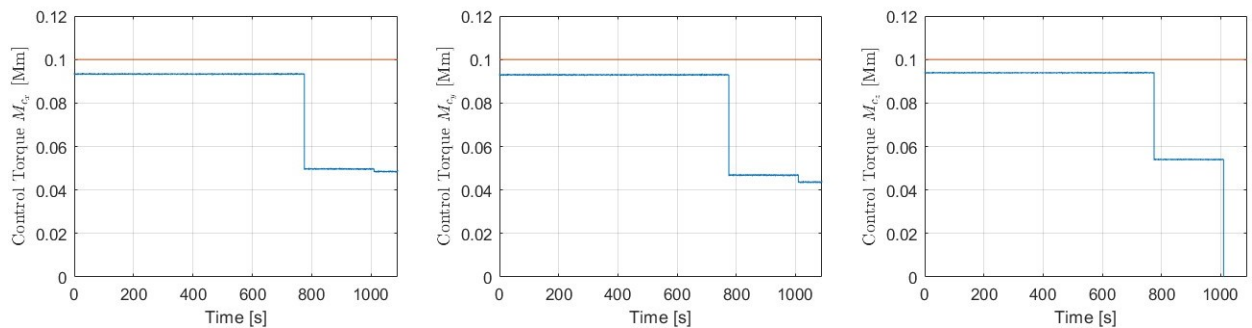


Figure 131: Commanded control torque (orange) and actual achieved torque by the reaction wheels (blue).

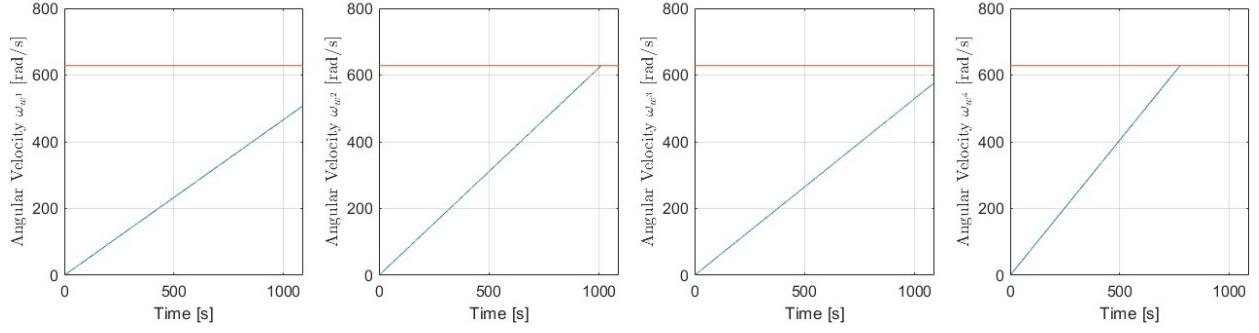


Figure 132: Angular speed of the four reaction wheels (blue) and maximum saturation angular speed (orange).

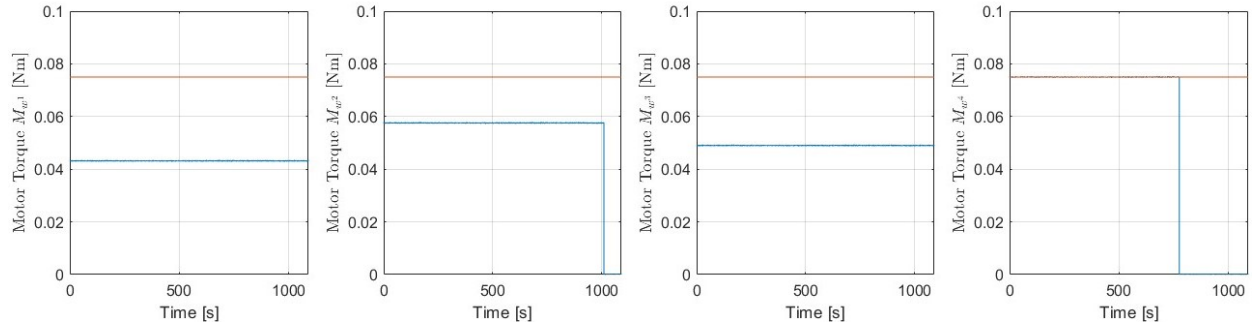


Figure 133: Motor torque of the four reaction wheels (blue) and maximum saturation torque (orange).

Next, the thrusters are modelled. The concept behind thrusters is simple, they produce force on the satellite by expelling material, at high velocity from their exit nozzle. If the moment arm between the exit nozzle and spacecraft center of mass is non-zero, then this thrust produces a torque. Therefore, given the thruster configuration, the torque exerted on the spacecraft by the thrusters is given by:

$$M_{thruster} = R_b^p \begin{bmatrix} lF_1 \\ lF_2 \\ lF_3 \end{bmatrix}$$

Hence given a commanded control torque, the thrust for each thruster can simply be calculated as:

$$\begin{bmatrix} F_1 \\ F_2 \\ F_3 \end{bmatrix} = R_p^b \frac{M_c}{b}$$

A noise term is added to the thrust to model random errors and the saturation limit is included to model thrust limits. The noise is taken to be Gaussian with bias of zero and standard deviation of $1 \times 10^{-4} [N]$. Figure 134 shows the achieved torque by the thrusters when the commanded control torque is $M_c = [5.0 \ 5.0 \ 5.0]^T [Nm]$ (again a torque which is not expected to be encountered in orbit). It can be seen that the thrusters are able to achieve this torque in x - and y -direction, but not in z -direction due to a thruster reaching its maximum capable thrust, as shown in Figure 135.

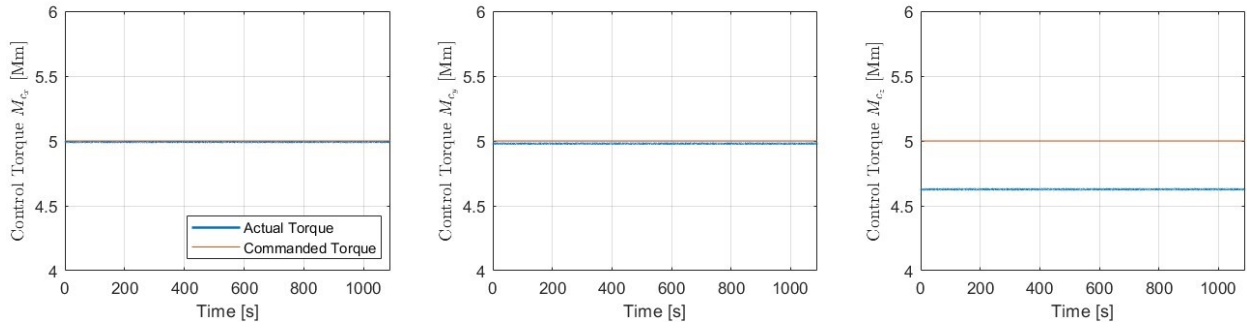


Figure 134: Commanded control torque (orange) and actual achieved torque by the reaction wheels (blue).

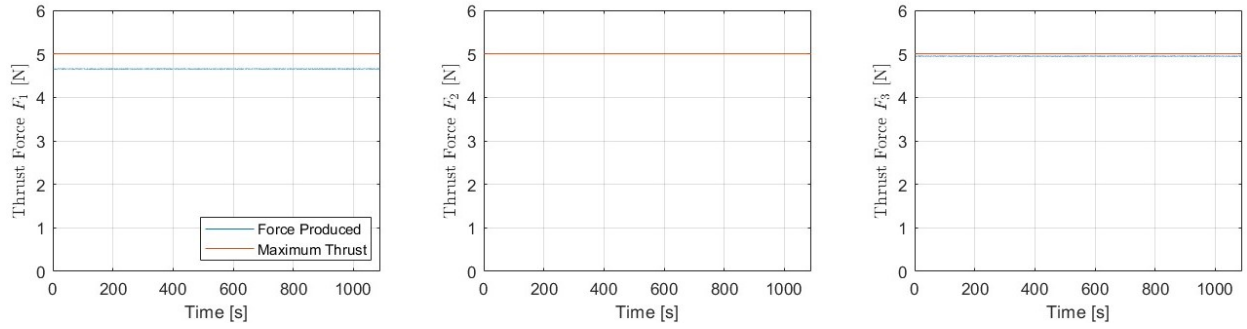


Figure 135: Angular speed of the four reaction wheels (blue) and maximum saturation angular speed (orange).

10.3 PROBLEM 3

The purpose of the ADCS is to stabilize the satellite around a desired nominal attitude. This comes down to counter-acting the disturbance torques experienced by the satellite. For this, the reaction wheels trade-off angular momentum such that all disturbance torques are absorbed over time. To verify that the actuators are capable over counter-acting the disturbances, the ground-truth disturbances are fed into the actuator module as control torque. The satellite is simulated over one orbital period from its nominal attitude and angular velocity equal to zero. This results in an attitude error as shown in Figure 136. As can be seen, the error is contained to within $0.15[deg]$ over one orbit. This indicates that the actuators are capable of counteracting the disturbance torques.

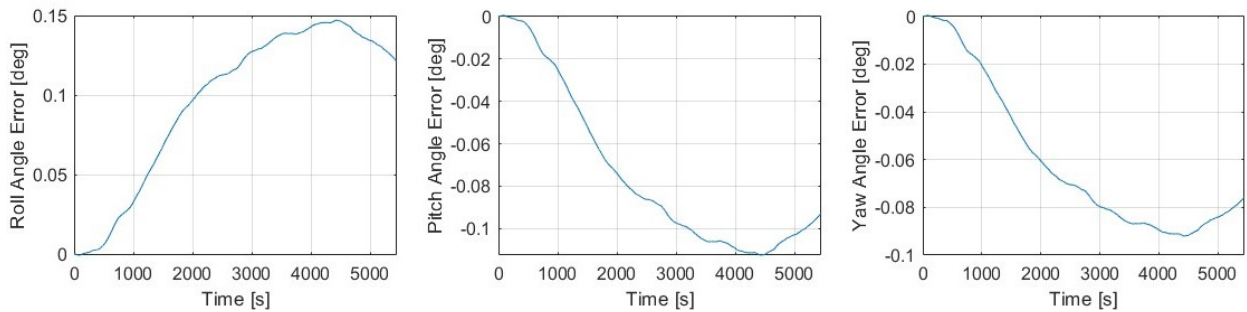


Figure 136: Attitude error when ground-truth disturbance torques are fed into actuator module as control torques.

To maintain attitude without access to ground-truth disturbance torques, a control law is required which determines the control torque based on the attitude tracking error. The dynamics are linearized using the first-order Taylor series approximation:

$$\begin{aligned}\dot{x} &\approx \frac{f}{\partial x}(x_0, u_0)(x - x_0) + \frac{\partial f}{\partial u}(x_0, u_0)(u - u_0) \\ &= \frac{f}{\partial x}(x_0, u_0)\Delta x + \frac{\partial f}{\partial u}(x_0, u_0)\Delta u \\ &= A\Delta x + B\Delta u\end{aligned}$$

where the dynamics consist of the attitude kinematics and Euler equations. These are linearized about the state $x_0 = [\eta_0 \ \omega_0]^T$ and $u_0 = [0 \ 0 \ 0]^T$, i.e. the nominal attitude and zero angular velocity. The matrices A and B are obtained using the *Time-Based Linearization* in *Simulink*. This result in the following matrices:

$$A = \begin{bmatrix} 0.0 & 0.0 & 0.0 & 1.0 & -0.0137 & -0.0914 \\ 0.0 & 0.0 & 0.0 & 0.0 & -0.9889 & 0.1483 \\ 0.0 & 0.0 & 0.0 & 0.0 & -0.1490 & -0.9932 \\ 0.0 & 0.0 & 0.0 & 0.0 & 0.0 & 0.0 \\ 0.0 & 0.0 & 0.0 & 0.0 & 0.0 & 0.0 \\ 0.0 & 0.0 & 0.0 & 0.0 & 0.0 & 0.0 \end{bmatrix} \quad B = \begin{bmatrix} 0.0 & 0.0 & 0.0 \\ 0.0 & 0.0 & 0.0 \\ 0.0 & 0.0 & 0.0 \\ -6.95 \times 10^{-4} & 0.0 & 0.0 \\ 0.0 & -5.28 \times 10^{-4} & 0.0 \\ 0.0 & 0.0 & -3.61 \times 10^{-4} \end{bmatrix}$$

In the state matrix A , the structure of the attitude kinematics, given in Equation 17 can be recognized, while in the input matrix B , the relation $M = I\omega$ can be recognized. The feedback control gain is designed based on the LQR control law for the system (A, B) . This control law regulates the satellite's state to the state about which the equations are linearized, i.e. $\Delta x = 0$. For the LQR weight matrices, they were designed such that the attitude converges to the nominal attitude and the actuators are not saturated. This following weight matrices are selected:

$$Q = \text{diag}([1 \times 10^2, 1 \times 10^2, 1 \times 10^2, 1 \times 10^{-1}, 1 \times 10^{-1}, 1 \times 10^{-1}]) \quad R = 1\mathbb{I}_3$$

Solving the algebraic Riccati equation for above specified LQR problem results in the optimal gain $K \in \mathbb{R}^{3 \times 6}$. The closed-loop system then becomes:

$$\dot{x} \approx (A - BK)\Delta x$$

The pol-zero map for this closed-loop system is given in Figure 137. It can be seen that all eigenvalues have negative real part, which indicates stable behaviour for all states. Additionally, Table 17 gives the characteristics of the zeros. From these, it can be seen that a desirable damping of ~ 0.7 is achieved. Furthermore, the mean motion of the satellite is $n = 1.2 \times 10^{-3}[\text{rad/s}]$, hence the frequencies are roughly $70\times$ faster, which is to ensure fast convergence to the linearized states as to keep them within the linear region. The implementation of the control law is given in Figure 190.

Table 17: Characteristics of the poles of the closed-loop linearized dynamics

Pole Location	Damping [–]	Frequency [rad/s]	Overshoot [%]
$-0.059 \pm 0.059i$	0.707	0.0834	4.32
$-0.0514 \pm 0.0514i$	0.707	0.0727	4.32
$-0.0425 \pm 0.0425i$	0.707	0.0602	4.32

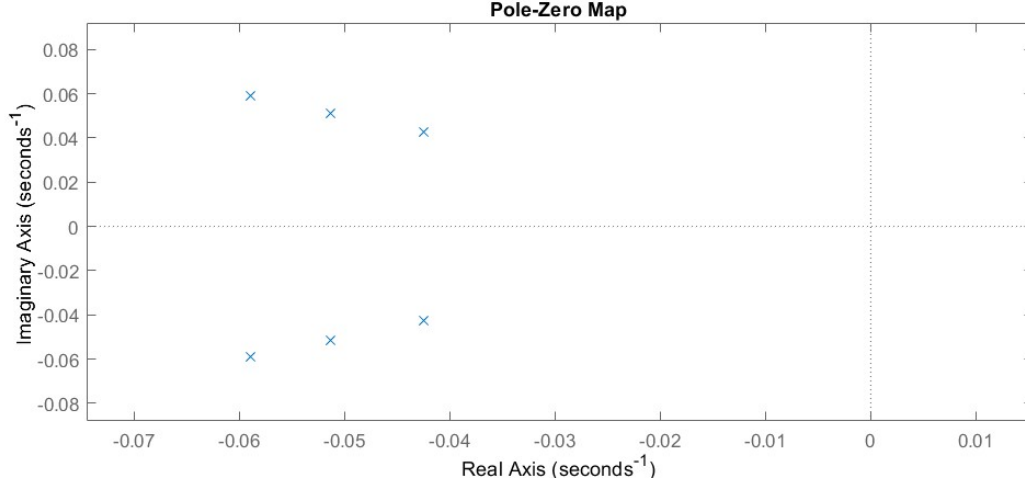


Figure 137: Pole-zero map for closed-loop linearized dynamics.

10.4 PROBLEM 4

To test the control law implementation in closed loop, the actuators, sensors and on-board computer are integrated with each other piece by piece according to following process:

- Closed-loop control with ground-truth state knowledge and perfect actuators.
- Closed-loop control with estimated state and perfect actuators.
- Closed-loop control with estimated state and actuator models.

At each stage, properties such as the state estimation errors, attitude control errors and control actions are evaluated. The satellite is simulated over one orbital period from its nominal attitude.

Stage 1 This stage verifies the functionality of the control law and its capability to regulate the satellite to its nominal attitude under the disturbance torques. Furthermore, it also allows for tuning of the weight matrices of the LQR control law. The control error, i.e. the error between desired attitude and actual attitude is given in Figure 138. As can be seen, the control law successfully maintains the attitude to within an error of $6 \times 10^{-2}[\text{deg}]$. The attitude estimation error for this stage is given in Figure 139. The state estimation is accurate to within $1.5[\text{deg}]$. This should be sufficiently small to achieve attitude regulation based on this estimated state. However, it is expected that regulation performance can only be as accurate as the state estimation. Lastly, Figure 140 shows the commanded torque, which in this case is immediately fed into the Euler equations, and the torque generated by the actuators. It can be seen that they are capable of producing the commanded torque.

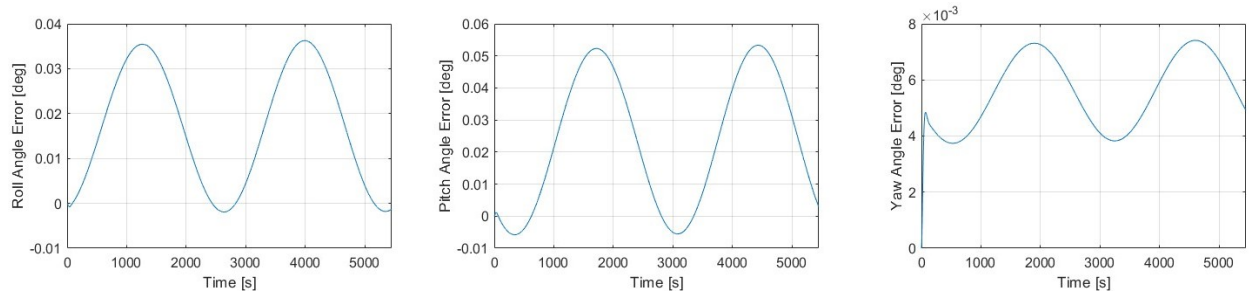


Figure 138: Stage 1 attitude control error, indicating the error between desired and true attitude.

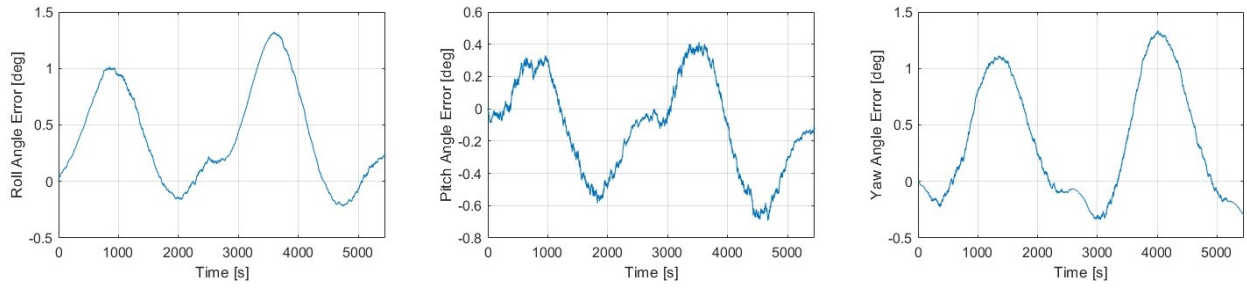


Figure 139: Stage 1 state estimation error, indicating the error between true attitude and estimate attitude.

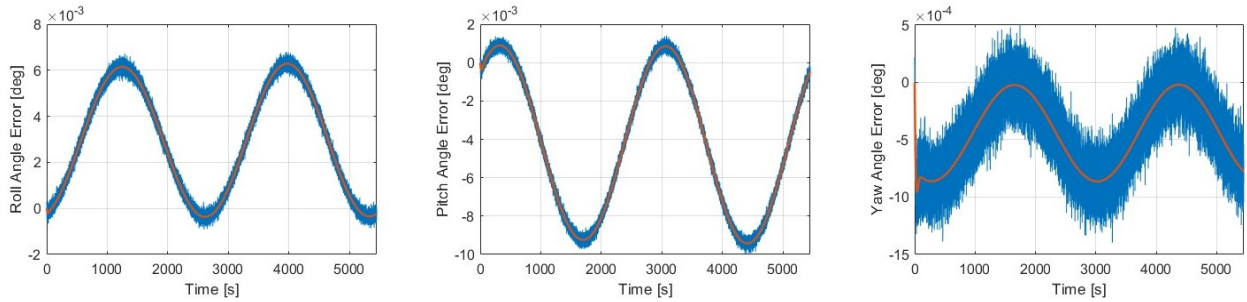


Figure 140: Stage 1 commanded control torque (orange) and realized control torque by actuators (blue).

Stage 2 This stage verifies the functionality of the control law in conjunction with the state estimation. It essentially tests the capability of the control law to regulate the attitude to the desired attitude despite the state estimation error. The control error is given in Figure 141. As can be seen, the control law successfully maintains the attitude to a maximum error of roughly $1.5[deg]$. This error is comparable to the attitude estimation error, which indicates that this estimation error does not cause the attitude to drift too far away from the nominal attitude which would cause the linear approximation to become invalid. The attitude estimation error for this stage is given in Figure 142. Again, state estimation is accurate to within $1.5[deg]$. Lastly, Figure 143 shows the commanded torque, where it can be seen that the commanded control torque has become noisy. This is due to the slightly noisy state estimation. Since the control law is a simple linear feedback gain, this noise is amplified. This is the reason it is not desirable for the frequency of the poles of the linear closed-loop system to be too high; a high frequency results in more aggressive control, which results in higher feedback gain, causing estimation noise to be amplified. However, for the current gain, the noise seems to match the inherent noise of the actuators. Therefore, it is expected that when including the actuator model (Stage 3), the control law will remain effective.

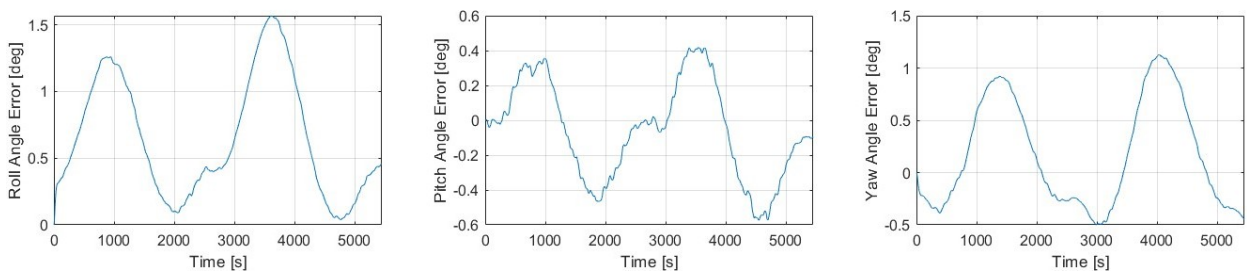


Figure 141: Stage 2 attitude control error, indicating the error between desired and true attitude.

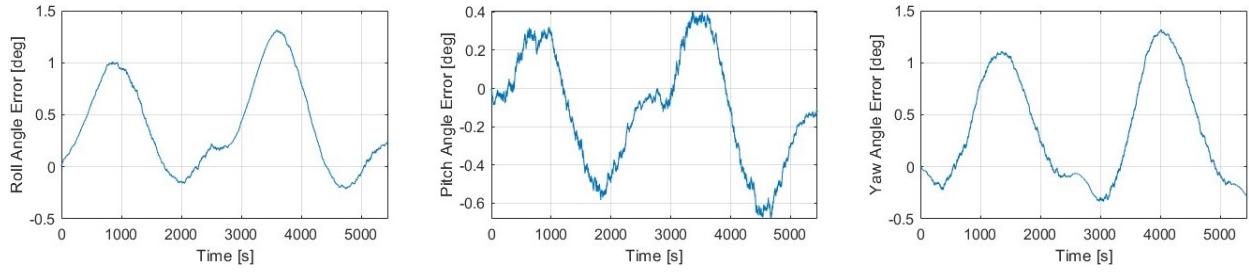


Figure 142: Stage 2 state estimation error, indicating the error between true attitude and estimate attitude.

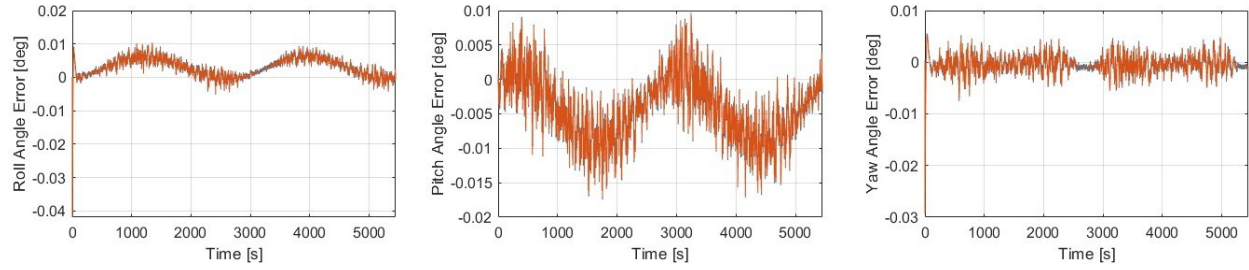


Figure 143: Stage 2 commanded control torque and realized control torque by actuators.

Stage 3 This stage integrates all components. The on-board computer, i.e. state estimation and control law, only has access to the measurements and the control torque is fed through actuator models. The control error, attitude estimation error and control torques are shown in Figure 149, Figure 150, Figure 151, respectively. These are remain virtually the same, which indicates that the actuator noise has no major effect on the regulation performance.

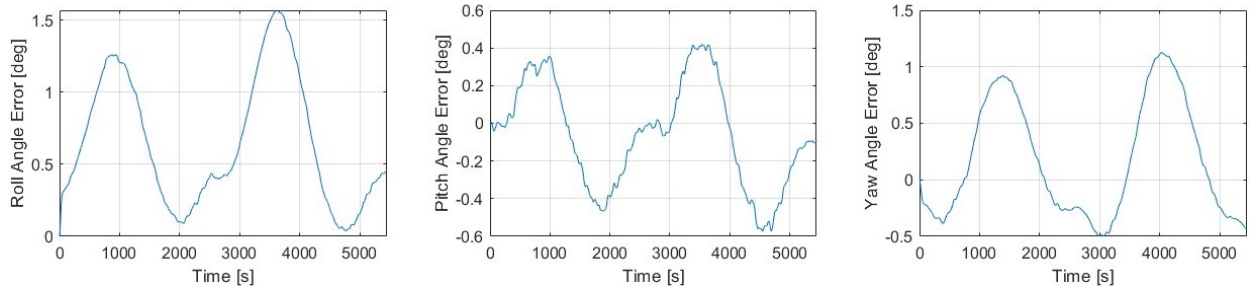


Figure 144: Stage 3 attitude control error, indicating the error between desired and true attitude.

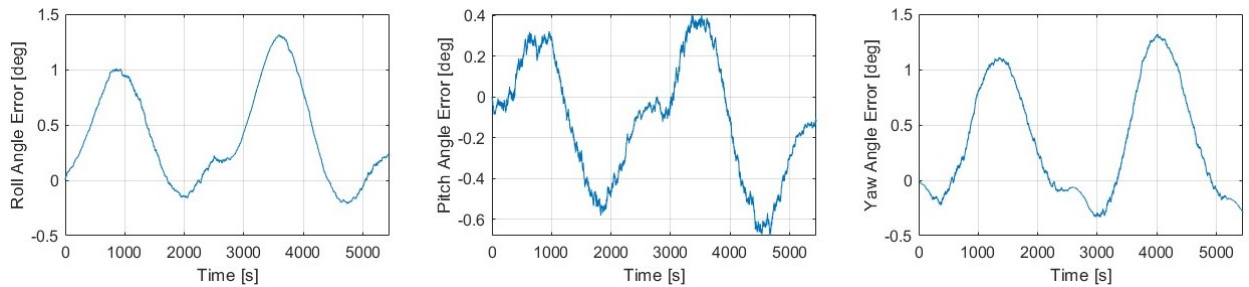


Figure 145: Stage 3 state estimation error, indicating the error between true attitude and estimate attitude.

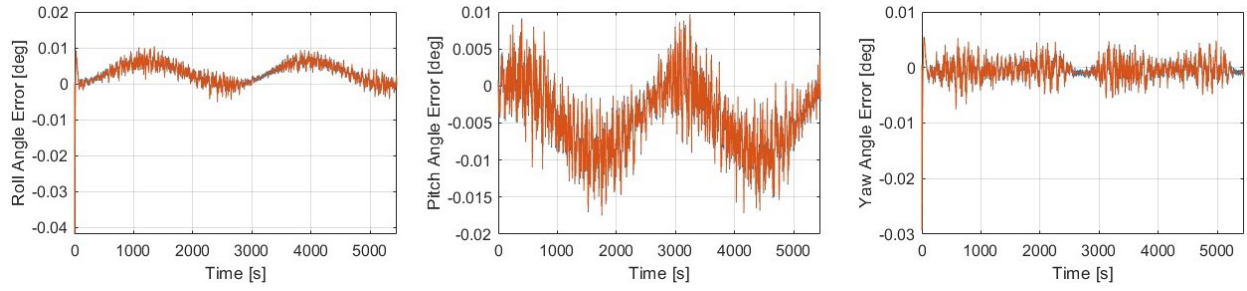


Figure 146: Stage 3 commanded control torque and realized control torque by actuators.

This finalizes the attitude regulation problem with a closed loop 3-axis control system. The implementation of all ADCS modules integrated is given in Figure 169. For the actuation, only the reaction wheels are used since the thruster are used for desaturating the reaction wheels. The angular speed of each reaction wheel and the motor torque supplied to the reaction wheels are shown in Figure 130 and Figure 153, respectively. From these, it can be seen that the angular velocity remains far away from saturation limit and motor torque remains below the maximum specified. However, while desaturation may not be strictly required yet, an implementation for desaturation using thrusters will be completed in next steps. Furthermore, the current control law can only regulate the state to an equilibrium and relies on small angles, hence cannot be used for the slew maneuver. Therefore, in next steps, a nonlinear control law is implemented¹.

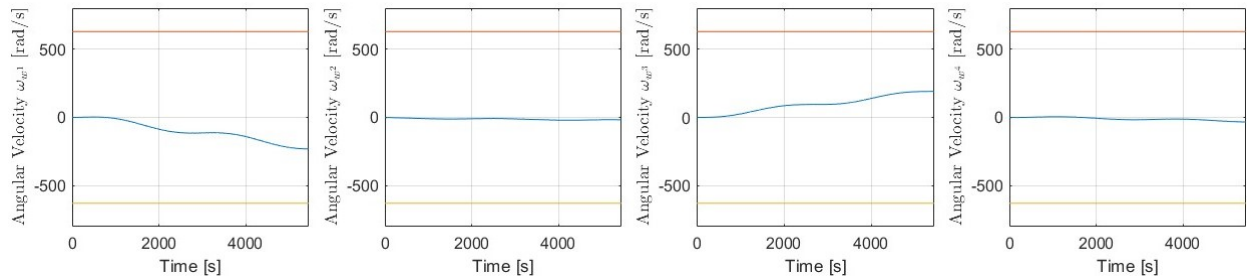


Figure 147: Angular speed of the four reaction wheels (blue) and maximum saturation angular speed (orange).

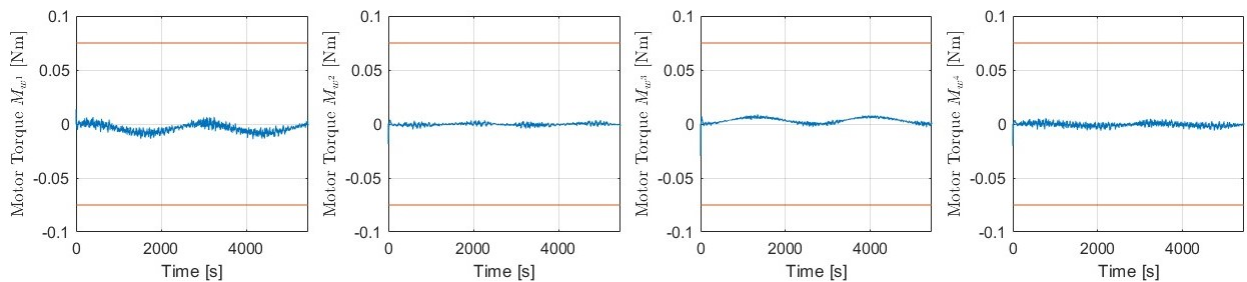


Figure 148: Motor torque of the four reaction wheels (blue) and maximum saturation torque (orange).

¹Next steps being the final project

11 Attitude Control Algorithm

11.1 PROBLEM 1/2

The coupling between actuators, control law and estimation was completed in previous section. Results are repeated. In this setting, the on-board computer, i.e. state estimation and control law, only has access to the measurements and the control torque is fed through actuator models. The control error, attitude estimation error and control torques are shown in Figure 149, Figure 150, Figure 151, respectively. From these figures, it can be seen that the attitude determination and control subsystem successfully manages to estimate the satellite's attitude up to an accuracy of $1.5[deg]$ in Euler angles, and stabilizes the satellite around its nominal attitude with an accuracy of less than $1.5[deg]$ in Euler angles. Moreover, the actuators are able to provide the required control action for an entire orbital period.

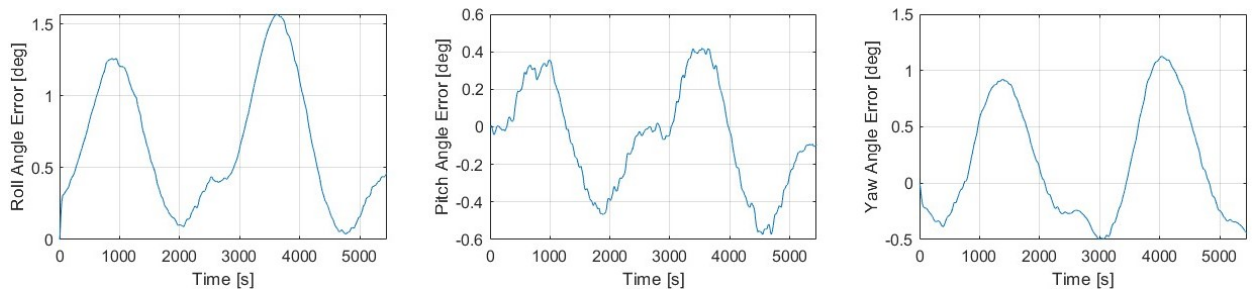


Figure 149: Attitude control error, indicating the error between desired and true attitude.

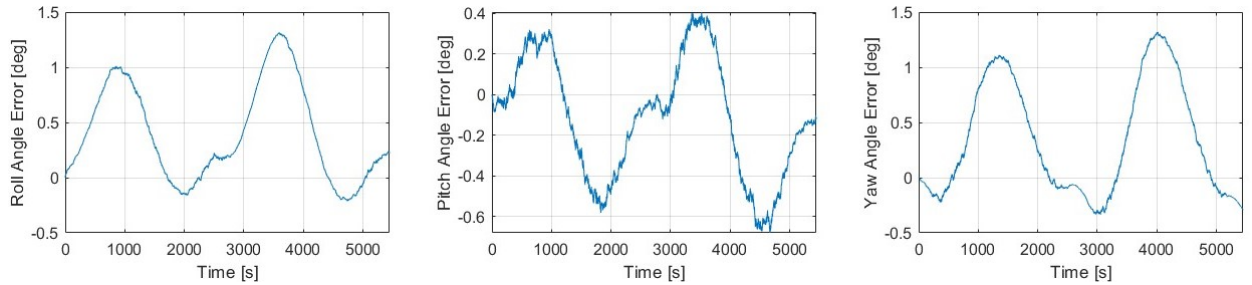


Figure 150: State estimation error, indicating the error between true attitude and estimate attitude.

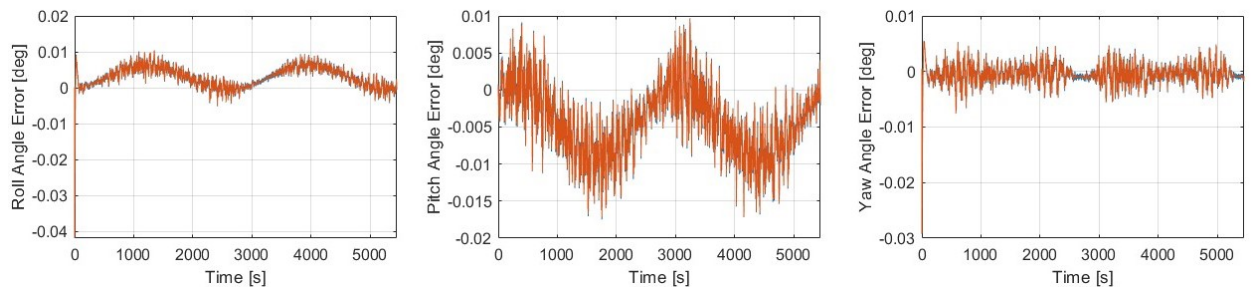


Figure 151: Stage 3 commanded control torque and realized control torque by actuators.

This finalizes the attitude regulation problem with a closed loop 3-axis control system. The implementation of all ADCS modules integrated is given in Figure 169. For the actuation, only the reaction wheels are used since the thruster are used for desaturating the reaction wheels. The angular speed of each reaction wheel and the motor torque supplied to the reaction wheels are shown in Figure 130 and Figure 153, respectively. From these, it can be seen that the angular velocity remains far away from saturation limit and motor torque remains below the maximum specified.

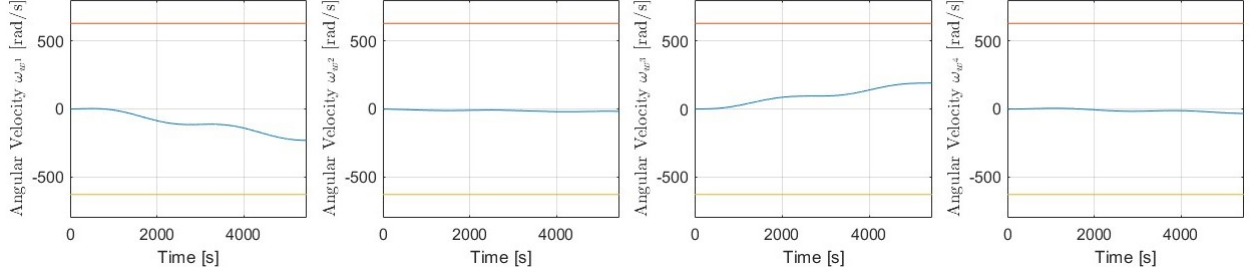


Figure 152: Angular speed of the four reaction wheels (blue) and maximum saturation angular speed (orange).

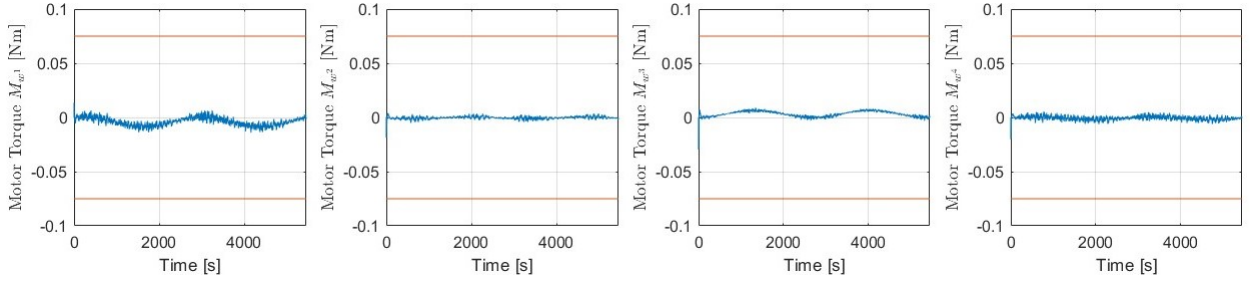


Figure 153: Motor torque of the four reaction wheels (blue) and maximum saturation torque (orange).

11.2 PROBLEM 3

In order to improve the accuracy of the implemented models, two adaptations are made to the sensors and state estimation process. The two considerations made for each sensor are the measurement availability and the update rate.

Firstly, the Sun sensor is only able to give sensor readings when the satellite is in eclipse. Furthermore, according to the datasheet of the Sun sensor [4], the update rate is $2Hz$. Hence, the state estimation is updated with Sun sensor measurements at a rate of $2Hz$, except while in eclipse. In case of the magnetometer, the satellite remains within Earth's magnetic field throughout its orbit, hence magnetometer measurements remain available. The state estimate is updated at a rate of $5Hz$ with magnetometer measurements [3]. For the Earth sensor, measurements are also available throughout the satellite's orbit with an update rate of $5Hz$ [12]. Lastly, the gyroscope provide angular velocity measurements at a data rate of $200Hz$, which are also available throughout the orbit. This is significantly higher than the other sensors, hence these serve as intermediate relative attitude estimation updates until the absolute attitude measurements are available. This is summarized in Table 18.

Table 18: Sensor availability and update rates.

Sensor	Availability	Update Rate [Hz]
Sun sensor	Not in eclipse	2
Magnetometer	Any time	5
Earth sensor	Any time	5
Gyroscope	Any time	200

11.3 PROBLEM 4

The satellite is required to perform attitude slew maneuvers between its nominal operational attitude and its calibration attitude. Moreover, both these attitude modes have time-varying trajectories due to their dependence on the RTN -frame. A 2D-view, looking from the normal direction, of the RTN -frame is shown in Figure 154. It can be seen the the RTN -frame completes one rotation about the normal axis over one orbital period.

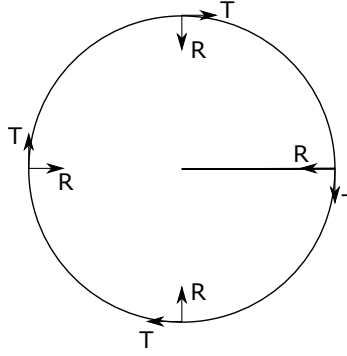


Figure 154: Evolution of RTN -frame over one orbit.

The calibration attitude is such that body z -axis is pointing in nadir direction, hence is aligned with the radial direction of the RTN -frame. Additionally, the body x -axis is aligned with normal direction, while body y -axis is aligned with transverse direction. Furthermore, the nominal attitude is such that the satellite is rotated by 35° about the transverse axis. This results in the nominal and calibration attitude trajectories of the body axes frame as shown in Figure 155. It can be seen that the calibration attitude involves one full rotation (from -90° to 270°) around the body x -axis, which is in line with the RTN -frame. The nominal attitude largely follows the full rotation about normal direction, but to maintain a 35° angle between nadir and body x -axis, there is an exchange between varying pitch and yaw angles.

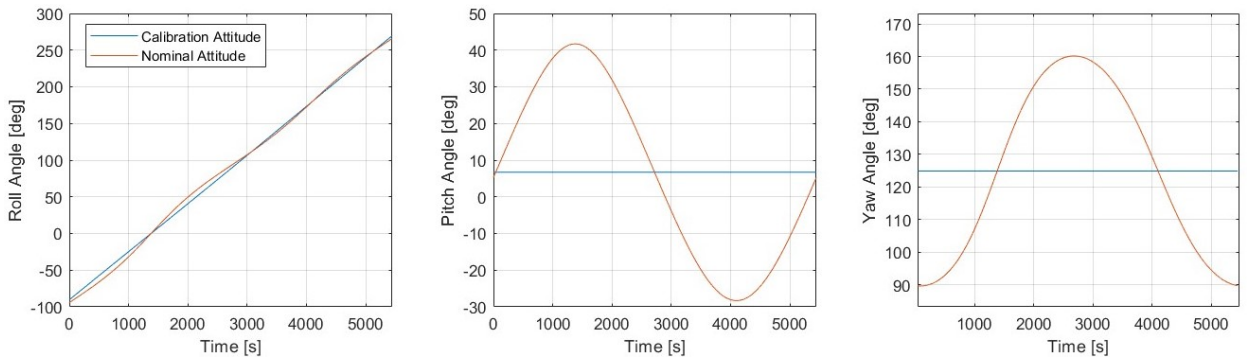


Figure 155: Body axes attitude trajectories for nominal and calibration mode.

The nominal and calibration attitude trajectories for the principal axes frame are shown in Figure 156. These are the trajectories which the control law needs to track, which can be seen to be highly varying over large angles. Therefore, a nonlinear tracking controller is required which is capable of tracking these attitude trajectories. Furthermore, it should also be able to switch between these two trajectories.

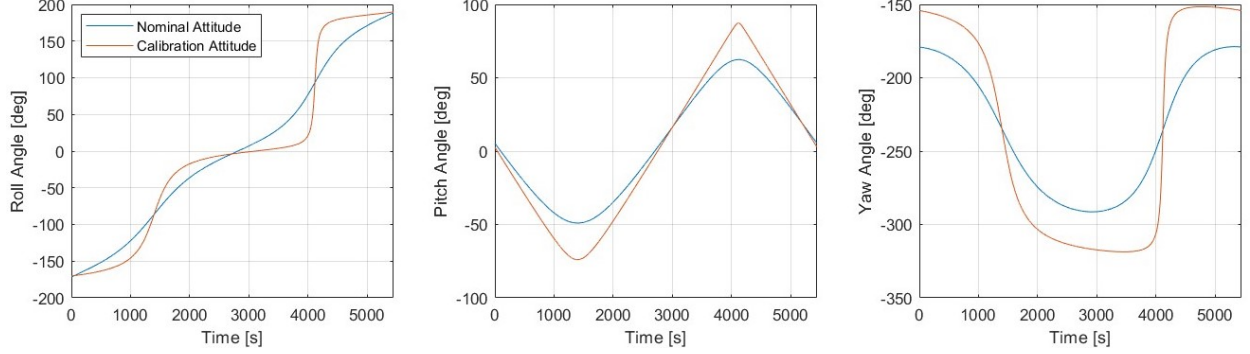


Figure 156: Principal axes attitude trajectories for nominal and calibration mode.

The currently implemented control law is not capable of tracking attitude reference trajectories. Therefore, another control law is implemented which is capable of performing attitude reference tracking. After performing some literature review, the control technique Incremental Nonlinear Dynamic Inversion [2] was found to be a viable option. This method relies on inverting the dynamics to obtain the control input. To control the attitude, the attitude kinematics can be inverted which results in a required angular velocity:

$$\omega_{cm} = \begin{bmatrix} 1 & 0 & -\sin(\theta) \\ 0 & \cos(\phi) & \cos(\theta)\sin(\phi) \\ 0 & -\sin(\theta) & \cos(\theta)\cos(\phi) \end{bmatrix} \begin{bmatrix} \dot{\phi} \\ \dot{\theta} \\ \dot{\psi} \end{bmatrix} \quad (34)$$

which is a first-order system in the attitude angles. The attitude angles can be controlled through closing the loop with a feedback gain; $\dot{\eta} = K_p(\eta_{ref} - \eta)$, where η represents the attitude angles. The required angular velocity ω_{cm} can be realized through the attitude dynamics:

$$I\dot{\omega} = M_c \quad (35)$$

which is a first-order system in angular velocity. To control angular velocity, the loop can be closed with a feedback gain: $\dot{\omega} = K_p(\omega_{cm} - \omega)$. This results in the control law given in Figure 157.

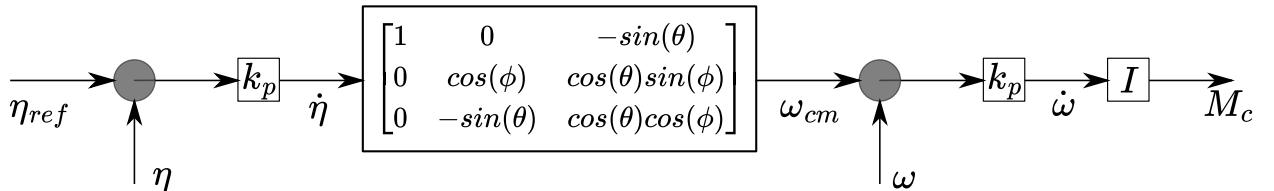


Figure 157: Attitude control law diagram.

where η and ω are the current estimated attitude angles and angular velocity. The gains were manually tuned to be 0.1 for the attitude angle control and 0.5 for the angular velocity control. The control law performance is analysed below and its implementation is given in Figure 191.

Nominal attitude reference tracking

Figure 158 shows the tracking of the nominal attitude trajectories. It can be seen that the controller manages to track the attitude trajectories. Additionally, the control error is shown in Figure 159, which indicates a maximum control error of $1.5[\text{deg}]$ across the Euler angles. Lastly, the attitude estimation error in tracking the nominal attitude trajectory is shown in Figure 160. This shows a maximum estimation error of $1.0[\text{deg}]$ across the Euler angles.

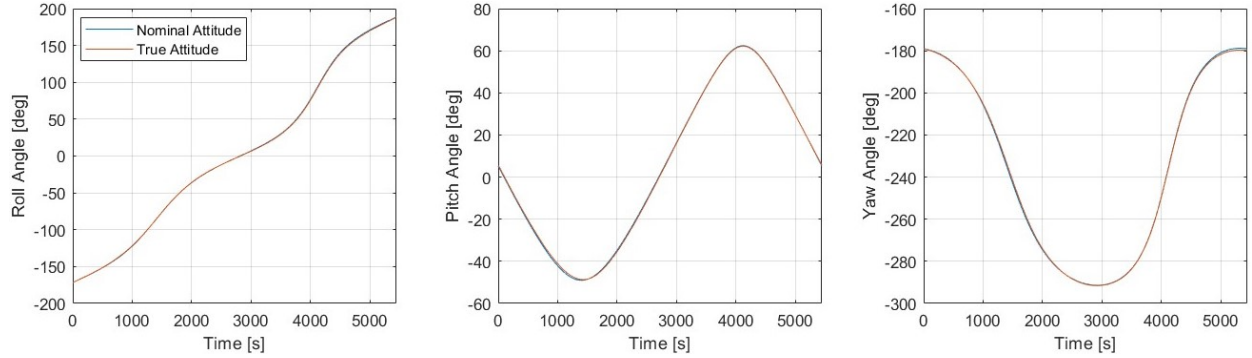


Figure 158: Attitude trajectory tracking for nominal attitude mode.

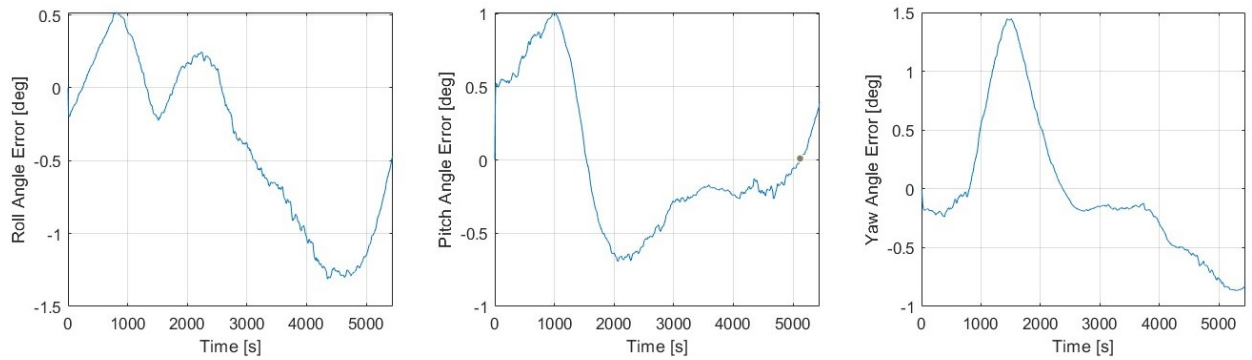


Figure 159: Control error in tracking the nominal attitude trajectories.

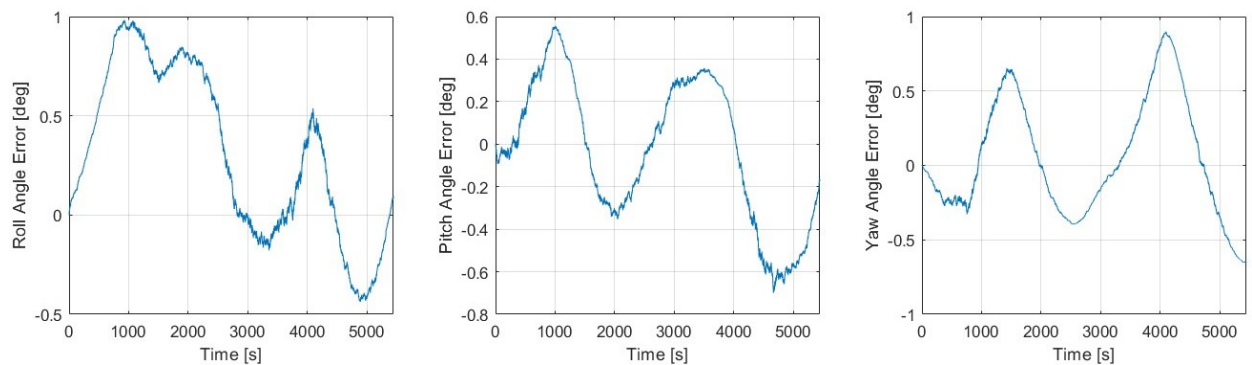


Figure 160: Estimation error in tracking the nominal attitude trajectories.

Calibration attitude reference tracking

Figure 161 shows the tracking of the calibration attitude trajectories. It can be seen that the controller also manages to track these attitude trajectories. Additionally, the control error is shown in Figure 162, which indicates a maximum control error of $12[deg]$ across the Euler angles at rapidly changing points in the trajectory, where otherwise the error is contained to $2[deg]$. Lastly, the attitude estimation error in tracking the nominal attitude trajectory is shown in Figure 163. This shows a maximum estimation error of around $3.0[deg]$ across the Euler angles.

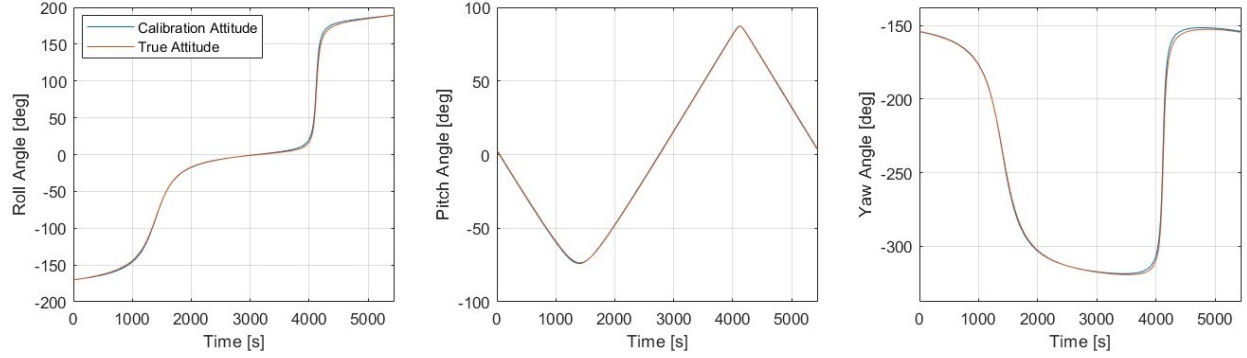


Figure 161: Attitude trajectory tracking for calibration attitude mode.

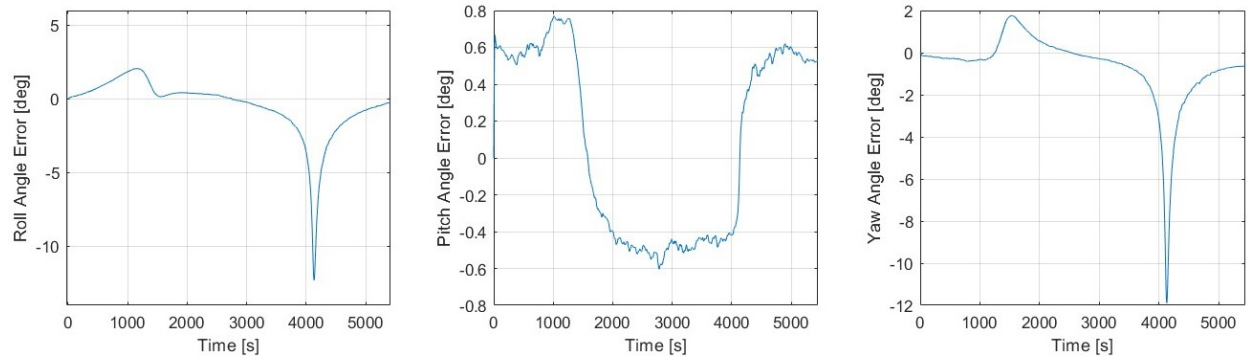


Figure 162: Control error in tracking the calibration attitude trajectories.

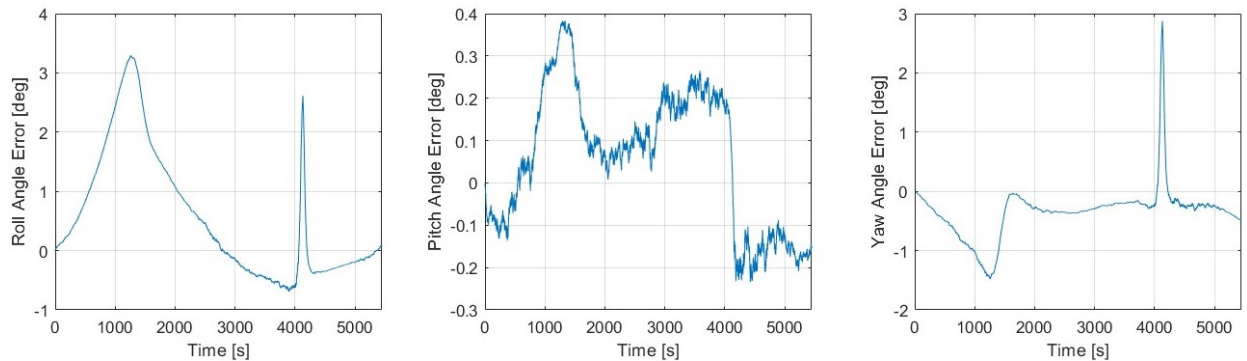


Figure 163: Estimation error in tracking the calibration attitude trajectories.

Attitude mode transfer

The satellite is required to transfer between the nominal and calibration attitude trajectories. To accomplish this, the reference trajectory for each attitude angle is switched independently for each angle when the trajectories are at the closest point. This ensure a smooth transition between attitude modes. The attitude switching is performed at the timings according to Table 19, where timing is with respect to the start of an orbit. From Figure 156, it can be seen that at these times, the trajectories for the given angle are closest.

Table 19: Attitude Mode Transfer Timings.

Attitude Angle	Switch Time [s]
Roll	1400
Pitch	3000
Yaw	1400

Figure 164 shows the attitude mode transfer from calibration attitude trajectories to nominal attitude trajectories. This is seen through the solid blue line initially tracking the orange dotted line, after which it switched to tracking the dotted yellow line based on the times indicated in Table 19. Therefore, the controller successfully manages to change from calibration mode to nominal mode.

Figure 165 shows the reverse attitude mode transfer. The satellite starts off in the nominal attitude mode, tracking the dotted yellow line, after which it switches to tracking the dotted orange line, which means the satellite successfully transferred attitude modes.

Calibration transfer sequence

A full attitude mode transfer sequence is performed to evaluate the satellite's capability to enter the calibration mode and exit it again to continue in nominal operational mode. The sequence is as follows:

- Initial nominal attitude mode
- In first orbit, transfer to calibration mode
- In second orbit, maintain calibration attitude
- In third orbit, transfer back to nominal mode
- At end of third orbit, the satellite has completed the sequence

The results for performing these attitude mode changes is shown in Figure 166, where it is seen that the satellite seamlessly transfers in between attitude modes. Additionally, Figure 167 and Figure 168 show the angular velocity and motor torque for each reaction wheel, respectively, throughout carrying out this sequence. From these figures it can be seen that the reaction wheels have still not reached their saturation limits. Furthermore, the commanded torque is generally also well within the limits capable of being achieved by these reaction wheels.

This concludes the design of the nonlinear control law which is capable of tracking a time-varying trajectory. Further testing of the robustness of the control law is required. Additional control features which would need to be implemented include the initial orbit insertion and attitude acquisition, as well as desaturation of the reaction wheels using the thrusters.

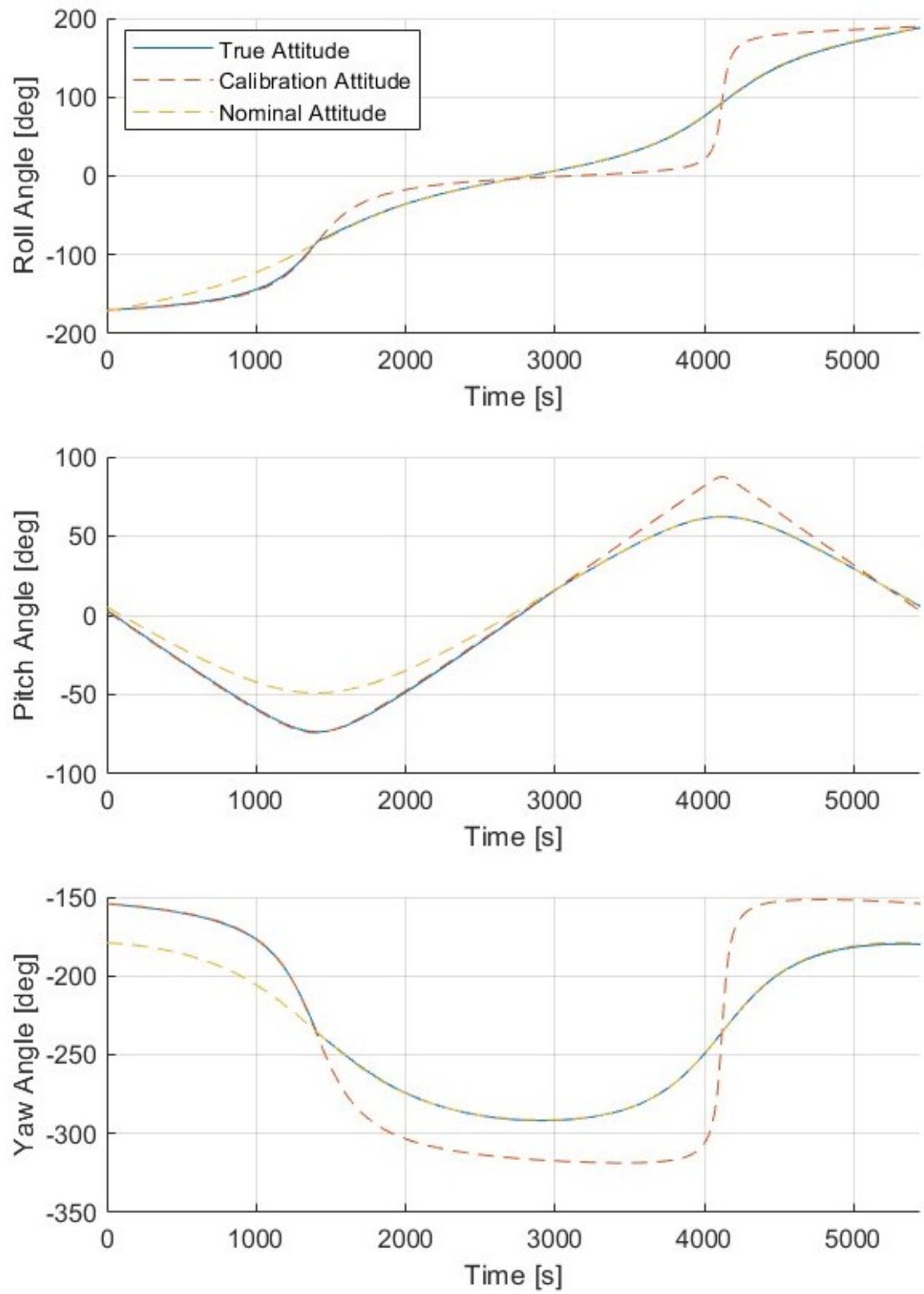


Figure 164: Attitude mode transfer from calibration attitude to nominal attitude mode.

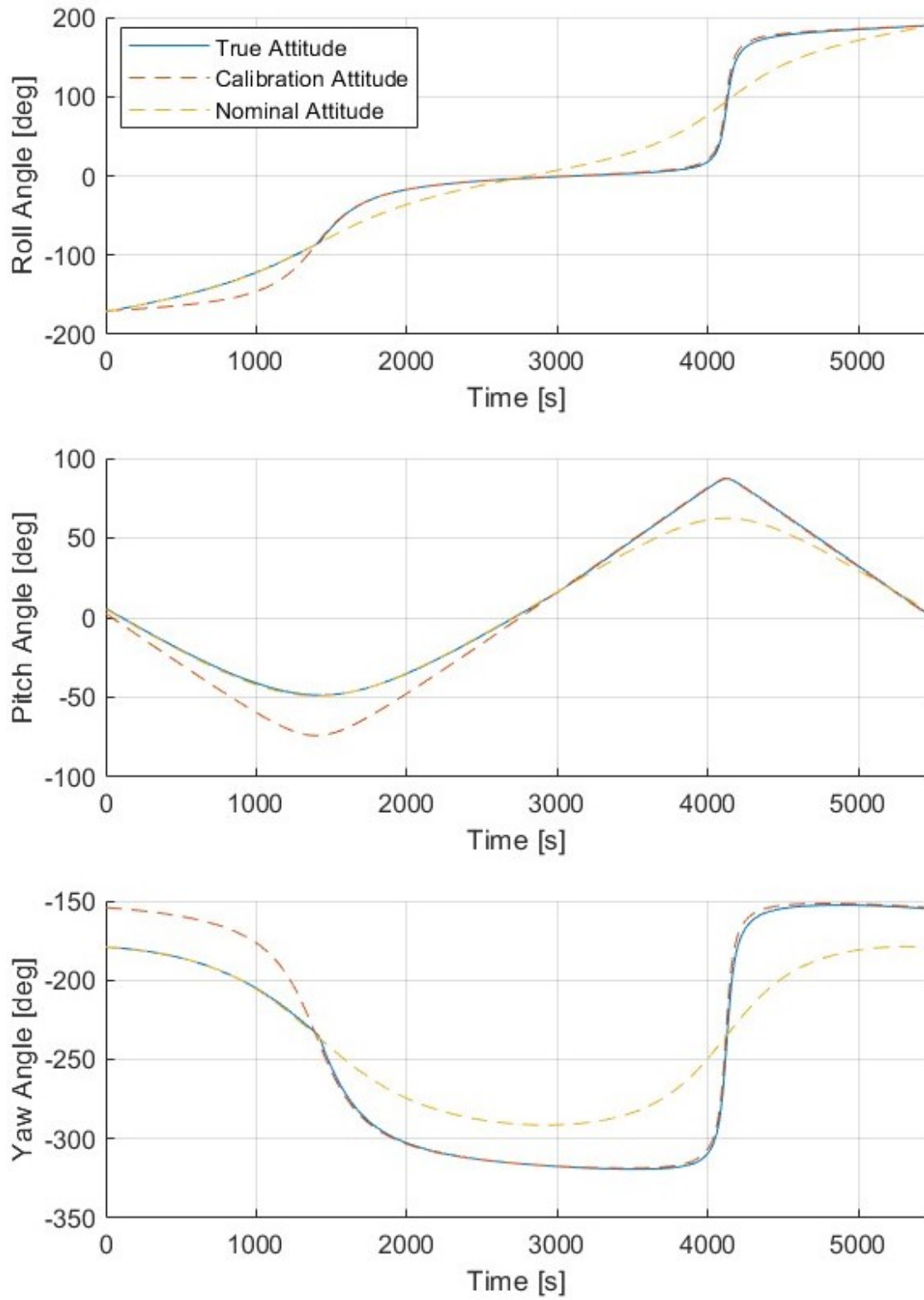


Figure 165: Attitude mode transfer from nominal attitude to calibration attitude mode.

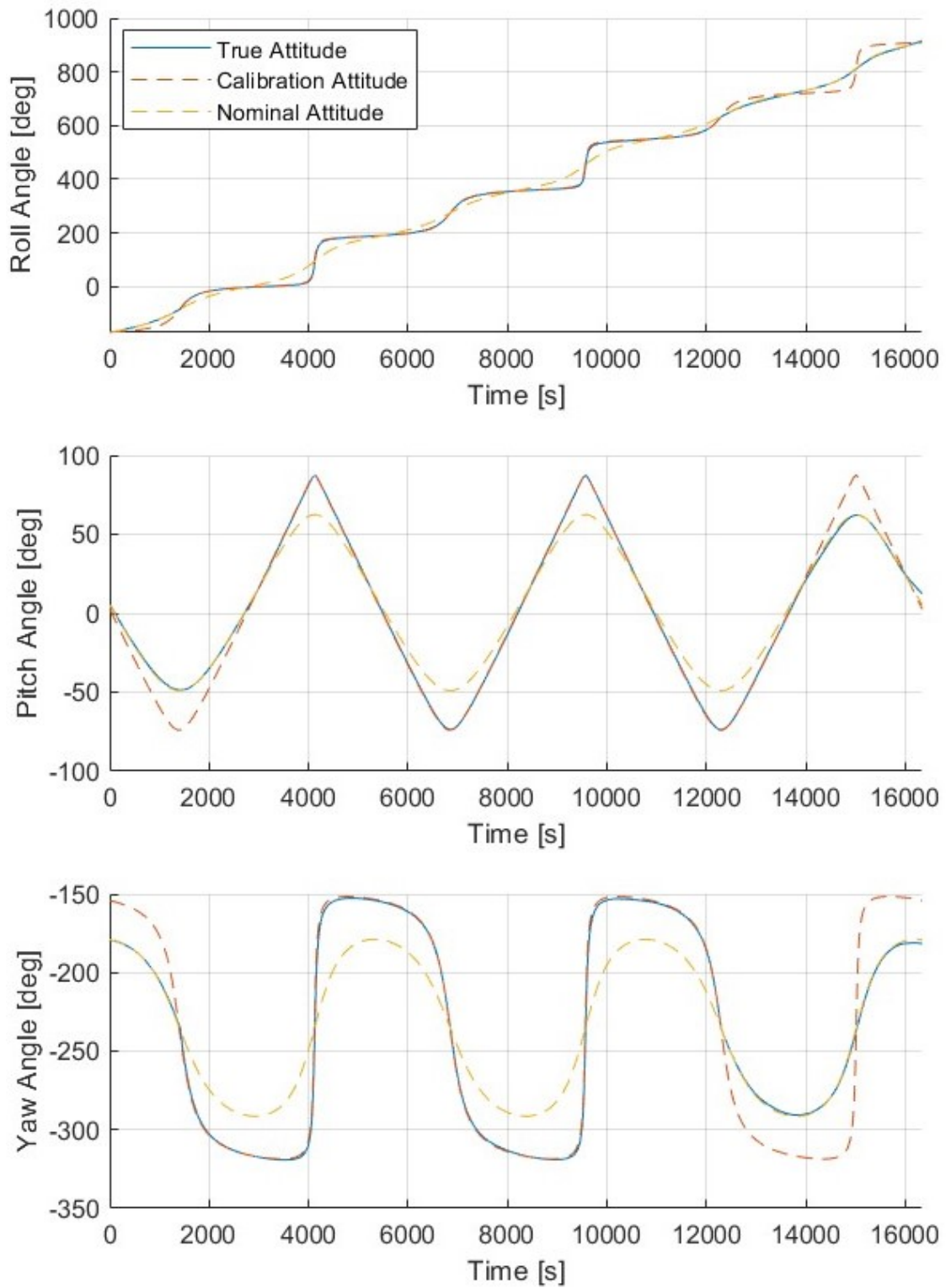


Figure 166: Sequence of attitude mode transfers from nominal to calibration to nominal attitude mode.

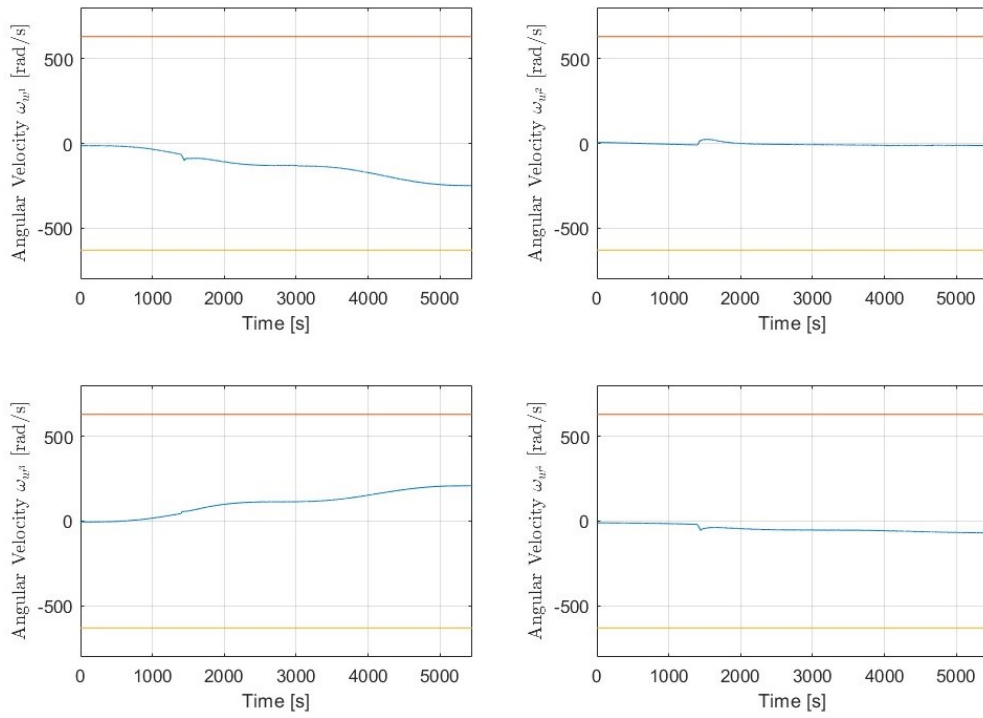


Figure 167: Angular speed of the four reaction wheels (blue) and maximum saturation angular speed (orange).

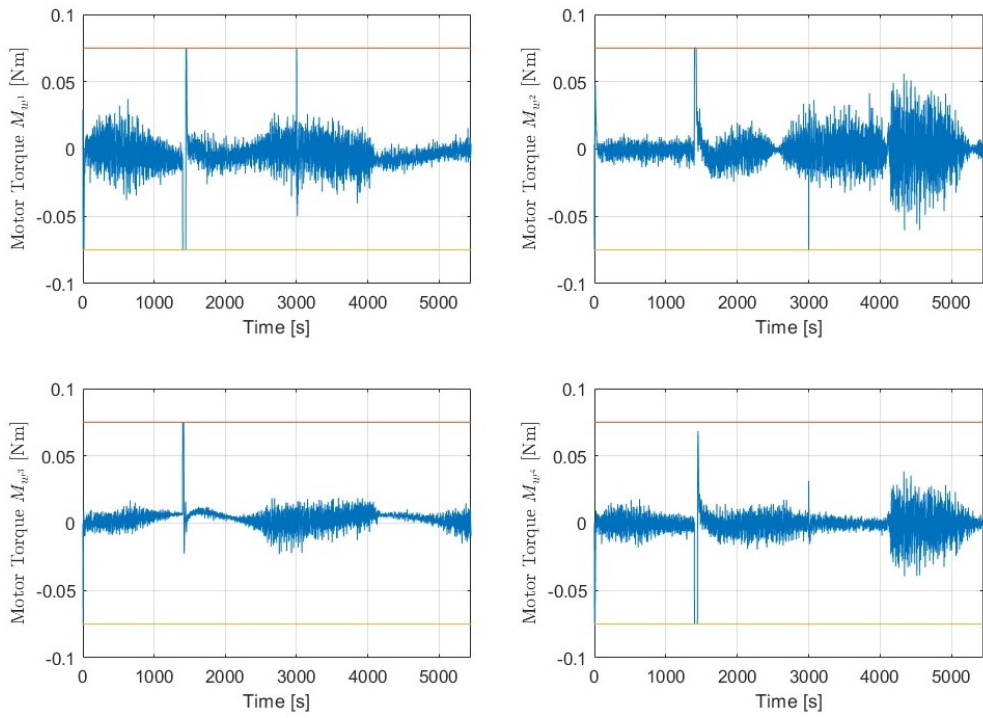


Figure 168: Motor torque of the four reaction wheels (blue) and maximum saturation torque (orange).

12 CONCLUSION

This design report detailed the design, implementation and testing of the attitude determination and control subsystem (ADCS) of a satellite. The satellite was based on the Aeolus mission which was an Earth observation mission, named after the ruler of winds from Greek mythology. The satellite was the first to acquire profiles of Earth's wind on a global scale. The document described the mission definition and ADCS requirements, after which the satellite's dynamics and kinematics were modeled and verified. The design of the ADCS included selection of an appropriate sensor and actuator suite, characterization of the environment in which the satellite was to be operational and implementation of the attitude determination and control algorithms. In the end, a functional implementation was achieved using coarse modeling of the sensors, actuators and external perturbations.

Through completing this design exercise, I have learned a great deal about the ADCS. While I already had the knowledge in the theory of state estimation and control, the real value was in learning how to select suitable sensors and actuators depending on mission specification and how to model these. How to account for the external factors depending on the orbit and which planet is orbited. Different methodologies used to stabilize or control a satellite, from passive to active methods. While working on this design exercise, I have also had to reference a range of different resources, such as [1] and [21], which I believe to be a very valuable skill for future work I might do.

References

- [1] *Spacecraft attitude determination and control*, volume 73 of *Astrophysics and Space Science Library*, January 1978.
- [2] Paul Acquatella, Wouter Falkena, Erik-Jan van Kampen, and Q Ping Chu. Robust nonlinear spacecraft attitude control using incremental nonlinear dynamic inversion. In *AIAA Guidance, Navigation, and Control Conference*, page 4623, 2012.
- [3] CubeSpace. Gen2: Cubemag (compact & deployable). <https://satsearch.co/products/cubespace-gen2-cubemag-compact-and-deployable>.
- [4] CubeSpace. Gen2: Cubesense earth. <https://satsearch.co/products/cubespace-gen2-cubesense-sun>.
- [5] CubeSpace. Gen2: Cubesense sun. <https://satsearch.co/products/cubespace-gen2-cubesense-earth>.
- [6] M Endemann. Adm-aeolus project: System requirements document. <https://earth.esa.int/eogateway/documents/20142/1564626/Aeolus-System-Requirements.pdf>, Sep 2010.
- [7] Dirk Henning. Rsi 45 momentum and reaction wheels 15 – 45 nms with integrated wheel drive electronics. https://satcatalog.s3.amazonaws.com/components/199/SatCatalog_-_Collins_Aerospace_-_RSI_15-215-20_-_Datasheet.pdf?lastmod=20210708032748.
- [8] Vaughn. Ihlen and Karen Zanter. *Landsat 7 (L7) Data Users Handbook*. LSDS-1927. Department of the Interior U.S. Geological Survey, 2019.
- [9] Herbert J. Kramer. Orbview-3 minisatellite. <https://www.eoportal.org/satellite-missions/orbview-3#mission-capabilities>, Jun 2012.
- [10] Gunter D Krebs. Eros-a 1, 2. https://space.skyrocket.de/doc_sdat/eros-a.htm, Jan 2023.
- [11] Rafael Ltd. 5n thruster. <https://satsearch.co/products/rafael-5n-thruster>.
- [12] NA. Earth's orbit. https://en.wikipedia.org/wiki/Earth%27s_orbit.
- [13] NA. Hg2801 fiber optic gyro inertial measurement unit. https://aerospace.honeywell.com/us/en/search?search=ring+laser+gyroscope&filters=document_type%2FProduct.
- [14] N.A. Satellite: Essa-9. https://space.oscar.wmo.int/satellites/view/essa_9, Jul 2015.
- [15] N.A. Aeolus / formerly adm (atmospheric dynamics mission). <https://www.eoportal.org/satellite-missions/aeolus>, Aug 2023.
- [16] N.A. Eos-07 mission. https://www.isro.gov.in/EOS_07.html, Feb 2023.
- [17] N.A. Ikonos data sheet. https://dg-cms-uploads-production.s3.amazonaws.com/uploads/document/file/96/DG_IKONOS_DS.pdf, Jun 2023.
- [18] N.A. Quickbird data sheet. <https://www.spaceimagingme.com/downloads/sensors/datasheets/QuickBird-DS-QB-Web.pdf>, Jan 2023.

- [19] N.A. Spot-6 and spot-7 (azersky). <https://www.eoportal.org/satellite-missions/spot-6-7#references>, Jun 2023.
- [20] Chris Peat. Aeolus - orbit. <https://www.heavens-above.com/orbit.aspx?satid=43600&lat=0&lng=0&loc=Unspecified&alt=0&tz=UCT>.
- [21] James Richard. Wertz, David F. Everett, and Jeffery John. Puschell. *Space mission engineering : the new SMAD*. Space technology library ; v. 28. Microcosm Press, Hawthorne, CA, 2011.

13 Appendices

13.1 Spacecraft Geometric Data

Table 20: Spacecraft Geometric Data

Surface Index	Unit surface normal vectors			Geometric barycenter coordinates			Surface Area [m ²]
	$\hat{i}$ [-]	$\hat{j}$ [-]	$\hat{k}$ [-]	x -coord [m]	y -coord [m]	z -coord [m]	
1	-1	0	0	1	0.418177006	-0.625302006	1.653
2	-1	0	0	1	-0.418177006	0.051052006	1.653
3	0	1	0	0.455481045	0.95	-0.612606045	1.74
4	0	1	0	-0.455481045	0.95	0.038356045	1.74
5	-1	0	0	-1	0.418177006	-0.625302006	1.653
6	-1	0	0	-1	-0.418177006	0.051052006	1.653
7	0	1	0	0.455481045	-0.95	-0.612606045	1.74
8	0	1	0	-0.455481045	-0.95	0.038356045	1.74
9	0	0	1	0.429311422	0.379311422	-1.157125	1.9
10	0	0	1	-0.429311422	-0.379311422	-1.157125	1.9
11	0	0	1	-0.429311422	0.379311422	0.582875	1.9
12	0	0	1	0.429311422	-0.379311422	0.582875	1.9
13	-1	0	0	0.05	6.942047922	-0.334172922	3.6075
14	-1	0	0	0.05	1.457952078	-0.240077078	3.6075
15	0	1	0	0.002247701	7.45	-0.794372701	0.0555
16	0	1	0	-0.002247701	7.45	0.220122701	0.0555
17	-1	0	0	-0.05	6.942047922	-0.334172922	3.6075
18	-1	0	0	-0.05	1.457952078	-0.240077078	3.6075
19	0	1	0	0.002247701	0.95	-0.794372701	0.0555
20	0	1	0	-0.002247701	0.95	0.220122701	0.0555
21	0	0	1	0.000384593	7.400384593	-0.842125	0.325
22	0	0	1	-0.000384593	0.999615407	-0.842125	0.325
23	0	0	1	-0.000384593	7.400384593	0.267875	0.325
24	0	0	1	0.000384593	0.999615407	0.267875	0.325
25	-1	0	0	0.05	-1.457952078	-0.334172922	3.6075
26	-1	0	0	0.05	-6.942047922	-0.240077078	3.6075
27	0	1	0	0.002247701	-0.95	-0.794372701	0.0555
28	0	1	0	-0.002247701	-0.95	0.220122701	0.0555
29	-1	0	0	-0.05	-1.457952078	-0.334172922	3.6075
30	-1	0	0	-0.05	-6.942047922	-0.240077078	3.6075
31	0	1	0	0.002247701	-7.45	-0.794372701	0.0555
32	0	1	0	-0.002247701	-7.45	0.220122701	0.0555
33	0	0	1	0.000384593	-0.999615407	-0.842125	0.325
34	0	0	1	-0.000384593	-7.400384593	-0.842125	0.325
35	0	0	1	-0.000384593	-0.999615407	0.267875	0.325
36	0	0	1	0.000384593	-7.400384593	0.267875	0.325
37	0	0	1	0.462987426	0.191775671	2.582875	0.120546176
38	0.923879533	0.382683432	0	0.655615165	0.227865149	2.33623554	0.329504871
39	-0.923879533	-0.382683432	0	0.624714921	0.302464937	0.82951446	0.329504871
40	0	0	1	0.191775671	0.462987426	2.582875	0.120546176
41	0.382683432	0.923879533	0	0.302464937	0.624714921	2.33623554	0.329504871
42	-0.382683432	-0.923879533	0	0.227865149	0.655615165	0.82951446	0.329504871
43	0	0	1	-0.191775671	0.462987426	2.582875	0.120546176
44	-0.382683432	0.923879533	0	-0.227865149	0.655615165	2.33623554	0.329504871
45	0.382683432	-0.923879533	0	-0.302464937	0.624714921	0.82951446	0.329504871
46	0	0	1	-0.462987426	0.191775671	2.582875	0.120546176
47	-0.923879533	0.382683432	0	-0.624714921	0.302464937	2.33623554	0.329504871
48	0.923879533	-0.382683432	0	-0.655615165	0.227865149	0.82951446	0.329504871
49	0	0	1	-0.462987426	-0.191775671	2.582875	0.120546176
50	-0.923879533	-0.382683432	0	-0.655615165	-0.227865149	2.33623554	0.329504871
51	0.923879533	0.382683432	0	-0.624714921	-0.302464937	0.82951446	0.329504871
52	0	0	1	-0.191775671	-0.462987426	2.582875	0.120546176
53	-0.382683432	-0.923879533	0	-0.302464937	-0.624714921	2.33623554	0.329504871
54	0.382683432	0.923879533	0	-0.227865149	-0.655615165	0.82951446	0.329504871
55	0	0	1	0.191775671	-0.462987426	2.582875	0.120546176
56	0.382683432	-0.923879533	0	0.227865149	-0.655615165	2.33623554	0.329504871
57	-0.382683432	0.923879533	0	0.302464937	-0.624714921	0.82951446	0.329504871
58	0	0	1	0.462987426	-0.191775671	2.582875	0.120546176
59	0.923879533	-0.382683432	0	0.624714921	-0.302464937	2.33623554	0.329504871
60	-0.923879533	0.382683432	0	0.655615165	-0.227865149	0.82951446	0.329504871

13.2 Problem Set 1 Main File

```
1 %% Spacecraft properties
2 run("spacecraft_properties.m");
3
4 %% Calculate Spacecraft geometric and mass properties
5 [TR, S] = discretize_sc(CG_centroid, attitude, 0, spacecraft);
6
7 centroids = TR.incenter;
8
9 figure
10 trimesh(TR.ConnectivityList, TR.Points(:,1), TR.Points(:,2), TR.Points(:,3))
11 hold on
12 scatter3(centroids(:, 1), centroids(:, 2), centroids(:, 3))
13
14 body_axes = angle2dcm(attitude(3), attitude(2), attitude(1))' * [10 0 0;
15                                                                    0 10 0;
16                                                                    0 0 10];
17 plot3([CG_centroid(1) CG_centroid(1) + body_axes(1,1)], [CG_centroid(2) ...
18     CG_centroid(2) + body_axes(2,1)], [CG_centroid(3) CG_centroid(3) + ...
19     body_axes(3,1)], 'r-^', 'LineWidth', 3);
20
21 plot3([CG_centroid(1) CG_centroid(1) + body_axes(1,2)], [CG_centroid(2) ...
22     CG_centroid(2) + body_axes(2,2)], [CG_centroid(3) CG_centroid(3) + ...
23     body_axes(3,2)], 'g-^', 'LineWidth', 3);
24
25 plot3([CG_centroid(1) CG_centroid(1) + body_axes(1,3)], [CG_centroid(2) ...
26     CG_centroid(2) + body_axes(2,3)], [CG_centroid(3) CG_centroid(3) + ...
27     body_axes(3,3)], 'b-^', 'LineWidth', 3);
28
29 principal_axes = angle2dcm(attitude(3), attitude(2), attitude(1))' * ...
30     spacecraft.Rpb * [10 0 0; 0 10 0; 0 0 10];
31
32 plot3([CG_centroid(1) CG_centroid(1) + ...
33     principal_axes(1,1)], [CG_centroid(2) CG_centroid(2) + ...
34     principal_axes(2,1)], [CG_centroid(3) CG_centroid(3) + ...
35     principal_axes(3,1)], 'y-^', 'LineWidth', 3);
36
37 plot3([CG_centroid(1) CG_centroid(1) + ...
38     principal_axes(1,2)], [CG_centroid(2) CG_centroid(2) + ...
39     principal_axes(2,2)], [CG_centroid(3) CG_centroid(3) + ...
40     principal_axes(3,2)], 'y-^', 'LineWidth', 3);
41
42 plot3([CG_centroid(1) CG_centroid(1) + ...
43     principal_axes(1,3)], [CG_centroid(2) CG_centroid(2) + ...
44     principal_axes(2,3)], [CG_centroid(3) CG_centroid(3) + ...
45     principal_axes(3,3)], 'y-^', 'LineWidth', 3);
46
47 axis equal
48 set(gca, 'Zdir', 'reverse')
49 set(gca, 'Ydir', 'reverse')
```

13.3 Problem Set 2 Main File

```
1 %% Spacecraft properties
2 run("spacecraft_properties.m");
3
4 %% Calculate spacecraft orbit trajectory
5
6 orbit.Re = 6371e3;
7 orbit.GM = 3.986004415e14; % Earth's Gravitational Constant [km3/s2]
```

```

8 orbit.e = 0.006;      % Eccentricity [-]
9 orbit.a = 320e3 + orbit.Re; % Semi-major axis [m]
10 orbit.incl = 96.7; % Inclination [deg]
11 orbit.RAAN = 214.9; % Longitude of ascending node [deg]
12 orbit.argp = 264.2; % Argument of periapsis [rad]
13 orbit.true_anomaly = 95.4; % Mean anomaly at epoch [rad]
14
15 [r_ijk, v_ijk] = keplerian2ijk(orbit.a, orbit.e, orbit.incl, ...
16                               orbit.RAAN, orbit.argp, ...
17                               orbit.true_anomaly);
18
19 x0 = [r_ijk; v_ijk];
20 dt = 1;
21 tf = (2*pi*sqrt(orbit.a^3/orbit.GM));
22
23 opts = odeset('RelTol',1e-13,'AbsTol',1e-14,'Stats','on');
24 tspan = 0:dt:tf;
25
26 [t,x] = ode113(orbital_equation(orbit), tspan, x0, opts);
27 [x_earth, y_earth, z_earth] = sphere;
28
29 figure
30 surf(x_earth* orbit.Re, y_earth* orbit.Re, z_earth* orbit.Re)
31 hold on
32 plot3(x(:, 1), x(:, 2), x(:, 3))
33 legend('Earth', 'Orbit Trajectory', 'Location', 'SouthEast')
34 xlabel('X-position')
35 ylabel('Y-position')
36 zlabel('Z-position')
37 axis equal
38
39 %% Integrate Euler equations
40
41 % Initial conditions
42 dt = 0.01;
43 eta_0 = [0; 0; 0];
44 omega_0 = [deg2rad(2); deg2rad(3); deg2rad(1)];
45 L2_0 = omega_0'.^2 * diag(spacecraft.Ip).^2;
46
47 % Numerical integration
48 tf = 650;
49 t = 0:dt:tf;
50 opts = odeset('RelTol',1e-13,'AbsTol',1e-14,'Stats','on');
51
52 [t, x] = ode45(euler_equation(spacecraft, "euler_angles"), t, [omega_0; ...
53                       eta_0], opts);
54
55 omega = x(:, 1:3);
56 eta = x(:, 4:6);
57
58 % Check conservation of energy
59 L2_ls = [];
60 for om=omega'
61     L2 = om'.^2 * diag(spacecraft.Ip).^2;
62     if abs(L2 - L2_0)>=1e-6
63         disp("Conservation of energy not maintained");
64     end
65     L2_ls = [L2_ls L2];
66 end

```

```

65
66 % Plot trajectories
67 %   Angular velocity
68 figure
69 subplot(1,3,1);
70 plot(t, omega(:,1)*180/pi);
71 grid on;
72 xlabel('Time [s]')
73 ylabel('Angular velocity along x-axis [deg/s]')
74 xlim([0 tf])
75
76 subplot(1,3,2);
77 plot(t, omega(:,2)*180/pi);
78 grid on;
79 xlabel('Time [s]')
80 ylabel('Angular velocity along y-axis [deg/s]')
81 xlim([0 tf])
82
83 subplot(1,3,3);
84 plot(t, omega(:,3)*180/pi);
85 grid on;
86 xlabel('Time [s]')
87 ylabel('Angular velocity along z-axis [deg/s]')
88 xlim([0 tf])
89
90 %   Attitude angle
91 figure
92 subplot(1,3,1);
93 plot(t, eta(:,1)*180/pi);
94 grid on;
95 xlabel('Time [s]')
96 ylabel('Roll Angle [deg]')
97 xlim([0 tf])
98
99 subplot(1,3,2);
100 plot(t, eta(:,2)*180/pi);
101 grid on;
102 xlabel('Time [s]')
103 ylabel('Pitch Angle [deg]')
104 xlim([0 tf])
105
106 subplot(1,3,3);
107 plot(t, eta(:,3)*180/pi);
108 grid on;
109 xlabel('Time [s]')
110 ylabel('Yaw Angle [deg]')
111 xlim([0 tf])
112
113 %   Angular Momentum
114 figure;
115 plot(t, sqrt(L2_ls));
116 grid on;
117 ylim([0 1.1*sqrt(L2_0)]);
118 xlabel('Time [s]')
119 ylabel('Angular Momentum [kg m^2/s]')
120 xlim([0 tf])
121
122 %% Ellipsoids and Polhode
123

```

```

124 % Energy ellipsoid
125 T = 1/2 * omega_0.^2' * diag(spacecraft.Ip);
126 a = sqrt(2*T / spacecraft.Ip(1,1));
127 b = sqrt(2*T / spacecraft.Ip(2,2));
128 c = sqrt(2*T / spacecraft.Ip(3,3));
129
130 [X_1,Y_1,Z_1] = ellipsoid(0,0,0,a,b,c, 50);
131
132 % Momentum ellipsoid
133 L = sqrt(omega_0.^2' * diag(spacecraft.Ip).^2);
134 a = L / spacecraft.Ip(1,1);
135 b = L / spacecraft.Ip(2,2);
136 c = L / spacecraft.Ip(3,3);
137
138 [X_2,Y_2,Z_2] = ellipsoid(0,0,0,a,b,c, 50);
139
140 figure
141 s1 = surf(X_1,Y_1,Z_1, 'FaceColor', 'r');
142 hold on;
143 s2 = surf(X_2,Y_2,Z_2, 'FaceColor', 'b');
144 plot3(omega(:,1), omega(:,2), omega(:,3), "-g", 'LineWidth', 1)
145 legend('Energy Ellipsoid', 'Momentum Ellipsoid', 'Polhode')
146 axis equal;

```

13.4 Problem Set 3 Main File

```

1 %% Spacecraft properties
2 run("spacecraft_properties.m");
3
4 %% Integrate Euler equations for Axial-Symmetric Satellite
5 spacecraft.Ip(1,1) = spacecraft.Ip(2,2); % axial-symmetric (Ix=Iy)
6
7 % Initial conditions
8 dt = 0.1;
9 eta_0 = [0; 0; 0];
10 omega_0 = [deg2rad(2); deg2rad(3); deg2rad(1)];
11 L2_0 = sqrt(omega_0'.^2 * diag(spacecraft.Ip).^2);
12
13 % Numerical integration
14 tf = 1500;
15 t = 0:dt:tf;
16 opts = odeset('RelTol',1e-13,'AbsTol',1e-14,'Stats','on');
17
18 [t, x] = ode45(euler_equation(spacecraft, "euler_angles"), t, [omega_0; ...
19     eta_0], opts);
20
21 omega = x(:, 1:3);
22
23 % Check conservation of energy
24 ang_momentum = zeros(size(omega));
25 for i=1:length(omega)
26     ang_momentum(i, :) = spacecraft.Ip * omega(i, :);
27     if abs(norm(ang_momentum(i, :)) - L2_0) >= 1e-6
28         disp("Conservation of energy not maintained");
29     end
30 end

```

```

31 % Plot angular velocity trajectories
32 figure
33 subplot(1,3,1);
34 plot(t, omega(:,1)*180/pi);
35 grid on;
36 xlabel('Time [s]')
37 ylabel('Angular velocity along x-axis [deg/s]')
38 xlim([0 tf])
39
40 subplot(1,3,2);
41 plot(t, omega(:,2)*180/pi);
42 grid on;
43 xlabel('Time [s]')
44 ylabel('Angular velocity along y-axis [deg/s]')
45 xlim([0 tf])
46
47 subplot(1,3,3);
48 plot(t, omega(:,3)*180/pi);
49 grid on;
50 xlabel('Time [s]')
51 ylabel('Angular velocity along z-axis [deg/s]')
52 xlim([0 tf])
53
54 %% Analytical Solution to Euler equation for axial-symmetric satellite
55
56 lambda = (spacecraft.Ip(3,3) - spacecraft.Ip(1,1)) / spacecraft.Ip(1,1) ...
          * omega_0(3);
57
58 t = 0:dt:tf;
59
60 omega_x = omega_0(1) * cos(lambda*t) - omega_0(2) * sin(lambda*t);
61 omega_y = omega_0(2) * cos(lambda*t) + omega_0(1) * sin(lambda*t);
62 omega_z = omega_0(3) * ones(size(t));
63
64 % Plot angular velocity
65 figure
66 subplot(1,3,1);
67 plot(t, omega_x*180/pi);
68 grid on;
69 xlabel('Time [s]')
70 ylabel('Angular velocity along x-axis [deg/s]')
71 xlim([0 tf])
72
73 subplot(1,3,2);
74 plot(t, omega_y*180/pi);
75 grid on;
76 xlabel('Time [s]')
77 ylabel('Angular velocity along y-axis [deg/s]')
78 xlim([0 tf])
79
80 subplot(1,3,3);
81 plot(t, omega_z*180/pi);
82 grid on;
83 xlabel('Time [s]')
84 ylabel('Angular velocity along z-axis [deg/s]')
85 xlim([0 tf])
86
87
88 %% Comparison between numerical and analytical angular velocity

```

```

89
90 % Difference between numerical and analytical angular velocity
91 figure
92 subplot(1,3,1);
93 plot(t, (omega_x' - omega(:, 1))*180/pi);
94 grid on;
95 xlabel('Time [s]')
96 ylabel('Difference along x-axis [deg/s]')
97 xlim([0 tf])
98
99 subplot(1,3,2);
100 plot(t, (omega_y' - omega(:, 2))*180/pi);
101 grid on;
102 xlabel('Time [s]')
103 ylabel('Difference along y-axis [deg/s]')
104 xlim([0 tf])
105
106 subplot(1,3,3);
107 plot(t, (omega_z' - omega(:, 3))*180/pi);
108 grid on;
109 xlabel('Time [s]')
110 ylabel('Difference z-axis [deg/s]')
111 xlim([0 tf])
112
113 % Decompose Angular Velocity
114 figure
115 subplot(1,3,1);
116 plot(t, vecnorm(omega(:, 1:2), 2, 2)*180/pi);
117 grid on;
118 xlabel('Time [s]')
119 ylabel('||\omega_{xy}|| [deg/s]')
120 xlim([0 tf])
121 ylim([0 4])
122
123 subplot(1,3,2);
124 plot(t, vecnorm(omega(:, 3), 2, 2)*180/pi);
125 grid on;
126 xlabel('Time [s]')
127 ylabel('||\omega_{z}|| [deg/s]')
128 xlim([0 tf])
129
130 subplot(1,3,3);
131 plot(t, vecnorm(omega, 2, 2)*180/pi);
132 grid on;
133 xlabel('Time [s]')
134 ylabel('||\omega|| [deg/s]')
135 xlim([0 tf])
136 ylim([0 4])
137
138 % Decompose Momentum vector
139 figure
140 subplot(1,3,1);
141 plot(t, vecnorm(ang_momentum(:, 1:2), 2, 2));
142 grid on;
143 xlabel('Time [s]')
144 ylabel('||L_{xy}|| [deg/s]')
145 xlim([0 tf])
146 ylim([0 150])
147

```

```

148 subplot(1,3,2);
149 plot(t, vecnorm(ang_momentum(:, 3), 2, 2));
150 grid on;
151 xlabel('Time [s]')
152 ylabel('||L-{z}|| [deg/s]')
153 xlim([0 tf])
154 ylim([0 150])
155
156 subplot(1,3,3);
157 plot(t, vecnorm(ang_momentum, 2, 2));
158 grid on;
159 xlabel('Time [s]')
160 ylabel('||L|| [deg/s]')
161 xlim([0 tf])
162 ylim([0 150])
163
164 %% Integrate Euler equations with Euler angle parameterization
165 run("spacecraft_properties.m");
166
167 % Initial conditions
168 dt = 1;
169 eta_0 = [0; 0; 0];
170 omega_0 = [deg2rad(2); deg2rad(3); deg2rad(1)];
171 L2_0 = sqrt(omega_0'.^2 * diag(spacecraft.Ip).^2);
172
173 % Numerical integration
174 tf = 650;
175 t = 0:dt:tf;
176 opts = odeset('RelTol',1e-13,'AbsTol',1e-14,'Stats','on');
177
178 [t, x] = ode45(euler_equation(spacecraft, "euler-angles"), t, [omega_0; ...
    eta_0], opts);
179
180 omega = x(:, 1:3);
181 eta_euler = x(:, 4:6);
182
183 % Check conservation of energy
184 ang_momentum = zeros(size(omega));
185 for i=1:length(omega)
186     ang_momentum(i, :) = spacecraft.Ip * omega(i, :);
187     if abs(norm(ang_momentum(i, :)) - L2_0) >= 1e-6
188         disp("Conservation of energy not maintained");
189     end
190 end
191
192 % Plot Angular velocity
193 figure
194 subplot(1,3,1);
195 plot(t, omega(:,1)*180/pi);
196 grid on;
197 xlabel('Time [s]')
198 ylabel('Angular velocity along x-axis [deg/s]')
199 xlim([0 tf])
200
201 subplot(1,3,2);
202 plot(t, omega(:,2)*180/pi);
203 grid on;
204 xlabel('Time [s]')
205 ylabel('Angular velocity along y-axis [deg/s]')

```

```

206 xlim([0 tf])
207
208 subplot(1,3,3);
209 plot(t, omega(:,3)*180/pi);
210 grid on;
211 xlabel('Time [s]')
212 ylabel('Angular velocity along z-axis [deg/s]')
213 xlim([0 tf])
214
215 %% Integrate Euler equations with quaternion attitude parameterization
216
217 % Initial conditions
218 dt = 1;
219 eta_0 = [1 0 0 0]';
220 omega_0 = [deg2rad(2) deg2rad(3) deg2rad(1)]';
221 L2_0 = sqrt(omega_0'.^2 * diag(spacecraft.Ip).^2);
222
223 % Numerical integration
224 tf = 650;
225 t = 0:dt:tf;
226 opts = odeset('RelTol',1e-13,'AbsTol',1e-14,'Stats','on');
227
228 [t, x] = ode45(euler_equation(spacecraft, "quaternion"), t, [omega_0; ...
    eta_0], opts);
229
230 omega = x(:, 1:3);
231 eta_quat = x(:, 4:7);
232
233 % Plot Angular velocity
234 figure
235 subplot(1,3,1);
236 plot(t, omega(:,1)*180/pi);
237 grid on;
238 xlabel('Time [s]')
239 ylabel('Angular velocity along x-axis [deg/s]')
240 xlim([0 tf])
241
242 subplot(1,3,2);
243 plot(t, omega(:,2)*180/pi);
244 grid on;
245 xlabel('Time [s]')
246 ylabel('Angular velocity along y-axis [deg/s]')
247 xlim([0 tf])
248
249 subplot(1,3,3);
250 plot(t, omega(:,3)*180/pi);
251 grid on;
252 xlabel('Time [s]')
253 ylabel('Angular velocity along z-axis [deg/s]')
254 xlim([0 tf])
255
256
257 %% Plot Attitudes
258
259 % Plot quaternion
260 quat_convert = euler2quat(eta-euler);
261
262 figure
263 subplot(1,4,1);

```

```

264 plot(t, eta_quat(:,1), t, quat_convert(:,1));
265 grid on;
266 xlabel('Time [s]')
267 ylabel('1th quaternion comp')
268 xlim([0 tf])
269
270 subplot(1,4,2);
271 plot(t, eta_quat(:,2), t, quat_convert(:,2));
272 grid on;
273 xlabel('Time [s]')
274 ylabel('2th quaternion comp')
275 xlim([0 tf])
276
277 subplot(1,4,3);
278 plot(t, eta_quat(:,3), t, quat_convert(:,3));
279 grid on;
280 xlabel('Time [s]')
281 ylabel('3th quaternion comp')
282 xlim([0 tf])
283
284 subplot(1,4,4);
285 plot(t, eta_quat(:,4), t, quat_convert(:,4));
286 grid on;
287 xlabel('Time [s]')
288 ylabel('4th quaternion comp')
289 xlim([0 tf])
290
291 legend('\eta-{quat}', '\eta-{euler}')
292
293
294 % Plot Euler angle
295 euler_convert = quat2euler(eta_quat);
296 eta_euler = wrapToPi(eta_euler);
297
298 figure
299 subplot(1,3,1);
300 plot(t, eta_euler(:,1)*180/pi, t, euler_convert(:,1)*180/pi);
301 grid on;
302 xlabel('Time [s]')
303 ylabel('Roll Angle [deg]')
304 xlim([0 tf])
305
306 subplot(1,3,2);
307 plot(t, eta_euler(:,2)*180/pi, t, euler_convert(:,2)*180/pi);
308 grid on;
309 xlabel('Time [s]')
310 ylabel('Pitch Angle [deg]')
311 xlim([0 tf])
312
313 subplot(1,3,3);
314 plot(t, eta_euler(:,3)*180/pi, t, euler_convert(:,3)*180/pi);
315 grid on;
316 xlabel('Time [s]')
317 ylabel('Yaw Angle [deg]')
318 xlim([0 tf])
319
320 legend('\eta-{euler}', '\eta-{quat}')
321
322

```

```

323 %% Changes in coordinate frame
324
325 % Convert angular momentum vector to inertial frame
326 ang_momentum_i = zeros(size(ang_momentum));
327 omega_i = zeros(size(omega));
328 for i=1:length(ang_momentum_i)
329     Rip = euler2rotm(eta_euler(i, :))';
330     ang_momentum_i(i, :) = Rip * ang_momentum(i, :);
331     omega_i(i, :) = Rip * omega(i, :);
332 end
333
334 % Plot Momentum vector
335 figure
336 subplot(1,3,1);
337 plot(t, ang_momentum_i(:, 1));
338 grid on;
339 xlabel('Time [s]')
340 ylabel('L-{x} [kgm^2/s]')
341 xlim([0 tf])
342 ylim([0, 60])
343
344 subplot(1,3,2);
345 plot(t, ang_momentum_i(:, 2));
346 grid on;
347 xlabel('Time [s]')
348 ylabel('L-{y} [kgm^2/s]')
349 xlim([0 tf])
350 ylim([0, 110])
351
352 subplot(1,3,3);
353 plot(t, ang_momentum_i(:, 3));
354 grid on;
355 xlabel('Time [s]')
356 ylabel('L-{z} [kgm^2/s]')
357 xlim([0 tf])
358 ylim([0, 60])
359
360
361 % Plot herpolhode
362 point = omega_i(1, :);
363 normal = ang_momentum_i(1, :)/norm(ang_momentum_i(1, :));
364 d = -point*normal';
365 [xx,yy]=ndgrid(0.:0.01:0.05,0.:0.01:0.1);
366 z = (-normal(1)*xx - normal(2)*yy - d)/normal(3);
367
368 figure
369 plot3(omega_i(:, 1), omega_i(:, 2), omega_i(:, 3));
370 hold on
371 plot3([0 ang_momentum_i(1, 1)/norm(ang_momentum_i(1, :))/10] , ...
372       [0 ang_momentum_i(1, 2)/norm(ang_momentum_i(1, :))/10], ...
373       [0 ang_momentum_i(1, 3)/norm(ang_momentum_i(1, :))/10]);
374 surf(xx,yy,z)
375 grid on
376 legend("Angular Velocity", "Angular Momentum", "Perpendicular Plane");
377
378 %% Create orbital motion pictures
379 orbit.Re = 6371e3; % Earth's radius [m]
380 orbit.GM = 3.986004415e14; % Earth's Gravitational Constant [km3/s2]
381 orbit.e = 0.006; % Eccentricity [-]

```

```

382 orbit.a = 320e3 + orbit.Re; % Semi-major axis [m]
383 orbit.incl = 96.7; % Inclination [deg]
384 orbit.RAAN = 214.9; % Longitude of ascending node [deg]
385 orbit.argp = 264.2; % Argument of periapsis [rad]
386 orbit.true_anomaly = 95.4; % Mean anomaly at epoch [rad]
387
388 % Initial conditions
389 [p_0, v_0] = keplerian2ijk(orbit.a, orbit.e, orbit.incl, ...
390                             orbit.RAAN, orbit.argp, ...
391                             orbit.true_anomaly);
392
393 eta_0 = [0; 0; 0];
394 omega_0 = [deg2rad(1); deg2rad(2); deg2rad(4)];
395
396 dt = 1;
397 tf = 110;
398 t = 0:dt:tf;
399
400 opts = odeset('RelTol',1e-13,'AbsTol',1e-14,'Stats','on');
401 [~, x1] = ode45(euler.equation(spacecraft, "euler_angles"), t, ...
402               [omega_0; eta_0], opts);
403 [~, x2] = ode113(orbital.equation(orbit), t, [p_0; v_0], opts);
404
405 omega = x1(:, 1:3);
406 eta_euler = x1(:, 4:6);
407 pos = x2(:, 1:3)/1000; % [km]
408 vel = x2(:, 4:6);
409
410 figure
411 for i = 1:10:length(t)
412     % Inertial reference frame
413     hold off
414     plot3(0, 0, 0);
415     hold on
416     plot3([0 1], [0 0], [0 0], "r-^"); % x-axis
417     plot3([0 0], [0 1], [0 0], "g-^"); % y-axis
418     plot3([0 0], [0 0], [0 1], "b-^"); % z-axis
419
420     % Plot principal axes frame
421     Rip = euler2rotrm(eta_euler(i, :));
422     plot3(pos(i, 1)+[0 Rip(1,1)], pos(i, 2)+[0 Rip(2,1)], pos(i, 3)+[0 ...
423           Rip(3,1)], "y-^"); % x-axis
424     plot3(pos(i, 1)+[0 Rip(1,2)], pos(i, 2)+[0 Rip(2,2)], pos(i, 3)+[0 ...
425           Rip(3,2)], "y-^"); % y-axis
426     plot3(pos(i, 1)+[0 Rip(1,3)], pos(i, 2)+[0 Rip(2,3)], pos(i, 3)+[0 ...
427           Rip(3,3)], "y-^"); % z-axis
428
429     % Plot body axes frame
430     Rib = Rip * spacecraft.Rpb;
431     plot3(pos(i, 1)+[0 Rib(1,1)], pos(i, 2)+[0 Rib(2,1)], pos(i, 3)+[0 ...
432           Rib(3,1)], "k-^"); % x-axis
433     plot3(pos(i, 1)+[0 Rib(1,2)], pos(i, 2)+[0 Rib(2,2)], pos(i, 3)+[0 ...
434           Rib(3,2)], "k-^"); % y-axis
435     plot3(pos(i, 1)+[0 Rib(1,3)], pos(i, 2)+[0 Rib(2,3)], pos(i, 3)+[0 ...
436           Rib(3,3)], "k-^"); % z-axis
437
438     % Plot orbital frame
439     n_hat = cross(pos(i, :)/norm(pos(i, :)), vel(i, :)/norm(vel(i, :)));
440     r_hat = pos(i, :)/norm(pos(i, :));
441     t_hat = cross(n_hat, r_hat);

```

```

433     plot3(pos(i, 1)+[0 r_hat(1)], pos(i, 2)+[0 r_hat(2)], pos(i, 3)+[0 ...
        r_hat(3)], "m-^");           % x-axis
434     plot3(pos(i, 1)+[0 t_hat(1)], pos(i, 2)+[0 t_hat(2)], pos(i, 3)+[0 ...
        t_hat(3)], "m-^");           % y-axis
435     plot3(pos(i, 1)+[0 n_hat(1)], pos(i, 2)+[0 n_hat(2)], pos(i, 3)+[0 ...
        n_hat(3)], "m-^");           % z-axis
436
437     grid on
438     axis equal
439     xlabel('Inertial x-axis [km]')
440     ylabel('Inertial y-axis [km]')
441     zlabel('Inertial z-axis [km]')
442     xlim(pos(i, 1)+[-2 2])
443     ylim(pos(i, 2)+[-2 2])
444     zlim(pos(i, 3)+[-2 2])
445
446     file_name = sprintf('reference_frames_%d.png', i-1);
447     exportgraphics(gcf, file_name);
448 end

```

13.5 Problem Set 4 Main File

```

1  %% Spacecraft properties
2  run("spacecraft_properties.m");
3  run("orbit_properties.m");
4
5  %% Numerical integration settings
6  dt = 0.1;
7  tf = 1000;
8  t = 0:dt:tf;
9  opts = odeset('RelTol',1e-13,'AbsTol',1e-14,'Stats','on');
10
11 %% Equilibrium/Stability analysis of single spin satellite (Q1)
12
13 % Initial conditions
14 perturbation = 1e-4 * ones(3, 1);
15 eta_0 = [0; 0; 0];
16 omega_0 = [deg2rad(0); deg2rad(1); deg2rad(0)] + perturbation;
17
18 % Numerical integration
19 [t, x] = ode45(euler.equation(spacecraft, "euler_angles", false), t, ...
    [omega_0; eta_0], opts);
20 omega = x(:, 1:3);
21 eta_euler = x(:, 4:6);
22
23 % Plot angular velocity trajectories
24 figure
25 subplot(1,3,1);
26 plot(t, omega(:,1)*180/pi);
27 grid on;
28 xlabel('Time [s]')
29 ylabel('Roll Rate [deg/s]')
30 xlim([0 tf])
31
32 subplot(1,3,2);
33 plot(t, omega(:,2)*180/pi);
34 grid on;

```

```

35 xlabel('Time [s]')
36 ylabel('Pitch Rate [deg/s]')
37 xlim([0 tf])
38
39 subplot(1,3,3);
40 plot(t, omega(:,3)*180/pi);
41 grid on;
42 xlabel('Time [s]')
43 ylabel('Yaw Rate [deg/s]')
44 xlim([0 tf])
45 ylim([0, 1.1]);
46
47 % Plot Euler angle
48 figure
49 subplot(1,3,1);
50 plot(t, eta_euler(:,1)*180/pi);
51 grid on;
52 xlabel('Time [s]')
53 ylabel('Roll Angle [deg]')
54 xlim([0 tf])
55
56 subplot(1,3,2);
57 plot(t, eta_euler(:,2)*180/pi);
58 grid on;
59 xlabel('Time [s]')
60 ylabel('Pitch Angle [deg]')
61 xlim([0 tf])
62
63 subplot(1,3,3);
64 plot(t, eta_euler(:,3)*180/pi);
65 grid on;
66 xlabel('Time [s]')
67 ylabel('Yaw Angle [deg]')
68 xlim([0 tf])
69
70
71 %% Rotation about N-axis (Q2)
72
73 % Initial conditions
74 n_hat = cross(p_0/norm(p_0), v_0/norm(v_0));
75 r_hat = p_0/norm(p_0);
76 t_hat = cross(n_hat, r_hat);
77 Rpi = [r_hat t_hat n_hat]'; % Principal axes equal to RTN frame
78
79 eta_0 = [atan2(Rpi(2, 3), Rpi(3, 3));
80         -asin(Rpi(1, 3));
81         atan2(Rpi(1, 2), Rpi(1, 1))];
82 omega_0 = [deg2rad(0); deg2rad(0); deg2rad(1)];
83
84 x0 = [p_0; v_0; omega_0; eta_0];
85
86 % Numerical integration
87 options.attitude_parameterization = "euler_angles";
88 options.rotor = false;
89 options.external_torque = false;
90
91 [t, x] = ode45(equations_of_motion(spacecraft, orbit, options), t, x0, ...
92               opts);
93 pos = x(:, 1:3);

```

```

93 vel = x(:, 4:6);
94 omega = x(:, 7:9);
95 eta = x(:, 10:end);
96
97 eta_o = zeros(size(eta));
98 for i=1:length(eta)
99     n_hat = cross(pos(i, :)/norm(pos(i, :)), vel(i, :)/norm(vel(i, :)));
100     r_hat = pos(i, :)/norm(pos(i, :));
101     t_hat = cross(n_hat, r_hat);
102
103     Roi = [r_hat' t_hat' n_hat']';
104     Rip = euler2rotm(eta(i, :))';
105     Rop = Roi * Rip;
106
107     eta_o(i, :) = [atan2(Rop(2, 3), Rop(3, 3));
108                   -asin(Rop(1, 3));
109                   atan2(Rop(1, 2), Rop(1, 1))];
110 end
111
112 % Plot angular velocity trajectories
113 figure
114 subplot(1,3,1);
115 plot(t, omega(:,1)*180/pi);
116 grid on;
117 xlabel('Time [s]')
118 ylabel('Roll Rate [deg/s]')
119 xlim([0 tf])
120
121 subplot(1,3,2);
122 plot(t, omega(:,2)*180/pi);
123 grid on;
124 xlabel('Time [s]')
125 ylabel('Pitch Rate [deg/s]')
126 xlim([0 tf])
127
128 subplot(1,3,3);
129 plot(t, omega(:,3)*180/pi);
130 grid on;
131 xlabel('Time [s]')
132 ylabel('Yaw Rate [deg/s]')
133 xlim([0 tf])
134
135 % Plot Euler angle
136 figure
137 subplot(1,3,1);
138 plot(t, eta_o(:,1)*180/pi);
139 grid on;
140 xlabel('Time [s]')
141 ylabel('Roll Angle [deg]')
142 xlim([0 tf])
143 ylim([-200 200])
144
145 subplot(1,3,2);
146 plot(t, eta_o(:,2)*180/pi);
147 grid on;
148 xlabel('Time [s]')
149 ylabel('Pitch Angle [deg]')
150 xlim([0 tf])
151 ylim([-200 200])

```

```

152
153 subplot(1,3,3);
154 plot(t, eta_o(:,3)*180/pi);
155 grid on;
156 xlabel('Time [s]')
157 ylabel('Yaw Angle [deg]')
158 xlim([0 tf])
159
160 %% Equilibrium/Stability analysis of dual spin satellite (Q3)
161
162 % Initial conditions
163 perturbation = 0 * ones(3, 1);
164 eta_0 = [0; 0; 0];
165 omega_0 = [deg2rad(0); deg2rad(0); deg2rad(1)] + perturbation;
166 L2_0 = norm(spacecraft.Ip * omega_0 + ...
    spacecraft.wheel.L*spacecraft.wheel.r');
167
168 % Numerical integration
169 [t, x] = ode45(euler_equation(spacecraft, "euler_angles", true), t, ...
    [omega_0; eta_0], opts);
170 omega = x(:, 1:3);
171 eta_euler = x(:, 4:6);
172
173 % Check conservation of energy
174 ang_momentum = zeros(size(omega));
175 for i=1:length(omega)
176     ang_momentum(i, :) = spacecraft.Ip * omega(i, :) + ...
        spacecraft.wheel.L*spacecraft.wheel.r';
177     if abs(norm(ang_momentum(i, :)) - L2_0) >= 1e-6
178         disp("Conservation of energy not maintained");
179     end
180 end
181
182 % Convert angular momentum vector to inertial frame
183 ang_momentum_i = zeros(size(ang_momentum));
184 for i=1:length(ang_momentum_i)
185     Rip = euler2rotrm(eta_euler(i, :));
186     ang_momentum_i(i, :) = Rip * ang_momentum(i, :);
187 end
188
189 % Plot Momentum vector in inertial frame
190 figure
191 subplot(1,3,1);
192 plot(t, ang_momentum_i(:, 1));
193 grid on;
194 xlabel('Time [s]')
195 ylabel('L-{x} [kgm^2/s]')
196 xlim([0 tf])
197
198 subplot(1,3,2);
199 plot(t, ang_momentum_i(:, 2));
200 grid on;
201 xlabel('Time [s]')
202 ylabel('L-{y} [kgm^2/s]')
203 xlim([0 tf])
204
205 subplot(1,3,3);
206 plot(t, ang_momentum_i(:, 3));
207 grid on;

```

```

208 xlabel('Time [s]')
209 ylabel('L-{z} [kgm^2/s]')
210 xlim([0 tf])
211
212
213 % Plot angular velocity trajectories
214 figure
215 subplot(1,3,1);
216 plot(t, omega(:,1)*180/pi);
217 grid on;
218 xlabel('Time [s]')
219 ylabel('Roll Rate [deg/s]')
220 xlim([0 tf])
221
222 subplot(1,3,2);
223 plot(t, omega(:,2)*180/pi);
224 grid on;
225 xlabel('Time [s]')
226 ylabel('Pitch Rate [deg/s]')
227 xlim([0 tf])
228
229 subplot(1,3,3);
230 plot(t, omega(:,3)*180/pi);
231 grid on;
232 xlabel('Time [s]')
233 ylabel('Yaw Rate [deg/s]')
234 xlim([0 tf])
235
236
237 % Plot Euler angle
238 figure
239 subplot(1,3,1);
240 plot(t, eta_euler(:,1)*180/pi);
241 grid on;
242 xlabel('Time [s]')
243 ylabel('Roll Angle [deg]')
244 xlim([0 tf])
245
246 subplot(1,3,2);
247 plot(t, eta_euler(:,2)*180/pi);
248 grid on;
249 xlabel('Time [s]')
250 ylabel('Pitch Angle [deg]')
251 xlim([0 tf])
252
253 subplot(1,3,3);
254 plot(t, eta_euler(:,3)*180/pi);
255 grid on;
256 xlabel('Time [s]')
257 ylabel('Yaw Angle [deg]')
258 xlim([0 tf])
259
260
261 %% Plot stable attitude region
262
263 om_y = 0:0.1:5; % [deg/s]
264 wheel_ang_mom = (spacecraft.Ip(3,3) - spacecraft.Ip(2,2))*deg2rad(om_y);
265 max_wheel_ang_mom = spacecraft.wheel.L * ones(size(om_y));

```

```

266 vertex = rad2deg(spacecraft.wheel.L / (spacecraft.Ip(3,3) - ...
    spacecraft.Ip(2,2)));
267
268 figure
269 hold on
270 fill([0, 0, vertex], [0, 45, 45], [0 0.4470 0.7410], 'FaceAlpha', 0.3, ...
    'EdgeColor', 'none');
271 scatter(vertex, spacecraft.wheel.L);
272 plot(om_y, wheel_ang_mom, om_y, max_wheel_ang_mom)
273 scatter(vertex, spacecraft.wheel.L);
274 label = sprintf('%0.2f, %0.1f', vertex, spacecraft.wheel.L);
275 text(vertex, spacecraft.wheel.L, label, 'Vert','bottom', ...
    'Horiz','right', 'FontSize',9)
276 xlabel('Angular Velocity \omega_y')
277 ylabel('Wheel Angular Momentum L_{wheel}')
278 legend('Region of Stability')
279
280
281 %% Dual spin: Equilibrium about arbitrary axis
282
283 % Initial conditions
284 perturbation = 1e-4 * ones(3, 1);
285 eta_0 = [0; 0; 0];
286 omega_0 = [deg2rad(1); deg2rad(1); deg2rad(1)] + perturbation;
287 spacecraft.wheel.L, spacecraft.wheel.r = dual_spin_equil(omega_0, ...
    spacecraft);
288 L2_0 = norm(spacecraft.Ip * omega_0 + ...
    spacecraft.wheel.L*spacecraft.wheel.r');
289
290 % Numerical integration
291 [t, x] = ode45(euler_equation(spacecraft, "euler-angles", true), t, ...
    [omega_0; eta_0], opts);
292 omega = x(:, 1:3);
293 eta_euler = x(:, 4:6);
294
295 % Check conservation of energy
296 ang_momentum = zeros(size(omega));
297 for i=1:length(omega)
298     ang_momentum(i, :) = spacecraft.Ip * omega(i, :) + ...
        spacecraft.wheel.L*spacecraft.wheel.r';
299     if abs(norm(ang_momentum(i, :)) - L2_0) >= 1e-6
300         disp("Conservation of energy not maintained");
301     end
302 end
303
304 % Plot angular velocity trajectories
305 figure
306 subplot(1,3,1);
307 plot(t, omega(:,1)*180/pi);
308 grid on;
309 xlabel('Time [s]')
310 ylabel('Roll Rate [deg/s]')
311 xlim([0 tf])
312 ylim([0 omega_0(1)*180/pi*1.1])
313
314 subplot(1,3,2);
315 plot(t, omega(:,2)*180/pi);
316 grid on;
317 xlabel('Time [s]')

```

```

318 ylabel('Pitch Rate [deg/s]')
319 xlim([0 tf])
320 ylim([0 omega_0(2)*180/pi*1.1])
321
322 subplot(1,3,3);
323 plot(t, omega(:,3)*180/pi);
324 grid on;
325 xlabel('Time [s]')
326 ylabel('Yaw Rate [deg/s]')
327 xlim([0 tf])
328 ylim([0 omega_0(3)*180/pi*1.1])
329
330
331 %% Gravity Gradient Torque (Q4)
332 orbit.e = 0.0; % Consider circular orbit
333
334 % Initial conditions
335 [p_0, v_0] = keplerian2ijk(orbit.a, orbit.e, orbit.incl, orbit.RAAN, ...
    orbit.argp, orbit.true_anomaly, 'arglat', 0.0);
336 eta_0 = [deg2rad(5); 0; 0];
337 omega_0 = [deg2rad(3); deg2rad(1); deg2rad(2)];
338 x0 = [p_0; v_0; omega_0; eta_0];
339
340
341 n_hat = cross(p_0/norm(p_0), v_0/norm(v_0));
342 r_hat = p_0/norm(p_0);
343 t_hat = cross(n_hat, r_hat);
344 Roi = [r_hat t_hat n_hat]';
345 Rpi = euler2rotm(eta_0);
346
347 % Numerical integration
348 options.attitude_parameterization = "euler_angles";
349 options.rotor = false;
350
351 [t, x] = ode45(equations_of_motion(spacecraft, orbit, options), t, x0, ...
    opts);
352 pos = x(:, 1:3);
353 vel = x(:, 4:6);
354 omega = x(:, 7:9);
355 eta = x(:, 10:end);
356
357
358 % Confirm gravity gradient torque in radial direction is 0.
359 M0_p = external_torque(x0, spacecraft, orbit, options);
360 M0_i = Roi * M0_p;
361
362 M_vec = zeros(size(pos));
363 for i=1:length(x)
364     pos = x(i, 1:3)';
365     vel = x(i, 4:6)';
366     omega = x(i, 7:9)';
367     eta = x(i, 10:end)';
368
369     n_hat = cross(pos/norm(pos), vel/norm(vel));
370     r_hat = pos/norm(pos);
371     t_hat = cross(n_hat, r_hat);
372     Roi = [r_hat t_hat n_hat]';
373
374     Rpi = euler2rotm(eta);

```

```

375
376     M_vec(i, :) = Roi * Rpi' * external_torque(x(i, :)', spacecraft, ...
        orbit, options);
377 end
378
379 figure
380 subplot(1,3,1);
381 plot(t, M_vec(:, 1));
382 grid on;
383 xlabel('Time [s]')
384 ylabel('Torque in radial direction [Nm]')
385 xlim([0 tf])
386 ylim([-1e-3 1e-3])
387
388 subplot(1,3,2);
389 plot(t, M_vec(:, 2));
390 grid on;
391 xlabel('Time [s]')
392 ylabel('Torque in tangential direction [Nm]')
393 xlim([0 tf])
394 ylim([min(M_vec(:, 2))*1.1 max(M_vec(:, 2))*1.1])
395
396 subplot(1,3,3);
397 plot(t, M_vec(:, 3));
398 grid on;
399 xlabel('Time [s]')
400 ylabel('Torque in normal direction [Nm]')
401 xlim([0 tf])
402 ylim([min(M_vec(:, 3))*1.1 max(M_vec(:, 3))*1.1])
403
404
405 %% Verify Gravity Gradient (proportionality with orbit radius and inertia)
406 run("spacecraft_properties.m");
407 run("orbit_properties.m");
408
409 % Proportionality with difference in principal moments of inertia
410 eta_0 = [deg2rad(20); 0; 0];
411 omega_0 = [deg2rad(0); deg2rad(0); deg2rad(0)];
412 x0 = [p_0; v_0; omega_0; eta_0];
413
414 I_axis = 3;
415 I_range = spacecraft.Ip(I_axis, I_axis):10:spacecraft.Ip(I_axis, I_axis)*2;
416 M = zeros(length(I_range), 3);
417 for i=1:length(I_range)
418     spacecraft.Ip(I_axis, I_axis) = I_range(i);
419     M(i, :) = external_torque(x0, spacecraft, orbit, options)';
420 end
421
422 figure
423 subplot(1,3,1);
424 scatter(I_range, M(:, 1));
425 ylabel('Torque M_{gg-x} [Nm]');
426 xlabel('Moment of Inertia [kgm^2]');
427 grid on
428
429 subplot(1,3,2);
430 scatter(I_range, M(:, 2));
431 ylabel('Torque M_{gg-y} [Nm]');
432 xlabel('Moment of Inertia [kgm^2]');

```

```

433 grid on
434
435 subplot(1,3,3);
436 scatter(I_range, M(:, 3));
437 ylabel('Torque M-{gg-z} [Nm]');
438 xlabel('Moment of Inertia [kgm^2]');
439 grid on
440
441
442 % Proportionality with altitude
443 eta_0 = [deg2rad(20); 0; 0];
444 omega_0 = [deg2rad(0); deg2rad(0); deg2rad(0)];
445
446 radius = orbit.a:100000:orbit.a*5;
447 M = zeros(length(radius), 3);
448 for i=1:length(radius)
449     orbit.a = radius(i);
450     [p_0, v_0] = keplerian2ijk(orbit.a, orbit.e, orbit.incl, ...
451         orbit.RAAN, orbit.argp, orbit.true_anomaly, 'arglat', 0.0);
452     disp(norm(p_0))
453     x0 = [p_0; v_0; omega_0; eta_0];
454     M(i, :) = external_torque(x0, spacecraft, orbit, options)';
455 end
456
457 figure
458 scatter(radius, vecnorm(M, 2, 2));
459 ylabel('Torque M-{gg} [Nm]');
460 xlabel('Orbit radius [m]');
461 grid on
462
463 %% Gravity Gradient Torque when principal axes aligned with RTN
464 orbit.e = 0.0; % Consider circular orbit
465
466 % Initial conditions
467 [p_0, v_0] = keplerian2ijk(orbit.a, orbit.e, orbit.incl, orbit.RAAN, ...
468     orbit.argp, orbit.true_anomaly, 'arglat', 0.0);
469
470 n_hat = cross(p_0/norm(p_0), v_0/norm(v_0));
471 r_hat = p_0/norm(p_0);
472 t_hat = cross(n_hat, r_hat);
473 Roi = [r_hat t_hat n_hat]';
474 eta_0 = [atan2(Roi(2, 3), Roi(3, 3));
475     -asin(Roi(1, 3));
476     atan2(Roi(1, 2), Roi(1, 1))]; % align prinicpal axes with RTN
477
478 n = sqrt(orbit.GM / orbit.a^3); % mean motion
479 omega_0 = [0; 0; n];
480
481 x0 = [p_0; v_0; omega_0; eta_0];
482
483 % Numerical integration
484 options.attitude_parameterization = "euler_angles";
485 options.rotor = false;
486
487 [t, x] = ode45(equations_of_motion(spacecraft, orbit, options), t, x0, ...
488     opts);
489 pos = x(:, 1:3);
490 vel = x(:, 4:6);

```

```

489 omega = x(:, 7:9);
490 eta = x(:, 10:end);
491
492
493 % Reconstruct gravity gradient torque
494 M_vec = zeros(size(pos));
495 for i=1:length(x)
496     M_vec(i, :) = external_torque(x(i, :)', spacecraft, orbit, options);
497 end
498
499 % Angular Velocity
500 figure
501 subplot(1,3,1);
502 plot(t, omega(:,1)*180/pi);
503 grid on;
504 xlabel('Time [s]')
505 ylabel('Roll Rate [deg/s]')
506 xlim([0 tf])
507 ylim([-0.01 0.07]);
508
509 subplot(1,3,2);
510 plot(t, omega(:,2)*180/pi);
511 grid on;
512 xlabel('Time [s]')
513 ylabel('Pitch Rate [deg/s]')
514 xlim([0 tf])
515 ylim([-0.01 0.07]);
516
517 subplot(1,3,3);
518 plot(t, omega(:,3)*180/pi);
519 grid on;
520 xlabel('Time [s]')
521 ylabel('Yaw Rate [deg/s]')
522 xlim([0 tf])
523 ylim([0 omega_0(3)*180/pi*1.1])
524
525 % Gravity Gradient Torque
526 figure
527 subplot(1,3,1);
528 plot(t, M_vec(:, 1));
529 grid on;
530 xlabel('Time [s]')
531 ylabel('Torque along principal x [Nm]')
532 xlim([0 tf])
533
534 subplot(1,3,2);
535 plot(t, M_vec(:, 2));
536 grid on;
537 xlabel('Time [s]')
538 ylabel('Torque along principal y [Nm]')
539 xlim([0 tf])
540
541 subplot(1,3,3);
542 plot(t, M_vec(:, 3));
543 grid on;
544 xlabel('Time [s]')
545 ylabel('Torque along principal z [Nm]')
546 xlim([0 tf])
547

```

```

548
549 %% Gravity Gradient Torque from arbitrary initial condition
550 run("spacecraft_properties.m");
551 run("orbit_properties.m");
552 tf = 3 * (2*pi*sqrt(orbit.a^3/orbit.GM));
553 t = 0:dt:tf;
554
555 % Initial conditions
556 n_hat = cross(p_0/norm(p_0), v_0/norm(v_0));
557 r_hat = p_0/norm(p_0);
558 t_hat = cross(n_hat, r_hat);
559 Roi = [r_hat t_hat n_hat]';
560 Rbo = euler2rotm([0 deg2rad(35) 0]) * [0 0 1; 0 -1 0; 1 0 0];
561 Rib = (Rbo * Roi)';
562 Rbi = Rib';
563 Rio = Roi';
564 Rip = Rib * spacecraft.Rpb';
565 Rpi = Rip';
566
567 eta_b = [atan2(Rbi(2, 3), Rbi(3, 3)); -asin(Rbi(1, 3)); atan2(Rbi(1, ...
    2), Rbi(1, 1))];
568 eta_0 = [atan2(Rpi(2, 3), Rpi(3, 3)); -asin(Rpi(1, 3)); atan2(Rpi(1, ...
    2), Rpi(1, 1))];
569 omega_0 = [0; 0; 0];
570
571 x0 = [p_0; v_0; omega_0; eta_0];
572
573 figure
574 [TR, S] = discretize_sc([0, 0, 0], eta_b, 0, spacecraft);
575
576 % Spacecraft geometry
577 trimesh(TR.ConnectivityList, TR.Points(:,1), TR.Points(:,2), ...
    TR.Points(:,3), "FaceColor", "none")
578 hold on
579 % Plot orbital frame
580 plot3([0 Rio(1,1)], [0 Rio(2,1)], [0 Rio(3,1)], "r-^"); % x-axis
581 plot3([0 Rio(1,2)], [0 Rio(2,2)], [0 Rio(3,2)], "g-^"); % y-axis
582 plot3([0 Rio(1,3)], [0 Rio(2,3)], [0 Rio(3,3)], "b-^"); % z-axis
583
584 % Plot body axes frame
585 plot3([0 Rib(1,1)], [0 Rib(2,1)], [0 Rib(3,1)], "y-^"); % x-axis
586 plot3([0 Rib(1,2)], [0 Rib(2,2)], [0 Rib(3,2)], "m-^"); % y-axis
587 plot3([0 Rib(1,3)], [0 Rib(2,3)], [0 Rib(3,3)], "k-^"); % z-axis
588
589 % Plot principal axes frame
590 plot3([0 Rip(1,1)], [0 Rip(2,1)], [0 Rip(3,1)], "c-^"); % x-axis
591 plot3([0 Rip(1,2)], [0 Rip(2,2)], [0 Rip(3,2)], "c-^"); % y-axis
592 plot3([0 Rip(1,3)], [0 Rip(2,3)], [0 Rip(3,3)], "c-^"); % z-axis
593
594 legend(' ', 'Radial', 'Transverse', 'Normal', 'x', 'y', 'z');
595 grid on
596 axis equal
597
598 % Numerical integration
599 options.attitude_parameterization = "euler_angles";
600 options.rotor = false;
601
602 [t, x] = ode45(equations_of_motion(spacecraft, orbit, options), t, x0, ...
    opts);

```

```

603 pos = x(:, 1:3);
604 vel = x(:, 4:6);
605 omega = x(:, 7:9);
606 eta = x(:, 10:end);
607
608 % Reconstruct gravity gradient torque
609 M_vec = zeros(size(pos));
610 for i=1:length(x)
611     M_vec(i, :) = external_torque(x(i, :)', spacecraft, orbit, options);
612 end
613
614 % Angular Velocity
615 figure
616 subplot(1,3,1);
617 plot(t, omega(:,1)*180/pi);
618 grid on;
619 xlabel('Time [s]')
620 ylabel('Roll Rate [deg/s]')
621 xlim([0 tf])
622
623 subplot(1,3,2);
624 plot(t, omega(:,2)*180/pi);
625 grid on;
626 xlabel('Time [s]')
627 ylabel('Pitch Rate [deg/s]')
628 xlim([0 tf])
629
630 subplot(1,3,3);
631 plot(t, omega(:,3)*180/pi);
632 grid on;
633 xlabel('Time [s]')
634 ylabel('Yaw Rate [deg/s]')
635 xlim([0 tf])
636
637 % Plot attitude
638 figure
639 subplot(1,3,1);
640 plot(t, eta(:,1)*180/pi);
641 grid on;
642 xlabel('Time [s]')
643 ylabel('Roll Angle [deg]')
644 xlim([0 tf])
645
646 subplot(1,3,2);
647 plot(t, eta(:,2)*180/pi);
648 grid on;
649 xlabel('Time [s]')
650 ylabel('Pitch Angle [deg]')
651 xlim([0 tf])
652
653 subplot(1,3,3);
654 plot(t, eta(:,3)*180/pi);
655 grid on;
656 xlabel('Time [s]')
657 ylabel('Yaw Angle [deg]')
658 xlim([0 tf])
659
660 % Gravity Gradient Torque
661 figure

```

```

662 subplot(1,3,1);
663 plot(t, M_vec(:, 1));
664 grid on;
665 xlabel('Time [s]')
666 ylabel('Torque along principal x [Nm]')
667 xlim([0 tf])
668
669 subplot(1,3,2);
670 plot(t, M_vec(:, 2));
671 grid on;
672 xlabel('Time [s]')
673 ylabel('Torque along principal y [Nm]')
674 xlim([0 tf])
675
676 subplot(1,3,3);
677 plot(t, M_vec(:, 3));
678 grid on;
679 xlabel('Time [s]')
680 ylabel('Torque along principal z [Nm]')
681 xlim([0 tf])

```

13.6 Problem Set 5 Main File

```

1  %% Spacecraft properties
2  run("spacecraft_properties.m");
3  run("orbit_properties.m");
4
5  %% Plot stability region for gravity gradient
6
7  % Unstable pitch
8  spacecraft.Ip(1,1) = 0.9*spacecraft.Ip(1,1);
9  spacecraft.Ip(2,2) = 0.6*spacecraft.Ip(2,2);
10 spacecraft.Ip(3,3) = 0.6*spacecraft.Ip(3,3);
11 % Add unstable yaw and roll
12 spacecraft.Ip(3,3) = 0.3*spacecraft.Ip(3,3);
13 kN = (spacecraft.Ip(2,2) - spacecraft.Ip(1,1)) / spacecraft.Ip(3,3);
14 kT = (spacecraft.Ip(3,3) - spacecraft.Ip(1,1)) / spacecraft.Ip(2,2);
15 kR = (spacecraft.Ip(3,3) - spacecraft.Ip(2,2)) / spacecraft.Ip(1,1);
16
17 kt = -1:0.1:1;
18 kr=-1:0.1:1;
19
20 figure
21 % Unstable pitch
22 fill([kt, fliplr(kt)], [kr, fliplr(ones(size(kr)))], 'y')
23 hold on
24
25 % Unstable yaw and roll
26 x = 0.:0.1:1.;
27 fill([x, fliplr(x)], [zeros(size(x)), fliplr(-ones(size(x)))], 'b')
28
29 % Unstable yaw, roll and pitch
30 x = -1.:0.1:0.;
31 fill([x, fliplr(x)], [zeros(size(x)), fliplr(ones(size(x)))], 'g')
32
33 x = -1.:0.001:0.;
34 y = zeros(size(x));

```

```

35 for i=1:length(x)
36     fun = @(kr) (1+3*x(i)+x(i)*kr)^2-16*kr*x(i);
37     y(i) = fsolve(fun, x(i));
38 end
39 fill([x, fliplr(x)], [y, fliplr(-ones(size(x)))], 'b')
40
41 x_0 = fsolve(@(x) (1+3*x+x^2)^2-16*x^2, 0.);
42 x = -1.:0.001:x_0;
43 y1 = y(1:length(x));
44 y2 = x;
45 fill([x, fliplr(x)], [y1, fliplr(y2)], 'g')
46
47 % Plot spacecraft
48 scatter(kT, kR, 'r')
49
50
51 legend('Unstable Pitch', 'Unstable yaw and roll', 'Unstable yaw, roll ...
    and pitch', '', '', 'Spacecraft')
52 axis equal
53 xlim([-1, 1])
54 ylim([-1, 1])
55 xlabel('K_T')
56 ylabel('K_R')
57
58 %% Numerical integration conditions
59 dt = 1;
60 tf = 5 * 2*pi*sqrt(orbit.a^3/orbit.GM);
61 t = 0:dt:tf;
62 opts = odeset('RelTol',1e-13,'AbsTol',1e-14,'Stats','on');
63
64 %% Stability
65 % Initial conditions
66 n = sqrt(orbit.GM / orbit.a^3); % mean motion
67 perturbation = deg2rad(n*1e-2) * [1; 1; 1]; % [a-x, a-y, a-z]
68 A = [1 perturbation(3) -perturbation(2);
69     -perturbation(3) 1 perturbation(1);
70     perturbation(2) -perturbation(1) 1];
71
72 n_hat = cross(p_0/norm(p_0), v_0/norm(v_0));
73 r_hat = p_0/norm(p_0);
74 t_hat = cross(n_hat, r_hat);
75 Roi = A * [r_hat t_hat n_hat]';
76 eta_0 = [atan2(Roi(2, 3), Roi(3, 3));
77     -asin(Roi(1, 3));
78     atan2(Roi(1, 2), Roi(1, 1))]; % align prinicpal axes ...
    with RTN
79 omega_0 = A * [0; 0; n];
80 x0 = [p_0; v_0; omega_0; eta_0; x0_earth];
81
82 % Numerical integration
83 options.attitude_parameterization = "euler_angles";
84 options.rotor = false;
85 options.external_torque = true;
86
87 [t, x] = ode45(equations_of_motion(spacecraft, orbit, options), t, x0, ...
    opts);
88 pos = x(:, 1:3);
89 vel = x(:, 4:6);
90 omega = x(:, 7:9);

```

```

91 eta = x(:, 10:12);
92
93 eta_o = zeros(size(eta));
94 for i=1:length(eta)
95     n_hat = cross(pos(i, :)/norm(pos(i, :)), vel(i, :)/norm(vel(i, :)));
96     r_hat = pos(i, :)/norm(pos(i, :));
97     t_hat = cross(n_hat, r_hat);
98
99     Roi = [r_hat' t_hat' n_hat'];
100     Rip = euler2rotm(eta(i, :));
101     Rop = Roi * Rip;
102
103     eta_o(i, :) = [atan2(Rop(2, 3), Rop(3, 3));
104                   -asin(Rop(1, 3));
105                   atan2(Rop(1, 2), Rop(1, 1))];
106 end
107 eta=unwrap(eta_o);
108
109 % Angular Velocity
110 figure
111 subplot(1,3,1);
112 plot(t, omega(:,1)*180/pi);
113 grid on;
114 xlabel('Time [s]')
115 ylabel('Rotation Rate about R [deg/s]')
116 xlim([0 tf])
117
118 subplot(1,3,2);
119 plot(t, omega(:,2)*180/pi);
120 grid on;
121 xlabel('Time [s]')
122 ylabel('Rotation Rate about T [deg/s]')
123 xlim([0 tf])
124
125 subplot(1,3,3);
126 plot(t, omega(:,3)*180/pi);
127 grid on;
128 xlabel('Time [s]')
129 ylabel('Rotation Rate about N [deg/s]')
130 xlim([0 tf])
131 ylim([0 1.1*max(omega(:, 3))*180/pi])
132
133 % Plot Euler angle
134 figure
135 subplot(1,3,1);
136 plot(t, eta(:,1)*180/pi);
137 grid on;
138 xlabel('Time [s]')
139 ylabel('Roll Angle [deg]')
140 xlim([0 tf])
141
142 subplot(1,3,2);
143 plot(t, eta(:,2)*180/pi);
144 grid on;
145 xlabel('Time [s]')
146 ylabel('Yaw Angle [deg]')
147 xlim([0 tf])
148
149 subplot(1,3,3);

```

```

150 plot(t, eta(:,3)*180/pi);
151 grid on;
152 xlabel('Time [s]')
153 ylabel('Pitch Angle [deg]')
154 xlim([0 tf])
155
156 %% Sun model
157 tf = 24 * 3600;
158 t = 0:tf;
159 x_sun = sun_orbit(t);
160
161 figure
162 subplot(1,3, 1)
163 plot(t, x_sun(:, 7)*180/pi)
164 xlabel('Time [s]')
165 ylabel('Roll angle [deg]')
166 grid on
167 xlim([0 tf])
168 subplot(1,3, 2)
169 plot(t, x_sun(:, 8)*180/pi)
170 xlabel('Time [s]')
171 ylabel('Pitch angle [deg]')
172 grid on
173 xlim([0 tf])
174 subplot(1,3, 3)
175 plot(t, x_sun(:, 9)*180/pi)
176 xlabel('Time [s]')
177 ylabel('Yaw angle [deg]')
178 grid on
179 xlim([0 tf])
180
181 figure
182 subplot(1,3, 1)
183 plot(t, x_sun(:, 10)*180/pi)
184 xlabel('Time [s]')
185 ylabel('Roll rate [deg/s]')
186 grid on
187 xlim([0 tf])
188 subplot(1,3, 2)
189 plot(t, x_sun(:, 11)*180/pi)
190 xlabel('Time [s]')
191 ylabel('Pitch rate [deg/s]')
192 grid on
193 xlim([0 tf])
194 subplot(1,3, 3)
195 plot(t, x_sun(:, 12)*180/pi)
196 xlabel('Time [s]')
197 ylabel('Yaw rate [deg/s]')
198 grid on
199 ylim([0 x_sun(1, 12)*180/pi*1.1])
200 xlim([0 tf])
201 %% Simulate satellite trajectory used to plot distrurbance torques
202
203 % Initial conditions
204 n_hat = cross(p_0/norm(p_0), v_0/norm(v_0));
205 r_hat = p_0/norm(p_0);
206 t_hat = cross(n_hat, r_hat);
207 Roi = [r_hat t_hat n_hat]';
208 Rbo = euler2rotm([0 deg2rad(35) 0]) * [0 0 1; 0 -1 0; 1 0 0];

```

```

209 Rib = (Rbo * Roi)';
210 Rbi = Rib';
211 Rip = Rib * spacecraft.Rpb';
212 Rpi = Rip';
213
214 eta_b = [atan2(Rbi(2, 3), Rbi(3, 3)); -asin(Rbi(1, 3)); atan2(Rbi(1, ...
    2), Rbi(1, 1))];
215 eta_0 = [atan2(Rpi(2, 3), Rpi(3, 3)); -asin(Rpi(1, 3)); atan2(Rpi(1, ...
    2), Rpi(1, 1))];
216 omega_0 = [0; 0; 0];
217
218 x0 = [p_0; v_0; omega_0; eta_0; x0_earth];
219
220 % Numerical integration
221 options.attitude.parameterization = "euler_angles";
222 options.rotor = false;
223 options.external_torque = true;
224
225 [t, x] = ode45(equations_of_motion(spacecraft, orbit, options), t, x0, ...
    opts);
226
227 %% ===== Disturbance Torques ===== %%
228 %% Magnetic Field Torque
229 Mm_max = spacecraft.mag.m * (orbit.B0 * orbit.Re^3/orbit.a^3 * 2);
230
231 % Reconstruct magnetic field torque
232 Mm_vec = zeros(length(x), 3);
233 for i=1:length(x)
234     Mm_vec(i, :) = magnetic_field_torque(x(i, :)', spacecraft, orbit, ...
        options);
235 end
236
237 figure
238 subplot(1,4,1);
239 plot(t, Mm_vec(:, 1));
240 grid on;
241 xlabel('Time [s]')
242 ylabel('Torque along principal x [Nm]')
243 xlim([0 tf])
244
245 subplot(1,4,2);
246 plot(t, Mm_vec(:, 2));
247 grid on;
248 xlabel('Time [s]')
249 ylabel('Torque along principal y [Nm]')
250 xlim([0 tf])
251
252 subplot(1,4,3);
253 plot(t, Mm_vec(:, 3));
254 grid on;
255 xlabel('Time [s]')
256 ylabel('Torque along principal z [Nm]')
257 xlim([0 tf])
258
259 subplot(1,4,4);
260 plot(t, vecnorm(Mm_vec, 2, 2));
261 hold on
262 plot(t, Mm_max*ones(size(Mm_vec)));
263 grid on;

```

```

264 xlabel('Time [s]')
265 ylabel('Torque Magnitude [Nm]')
266 xlim([0 tf])
267
268 %% Solar Radiation Torque
269 As = 14.5 + 1.9*1.74 + 2*1.5; % Solar arrays + bus + payload [m^2]
270 Ms_max = orbit.flux*As*(1+spacecraft.solar.Cs+spacecraft.solar.Cd) * 0.15;
271
272 % Reconstruct solar radiation torque
273 Ms_vec = zeros(length(x), 3);
274 for i=1:length(x)
275     Ms_vec(i, :) = solar_radiation_torque(x(i, :)', spacecraft, orbit, ...
276         options);
277 end
278 figure
279 subplot(1,4,1);
280 plot(t, Ms_vec(:, 1));
281 grid on;
282 xlabel('Time [s]')
283 ylabel('Torque along principal x [Nm]')
284 xlim([0 tf])
285
286 subplot(1,4,2);
287 plot(t, Ms_vec(:, 2));
288 grid on;
289 xlabel('Time [s]')
290 ylabel('Torque along principal y [Nm]')
291 xlim([0 tf])
292
293 subplot(1,4,3);
294 plot(t, Ms_vec(:, 3));
295 grid on;
296 xlabel('Time [s]')
297 ylabel('Torque along principal z [Nm]')
298 xlim([0 tf])
299
300 subplot(1,4,4);
301 plot(t, vecnorm(Ms_vec, 2, 2));
302 hold on
303 plot(t, Ms_max*ones(size(Ms_vec)));
304 grid on;
305 xlabel('Time [s]')
306 ylabel('Torque Magnitude [Nm]')
307 xlim([0 tf])
308
309
310 %% Atmospheric Drag Torque
311 A_total = sum(spacecraft.S);
312 Vmean = sqrt(orbit.GM / orbit.a);
313 Md_max = 1/2*orbit.density*Vmean^2*A_total*spacecraft.aero.Cd * 0.15;
314
315 % Reconstruct atmospheric drag torque
316 Md_vec = zeros(length(x), 3);
317 for i=1:length(x)
318     Md_vec(i, :) = drag_torque(x(i, :)', spacecraft, orbit, options);
319 end
320
321 figure

```

```

322 subplot(1,4,1);
323 plot(t, Md_vec(:, 1));
324 grid on;
325 xlabel('Time [s]')
326 ylabel('Torque along principal x [Nm]')
327 xlim([0 tf])
328
329 subplot(1,4,2);
330 plot(t, Md_vec(:, 2));
331 grid on;
332 xlabel('Time [s]')
333 ylabel('Torque along principal y [Nm]')
334 xlim([0 tf])
335
336 subplot(1,4,3);
337 plot(t, Md_vec(:, 3));
338 grid on;
339 xlabel('Time [s]')
340 ylabel('Torque along principal z [Nm]')
341 xlim([0 tf])
342
343 subplot(1,4,4);
344 plot(t, vecnorm(Md_vec, 2, 2));
345 hold on
346 plot(t, Md_max*ones(size(Md_vec)));
347 grid on;
348 xlabel('Time [s]')
349 ylabel('Torque Magnitude [Nm]')
350 xlim([0 tf])
351
352 %% Gravity Gradient Torque
353 Mgg_max = ...
           3/2*orbit.GM/orbit.a^3*abs(spacecraft.Ip(3,3)-spacecraft.Ip(1,1));
354
355 % Reconstruct gravity gradient torque
356 Mgg_vec = zeros(length(x), 3);
357 for i=1:length(x)
358     Mgg_vec(i, :) = gravity-gradient-torque(x(i, :)', spacecraft, ...
           orbit, options);
359 end
360
361 figure
362 subplot(1,4,1);
363 plot(t, Mgg_vec(:, 1));
364 grid on;
365 xlabel('Time [s]')
366 ylabel('Torque along principal x [Nm]')
367 xlim([0 tf])
368
369 subplot(1,4,2);
370 plot(t, Mgg_vec(:, 2));
371 grid on;
372 xlabel('Time [s]')
373 ylabel('Torque along principal y [Nm]')
374 xlim([0 tf])
375
376 subplot(1,4,3);
377 plot(t, Mgg_vec(:, 3));
378 grid on;

```

```

379 xlabel('Time [s]')
380 ylabel('Torque along principal z [Nm]')
381 xlim([0 tf])
382
383 subplot(1,4,4);
384 plot(t, vecnorm(Mgg_vec, 2, 2));
385 hold on
386 plot(t, Mgg_max*ones(size(Mgg_vec)));
387 grid on;
388 xlabel('Time [s]')
389 ylabel('Torque Magnitude [Nm]')
390 xlim([0 tf])
391
392
393 %% Plot all disturbance torques
394
395 % Reconstruct disturbance torque
396 M_vec = zeros(length(x), 3);
397 for i=1:length(x)
398     M_vec(i, :) = external_torque(x(i, :)', spacecraft, orbit, options);
399 end
400
401 figure
402 subplot(1,3,1)
403 plot(t , M_vec(:, 1))
404 hold on
405 plot(t , Mgg_vec(:, 1))
406 plot(t , Md_vec(:, 1))
407 plot(t , Ms_vec(:, 1))
408 plot(t , Mm_vec(:, 1))
409 grid on;
410 xlabel('Time [s]')
411 ylabel('Torque along principal x [Nm]')
412 xlim([0 tf])
413 legend('Resultant Torque', 'Gravity Gradient Torque', 'Atmospheric Drag ...
        Torque', ...
        'Solar Radiation Torque', 'Magnetic Field Torque')
414
415
416 subplot(1,3,2)
417 plot(t , M_vec(:, 2))
418 hold on
419 plot(t , Mgg_vec(:, 2))
420 plot(t , Md_vec(:, 2))
421 plot(t , Ms_vec(:, 2))
422 plot(t , Mm_vec(:, 2))
423 grid on;
424 xlabel('Time [s]')
425 ylabel('Torque along principal y [Nm]')
426 xlim([0 tf])
427
428 subplot(1,3,3)
429 plot(t , M_vec(:, 3))
430 hold on
431 plot(t , Mgg_vec(:, 3))
432 plot(t , Md_vec(:, 3))
433 plot(t , Ms_vec(:, 3))
434 plot(t , Mm_vec(:, 3))
435 grid on;
436 xlabel('Time [s]')

```

```

437 ylabel('Torque along principal z [Nm]')
438 xlim([0 tf])
439
440 figure
441 plot(t , vecnorm(M_vec, 2, 2))
442 hold on
443 plot(t , vecnorm(Mgg_vec, 2, 2))
444 plot(t , vecnorm(Md_vec, 2, 2))
445 plot(t , vecnorm(Ms_vec, 2, 2))
446 plot(t , vecnorm(Mm_vec, 2, 2))
447 grid on;
448 xlabel('Time [s]')
449 ylabel('Torque Magnitude [Nm]')
450 xlim([0 tf])
451 legend('Resultant Torque', 'Gravity Gradient Torque', 'Atmospheric Drag ...
         Torque', ...
452         'Solar Radiation Torque', 'Magnetic Field Torque')

```

13.7 Problem Set 6 Main File

```

1 close all;
2 clear;
3 clc;
4
5 %% Spacecraft properties
6 run("spacecraft_properties.m");
7 run("orbit_properties.m");
8
9 %% Numerical integration conditions
10 dt = 1;
11 tf = (2*pi*sqrt(orbit.a^3/orbit.GM));
12 t = 0:dt:tf;
13 opts = odeset('RelTol',1e-13,'AbsTol',1e-14,'Stats','on');
14
15 %% Simulate
16 % Initial conditions
17 n_hat = cross(p_0/norm(p_0), v_0/norm(v_0));
18 r_hat = p_0/norm(p_0);
19 t_hat = cross(n_hat, r_hat);
20 Roi = [r_hat t_hat n_hat]';
21 Rbo = euler2rotm([0 deg2rad(35) 0]) * [0 0 1; 0 -1 0; 1 0 0];
22 Rpi = spacecraft.Rpb * Rbo * Roi;
23 eta_0 = rotm2euler(Rpi);
24 omega_0 = [0; 0; 0];
25
26 x0 = [p_0; v_0; omega_0; eta_0; x0_earth];
27
28 % Numerical integration
29 options.attitude_parameterization = "euler_angles";
30 options.rotor = false;
31 options.external_torque = true;
32
33 [t, x] = ode45(equations_of_motion(spacecraft, orbit, options), t, x0, ...
               opts);
34 pos = x(:, 1:3);
35 vel = x(:, 4:6);
36 omega = x(:, 7:9);

```

```

37 eta = x(:, 10:12);
38
39 %% Control error
40
41 eta_d = zeros(size(eta));
42 eta_err = zeros(size(eta));
43 eta_err_1 = zeros(length(eta), 4);
44 for i=1:length(eta_d)
45     n_hat = cross(pos(i, :)/norm(pos(i, :)), vel(i, :)/norm(vel(i, :)));
46     r_hat = pos(i, :)/norm(pos(i, :));
47     t_hat = cross(n_hat, r_hat);
48
49     Roi = [r_hat' t_hat' n_hat'];
50     Rpi_d = spacecraft.Rpb * Rbo * Roi;
51     Rpi = euler2rotm(eta(i, :));
52
53     eta_d(i, :) = rotm2euler(Rpi_d);
54     eta_err(i, :) = rotm2euler(Rpi_d * Rpi');
55     eta_err_1(i, :) = rotm2axang(Rpi_d * Rpi');
56 end
57 eta_d = unwrap(eta_d);
58 eta_err = unwrap(eta_err);
59
60 % Plot desired and actual attitude angles
61 figure
62 subplot(1,3,1);
63 plot(t, eta(:,1)*180/pi, t, eta_d(:,1)*180/pi);
64 grid on;
65 xlabel('Time [s]')
66 ylabel('Roll Angle [deg]')
67 xlim([0 tf])
68
69 subplot(1,3,2);
70 plot(t, eta(:,2)*180/pi, t, eta_d(:,2)*180/pi);
71 grid on;
72 xlabel('Time [s]')
73 ylabel('Yaw Angle [deg]')
74 xlim([0 tf])
75
76 subplot(1,3,3);
77 plot(t, eta(:,3)*180/pi, t, eta_d(:,3)*180/pi);
78 grid on;
79 xlabel('Time [s]')
80 ylabel('Pitch Angle [deg]')
81 xlim([0 tf])
82 legend('Actual attitude', 'Desired attitude')
83
84 % Plot control error 1
85 figure
86 subplot(1,3,1);
87 plot(t, eta_err(:,1)*180/pi);
88 grid on;
89 xlabel('Time [s]')
90 ylabel('Roll Angle [deg]')
91 xlim([0 tf])
92
93 subplot(1,3,2);
94 plot(t, eta_err(:,2)*180/pi);
95 grid on;

```

```

96 xlabel('Time [s]')
97 ylabel('Yaw Angle [deg]')
98 xlim([0 tf])
99
100 subplot(1,3,3);
101 plot(t, eta_err(:,2)*180/pi);
102 grid on;
103 xlabel('Time [s]')
104 ylabel('Pitch Angle [deg]')
105 xlim([0 tf])
106
107 % Plot control error 2
108 figure
109 plot(t, eta_err_1(:, 4)*180/pi);
110 grid on;
111 xlabel('Time [s]')
112 ylabel('Axis Angle [deg]')
113 xlim([0 tf])
114
115
116 %% Simulink simulation
117 % Initial conditions
118 n_hat = cross(p_0/norm(p_0), v_0/norm(v_0));
119 r_hat = p_0/norm(p_0);
120 t_hat = cross(n_hat, r_hat);
121 Roi = [r_hat t_hat n_hat]';
122 Rbo = euler2rotm([0 deg2rad(35) 0]) * [0 0 1; 0 -1 0; 1 0 0];
123 Rpi = spacecraft.Rpb * Rbo * Roi;
124 eta_0 = rotm2euler(Rpi);
125 omega_0 = [0; 0; 0];
126
127 x0 = [p_0; v_0; eta_0; omega_0];
128 out = sim('simulation');
129
130 %% Get state estimation
131
132 t = out.true_att.Time;
133 det_att_est = out.det_att_est.Data;
134 stat_att_est = out.stat_att_est.Data;
135 int_att_est = wrapTol80(out.int_att_est.Data);
136 true_att = wrapTol80(out.true_att.Data);
137
138
139 % Plot deterministic estimate
140 figure
141 subplot(1,3,1);
142 plot(t, det_att_est(:,1), t, true_att(:, 1));
143 grid on;
144 xlabel('Time [s]')
145 ylabel('Roll Angle [deg]')
146 xlim([0 tf])
147
148 subplot(1,3,2);
149 plot(t, det_att_est(:,2), t, true_att(:, 2));
150 grid on;
151 xlabel('Time [s]')
152 ylabel('Pitch Angle [deg]')
153 xlim([0 tf])
154

```

```

155 subplot(1,3,3);
156 plot(t, det_att_est(:,3), t, true_att(:, 3));
157 grid on;
158 xlabel('Time [s]')
159 ylabel('Yaw Angle [deg]')
160 xlim([0 tf])
161 legend('Ground truth', 'Deterministic Estimate')
162
163
164 % Plot statistical estimate
165 figure
166 subplot(1,3,1);
167 plot(t, int_att_est(:,1), t, true_att(:, 1));
168 grid on;
169 xlabel('Time [s]')
170 ylabel('Roll Angle [deg]')
171 xlim([0 tf])
172
173 subplot(1,3,2);
174 plot(t, int_att_est(:,2), t, true_att(:, 2));
175 grid on;
176 xlabel('Time [s]')
177 ylabel('Pitch Angle [deg]')
178 xlim([0 tf])
179
180 subplot(1,3,3);
181 plot(t, int_att_est(:,3), t, true_att(:, 3));
182 grid on;
183 xlabel('Time [s]')
184 ylabel('Yaw Angle [deg]')
185 xlim([0 tf])
186 legend('Ground truth', 'Statistical Estimate')
187
188
189 % Plot integrated estimate
190 figure
191 subplot(1,3,1);
192 plot(t, stat_att_est(:,1), t, true_att(:, 1));
193 grid on;
194 xlabel('Time [s]')
195 ylabel('Roll Angle [deg]')
196 xlim([0 tf])
197
198 subplot(1,3,2);
199 plot(t, stat_att_est(:,2), t, true_att(:, 2));
200 grid on;
201 xlabel('Time [s]')
202 ylabel('Pitch Angle [deg]')
203 xlim([0 tf])
204
205 subplot(1,3,3);
206 plot(t, stat_att_est(:,3), t, true_att(:, 3));
207 grid on;
208 xlabel('Time [s]')
209 ylabel('Yaw Angle [deg]')
210 xlim([0 tf])
211 legend('Ground truth', 'Integrated Estimate')

```

13.8 Problem Set 7 Main File

```
1 close all;
2 clear;
3 clc;
4
5 %% Spacecraft properties
6 run("spacecraft_properties.m");
7 run("orbit_properties.m");
8
9 %% Simulink simulation
10 % Initial conditions
11 n_hat = cross(p_0/norm(p_0), v_0/norm(v_0));
12 r_hat = p_0/norm(p_0);
13 t_hat = cross(n_hat, r_hat);
14 Roi = [r_hat t_hat n_hat]';
15 Rbo = euler2rotm([0 deg2rad(35) 0]) * [0 0 1; 0 -1 0; 1 0 0];
16 Rpi = spacecraft.Rpb * Rbo * Roi;
17 eta_0 = rotm2euler(Rpi);
18 omega_0 = [deg2rad(1); deg2rad(1); deg2rad(1)];
19
20 dt = 0.1;
21 tf = 1/5*(2*pi*sqrt(orbit.a^3/orbit.GM));
22
23 x0 = [p_0; v_0; eta_0; omega_0];
24 out = sim('simulation');
25
26 %% Get state estimation
27
28 t = out.true_att.Time;
29 det_att_est = out.det_att_est.Data;
30 stat_att_est = out.stat_att_est.Data;
31 int_att_est = wrapTo180(out.int_att_est.Data);
32 true_att = wrapTo180(out.true_att.Data);
33
34
35 % Plot deterministic estimate
36 figure
37 subplot(1,3,1);
38 plot(t, det_att_est(:,1), t, true_att(:, 1));
39 grid on;
40 xlabel('Time [s]')
41 ylabel('Roll Angle [deg]')
42 xlim([0 tf])
43
44 subplot(1,3,2);
45 plot(t, det_att_est(:,2), t, true_att(:, 2));
46 grid on;
47 xlabel('Time [s]')
48 ylabel('Pitch Angle [deg]')
49 xlim([0 tf])
50
51 subplot(1,3,3);
52 plot(t, det_att_est(:,3), t, true_att(:, 3));
53 grid on;
54 xlabel('Time [s]')
55 ylabel('Yaw Angle [deg]')
56 xlim([0 tf])
```

```

57 legend('Deterministic Estimate', 'Ground truth')
58
59
60 % Plot statistical estimate
61 figure
62 subplot(1,3,1);
63 plot(t, stat_att_est(:,1), t, true_att(:, 1));
64 grid on;
65 xlabel('Time [s]')
66 ylabel('Roll Angle [deg]')
67 xlim([0 tf])
68
69 subplot(1,3,2);
70 plot(t, stat_att_est(:,2), t, true_att(:, 2));
71 grid on;
72 xlabel('Time [s]')
73 ylabel('Pitch Angle [deg]')
74 xlim([0 tf])
75
76 subplot(1,3,3);
77 plot(t, stat_att_est(:,3), t, true_att(:, 3));
78 grid on;
79 xlabel('Time [s]')
80 ylabel('Yaw Angle [deg]')
81 xlim([0 tf])
82 legend('Statistical Estimate', 'Ground truth')
83
84
85 % Plot integrated estimate
86 figure
87 subplot(1,3,1);
88 plot(t, int_att_est(:,1), t, true_att(:, 1));
89 grid on;
90 xlabel('Time [s]')
91 ylabel('Roll Angle [deg]')
92 xlim([0 tf])
93
94 subplot(1,3,2);
95 plot(t, int_att_est(:,2), t, true_att(:, 2));
96 grid on;
97 xlabel('Time [s]')
98 ylabel('Pitch Angle [deg]')
99 xlim([0 tf])
100
101 subplot(1,3,3);
102 plot(t, int_att_est(:,3), t, true_att(:, 3));
103 grid on;
104 xlabel('Time [s]')
105 ylabel('Yaw Angle [deg]')
106 xlim([0 tf])
107 legend('Integrated Estimate', 'Ground truth')
108
109
110 %% Error DCM
111
112
113 % Plot deterministic estimate
114 figure
115 subplot(1,3,1);

```

```

116 plot(t, det_att_est(:,1) - true_att(:, 1));
117 grid on;
118 xlabel('Time [s]')
119 ylabel('Roll Angle Error [deg]')
120 xlim([0 tf])
121 ylim([-15, 15])
122
123 subplot(1,3,2);
124 plot(t, det_att_est(:,2) - true_att(:, 2));
125 grid on;
126 xlabel('Time [s]')
127 ylabel('Pitch Angle Error [deg]')
128 xlim([0 tf])
129 ylim([-15, 15])
130
131 subplot(1,3,3);
132 plot(t, det_att_est(:,3) - true_att(:, 3));
133 grid on;
134 xlabel('Time [s]')
135 ylabel('Yaw Angle Error [deg]')
136 xlim([0 tf])
137 ylim([-15, 15])
138
139 cond = (det_att_est - true_att) < 10;
140 disp(sum(sum(cond))/(length(cond)*3))
141
142 cond = (det_att_est - true_att) < 5;
143 disp(sum(sum(cond))/(length(cond)*3))
144
145 cond = (det_att_est - true_att) < 1;
146 disp(sum(sum(cond))/(length(cond)*3))
147
148 %% State transition matrix
149
150 t = out.true_att.Time;
151 true_att = deg2rad(out.true_att.Data);
152 true_ang_vel = deg2rad(out.true_ang_vel.Data);
153
154 x_est = zeros(length(t), 6);
155 x_est(1,:) = [eta_0; omega_0];
156 eta_err = zeros(length(t), 4);
157 for i=1:length(t)
158     Rpi = euler2rotm(true_att(i, :));
159     Rpri = euler2rotm(x_est(i, 1:3));
160     Rppr = Rpi * Rpri'; % Propagated att to true att
161
162     eta_err(i, :) = rotm2axang(Rppr);
163     if i ~= length(t)
164         x_est(i+1, :) = discrete_linear(x_est(i,:), spacecraft.Ip, ...
            t(i+1)-t(i));
165     end
166 end
167
168 % Plot linearized propagated attitude with true attitude
169 figure
170 subplot(1,3,1);
171 plot(t, x_est(:,1)*180/pi, t, true_att(:, 1)*180/pi);
172 grid on;
173 xlabel('Time [s]')

```

```

174 ylabel('Roll Angle [deg]')
175 xlim([0 tf])
176
177 subplot(1,3,2);
178 plot(t, x_est(:,2)*180/pi, t, true_att(:, 2)*180/pi);
179 grid on;
180 xlabel('Time [s]')
181 ylabel('Pitch Angle [deg]')
182 xlim([0 tf])
183
184 subplot(1,3,3);
185 plot(t, x_est(:,3)*180/pi, t, true_att(:, 3)*180/pi);
186 grid on;
187 xlabel('Time [s]')
188 ylabel('Yaw Angle [deg]')
189 xlim([0 tf])
190 legend('Propagated', 'Ground truth')
191
192
193 % Plot linearized propagated angular velocity with true angular velocity
194 figure
195 subplot(1,3,1);
196 plot(t, x_est(:,4)*180/pi, t, true_ang_vel(:, 1)*180/pi);
197 grid on;
198 xlabel('Time [s]')
199 ylabel('Roll Rate [deg/s]')
200 xlim([0 tf])
201
202 subplot(1,3,2);
203 plot(t, x_est(:,5)*180/pi, t, true_ang_vel(:, 2)*180/pi);
204 grid on;
205 xlabel('Time [s]')
206 ylabel('Pitch Rate [deg/s]')
207 xlim([0 tf])
208
209 subplot(1,3,3);
210 plot(t, x_est(:,6)*180/pi, t, true_ang_vel(:, 3)*180/pi);
211 grid on;
212 xlabel('Time [s]')
213 ylabel('Yaw Rate [deg/s]')
214 xlim([0 tf])
215 legend('Propagated', 'Ground truth')
216
217 % Plot control error
218 figure
219 plot(t, eta_err(:, 4)*180/pi);
220 grid on;
221 xlabel('Time [s]')
222 ylabel('Axis Angle [deg]')
223 xlim([0 tf])
224
225
226 %% KF estimation
227
228 t = out.true_att.Time;
229 true_att = deg2rad(out.true_att.Data);
230 true_ang_vel = deg2rad(out.true_ang_vel.Data);
231 P = out.P.Data;
232 KF_est = out.KF_est.Data;

```

```

233 sigmas = zeros(length(t), 6);
234
235 for i=1:length(t)
236     sigmas(i, :) = sqrt(diag(P(:, :, i)));
237 end
238
239 % Plot KF estimation
240 % attitude Angles
241 figure
242 subplot(1,3,1);
243 hold on
244 plot(t, (KF_est(:,1) + sigmas(:, 1)) * 180/pi)
245 plot(t, KF_est(:,1)*180/pi, t, true_att(:, 1)*180/pi);
246 plot(t, (KF_est(:,1) - sigmas(:, 1)) * 180/pi)
247 hold off
248 grid on;
249 xlabel('Time [s]')
250 ylabel('Roll Angle [deg]')
251 xlim([0 tf])
252
253 subplot(1,3,2);
254 hold on
255 plot(t, (KF_est(:,2) + sigmas(:, 2)) * 180/pi)
256 plot(t, KF_est(:,2)*180/pi, t, true_att(:, 2)*180/pi);
257 plot(t, (KF_est(:,2) - sigmas(:, 2)) * 180/pi)
258 hold off
259 grid on;
260 xlabel('Time [s]')
261 ylabel('Pitch Angle [deg]')
262 xlim([0 tf])
263
264 subplot(1,3,3);
265 hold on
266 plot(t, (KF_est(:,3) + sigmas(:, 3)) * 180/pi)
267 plot(t, KF_est(:,3)*180/pi, t, true_att(:, 3)*180/pi);
268 plot(t, (KF_est(:,3) - sigmas(:, 3)) * 180/pi)
269 hold off
270 grid on;
271 xlabel('Time [s]')
272 ylabel('Yaw Angle [deg]')
273 xlim([0 tf])
274 legend('Upper Estimation Error', 'KF Estimate', 'Ground truth', 'Lower ...
        Estimation Error')
275
276 % Angular Velocity
277 figure
278 subplot(1,3,1);
279 hold on
280 plot(t, (KF_est(:,4) + sigmas(:, 4)) * 180/pi)
281 plot(t, KF_est(:,4)*180/pi, t, true_ang_vel(:, 1)*180/pi);
282 plot(t, (KF_est(:,4) - sigmas(:, 4)) * 180/pi)
283 hold off
284 grid on;
285 xlabel('Time [s]')
286 ylabel('Roll Rate [deg/s]')
287 xlim([0 tf])
288 ylim([-0.2 1.6])
289
290 subplot(1,3,2);

```

```

291 hold on
292 plot(t, (KF_est(:,5) + sigmas(:, 5)) * 180/pi)
293 plot(t, KF_est(:,5)*180/pi, t, true_ang_vel(:, 2)*180/pi);
294 plot(t, (KF_est(:,5) - sigmas(:, 5)) * 180/pi)
295 hold off
296 grid on;
297 xlabel('Time [s]')
298 ylabel('Pitch Rate [deg/s]')
299 xlim([0 tf])
300
301 subplot(1,3,3);
302 hold on
303 plot(t, (KF_est(:,6) + sigmas(:, 6)) * 180/pi)
304 plot(t, KF_est(:,6)*180/pi, t, true_ang_vel(:, 3)*180/pi);
305 plot(t, (KF_est(:,6) - sigmas(:, 6)) * 180/pi)
306 hold off
307 grid on;
308 xlabel('Time [s]')
309 ylabel('Yaw Rate [deg/s]')
310 xlim([0 tf])
311 legend('Upper Estimation Error', 'KF Estimate', 'Ground truth', 'Lower ...
    Estimation Error')
312
313
314
315 % Plot Errors
316 % attitude Angles
317 figure
318 subplot(1,3,1);
319 hold on
320 plot(t, sigmas(:, 1) * 180/pi)
321 plot(t, abs(KF_est(:,1) - true_att(:, 1))*180/pi);
322 hold off
323 grid on;
324 xlabel('Time [s]')
325 ylabel('Roll Angle Error [deg]')
326 xlim([0 tf])
327
328 subplot(1,3,2);
329 hold on
330 plot(t, sigmas(:, 2) * 180/pi)
331 plot(t, abs(KF_est(:,2) - true_att(:, 2))*180/pi);
332 hold off
333 grid on;
334 xlabel('Time [s]')
335 ylabel('Pitch Angle Error [deg]')
336 xlim([0 tf])
337
338 subplot(1,3,3);
339 hold on
340 plot(t, sigmas(:, 3) * 180/pi)
341 plot(t, abs(KF_est(:,3) - true_att(:, 3))*180/pi);
342 hold off
343 grid on;
344 xlabel('Time [s]')
345 ylabel('Yaw Angle Error [deg]')
346 xlim([0 tf])
347 legend('Formal Error', 'True Error')
348

```

```

349 % Angular Velocity
350 figure
351 subplot(1,3,1);
352 hold on
353 plot(t, sigmas(:, 4) * 180/pi)
354 plot(t, abs(KF_est(:,4) - true_ang_vel(:, 1))*180/pi);
355 hold off
356 grid on;
357 xlabel('Time [s]')
358 ylabel('Roll Rate Error [deg/s]')
359 xlim([0 tf])
360 ylim([-0.2 1.6])
361
362 subplot(1,3,2);
363 hold on
364 plot(t, sigmas(:, 5) * 180/pi)
365 plot(t, abs(KF_est(:,5) - true_ang_vel(:, 2))*180/pi);
366 hold off
367 grid on;
368 xlabel('Time [s]')
369 ylabel('Pitch Rate Error [deg/s]')
370 xlim([0 tf])
371
372 subplot(1,3,3);
373 hold on
374 plot(t, sigmas(:, 6) * 180/pi)
375 plot(t, abs(KF_est(:, 6) - true_ang_vel(:, 3))*180/pi);
376 hold off
377 grid on;
378 xlabel('Time [s]')
379 ylabel('Yaw Rate Error [deg/s]')
380 xlim([0 tf])
381 legend('Upper Estimation Error', 'KF Estimate', 'Ground truth', 'Lower ...
        Estimation Error')

```

13.9 Problem Set 8 Main File

```

1 close all;
2 clear;
3 clc;
4
5 %% Spacecraft properties
6 run("spacecraft_properties.m");
7 run("orbit_properties.m");
8
9 %% Simulink simulation
10 % Initial conditions
11 n_hat = cross(p_0/norm(p_0), v_0/norm(v_0));
12 r_hat = p_0/norm(p_0);
13 t_hat = cross(n_hat, r_hat);
14 Roi = [r_hat t_hat n_hat]';
15 Rbo = euler2rotm([0 deg2rad(35) 0]) * [0 0 1; 0 -1 0; 1 0 0];
16 Rpi = spacecraft.Rpb * Rbo * Roi;
17 eta_0 = rotm2euler(Rpi);
18 omega_0 = [deg2rad(1); deg2rad(1); deg2rad(1)];
19
20 dt = 0.1;

```

```

21 tf = 1/5*(2*pi*sqrt(orbit.a^3/orbit.GM));
22
23 x0 = [p_0; v_0; eta_0; omega_0];
24 out = sim('simulation');
25
26 %% Pre-fit residuals
27 t = out.tout;
28 pre_res_ang_vel = out.pre_res_ang_vel;
29 pre_res_earth = out.pre_res_earth;
30 pre_res_magnetic = out.pre_res_magnetic;
31 pre_res_sun = out.pre_res_sun;
32
33 % Angular velocity
34 figure
35 subplot(1,3,1);
36 plot(t, pre_res_ang_vel.Data(1, :) * 180/pi);
37 grid on;
38 xlabel('Time [s]')
39 ylabel('Roll Rate Residual [deg/s]')
40 xlim([0 tf])
41
42 subplot(1,3,2);
43 plot(t, pre_res_ang_vel.Data(2, :) * 180/pi);
44 grid on;
45 xlabel('Time [s]')
46 ylabel('Pitch Rate Residual [deg/s]')
47 xlim([0 tf])
48
49 subplot(1,3,3);
50 plot(t, pre_res_ang_vel.Data(3, :) * 180/pi);
51 grid on;
52 xlabel('Time [s]')
53 ylabel('Yaw rate Residual [deg/s]')
54 xlim([0 tf])
55
56 % Earth Direction
57 figure
58 subplot(1,3,1);
59 plot(t, pre_res_earth.Data(1, :));
60 grid on;
61 xlabel('Time [s]')
62 ylabel('X-Earth Residual [-]')
63 xlim([0 tf])
64
65 subplot(1,3,2);
66 plot(t, pre_res_earth.Data(2, :));
67 grid on;
68 xlabel('Time [s]')
69 ylabel('Y-Earth Residual [-]')
70 xlim([0 tf])
71
72 subplot(1,3,3);
73 plot(t, pre_res_earth.Data(3, :));
74 grid on;
75 xlabel('Time [s]')
76 ylabel('Z-Earth Residual [-]')
77 xlim([0 tf])
78
79 % Magnetic Field Direction

```

```

80 figure
81 subplot(1,3,1);
82 plot(t, pre_res.magnetic.Data(1, :));
83 grid on;
84 xlabel('Time [s]')
85 ylabel('X-Magnetic Field Residual [-]')
86 xlim([0 tf])
87
88 subplot(1,3,2);
89 plot(t, pre_res.magnetic.Data(2, :));
90 grid on;
91 xlabel('Time [s]')
92 ylabel('Y-Magnetic Field Residual [-]')
93 xlim([0 tf])
94
95 subplot(1,3,3);
96 plot(t, pre_res.magnetic.Data(3, :));
97 grid on;
98 xlabel('Time [s]')
99 ylabel('Z-Magnetic Field Residual [-]')
100 xlim([0 tf])
101
102 % Sun Direction
103 figure
104 subplot(1,3,1);
105 plot(t, pre_res.sun.Data(1, :));
106 grid on;
107 xlabel('Time [s]')
108 ylabel('X-Sun Residual [-]')
109 xlim([0 tf])
110
111 subplot(1,3,2);
112 plot(t, pre_res.sun.Data(2, :));
113 grid on;
114 xlabel('Time [s]')
115 ylabel('Y-Sun Residual [-]')
116 xlim([0 tf])
117
118 subplot(1,3,3);
119 plot(t, pre_res.sun.Data(3, :));
120 grid on;
121 xlabel('Time [s]')
122 ylabel('Z-Sun Residual [-]')
123 xlim([0 tf])
124
125 %% Post-fit residuals
126
127 post_res_ang_vel = out.post_res_ang_vel;
128 post_res_earth = out.post_res_earth;
129 post_res_magnetic = out.post_res_magnetic;
130 post_res_sun = out.post_res_sun;
131
132 % Angular velocity
133 figure
134 subplot(1,3,1);
135 hold on;
136 plot(t, 3 * sqrt(2.63e-3) * pi/180 * ones(size(t)), 'r')
137 plot(t, post_res_ang_vel.Data(1, :) * 180/pi, 'b');
138 plot(t, -3 * sqrt(2.63e-3) * pi/180 * ones(size(t)), 'r')

```

```

139 grid on;
140 xlabel('Time [s]')
141 ylabel('Roll Rate Residual [rad/s]')
142 xlim([0 tf])
143 legend({'3- $\sigma$  Confidence', 'Post-fit ...
        Residual'}, 'Interpreter', 'latex')
144
145 subplot(1,3,2);
146 hold on;
147 plot(t, 3 * sqrt(2.63e-3) * pi/180 * ones(size(t)), 'r')
148 plot(t, post_res_ang_vel.Data(2, :) * 180/pi, 'b');
149 plot(t, -3 * sqrt(2.63e-3) * pi/180 * ones(size(t)), 'r')
150 grid on;
151 xlabel('Time [s]')
152 ylabel('Pitch Rate Residual [rad/s]')
153 xlim([0 tf])
154
155 subplot(1,3,3);
156 hold on;
157 plot(t, 3 * sqrt(2.63e-3) * pi/180 * ones(size(t)), 'r')
158 plot(t, post_res_ang_vel.Data(3, :) * 180/pi, 'b');
159 plot(t, -3 * sqrt(2.63e-3) * pi/180 * ones(size(t)), 'r')
160 grid on;
161 xlabel('Time [s]')
162 ylabel('Yaw rate Residual [rad/s]')
163 xlim([0 tf])
164
165 % Earth Direction
166 figure
167 subplot(1,3,1);
168 hold on;
169 plot(t, 3 * sqrt(0.33) * pi/180 * ones(size(t)), 'r')
170 plot(t, post_res_earth.Data(1, :) * 180/pi, 'b');
171 plot(t, -3 * sqrt(0.33) * pi/180 * ones(size(t)), 'r')
172 grid on;
173 xlabel('Time [s]')
174 ylabel('X-Earth Residual [-]')
175 xlim([0 tf])
176 legend({'3- $\sigma$  Confidence', 'Post-fit ...
        Residual'}, 'Interpreter', 'latex')
177
178 subplot(1,3,2);
179 hold on;
180 plot(t, 3 * sqrt(0.33) * pi/180 * ones(size(t)), 'r')
181 plot(t, post_res_earth.Data(2, :) * 180/pi, 'b');
182 plot(t, -3 * sqrt(0.33) * pi/180 * ones(size(t)), 'r')
183 grid on;
184 xlabel('Time [s]')
185 ylabel('Y-Earth Residual [-]')
186 xlim([0 tf])
187
188 subplot(1,3,3);
189 hold on;
190 plot(t, 3 * sqrt(0.33) * pi/180 * ones(size(t)), 'r')
191 plot(t, post_res_earth.Data(3, :) * 180/pi, 'b');
192 plot(t, -3 * sqrt(0.33) * pi/180 * ones(size(t)), 'r')
193 grid on;
194 xlabel('Time [s]')
195 ylabel('Z-Earth Residual [-]')

```

```

196 xlim([0 tf])
197
198 % Magnetic Field Direction
199 figure
200 subplot(1,3,1);
201 hold on;
202 plot(t, 3 * sqrt(3.0) * pi/180 * ones(size(t)), 'r')
203 plot(t, post_res.magnetic.Data(1, :)', 'b');
204 plot(t, -3 * sqrt(3.0) * pi/180 * ones(size(t)), 'r')
205 grid on;
206 xlabel('Time [s]')
207 ylabel('X-Magnetic Field Residual [-]')
208 ylim([-0.15 0.15])
209 xlim([0 tf])
210 legend({'3-\sigma$ Confidence', 'Post-fit ...
        Residual'}, 'Interpreter', 'latex')
211
212 subplot(1,3,2);
213 hold on;
214 plot(t, 3 * sqrt(3.0) * pi/180 * ones(size(t)), 'r')
215 plot(t, post_res.magnetic.Data(2, :)', 'b');
216 plot(t, -3 * sqrt(3.0) * pi/180 * ones(size(t)), 'r')
217 grid on;
218 xlabel('Time [s]')
219 ylabel('Y-Magnetic Field Residual [-]')
220 ylim([-0.15 0.15])
221 xlim([0 tf])
222
223 subplot(1,3,3);
224 hold on
225 plot(t, 3 * sqrt(3.0) * pi/180 * ones(size(t)), 'r')
226 plot(t, post_res.magnetic.Data(3, :)', 'b');
227 plot(t, -3 * sqrt(3.0) * pi/180 * ones(size(t)), 'r')
228 grid on;
229 xlabel('Time [s]')
230 ylabel('Z-Magnetic Field Residual [-]')
231 ylim([-0.15 0.15])
232 xlim([0 tf])
233
234 % Sun Direction
235 figure
236 subplot(1,3,1);
237 hold on
238 plot(t, 3 * sqrt(0.1) * pi/180 * ones(size(t)), 'r')
239 plot(t, post_res.sun.Data(1, :)', 'b');
240 plot(t, -3 * sqrt(0.1) * pi/180 * ones(size(t)), 'r')
241 grid on;
242 xlabel('Time [s]')
243 ylabel('X-Sun Residual [-]')
244 ylim([-0.04 0.04])
245 xlim([0 tf])
246 legend({'3-\sigma$ Confidence', 'Post-fit ...
        Residual'}, 'Interpreter', 'latex')
247
248 subplot(1,3,2);
249 hold on
250 plot(t, 3 * sqrt(0.1) * pi/180 * ones(size(t)), 'r')
251 plot(t, post_res.sun.Data(2, :)', 'b');
252 plot(t, -3 * sqrt(0.1) * pi/180 * ones(size(t)), 'r')

```

```

253 grid on;
254 xlabel('Time [s]')
255 ylabel('Y-Sun Residual [-]')
256 ylim([-0.04 0.04])
257 xlim([0 tf])
258
259 subplot(1,3,3);
260 hold on
261 plot(t, 3 * sqrt(0.1) * pi/180 * ones(size(t)), 'r')
262 plot(t, post_res.sun.Data(3, :)', 'b');
263 plot(t, -3 * sqrt(0.1) * pi/180 * ones(size(t)), 'r')
264 grid on;
265 xlabel('Time [s]')
266 ylabel('Z-Sun Residual [-]')
267 ylim([-0.04 0.04])
268 xlim([0 tf])
269
270 disp("Angular velocity residuals:")
271 disp("Mean:")
272 disp(mean(post_res.ang_vel.Data(1, :)))
273 disp(mean(post_res.ang_vel.Data(2, :)))
274 disp(mean(post_res.ang_vel.Data(3, :)))
275 disp("Std:")
276 disp(std(post_res.ang_vel.Data(1, :)))
277 disp(std(post_res.ang_vel.Data(2, :)))
278 disp(std(post_res.ang_vel.Data(3, :)))
279
280 disp("Earth direction residuals:")
281 disp("Mean:")
282 disp(mean(post_res.earth.Data(1, :)))
283 disp(mean(post_res.earth.Data(2, :)))
284 disp(mean(post_res.earth.Data(3, :)))
285 disp("Std:")
286 disp(std(post_res.earth.Data(1, :)))
287 disp(std(post_res.earth.Data(2, :)))
288 disp(std(post_res.earth.Data(3, :)))
289
290 disp("Magnetic field direction residuals:")
291 disp("Mean:")
292 disp(mean(post_res.magnetic.Data(1, :)))
293 disp(mean(post_res.magnetic.Data(2, :)))
294 disp(mean(post_res.magnetic.Data(3, :)))
295 disp("Std:")
296 disp(std(post_res.magnetic.Data(1, :)))
297 disp(std(post_res.magnetic.Data(2, :)))
298 disp(std(post_res.magnetic.Data(3, :)))
299
300 disp("Sun direction residuals:")
301 disp("Mean:")
302 disp(mean(post_res.sun.Data(1, :)))
303 disp(mean(post_res.sun.Data(2, :)))
304 disp(mean(post_res.sun.Data(3, :)))
305 disp("Std:")
306 disp(std(post_res.sun.Data(1, :)))
307 disp(std(post_res.sun.Data(2, :)))
308 disp(std(post_res.sun.Data(3, :)))
309
310
311 %% KF estimation

```

```

312
313 t = out.true_att.Time;
314 true_att = deg2rad(out.true_att.Data);
315 true_ang_vel = deg2rad(out.true_ang_vel.Data);
316 P = out.P.Data;
317 KF_est = out.KF_est.Data;
318 sigmas = zeros(length(t), 6);
319
320 for i=1:length(t)
321     sigmas(i, :) = sqrt(diag(P(:, :, i)));
322 end
323
324 % Plot KF estimation
325 % attitude Angles
326 figure
327 subplot(1,3,1);
328 hold on
329 plot(t, (KF_est(:,1) + sigmas(:, 1)) * 180/pi)
330 plot(t, KF_est(:,1)*180/pi, t, true_att(:, 1)*180/pi);
331 plot(t, (KF_est(:,1) - sigmas(:, 1)) * 180/pi)
332 hold off
333 grid on;
334 xlabel('Time [s]')
335 ylabel('Roll Angle [deg]')
336 xlim([0 tf])
337
338 subplot(1,3,2);
339 hold on
340 plot(t, (KF_est(:,2) + sigmas(:, 2)) * 180/pi)
341 plot(t, KF_est(:,2)*180/pi, t, true_att(:, 2)*180/pi);
342 plot(t, (KF_est(:,2) - sigmas(:, 2)) * 180/pi)
343 hold off
344 grid on;
345 xlabel('Time [s]')
346 ylabel('Pitch Angle [deg]')
347 xlim([0 tf])
348
349 subplot(1,3,3);
350 hold on
351 plot(t, (KF_est(:,3) + sigmas(:, 3)) * 180/pi)
352 plot(t, KF_est(:,3)*180/pi, t, true_att(:, 3)*180/pi);
353 plot(t, (KF_est(:,3) - sigmas(:, 3)) * 180/pi)
354 hold off
355 grid on;
356 xlabel('Time [s]')
357 ylabel('Yaw Angle [deg]')
358 xlim([0 tf])
359 legend('Upper Estimation Error', 'KF Estimate', 'Ground truth', 'Lower ...
    Estimation Error')
360
361 % Angular Velocity
362 figure
363 subplot(1,3,1);
364 hold on
365 plot(t, (KF_est(:,4) + sigmas(:, 4)) * 180/pi)
366 plot(t, KF_est(:,4)*180/pi, t, true_ang_vel(:, 1)*180/pi);
367 plot(t, (KF_est(:,4) - sigmas(:, 4)) * 180/pi)
368 hold off
369 grid on;

```

```

370 xlabel('Time [s]')
371 ylabel('Roll Rate [deg/s]')
372 xlim([0 tf])
373 ylim([-0.2 1.6])
374
375 subplot(1,3,2);
376 hold on
377 plot(t, (KF_est(:,5) + sigmas(:, 5)) * 180/pi)
378 plot(t, KF_est(:,5)*180/pi, t, true_ang_vel(:, 2)*180/pi);
379 plot(t, (KF_est(:,5) - sigmas(:, 5)) * 180/pi)
380 hold off
381 grid on;
382 xlabel('Time [s]')
383 ylabel('Pitch Rate [deg/s]')
384 xlim([0 tf])
385
386 subplot(1,3,3);
387 hold on
388 plot(t, (KF_est(:,6) + sigmas(:, 6)) * 180/pi)
389 plot(t, KF_est(:,6)*180/pi, t, true_ang_vel(:, 3)*180/pi);
390 plot(t, (KF_est(:,6) - sigmas(:, 6)) * 180/pi)
391 hold off
392 grid on;
393 xlabel('Time [s]')
394 ylabel('Yaw Rate [deg/s]')
395 xlim([0 tf])
396 ylim([0.7 1.5])
397 legend('Upper Estimation Error', 'KF Estimate', 'Ground truth', 'Lower ...
        Estimation Error')
398
399
400
401 % Plot Errors
402 % attitude Angles
403 figure
404 subplot(1,3,1);
405 hold on
406 plot(t, sigmas(:, 1) * 180/pi)
407 plot(t, abs(KF_est(:,1) - true_att(:, 1))*180/pi);
408 hold off
409 grid on;
410 xlabel('Time [s]')
411 ylabel('Roll Angle Error [deg]')
412 xlim([0 tf])
413
414 subplot(1,3,2);
415 hold on
416 plot(t, sigmas(:, 2) * 180/pi)
417 plot(t, abs(KF_est(:,2) - true_att(:, 2))*180/pi);
418 hold off
419 grid on;
420 xlabel('Time [s]')
421 ylabel('Pitch Angle Error [deg]')
422 xlim([0 tf])
423
424 subplot(1,3,3);
425 hold on
426 plot(t, sigmas(:, 3) * 180/pi)
427 plot(t, abs(KF_est(:,3) - true_att(:, 3))*180/pi);

```

```

428 hold off
429 grid on;
430 xlabel('Time [s]')
431 ylabel('Yaw Angle Error [deg]')
432 xlim([0 tf])
433 legend('Formal Error', 'True Error')
434
435 % Angular Velocity
436 figure
437 subplot(1,3,1);
438 hold on
439 plot(t, sigmas(:, 4) * 180/pi)
440 plot(t, abs(KF_est(:,4) - true_ang_vel(:, 1))*180/pi);
441 hold off
442 grid on;
443 xlabel('Time [s]')
444 ylabel('Roll Rate Error [deg/s]')
445 xlim([0 tf])
446 ylim([-0.2 1.6])
447
448 subplot(1,3,2);
449 hold on
450 plot(t, sigmas(:, 5) * 180/pi)
451 plot(t, abs(KF_est(:,5) - true_ang_vel(:, 2))*180/pi);
452 hold off
453 grid on;
454 xlabel('Time [s]')
455 ylabel('Pitch Rate Error [deg/s]')
456 xlim([0 tf])
457
458 subplot(1,3,3);
459 hold on
460 plot(t, sigmas(:, 6) * 180/pi)
461 plot(t, abs(KF_est(:, 6) - true_ang_vel(:, 3))*180/pi);
462 hold off
463 grid on;
464 xlabel('Time [s]')
465 ylabel('Yaw Rate Error [deg/s]')
466 xlim([0 tf])
467 legend('Formal Error', 'True Error')
468
469
470 %% Estimation statistics
471 disp("Attitude angles:")
472 disp("Mean:")
473 disp(mean(KF_est(:,1)*180/pi - true_att(:, 1)*180/pi))
474 disp(mean(KF_est(:,2)*180/pi - true_att(:, 2)*180/pi))
475 disp(mean(KF_est(:,3)*180/pi - true_att(:, 3)*180/pi))
476 disp("Std:")
477 disp(std(KF_est(:,1)*180/pi - true_att(:, 1)*180/pi))
478 disp(std(KF_est(:,2)*180/pi - true_att(:, 2)*180/pi))
479 disp(std(KF_est(:,3)*180/pi - true_att(:, 3)*180/pi))
480
481 disp("Angular velocity:")
482 disp("Mean:")
483 disp(mean(KF_est(:,4)*180/pi - true_ang_vel(:, 1)*180/pi))
484 disp(mean(KF_est(:,5)*180/pi - true_ang_vel(:, 2)*180/pi))
485 disp(mean(KF_est(:,6)*180/pi - true_ang_vel(:, 3)*180/pi))
486 disp("Std:")

```

```

487 disp(std(KF_est(:,4)*180/pi - true_ang_vel(:, 1)*180/pi))
488 disp(std(KF_est(:,5)*180/pi - true_ang_vel(:, 2)*180/pi))
489 disp(std(KF_est(:,6)*180/pi - true_ang_vel(:, 3)*180/pi))

```

13.10 Problem Set 9 Main File

```

1  %% Spacecraft properties
2  run("spacecraft_properties.m");
3  run("orbit_properties.m");
4
5  %% Simulink simulation
6  % Initial conditions
7  n_hat = cross(p_0/norm(p_0), v_0/norm(v_0));
8  r_hat = p_0/norm(p_0);
9  t_hat = cross(n_hat, r_hat);
10 Roi = [r_hat t_hat n_hat]';
11 Rbo = euler2rotm([0 deg2rad(35) 0]) * [0 0 1; 0 -1 0; 1 0 0];
12 Rpi = spacecraft.Rpb * Rbo * Roi;
13 eta_0 = rotm2euler(Rpi);
14 omega_0 = [deg2rad(0); deg2rad(0); deg2rad(0)];
15
16 dt = 0.1;
17 tf = (2*pi*sqrt(orbit.a^3/orbit.GM));
18
19 x0 = [p_0; v_0; eta_0; omega_0];
20 out = sim('simulation');
21
22 %% Reaction Wheel Model
23
24 figure;
25
26 subplot(1, 3, 1)
27 plot(out.tout, out.Mc_out.Data(:, 1), out.tout, out.Mc_in.Data(:, 1))
28 grid on
29 ylim([0, 0.12])
30 xlim([0 tf])
31 legend('Actual Torque', 'Commanded Torque')
32 xlabel('Time [s]')
33 ylabel('Control Torque  $M_{c-x}$  [Mm]', 'Interpreter', 'latex')
34
35 subplot(1, 3, 2)
36 plot(out.tout, out.Mc_out.Data(:, 2), out.tout, out.Mc_in.Data(:, 2))
37 ylim([0, 0.12])
38 xlim([0 tf])
39 grid on
40 xlabel('Time [s]')
41 ylabel('Control Torque  $M_{c-y}$  [Mm]', 'Interpreter', 'latex')
42
43 subplot(1, 3, 3)
44 plot(out.tout, out.Mc_out.Data(:, 3), out.tout, out.Mc_in.Data(:, 3))
45 ylim([0, 0.12])
46 xlim([0 tf])
47 grid on
48 xlabel('Time [s]')
49 ylabel('Control Torque  $M_{c-z}$  [Mm]', 'Interpreter', 'latex')
50
51

```

```

52 figure;
53
54 subplot(1, 4, 1)
55 plot(out.tout, out.omega_w.Data(:, 1), out.tout, ...
      628.3*ones(size(out.tout)), out.tout, -628.3*ones(size(out.tout)))
56 grid on
57 xlim([0 tf])
58 ylim([-800 800])
59 xlabel('Time [s]')
60 ylabel('Angular Velocity  $\omega_1$  [rad/s]', 'Interpreter', 'latex')
61 legend('Wheel Angular Speed', 'Wheel Saturation')
62
63 subplot(1, 4, 2)
64 plot(out.tout, out.omega_w.Data(:, 2), out.tout, ...
      628.3*ones(size(out.tout)), out.tout, -628.3*ones(size(out.tout)))
65 xlim([0 tf])
66 ylim([-800 800])
67 grid on
68 xlabel('Time [s]')
69 ylabel('Angular Velocity  $\omega_2$  [rad/s]', 'Interpreter', 'latex')
70
71 subplot(1, 4, 3)
72 plot(out.tout, out.omega_w.Data(:, 3), out.tout, ...
      628.3*ones(size(out.tout)), out.tout, -628.3*ones(size(out.tout)))
73 xlim([0 tf])
74 ylim([-800 800])
75 grid on
76 xlabel('Time [s]')
77 ylabel('Angular Velocity  $\omega_3$  [rad/s]', 'Interpreter', 'latex')
78
79 subplot(1, 4, 4)
80 plot(out.tout, out.omega_w.Data(:, 4), out.tout, ...
      628.3*ones(size(out.tout)), out.tout, -628.3*ones(size(out.tout)))
81 xlim([0 tf])
82 ylim([-800 800])
83 grid on
84 xlabel('Time [s]')
85 ylabel('Angular Velocity  $\omega_4$  [rad/s]', 'Interpreter', 'latex')
86
87
88 figure;
89
90 subplot(1, 4, 1)
91 plot(out.tout, out.Mw.Data(:, 1), out.tout, 0.075*ones(size(out.tout)), ...
      out.tout, -0.075*ones(size(out.tout)))
92 grid on
93 xlim([0 tf])
94 xlabel('Time [s]')
95 ylabel('Motor Torque  $M_1$  [Nm]', 'Interpreter', 'latex')
96 legend('Wheel Angular Speed', 'Wheel Saturation')
97
98 subplot(1, 4, 2)
99 plot(out.tout, out.Mw.Data(:, 2), out.tout, 0.075*ones(size(out.tout)), ...
      out.tout, -0.075*ones(size(out.tout)))
100 xlim([0 tf])
101 grid on
102 xlabel('Time [s]')
103 ylabel('Motor Torque  $M_2$  [Nm]', 'Interpreter', 'latex')
104

```

```

105 subplot(1, 4, 3)
106 plot(out.tout, out.Mw.Data(:, 3), out.tout, 0.075*ones(size(out.tout)), ...
      out.tout, -0.075*ones(size(out.tout)))
107 xlim([0 tf])
108 grid on
109 xlabel('Time [s]')
110 ylabel('Motor Torque  $M_{w^3}$  [Nm]', 'Interpreter', 'latex')
111
112 subplot(1, 4, 4)
113 plot(out.tout, out.Mw.Data(:, 4), out.tout, 0.075*ones(size(out.tout)), ...
      out.tout, -0.075*ones(size(out.tout)))
114 xlim([0 tf])
115 grid on
116 xlabel('Time [s]')
117 ylabel('Motor Torque  $M_{w^4}$  [Nm]', 'Interpreter', 'latex')
118
119
120 %% Thruster Model
121
122 figure;
123
124 subplot(1, 3, 1)
125 plot(out.tout, out.Mc_out_th.Data(:, 1), out.tout, out.Mc_in_th.Data(:, 1))
126 grid on
127 xlim([0 tf])
128 ylim([4, 6])
129 legend('Actual Torque', 'Commanded Torque')
130 xlabel('Time [s]')
131 ylabel('Control Torque  $M_{c-x}$  [Mm]', 'Interpreter', 'latex')
132
133 subplot(1, 3, 2)
134 plot(out.tout, out.Mc_out_th.Data(:, 2), out.tout, out.Mc_in_th.Data(:, 2))
135 xlim([0 tf])
136 ylim([4, 6])
137 grid on
138 xlabel('Time [s]')
139 ylabel('Control Torque  $M_{c-y}$  [Mm]', 'Interpreter', 'latex')
140
141 subplot(1, 3, 3)
142 plot(out.tout, out.Mc_out_th.Data(:, 3), out.tout, out.Mc_in_th.Data(:, 3))
143 xlim([0 tf])
144 ylim([4, 6])
145 grid on
146 xlabel('Time [s]')
147 ylabel('Control Torque  $M_{c-z}$  [Mm]', 'Interpreter', 'latex')
148
149
150 figure;
151
152 subplot(1, 3, 1)
153 plot(out.tout, out.F_th.Data(:, 1), out.tout, 5*ones(size(out.tout)))
154 grid on
155 xlim([0 tf])
156 ylim([0, 6])
157 xlabel('Time [s]')
158 ylabel('Thrust Force  $F_{1}$  [N]', 'Interpreter', 'latex')
159 legend('Force Produced', 'Maximum Thrust')
160
161 subplot(1, 3, 2)

```

```

162 plot(out.tout, out.F_th.Data(:, 2), out.tout, 5*ones(size(out.tout)))
163 xlim([0 tf])
164 ylim([0, 6])
165 grid on
166 xlabel('Time [s]')
167 ylabel('Thrust Force  $F_2$  [N]', 'Interpreter', 'latex')
168
169 subplot(1, 3, 3)
170 plot(out.tout, out.F_th.Data(:, 3), out.tout, 5*ones(size(out.tout)))
171 xlim([0 tf])
172 ylim([0, 6])
173 grid on
174 xlabel('Time [s]')
175 ylabel('Thrust Force  $F_3$  [N]', 'Interpreter', 'latex')
176
177
178 %% Attitude Error 1
179
180
181 figure;
182
183 subplot(1, 3, 1)
184 plot(out.tout, out.true_att.Data(:, 1) - eta_0(1) * 180 / pi * ...
      ones(size(out.tout)))
185 grid on
186 xlim([0 tf])
187 xlabel('Time [s]')
188 ylabel('Roll Angle Error [deg]')
189
190 subplot(1, 3, 2)
191 plot(out.tout, out.true_att.Data(:, 2) - eta_0(2) * 180 / pi * ...
      ones(size(out.tout)))
192 xlim([0 tf])
193 grid on
194 xlabel('Time [s]')
195 ylabel('Pitch Angle Error [deg]')
196
197 subplot(1, 3, 3)
198 plot(out.tout, out.true_att.Data(:, 3) - eta_0(3) * 180 / pi * ...
      ones(size(out.tout)))
199 xlim([0 tf])
200 grid on
201 xlabel('Time [s]')
202 ylabel('Yaw Angle Error [deg]')
203
204
205 %% Control design (Linearized dynamics)
206
207 % Linearized system
208 A = [0 0 0 1.000000000000000 -0.0137063017876843 ...
      -0.0913863458218279;
209      0 0 0 0 -0.988938972239385 0.148322989405249;
210      0 0 0 0 -0.148954932428586 -0.993152432920826;
211      0 0 0 0 0 0;
212      0 0 0 0 0 0;
213      0 0 0 0 0 0];
214
215 B = [0 0 0;
216      0 0 0;

```

```

217     0    0    0;
218     -0.000694819902953446    0    0;
219     0    -0.000528190167596238    0;
220     0    0    -0.000360979401170922];
221
222 % LQR problem
223 Q = diag([1e2, 1e2, 1e2, 1e-1, 1e-1, 1e-1]);
224 R = 1e0 * eye(3);
225 [K, S, P] = lqr(A, B, Q, R);
226
227 % Closed-loop system
228 sys = ss(A-B*K, zeros(6, 3), eye(6), zeros(6, 3));
229
230 pzmap(sys)
231
232
233 %% Closed-loop control implementation
234
235 % Attitude Error
236 figure;
237
238 subplot(1, 3, 1)
239 plot(out.tout, out.true_att.Data(:, 1) - eta_0(1) * 180 / pi * ...
      ones(size(out.tout)))
240 grid on
241 xlim([0 tf])
242 xlabel('Time [s]')
243 ylabel('Roll Angle Error [deg]')
244
245 subplot(1, 3, 2)
246 plot(out.tout, out.true_att.Data(:, 2) - eta_0(2) * 180 / pi * ...
      ones(size(out.tout)))
247 xlim([0 tf])
248 grid on
249 xlabel('Time [s]')
250 ylabel('Pitch Angle Error [deg]')
251
252 subplot(1, 3, 3)
253 plot(out.tout, out.true_att.Data(:, 3) - eta_0(3) * 180 / pi * ...
      ones(size(out.tout)))
254 xlim([0 tf])
255 grid on
256 xlabel('Time [s]')
257 ylabel('Yaw Angle Error [deg]')
258
259
260 % State estimation error
261 figure;
262
263 subplot(1, 3, 1)
264 plot(out.tout, out.estimation_error.Data(:, 1))
265 grid on
266 xlim([0 tf])
267 xlabel('Time [s]')
268 ylabel('Roll Angle Error [deg]')
269
270 subplot(1, 3, 2)
271 plot(out.tout, out.estimation_error.Data(:, 2))
272 xlim([0 tf])

```

```

273 grid on
274 xlabel('Time [s]')
275 ylabel('Pitch Angle Error [deg]')
276
277 subplot(1, 3, 3)
278 plot(out.tout, out.estimation_error.Data(:, 3))
279 xlim([0 tf])
280 grid on
281 xlabel('Time [s]')
282 ylabel('Yaw Angle Error [deg]')
283
284
285 % Control action
286 figure;
287
288 subplot(1, 3, 1)
289 plot(out.tout, out.Mc_out.Data(:, 1), out.tout, out.Mc_in.Data(:, 1))
290 grid on
291 xlim([0 tf])
292 xlabel('Time [s]')
293 ylabel('Roll Angle Error [deg]')
294
295 subplot(1, 3, 2)
296 plot(out.tout, out.Mc_out.Data(:, 2), out.tout, out.Mc_in.Data(:, 2))
297 xlim([0 tf])
298 grid on
299 xlabel('Time [s]')
300 ylabel('Pitch Angle Error [deg]')
301
302 subplot(1, 3, 3)
303 plot(out.tout, out.Mc_out.Data(:, 3), out.tout, out.Mc_in.Data(:, 3))
304 xlim([0 tf])
305 grid on
306 xlabel('Time [s]')
307 ylabel('Yaw Angle Error [deg]')

```

13.11 Problem Set 10 Main File

```

1 %% Spacecraft properties
2 run("spacecraft_properties.m");
3 run("orbit_properties.m");
4
5 %% Simulink simulation
6 % Initial conditions
7 n_hat = cross(p_0/norm(p_0), v_0/norm(v_0));
8 r_hat = p_0/norm(p_0);
9 t_hat = cross(n_hat, r_hat);
10 Roi = [r_hat t_hat n_hat]';
11 Rbo = euler2rotm([0 deg2rad(35) 0]) * [0 0 1; 0 -1 0; 1 0 0];
12 Rpi = spacecraft.Rpb * Rbo * Roi;
13 eta_0 = rotm2euler(Rpi);
14 omega_0 = [deg2rad(0); deg2rad(0); deg2rad(0)];
15
16 dt = 0.1;
17 tf = (2*pi*sqrt(orbit.a^3/orbit.GM));
18
19 x0 = [p_0; v_0; eta_0; omega_0];

```

```

20 out = sim('simulation');
21
22 %% Attitude Mode Trajectories
23
24 calibration_att = unwrap(squeeze(out.calibration_att.Data));
25 nominal_att = unwrap(squeeze(out.nominal_att.Data));
26
27
28 % Attitude RTN
29 figure;
30
31 subplot(1, 3, 1)
32 plot(out.tout, nominal_att(:, 1)*180/pi, out.tout, calibration_att(:, ...
    1)*180/pi)
33 grid on
34 xlim([0 tf])
35 xlabel('Time [s]')
36 ylabel('Roll Angle [deg]')
37
38 subplot(1, 3, 2)
39 plot(out.tout, nominal_att(:, 2)*180/pi, out.tout, calibration_att(:, ...
    2)*180/pi)
40 xlim([0 tf])
41 grid on
42 xlabel('Time [s]')
43 ylabel('Pitch Angle [deg]')
44
45 subplot(1, 3, 3)
46 plot(out.tout, nominal_att(:, 3)*180/pi, out.tout, calibration_att(:, ...
    3)*180/pi)
47 xlim([0 tf])
48 grid on
49 xlabel('Time [s]')
50 ylabel('Yaw Angle [deg]')
51 legend('Nominal Attitude', 'Calibration Attitude')
52
53 %% Nominal trajectory
54
55 calibration_att = unwrap(squeeze(out.calibration_att.Data));
56 nominal_att = unwrap(squeeze(out.nominal_att.Data));
57 true_att = out.true_att.Data;
58
59
60 % Attitude RTN
61 figure;
62
63 subplot(1, 3, 1)
64 plot(out.tout, nominal_att(:, 1)*180/pi, out.tout, true_att(:, 1))
65 grid on
66 xlim([0 tf])
67 xlabel('Time [s]')
68 ylabel('Roll Angle [deg]')
69
70 subplot(1, 3, 2)
71 plot(out.tout, nominal_att(:, 2)*180/pi, out.tout, true_att(:, 2))
72 xlim([0 tf])
73 grid on
74 xlabel('Time [s]')
75 ylabel('Pitch Angle [deg]')

```

```

76
77 subplot(1, 3, 3)
78 plot(out.tout, nominal_att(:, 3)*180/pi, out.tout, true_att(:, 3))
79 xlim([0 tf])
80 grid on
81 xlabel('Time [s]')
82 ylabel('Yaw Angle [deg]')
83 legend('Nominal Attitude', 'True Attitude')
84
85 %% Calibration trajectory
86
87 calibration_att = unwrap(squeeze(out.calibration_att.Data));
88 nominal_att = unwrap(squeeze(out.nominal_att.Data));
89 true_att = out.true_att.Data;
90
91
92 % Attitude RTN
93 figure;
94
95 subplot(1, 3, 1)
96 plot(out.tout, calibration_att(:, 1)*180/pi, out.tout, true_att(:, 1))
97 grid on
98 xlim([0 tf])
99 xlabel('Time [s]')
100 ylabel('Roll Angle [deg]')
101
102 subplot(1, 3, 2)
103 plot(out.tout, calibration_att(:, 2)*180/pi, out.tout, true_att(:, 2))
104 xlim([0 tf])
105 grid on
106 xlabel('Time [s]')
107 ylabel('Pitch Angle [deg]')
108
109 subplot(1, 3, 3)
110 plot(out.tout, calibration_att(:, 3)*180/pi, out.tout, true_att(:, 3))
111 xlim([0 tf])
112 grid on
113 xlabel('Time [s]')
114 ylabel('Yaw Angle [deg]')
115 legend('Calibration Attitude', 'True Attitude')
116
117 %% Switching trajectories
118
119 calibration_att = unwrap(squeeze(out.calibration_att.Data));
120 nominal_att = unwrap(squeeze(out.nominal_att.Data));
121 true_att = squeeze(out.true_att.Data);
122
123
124 % Attitude RTN
125 figure;
126
127 subplot(3, 1, 1)
128 hold on
129 plot(out.tout, true_att(:, 1), '-')
130 plot(out.tout, calibration_att(:, 1)*180/pi, '--')
131 plot(out.tout, nominal_att(:, 1)*180/pi, '--')
132 hold off
133 grid on
134 xlim([0 3*tf])

```

```

135 xlabel('Time [s]')
136 ylabel('Roll Angle [deg]')
137
138 subplot(3, 1, 2)
139 hold on
140 plot(out.tout, true_att(:, 2), '-')
141 plot(out.tout, calibration_att(:, 2)*180/pi, '--')
142 plot(out.tout, nominal_att(:, 2)*180/pi, '--')
143 hold off
144 xlim([0 3*tf])
145 grid on
146 xlabel('Time [s]')
147 ylabel('Pitch Angle [deg]')
148
149 subplot(3, 1, 3)
150 hold on
151 plot(out.tout, true_att(:, 3), '-')
152 plot(out.tout, calibration_att(:, 3)*180/pi, '--')
153 plot(out.tout, nominal_att(:, 3)*180/pi, '--')
154 hold off
155 xlim([0 3*tf])
156 grid on
157 xlabel('Time [s]')
158 ylabel('Yaw Angle [deg]')
159 legend('True Attitude', 'Calibration Attitude', 'Nominal Attitude')
160
161
162
163 %% Performance analysis
164
165 calibration_att = unwrap(squeeze(out.calibration_att.Data));
166 nominal_att = unwrap(squeeze(out.nominal_att.Data));
167 attitude_ref = unwrap(out.attitude_ref.Data);
168 true_att = out.true_att.Data;
169
170 % Attitude Error
171 figure;
172
173 subplot(1, 3, 1)
174 plot(out.tout, true_att(:, 1) - attitude_ref(:, 1) * 180 / pi)
175 grid on
176 xlim([0 tf])
177 xlabel('Time [s]')
178 ylabel('Roll Angle Error [deg]')
179
180 subplot(1, 3, 2)
181 plot(out.tout, true_att(:, 2) - attitude_ref(:, 2) * 180 / pi)
182 xlim([0 tf])
183 grid on
184 xlabel('Time [s]')
185 ylabel('Pitch Angle Error [deg]')
186
187 subplot(1, 3, 3)
188 plot(out.tout, true_att(:, 3) - attitude_ref(:, 3) * 180 / pi)
189 xlim([0 tf])
190 grid on
191 xlabel('Time [s]')
192 ylabel('Yaw Angle Error [deg]')
193

```

```

194
195 % State estimation error
196 figure;
197
198 subplot(1, 3, 1)
199 plot(out.tout, out.estimated_error.Data(:, 1))
200 grid on
201 xlim([0 tf])
202 xlabel('Time [s]')
203 ylabel('Roll Angle Error [deg]')
204
205 subplot(1, 3, 2)
206 plot(out.tout, out.estimated_error.Data(:, 2))
207 xlim([0 tf])
208 grid on
209 xlabel('Time [s]')
210 ylabel('Pitch Angle Error [deg]')
211
212 subplot(1, 3, 3)
213 plot(out.tout, out.estimated_error.Data(:, 3))
214 xlim([0 tf])
215 grid on
216 xlabel('Time [s]')
217 ylabel('Yaw Angle Error [deg]')
218
219
220 % Control action
221 figure;
222
223 subplot(1, 3, 1)
224 plot(out.tout, out.Mc_out.Data(:, 1), out.tout, out.Mc_in.Data(:, 1))
225 grid on
226 xlim([0 tf])
227 xlabel('Time [s]')
228 ylabel('Roll Angle Error [deg]')
229
230 subplot(1, 3, 2)
231 plot(out.tout, out.Mc_out.Data(:, 2), out.tout, out.Mc_in.Data(:, 2))
232 xlim([0 tf])
233 grid on
234 xlabel('Time [s]')
235 ylabel('Pitch Angle Error [deg]')
236
237 subplot(1, 3, 3)
238 plot(out.tout, out.Mc_out.Data(:, 3), out.tout, out.Mc_in.Data(:, 3))
239 xlim([0 tf])
240 grid on
241 xlabel('Time [s]')
242 ylabel('Yaw Angle Error [deg]')

```

13.12 Utility Function Code

13.12.1 Spacecraft Properties

```

1 % Spacecraft geometry and mass
2 spacecraft.wet_mass = 1360;
3

```

```

4 spacecraft.solar_arrays.mass = 96;
5 spacecraft.solar_arrays.length = 13;
6 spacecraft.solar_arrays.width = 1.11;
7 spacecraft.solar_arrays.thickness = 0.1;
8
9 spacecraft.bus.mass = 709;
10 spacecraft.bus.length = 1.74;
11 spacecraft.bus.width = 1.9;
12 spacecraft.bus.depth = 2.0;
13
14 spacecraft.propellant.mass = 289;
15 spacecraft.tank.diameter = 0.66;
16 spacecraft.tank.length = 1.0;
17
18 spacecraft.payload.mass = 266;
19 spacecraft.payload.diameter = 1.5;
20 spacecraft.payload.length = 2.0;
21
22 % Spacecraft rotor properties
23 spacecraft.wheel.r = [0 0 1]; % axis of rotation w.r.t. principal axes
24 spacecraft.wheel.Ir = 0.07162; % inertia of rotor wheel [kgm^2]
25 spacecraft.wheel.L = 45; % angular momentum of wheel [kgm^2/s]
26
27 % Other spacecraft properties
28 spacecraft.mag.m = 10; % magnetic dipole of spacecraft [Tm^2]
29 spacecraft.aero.Cd = 2.0; % spacecraft drag coefficient
30 spacecraft.solar.Cs = 0.25; % Coefficient of specular reflection
31 spacecraft.solar.Cd = 0.25; % Coefficient of diffuse reflection
32
33 % Spacecraft inertia tensors
34 spacecraft.d_cg = get_cg_coords(spacecraft);
35 [spacecraft.Ib, spacecraft.Ip, spacecraft.Rpb] = ...
    get_inertia_tensor("operation", spacecraft);
36
37 % Spacecraft position and attitude in inertial frame
38 cg_centroid = [0 0 0];
39 attitude = [0, 0, 0];
40
41 % Discretize spacecraft
42 [spacecraft.TR, spacecraft.S] = discretize_sc(cg_centroid, attitude, 0, ...
    spacecraft);

```

13.12.2 Orbit Properties

```

1 % Orbit Initial conditions
2 orbit.Re = 6371e3; % Earth radius [m]
3 orbit.GM = 3.986004415e14; % Earth's Gravitational Constant [m3/s2]
4 orbit.e = 0.006; % Eccentricity [-]
5 orbit.a = 320e3 + orbit.Re; % Semi-major axis [m]
6 orbit.incl = 96.7; % Inclination [deg]
7 orbit.RAAN = 214.9; % Longitude of ascending node [deg]
8 orbit.argp = 264.2; % Argument of periapsis [deg]
9 orbit.true_anomaly = 95.4; % True anomaly [deg]
10
11 orbit.B0 = 3.12e-5; % Earth's magnetic field at equator
12 orbit.density = 2.418e-11; % Atmospheric density
13 orbit.flux = 1358/3e8; % Mean momentum flux

```

```

14
15 [p_0, v_0] = keplerian2ijk(orbit.a, orbit.e, orbit.incl, ...
16                             orbit.RAAN, orbit.argp, ...
17                             orbit.true_anomaly, ...
18                             'arglat', 0.0);
19
20 % Initial Keplerian elements of Earth's orbit
21 e = 0.0167086; % Eccentricity [-]
22 a = 149.60e9; % Semi-major axis [m]
23 incl = 7.155; % Inclination [deg]
24 RAAN = 174.9; % Longitude of ascending node [deg]
25 argp = 288.1; % Argument of periapsis [deg]
26 true_anomaly = 0; % True anomaly [deg]
27
28 mu_sun = 1.327e20; % Gravitational paramter of sun
29
30 [p_earth, v_earth] = keplerian2ijk(a, e, incl, RAAN, argp, true_anomaly);
31 v_earth = sqrt(mu_sun/a) * v_earth/norm(v_earth);
32 att_earth = [deg2rad(23.5); 0; 0];
33 omega_earth = [0; 0; 7.29e-5]; % Rotation of Earth in principal axes
34 x0_earth = [p_earth; v_earth; att_earth; omega_earth];

```

13.12.3 Center of Gravity Calculation

```

1 function cg = get_cg_coords(sc_prop)
2 %CG_COORDS This function calculates the center of gravity with respect to
3 % spacecraft bus center
4
5 bus = sc_prop.bus;
6 solar_arrays = sc_prop.solar_arrays;
7 payload = sc_prop.payload;
8 propellant = sc_prop.propellant;
9 tank = sc_prop.tank;
10
11 % Deviation in from bus center along x-axis
12 dcg_x = (0 * bus.mass + 0 * payload.mass + 0 * propellant.mass + 0 ...
13          * solar_arrays.mass) ...
14          / (sc_prop.wet_mass);
15 dcg_y = (0 * bus.mass + 0 * payload.mass + 0 * propellant.mass + 0 ...
16          * solar_arrays.mass) ...
17          / (sc_prop.wet_mass);
18 dcg_z = (0 * bus.mass + (payload.length/2 + bus.length/2) * ...
19          payload.mass + (-bus.length/2 + tank.length/2) ...
20          * propellant.mass + 0 * solar_arrays.mass) / ...
21          (sc_prop.wet_mass);
22
23 cg = [dcg_x dcg_y dcg_z];
24 end

```

13.12.4 Inertia Tensor Calculation

```

1 function [Ib, Ip, Rpb] = get_inertia_tensor(phase, sc_prop)
2 %INERTIA_TENSOR Calculates the inertia tensor w.r.t. the body axes frame.
3
4 % Distance of origin of body axes to center of S/C bus

```

```

5 dx_cg = sc_prop.d_cg(1);
6 dy_cg = sc_prop.d_cg(2);
7 dz_cg = sc_prop.d_cg(3);
8
9 bus = sc_prop.bus;
10 payload = sc_prop.payload;
11 solar_arrays = sc_prop.solar_arrays;
12 propellant = sc_prop.propellant;
13 tank = sc_prop.tank;
14
15 % Inertia tensor of bus w.r.t. local reference frame
16 Ibus_x = 1/12 * bus.mass * (bus.length^2 + bus.width^2);
17 Ibus_xy = -15;
18 Ibus_y = 1/12 * bus.mass * (bus.length^2 + bus.depth^2);
19 Ibus_yz = 25;
20 Ibus_z = 1/12 * bus.mass * (bus.depth^2 + bus.width^2);
21 Ibus_xz = 19;
22
23 % Inertia tensor of solar arrays w.r.t. local reference frame
24 Ipanel_x = 1/12 * solar_arrays.mass * (solar_arrays.length^2 + ...
    solar_arrays.width^2);
25 Ipanel_y = 1/12 * solar_arrays.mass * (solar_arrays.thickness^2 + ...
    solar_arrays.width^2);
26 Ipanel_z = 1/12 * solar_arrays.mass * (solar_arrays.length^2 + ...
    solar_arrays.thickness^2);
27
28 % Inertia tensor of payload w.r.t. local reference frame
29 Ipayload_x = payload.mass * (1/4 * (payload.diameter/2)^2 + 1/12 * ...
    payload.length^2);
30 Ipayload_y = payload.mass * (1/4 * (payload.diameter/2)^2 + 1/12 * ...
    payload.length^2);
31 Ipayload_z = 1/2 * payload.mass * (payload.diameter/2)^2;
32
33 % Inertia tensor of propellant w.r.t. local reference frame
34 Ipropellant_x = propellant.mass * (1/4 * (tank.diameter/2)^2 + 1/12 * ...
    tank.length^2);
35 Ipropellant_y = propellant.mass * (1/4 * (tank.diameter/2)^2 + 1/12 * ...
    tank.length^2);
36 Ipropellant_z = 1/2 * propellant.mass * (tank.diameter/2)^2;
37
38 % Inertia tensor w.r.t body axes
39 Ixx = (Ibus_x + bus.mass * (dy_cg^2 + dz_cg^2)) ... % bus contribution
40     + (Ipanel_x + solar_arrays.mass * (dy_cg^2 + dz_cg^2)) ... % solar ...
    panel contribution
41     + (Ipropellant_x + propellant.mass * (dy_cg^2 + (bus.length/2 - ...
    tank.length/2 + dz_cg)^2)) ... % propellant contribution
42     + (Ipayload_x + payload.mass * (dy_cg^2 + (bus.length/2 - dz_cg + ...
    payload.length/2)^2)); % payload contribution
43
44 Iyy = (Ibus_y + bus.mass * (dx_cg^2 + dz_cg^2)) ... % bus contribution
45     + (Ipanel_y + solar_arrays.mass * (dx_cg^2 + dz_cg^2)) ... % solar ...
    panel contribution
46     + (Ipropellant_y + propellant.mass * (dx_cg^2 + (bus.length/2 - ...
    tank.length/2 + dz_cg)^2)) ... % propellant contribution
47     + (Ipayload_y + payload.mass * (dx_cg^2 + (bus.length/2 - dz_cg + ...
    payload.length/2)^2)); % payload contribution
48
49 Izz = (Ibus_z + bus.mass * (dx_cg^2 + dy_cg^2)) ... % bus contribution

```

```

50     + Ipanel_z + solar_arrays.mass * (dx_cg^2 + dy_cg^2) ...    % solar ...
        panel contribution
51     + Ipropellant_z + propellant.mass * (dx_cg^2 + dy_cg^2) ...    % ...
        propellant contribution
52     + Ipayload_z + payload.mass * (dx_cg^2 + dy_cg^2));        % payload ...
        contribution
53
54     Ixy = Ibus_xy;
55     Iyz = Ibus_yz;
56     Ixz = Ibus_xz;
57
58     Ib = [Ixx Ixy Ixz;
59           Ixy Iyy Iyz;
60           Ixz Iyz Izz];
61     [Rpb, Ip] = eig(Ib);
62 end

```

13.12.5 Spacecraft Discretization

```

1 function [TR, S] = discretize_sc(cg_coord, attitude, panel_rotation, ...
    sc_prop)
2 %DISCRETIZE Summary of this function goes here
3
4 S = [];
5 %   Discretize spacecraft bus
6 bus = sc_prop.bus;
7 bus_coords = [bus.depth/2, bus.width/2, bus.length/2;
8               bus.depth/2, bus.width/2, -bus.length/2;
9               bus.depth/2, -bus.width/2, -bus.length/2;
10              bus.depth/2, -bus.width/2, bus.length/2;
11              -bus.depth/2, bus.width/2, bus.length/2;
12              -bus.depth/2, bus.width/2, -bus.length/2;
13              -bus.depth/2, -bus.width/2, -bus.length/2;
14              -bus.depth/2, -bus.width/2, bus.length/2];
15 bus_coords = bus_coords - sc_prop.d_cg;
16
17 T_bus = [1 2 3; 1 3 4; 1 2 6; 1 6 5; 5 6 7; 5 7 8; 4 3 7; 4 7 8; 3 2 6; ...
18          3 6 7; 1 5 8; 1 8 4];
19 S = [S;
20      bus.width * bus.length / 2; bus.width * bus.length / 2;
21      bus.depth * bus.length / 2; bus.depth * bus.length / 2;
22      bus.width * bus.length / 2; bus.width * bus.length / 2;
23      bus.depth * bus.length / 2; bus.depth * bus.length / 2;
24      bus.depth * bus.width / 2; bus.depth * bus.width / 2;
25      bus.depth * bus.width / 2; bus.depth * bus.width / 2];
26 %   Discretize spacecraft solar arrays
27 R = [cos(panel_rotation) 0 sin(panel_rotation);
28      0 1 0;
29      -sin(panel_rotation) 0 cos(panel_rotation)];
30 solar_arrays = sc_prop.solar_arrays;
31 solar_array_coords = [solar_arrays.thickness/2, solar_arrays.length/4, ...
32                       solar_arrays.thickness/2, ...
33                       solar_arrays.length/4, -solar_arrays.width/2;
34                       solar_arrays.thickness/2, ...
35                       -solar_arrays.length/4, -solar_arrays.width/2;

```

```

34         solar_arrays.thickness/2, ...
35         -solar_arrays.length/4, solar_arrays.width/2;
36     -solar_arrays.thickness/2, ...
37         solar_arrays.length/4, solar_arrays.width/2;
38     -solar_arrays.thickness/2, ...
39         solar_arrays.length/4, -solar_arrays.width/2;
40     -solar_arrays.thickness/2, ...
41         -solar_arrays.length/4, -solar_arrays.width/2;
42     -solar_arrays.thickness/2, ...
43         -solar_arrays.length/4, ...
44         solar_arrays.width/2] * R;
45 left_array_coords = solar_array_coords + [0, bus.width/2 + ...
46     solar_arrays.length/4, 0] - sc_prop.d_cg;
47 right_array_coords = solar_array_coords - [0, bus.width/2 + ...
48     solar_arrays.length/4, 0] - sc_prop.d_cg;
49 T_array = [T_bus + 8; T_bus + 16];
50
51 S = [S;
52     solar_arrays.width * solar_arrays.length / 4; solar_arrays.width * ...
53     solar_arrays.length / 4;
54     solar_arrays.thickness * solar_arrays.width / 2; ...
55     solar_arrays.thickness * solar_arrays.width / 2;
56     solar_arrays.width * solar_arrays.length / 4; solar_arrays.width * ...
57     solar_arrays.length / 4;
58     solar_arrays.thickness * solar_arrays.width / 2; ...
59     solar_arrays.thickness * solar_arrays.width / 2;
60     solar_arrays.width * solar_arrays.length / 4; solar_arrays.width * ...
61     solar_arrays.length / 4;
62     solar_arrays.thickness * solar_arrays.length / 4; ...
63     solar_arrays.thickness * solar_arrays.length / 4;
64     solar_arrays.thickness * solar_arrays.length / 4; ...
65     solar_arrays.thickness * solar_arrays.length / 4;
66     solar_arrays.width * solar_arrays.length / 4; solar_arrays.width * ...
67     solar_arrays.length / 4;
68     solar_arrays.thickness * solar_arrays.width / 2; ...
69     solar_arrays.thickness * solar_arrays.width / 2;
70     solar_arrays.width * solar_arrays.length / 4; solar_arrays.width * ...
71     solar_arrays.length / 4;
72     solar_arrays.thickness * solar_arrays.length / 4; ...
73     solar_arrays.thickness * solar_arrays.length / 4;
74     solar_arrays.thickness * solar_arrays.length / 4];
75
76 % Discretize payload
77 N = 8; % Discretization fidelity of cylinder
78 payload = sc_prop.payload;
79
80 alpha = 2*pi / N; % angle of triangle in base polygon
81 r = sc_prop.payload.diameter / 2; % cylinder radius
82 b = 2*r^2*(1-cos(alpha)); % base of triangle in base polygon
83
84 payload_coords = [0, 0, bus.length/2 + payload.length];
85 for i=0:N-1
86     payload_coords = [payload_coords;
87         r*cos(i*alpha), r*sin(i*alpha), bus.length/2 + ...
88         payload.length;
89         r*cos(i*alpha), r*sin(i*alpha), bus.length/2];
90 end
91 payload_coords = payload_coords - sc_prop.d_cg;

```

```

72
73 T_payload = [];
74 for i=1:N-1
75     T_payload = [T_payload
76                  1 2*i 2*i+2;           % triangulate bottom polygon
77                  2*i 2*i+1 2*i+2;       % triangulate side
78                  2*i+1 2*i+2 2*i+3];
79     S = [S; b/2*sqrt(r^2-b^2/4);
80          b * payload.length / 2; b * payload.length / 2];
81 end
82 % wrap cylinder back to starting point
83 T_payload = [T_payload
84              1 2*N 2;           % triangulate bottom polygon
85              2*N 2*N+1 2;       % triangulate side
86              2*N+1 2 3];
87 S = [S; b/2*sqrt(r^2-b^2/4);
88      b * payload.length / 2; b * payload.length / 2];
89
90 T_payload = T_payload + 24;
91
92 % Compose geomerty
93 RIb = angle2dcm(attitude(3), attitude(2), attitude(1), 'ZYX');
94
95 T = [T_bus; T_array; T_payload];
96 coords = [bus_coords;
97            left_array_coords;
98            right_array_coords;
99            payload_coords];
100 coords = coords * RIb + cg_coord;
101
102 TR = triangulation(T, coords);
103
104 end

```

13.12.6 Orbit Dynamics

```

1 function fun = orbital.equation(orbit)
2 %ORBITAL.EQUATION equations of motion for position of orbital satellite
3     fun = @ode;
4
5     function dxdt = ode(t, x)
6         acc = -orbit.GM / norm(x(1:3))^3;
7
8         dxdt = [x(4:6);           % Time-derivative of position
9                acc * x(1:3)];    % Acceleration
10    end
11 end

```

13.12.7 Attitude Dynamics

```

1 function fun = euler.equation(sc_prop, attitude_parameterization)
2 %EULER.EQUATION Attitude dynamics and kinematics of principal axes frame
3     fun = @ode;
4
5     Ip = sc_prop.Ip;

```

```

6     function dxdt = ode(t, x)
7         omega = x(1:3);
8         % Derivative of angular velocity vector in principal axes frame
9         dwdt = [(Ip(2,2) - Ip(3,3))*omega(2)*omega(3)/Ip(1,1);
10                (Ip(3,3) - Ip(1,1))*omega(3)*omega(1)/Ip(2,2);
11                (Ip(1,1) - Ip(2,2))*omega(1)*omega(2)/Ip(3,3)];
12
13         % Derivative of attitude vector in principal axes frame
14         if attitude_parameterization == "euler_angles"
15             phi = x(4); theta = x(5);
16
17             detadt = [1 sin(phi)*tan(theta) cos(phi)*tan(theta); ...
18                     0 cos(phi) -sin(phi); ...
19                     0 sin(phi)*sec(theta) cos(phi)*sec(theta)] * omega;
20         elseif attitude_parameterization == "quaternion"
21             att = x(4:7)';
22             att = att / norm(att);
23
24             detadt = 1/2 * [-att(2) -att(3) -att(4);
25                             att(1) -att(4) att(3);
26                             att(4) att(1) -att(2);
27                             -att(3) att(2) att(1)] * omega;
28         end
29
30         dxdt = [dwdt; detadt];
31     end
32 end

```

13.12.8 Equations of Motion

```

1 function EOM = equations_of_motion(sc_prop, orbit, options)
2 %EQUATIONS_OF_MOTION Return equations of motion functional dxdt
3     EOM = @func;
4
5     Ip = sc_prop.Ip;
6     Ip_x = Ip(1,1); Ip_y = Ip(2,2); Ip_z = Ip(3,3);
7     function dxdt = func(t, x)
8         pos = x(1:3);
9         vel = x(4:6);
10        omega = x(7:9);
11        eta = x(10:end);
12        %% Get external torques and forces
13        if options.external_torque
14            M = external_torque(x, sc_prop, orbit, options);
15        else
16            M = zeros(3, 1);
17        end
18        %% Position Dynamics
19        acc = -orbit.GM / norm(x(1:3))^3;
20
21        dpdt = vel;           % Time-derivative of position
22        dvdt = acc * pos;     % Time-derivative of velocity
23
24        %% Attitude Dynamics
25        % Rotor configuration
26        r = sc_prop.wheel.r; % axis of rotation w.r.t. ...
27        principal axes

```

```

27     Ir = sc.prop.wheel.Ir;           % inertia of rotor wheel [kgm^2]
28     Lr = sc.prop.wheel.L;           % angular momentum of wheel [kgm^2/s]
29     if options.rotor
30         om_r = Lr/Ir;                % angular velocity magnitude [rad/s]
31     else
32         om_r = 0;                    % disable rotor wheel
33     end
34     % Derivative of angular velocity vector in principal axes frame
35     om_x = omega(1); om_y = omega(2); om_z = omega(3);
36     dwdt = [((Ip_y - Ip_z)*om_y*om_z + Ir*om_r*(om_z*r(2) - ...
37             om_y*r(3)) + M(1))/Ip_x;
38             ((Ip_z - Ip_x)*om_z*om_x + Ir*om_r*(om_x*r(3) - ...
39             om_z*r(1)) + M(2))/Ip_y;
40             ((Ip_x - Ip_y)*om_x*om_y + Ir*om_r*(om_y*r(1) - ...
41             om_x*r(2)) + M(3))/Ip_z];
42     % Derivative of attitude vector in principal axes frame
43     if options.attitude.parameterization == "euler_angles"
44         phi = eta(1); theta = eta(2);
45
46         detadt = [1 sin(phi)*tan(theta) cos(phi)*tan(theta); ...
47                 0 cos(phi) -sin(phi); ...
48                 0 sin(phi)*sec(theta) cos(phi)*sec(theta)] * omega;
49     elseif options.attitude.parameterization == "quaternion"
50         att = eta';
51         att = att / norm(att);
52
53         detadt = 1/2 * [-att(2) -att(3) -att(4);
54                         att(1) -att(4) att(3);
55                         att(4) att(1) -att(2);
56                         -att(3) att(2) att(1)] * omega;
57     end
58
59     % Propagate Earth orbit
60     dx_sundt = EOM_earth(t, x);
61
62     dxdt = [dpdt; dvdt; dwdt; detadt; dx_sundt];
63 end
64 end

```

13.12.9 Earth's Equations of Motion

```

1 function dxdt = EOM_earth(t, x)
2     mu_sun = 1.327e20;                % Gravitational paramter of sun
3
4     x = x(13:end);
5     phi = x(7); theta = x(8);
6     detadt = [1 sin(phi)*tan(theta) cos(phi)*tan(theta); ...
7             0 cos(phi) -sin(phi); ...
8             0 sin(phi)*sec(theta) cos(phi)*sec(theta)] * x(10:12);
9
10    dxdt = [x(4:6);                    % Time-derivative of position
11            -mu_sun/norm(x(1:3))^3*x(1:3); % Graviational acceleration
12            detadt;                    % Attitude kinematics
13            zeros(3,1)                % Constant angular velocity
14            ];
15 end

```

13.12.10 Sensor measurement models

13.12.11 External Torque Calculation

```
1 function M = external_torque(x, sc_prop, orbit, options)
2 %EXTERNAL_TORQUE Determine the external torque based on spacecraft state
3
4 % Gravity Gradient Torque
5 Mgg = gravity_gradient_torque(x, sc_prop, orbit, options);
6
7 % Magnetic Field Torque
8 Mm = magnetic_field_torque(x, sc_prop, orbit, options);
9
10 % Atmospheric Drag Torque
11 Md = drag_torque(x, sc_prop, orbit, options);
12
13 % Solar Radiation Torque
14 Ms = solar_radiation_torque(x, sc_prop, orbit, options);
15
16 % Final External torque
17 M = Mgg + Mm + Md + Ms;
18 end
```

13.12.12 Gravity Gradient Torque

```
1 function Mgg = gravity_gradient_torque(x, sc_prop, orbit, options)
2 % Gravity Gradient
3 Ip = sc_prop.Ip;
4 pos = x(1:3);
5 eta = x(10:end);
6
7 if options.attitude_parameterization == "euler_angles"
8     Rpi = euler2rotm(eta);
9 elseif options.attitude_parameterization == "quaternions"
10    Rpi = euler2rotm(quat2euler(eta));
11 end
12
13 r_hat = Rpi * pos / norm(pos); % unit vector of pos vector in ...
    principal axes
14 Mgg = 3*orbit.GM / orbit.a^3 * cross(r_hat, Ip*r_hat);
15 end
```

13.12.13 Magnetic Field Torque

```
1 function Mm = magnetic_field_torque(x, sc_prop, orbit, options)
2 % Magnetic Field Torque
3 pos = x(1:3);
4 eta = x(10:end);
5
6 % Satellite's magnetic dipole
7 m_max = sc_prop.mag.m * sc_prop.Rpb * [0; 0; 1];
8
9 % Earth's magnetic field strength
10 r_hat = pos / norm(pos);
```

```

11     theta_m = 168.6;                % coelevation of Earth's dipole [deg]
12     alpha_m = 109.3;               % east longitude of dipole [deg]
13     m_hat = [sin(deg2rad(theta_m)) * cos(deg2rad(alpha_m));
14              sin(deg2rad(theta_m)) * sin(deg2rad(alpha_m));
15              cos(deg2rad(theta_m))];
16     B = -orbit.Re^3*orbit.B0/norm(pos)^3*(3*dot(m_hat,r_hat)*r_hat-m_hat);
17     Bp = euler2rotm(eta) * B;      % Express in principal axes
18
19     % Resulting torque
20     Mm = cross(m_max, Bp);         % Expressed in principal axes
21 end

```

13.12.14 Atmospheric Drag Torque

```

1 function Md = drag_torque(x, sc_prop, orbit, options)
2 %   Atmospheric Drag Torque
3     n_hats = sc_prop.TR.faceNormal;
4     r = sc_prop.TR.incenter;
5
6     Md = [0; 0; 0];
7     for i=1:length(sc_prop.S)
8         % Determine aerodynamic force on surface element
9         vel = x(4:6) + cross(x(7:9), r(i, :)');
10        v_hat = vel / norm(vel);
11
12        p_dynamic = 1/2*orbit.density*norm(vel)^2;
13        df_aero = -p_dynamic*sc_prop.aero.Cd*dot(v_hat, n_hats(i, ...
14            :))'*sc_prop.S(i);
15
16        % Torque contribution by surface element
17        Md = Md + df_aero * cross(r(i, :)', v_hat);
18    end
19 end

```

13.12.15 Solar Radiation Torque

```

1 function Ms = solar_radiation_torque(x, sc_prop, orbit, options)
2 %   Solar Radiation Torque
3     r_sun = -euler2rotm(x(19:21))*x(13:15); % Sun position in ECI
4     r_relative = x(1:3) - r_sun;
5
6     s_hat = euler2rotm(x(10:12)) * r_relative/norm(r_relative);
7     n_hats = sc_prop.TR.faceNormal;
8     r = sc_prop.TR.incenter;
9
10    Ms = [0; 0; 0];
11    for i=1:length(sc_prop.S)
12        % Determine solar radiation force on surface element
13        costh = dot(n_hats(i, :), s_hat);
14        if costh > 0
15            dA = sc_prop.S(i);
16            df_solar = -orbit.flux*((1-sc_prop.solar.Cs)*s_hat+ ...
17                2*(sc_prop.solar.Cs*costh+1/3*sc_prop.solar.Cd)*n_hats(i, ...
18                    :))*costh*dA;
19        end
20    end

```

```

19         % Torque contribution by surface element
20         Ms = Ms + cross(r(i, :)', df_solar);
21
22     end
23 end
24 end

```

13.12.16 Determine equilibrium configuration for dual spin satellite

```

1 function [r_hat, L] = dual_spin_equil(om_0, spacecraft)
2 %DUAL_SPIN_EQUIL Determine the axis of rotation of the momentum wheel
3     Ip = spacecraft.Ip;
4 % Solve nonlinear system of equations
5     function f = func(x)
6         r = x(1:3);
7         L = x(4);
8         f = [
9             (Ip(3,3) - Ip(2,2))*om_0(2)*om_0(3) + L * (om_0(2)*r(3) - ...
10                om_0(3)*r(2));
11             (Ip(1,1) - Ip(3,3))*om_0(3)*om_0(1) + L * (om_0(3)*r(1) - ...
12                om_0(1)*r(3));
13             (Ip(2,2) - Ip(1,1))*om_0(1)*om_0(2) + L * (om_0(1)*r(2) - ...
14                om_0(2)*r(1));
15             sum(r.^2) - 1
16         ];
17     end
18     options = optimoptions('fsolve','MaxIterations', 4e3, ...
19        'MaxFunctionEvaluations', 4e3);
20     x = fsolve(@func, [spacecraft.wheel.r'; spacecraft.wheel.L], options);
21     r_hat = x(1:3)';
22     L = x(4);
23 end

```

13.13 Discrete-time linear state propagation

```

1 function x_att_next = discrete_linear(x_att, Ip, dt)
2 %DISCRETE_LINEAR Propagate the state through a discrete linearization ...
3   of dynamics
4 % Get state variables
5     phi = x_att(1); theta = x_att(2); psi = x_att(3);
6     p = x_att(4); q = x_att(5); r = x_att(6);
7     Ix = Ip(1,1); Iy = Ip(2,2); Iz = Ip(3,3);
8
9     Psi = [0 0 0 1 sin(phi)*tan(theta) cos(phi)*tan(theta);
10            0 0 0 0 cos(phi) -sin(phi);
11            0 0 0 0 sin(phi)*sec(theta) cos(phi)*sec(theta);
12            0 0 0 0 (Iy-Iz)/Ix*r (Iy-Iz)/Ix*q;
13            0 0 0 (Iz-Ix)/Iy*r 0 (Iz-Ix)/Iy*p;
14            0 0 0 (Ix-Iy)/Iz*q (Ix-Iy)/Iz*p 0];
15     Psi = eye(6) + dt * Psi;
16
17     x_att_next = Psi * x_att;
18 end

```

13.13.1 Convert Euler Angles to Rotation Matrix

```
1 function Rpi = euler2rotm(euler_angles)
2 %EULER2ROTM Return the rotation matrix from inertial to principal frame
3     phi = euler_angles(1); theta = euler_angles(2); psi = euler_angles(3);
4     Rpi = [cos(theta)*cos(psi) cos(theta)*sin(psi) -sin(theta);
5            sin(phi)*sin(theta)*cos(psi)-cos(phi)*sin(psi) ...
6            sin(phi)*sin(theta)*sin(psi)+cos(phi)*cos(psi) cos(theta)*sin(phi);
7            cos(phi)*sin(theta)*cos(psi)+sin(phi)*sin(psi) ...
8            cos(phi)*sin(theta)*sin(psi)-sin(phi)*cos(psi) ...
              cos(theta)*cos(phi)];
9 end
```

13.13.2 Convert Euler Angles to Quaternion

```
1 function quat = euler2quat(euler_angles)
2 %EULER2QUAT Convert Euler angles to quaternion
3     phi = euler_angles(:,1); theta = euler_angles(:,2); psi = ...
        euler_angles(:,3);
4
5     quat = [cos(phi/2).*cos(theta/2).*cos(psi/2) + ...
6            sin(phi/2).*sin(theta/2).*sin(psi/2) ...
7            -cos(phi/2).*sin(theta/2).*sin(psi/2) + ...
8            cos(theta/2).*cos(psi/2).*sin(phi/2) ...
9            cos(phi/2).*cos(psi/2).*sin(theta/2) + ...
10           sin(phi/2).*cos(theta/2).*sin(psi/2) ...
11           cos(phi/2).*cos(theta/2).*sin(psi/2) - ...
12           sin(phi/2).*cos(psi/2).*sin(theta/2)];
13 end
```

13.13.3 Convert Quaternion to Euler Angles

```
1 function euler = quat2euler(quat)
2 %QUAT2EULER Convert quaternion to Euler angles
3
4 euler = [atan2(2*quat(:, 3).*quat(:, 4) + 2*quat(:, 1).*quat(:, 2), ...
5            quat(:, 4).^2-quats(:, 3).^2-quats(:, 2).^2+quat(:, 1).^2) ...
6            -asin(2*quat(:, 2).*quat(:, 4)-2*quat(:, 1).*quat(:, 3)) ...
7            atan2(2*quat(:, 2).*quat(:, 3)+2*quat(:, 1).*quat(:, 4), ...
8            quat(:, 2).^2+quat(:, 1).^2-quats(:, 4).^2-quats(:, 3).^2)];
9 end
```

13.14.1 Integrated Satellite ADCS Model

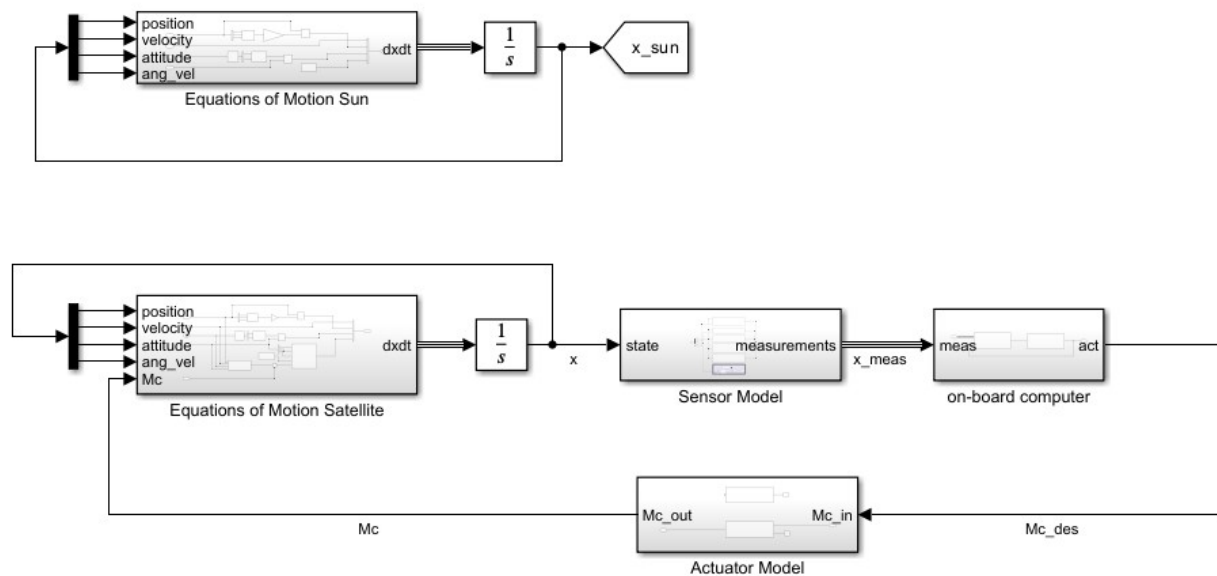


Figure 169: Integrated Satellite ADCS Model

13.14.2 Equations of Motion

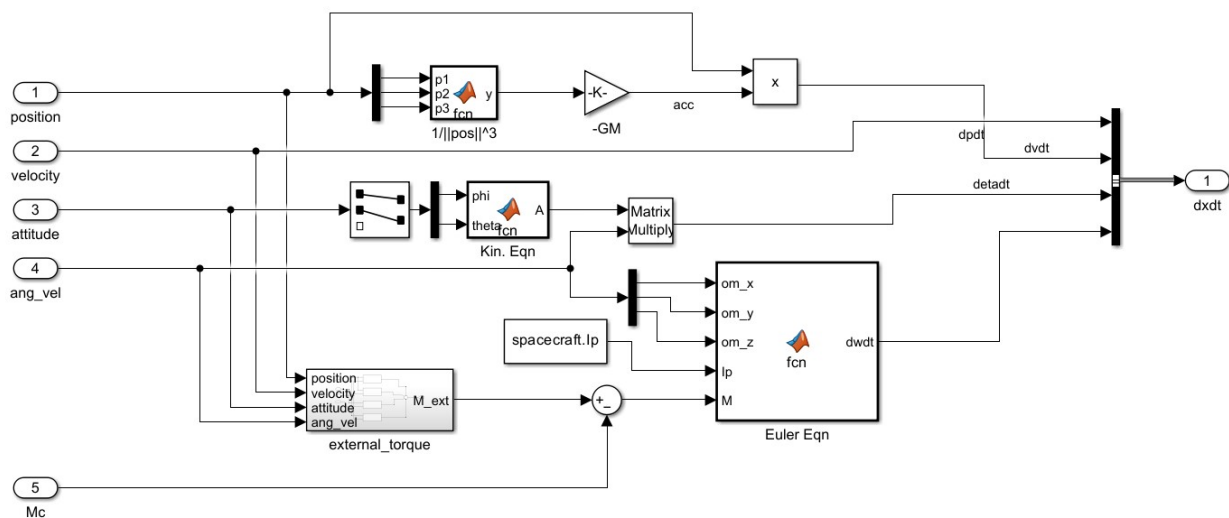


Figure 170: Equations of Motion of the Satellite.

```

1 function dwdt = fcn(om-x, om-y, om-z, Ip, M)
2 % Determine angular velocity derivative
3 M_1 = M(1); M_2 = M(2); M_3 = M(3);
4 Ip-x = Ip(1,1); Ip-y = Ip(2,2); Ip-z = Ip(3,3);
5
6 dwdt = [((Ip-y - Ip-z)*om-y*om-z + M_1)/Ip-x;
7         ((Ip-z - Ip-x)*om-z*om-x + M_2)/Ip-y;
8         ((Ip-x - Ip-y)*om-x*om-y + M_3)/Ip-z];

```

```

1 function A = fcn(phi, theta)
2 % Attitude kinematics equations for Euler angles
3 A = [1 sin(phi)*tan(theta) cos(phi)*tan(theta); ...
4      0 cos(phi) -sin(phi); ...
5      0 sin(phi)*sec(theta) cos(phi)*sec(theta)];

```

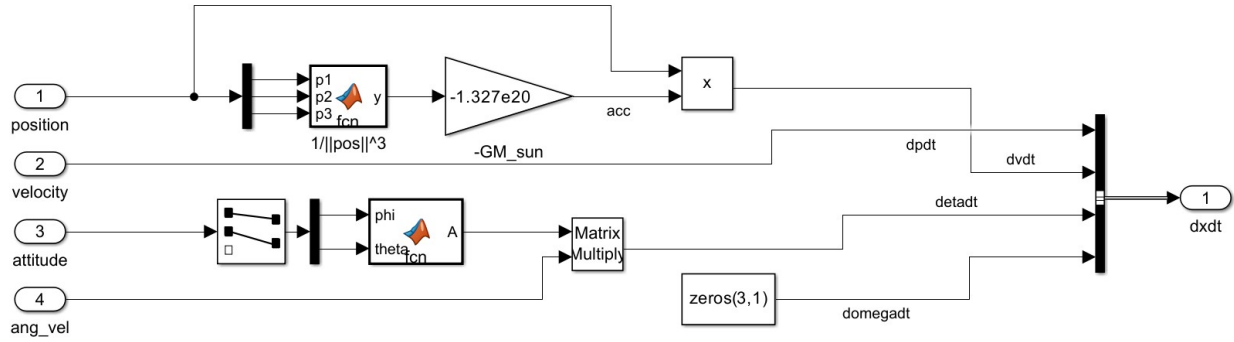


Figure 171: Equations of Motion of the Sun.

13.14.3 Sensor Model

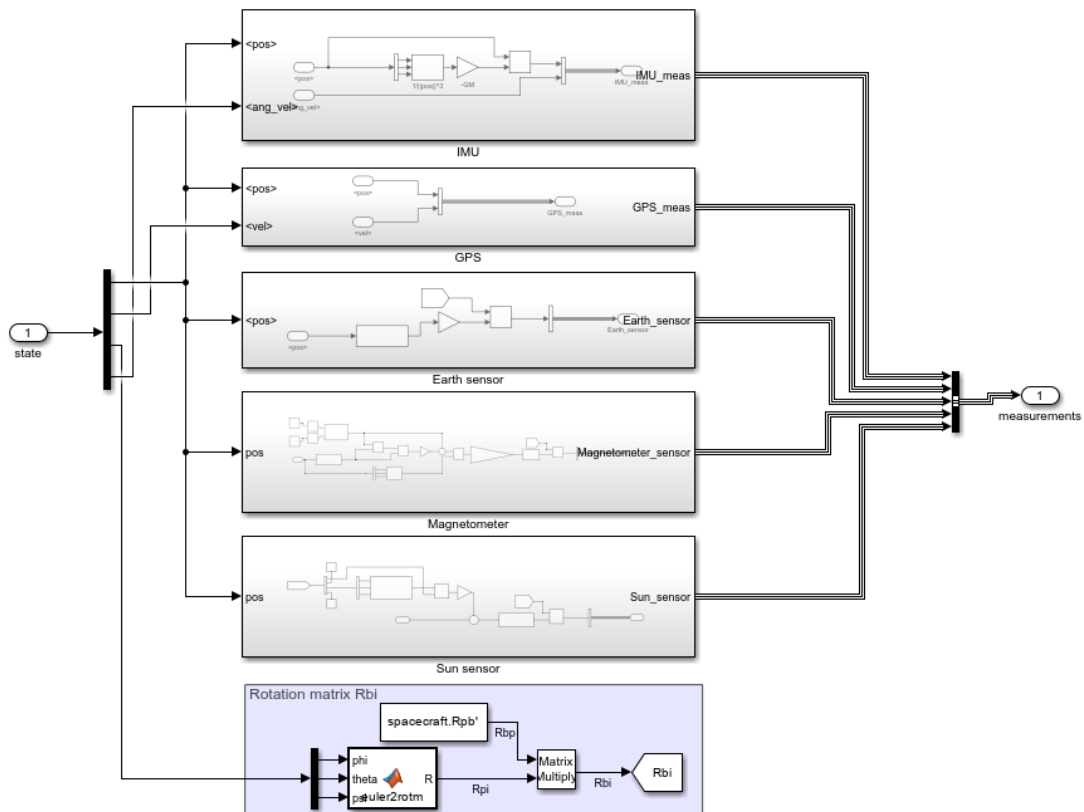


Figure 172: Sensor model implementation

13.14.4 Actuator Model

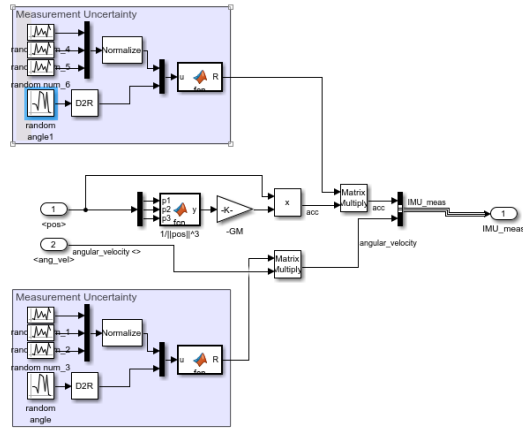


Figure 173: IMU model implementation

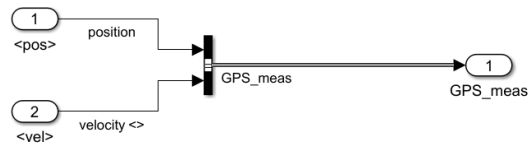


Figure 174: GPS model implementation

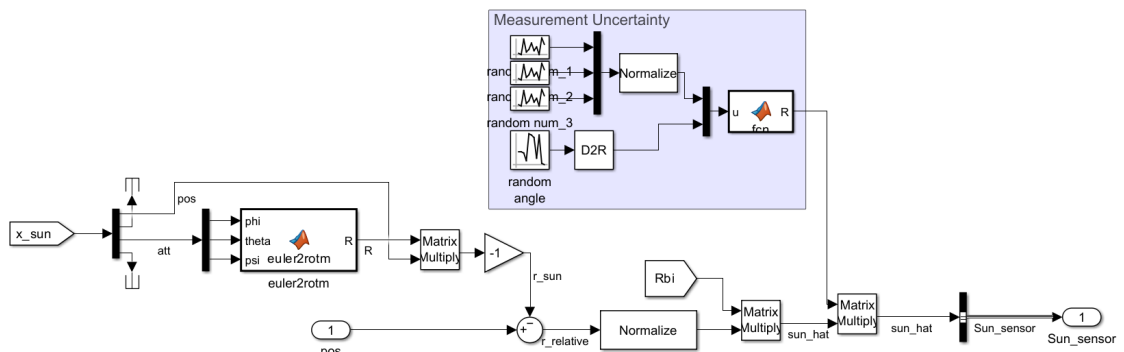


Figure 175: Sun Sensor model implementation

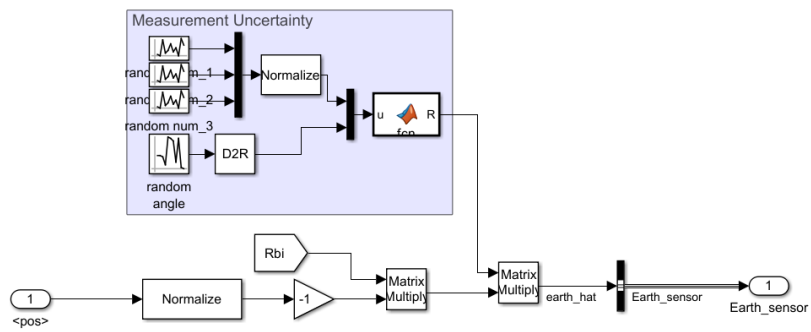


Figure 176: Earth Sensor model implementation

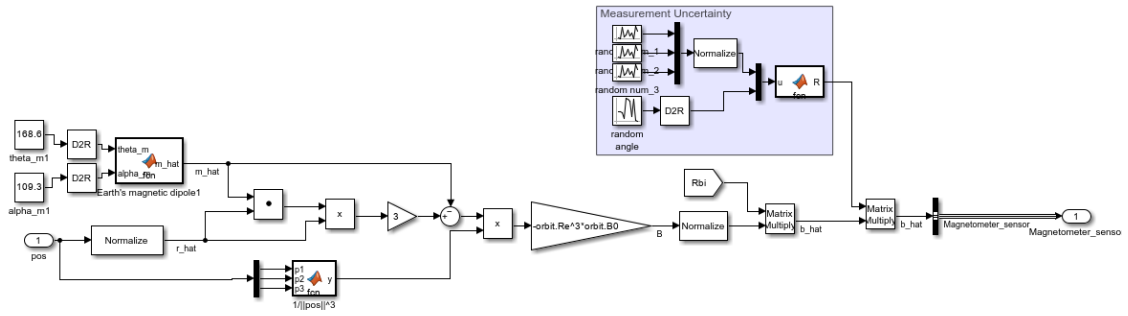


Figure 177: Magnetometer model implementation

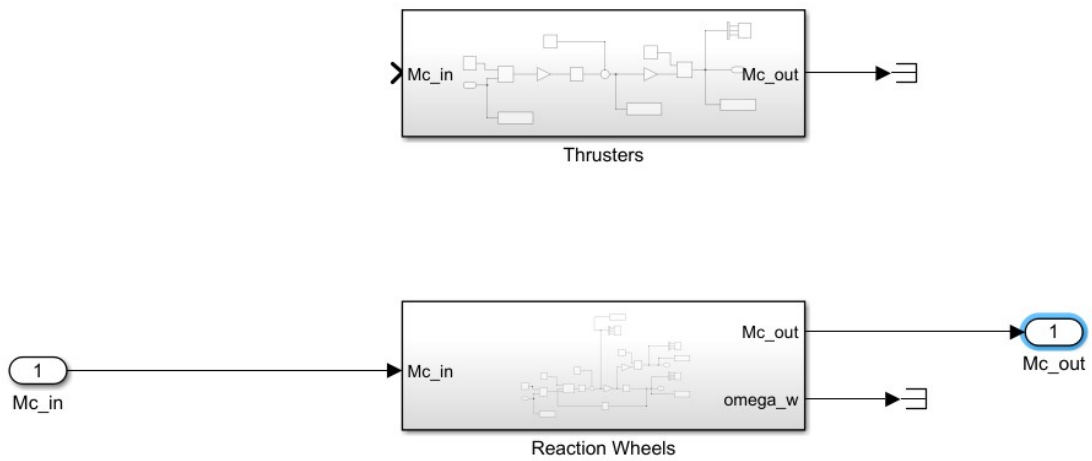


Figure 178: Actuator model implementation

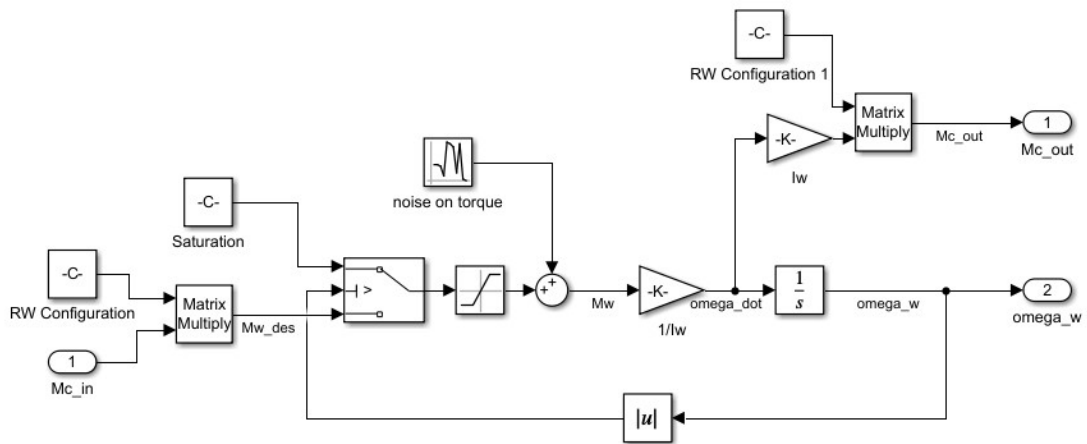


Figure 179: Reaction wheel model implementation

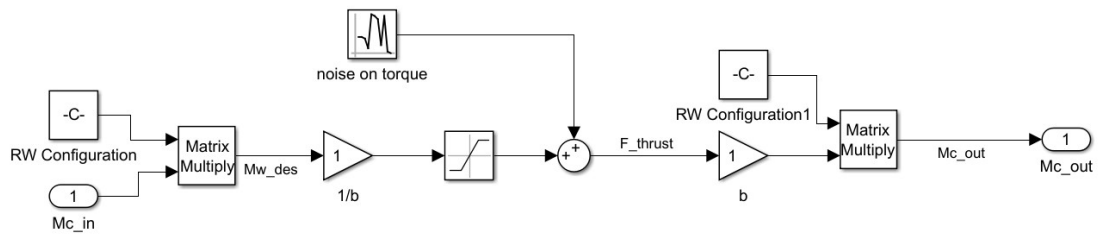


Figure 180: Thruster model implementation

13.14.5 On-board Computer

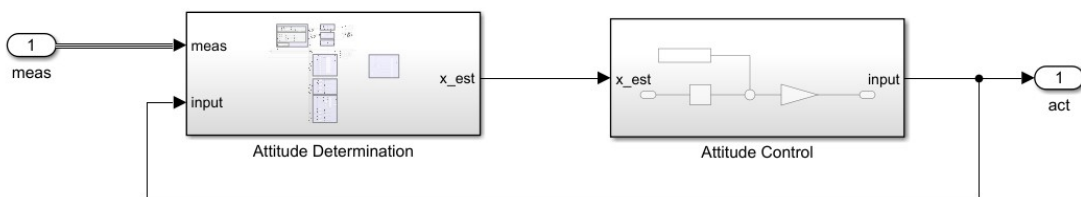


Figure 181: On-board computer implementation

13.14.6 Attitude Determination

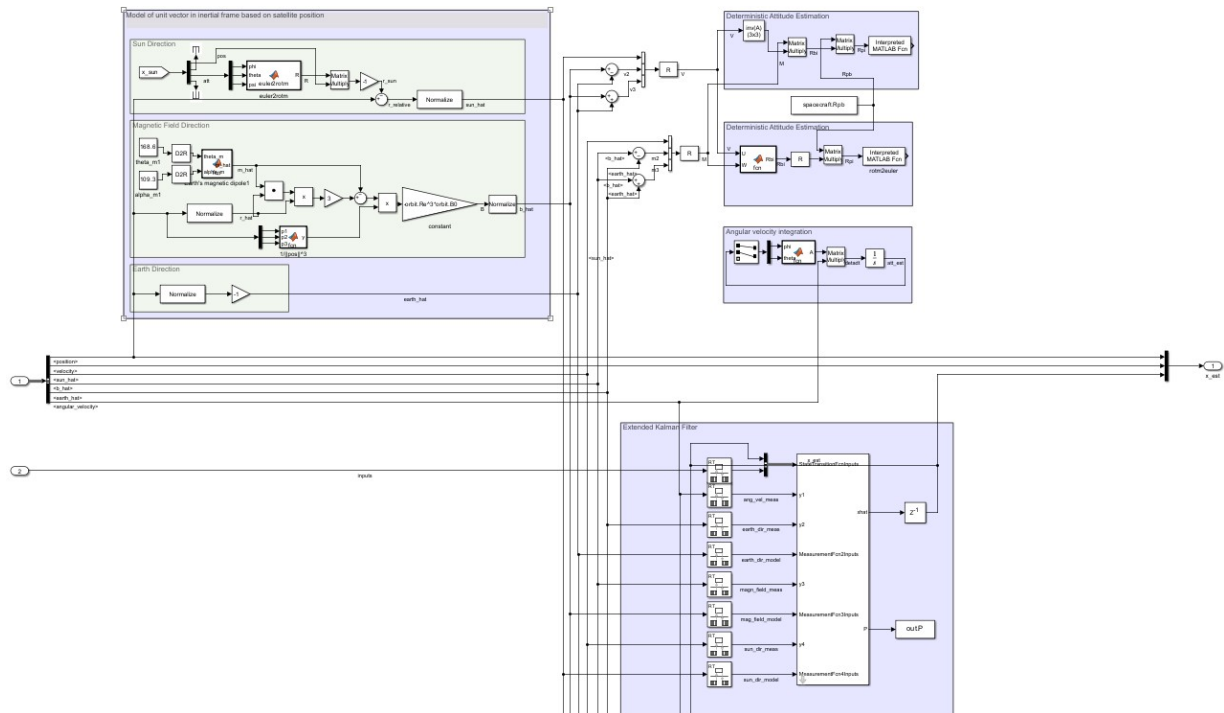


Figure 182: Attitude determination model implementation

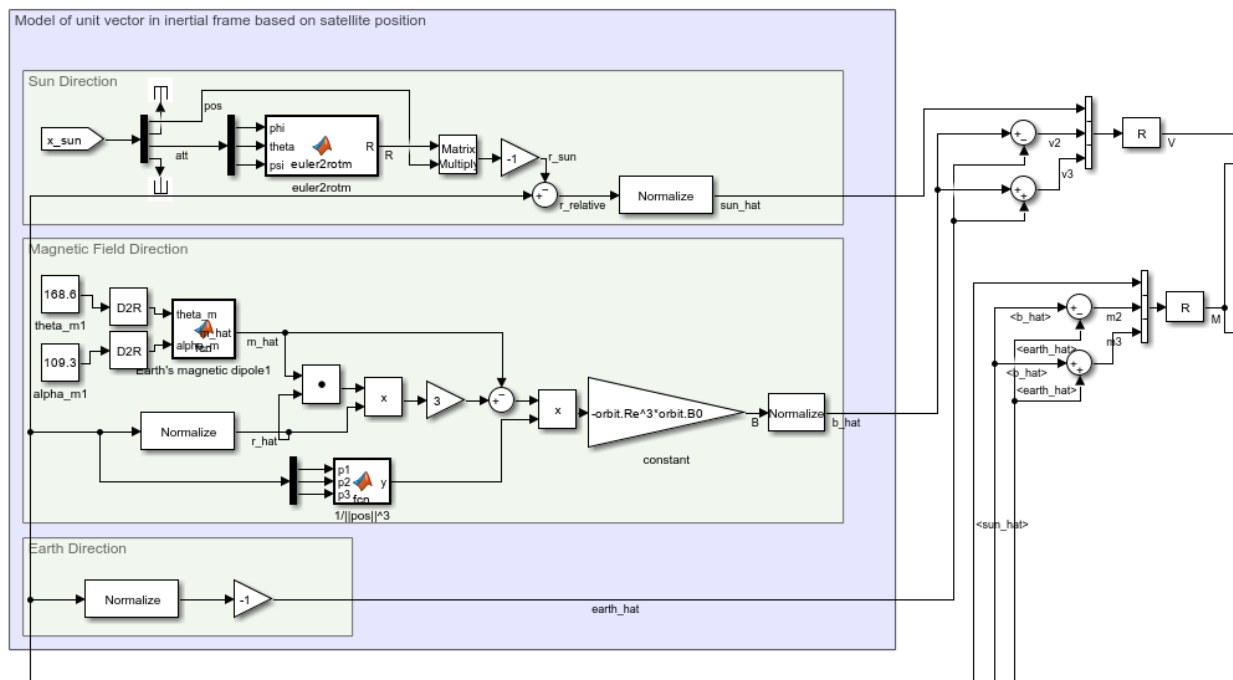


Figure 183: Measurement model implementation

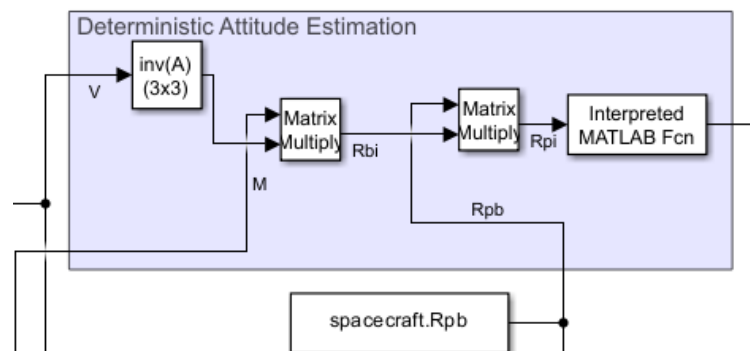


Figure 184: Deterministic attitude estimation model implementation

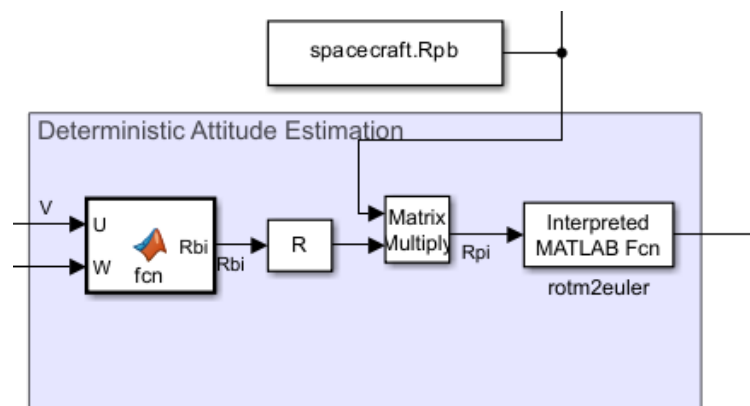


Figure 185: Statistical attitude estimation model implementation (Matlab function given below)

```

1 function Rbi = stat_att_est(U, W)
2     B = W * U';
3     S = B + B';
4     Z = [B(2,3)-B(3,2) B(3,1)-B(1,3) B(1,2)-B(2,1)]';
5     sigma = trace(B);
6     K = [S-sigma*eye(3) Z; Z' sigma];
7
8     [V, D] = eig(K);
9     [~, idx] = max(diag(D));
10    quat = real(V(:, idx));
11    quat = real([quat(4); quat(1:3)]);
12    Rbi = euler2rotm(quat2euler(quat'));

```

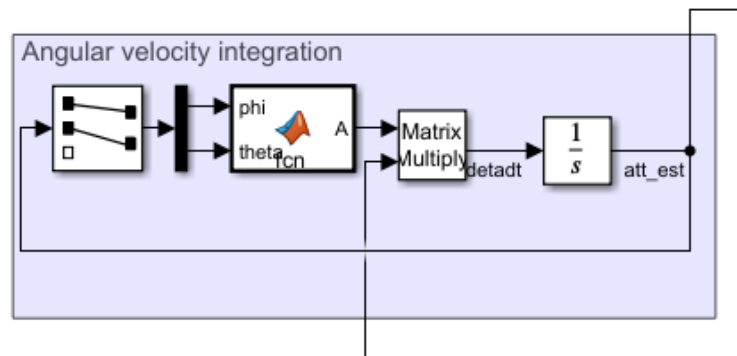


Figure 186: Integrated angular velocity attitude estimation model implementation (Matlab function given below)

```

1 function A = att_kinematics(phi, theta)
2 % Attitude kinematics equations for Euler angles
3 A = [1 sin(phi)*tan(theta) cos(phi)*tan(theta); ...
4      0 cos(phi) -sin(phi); ...
5      0 sin(phi)*sec(theta) cos(phi)*sec(theta)];

```

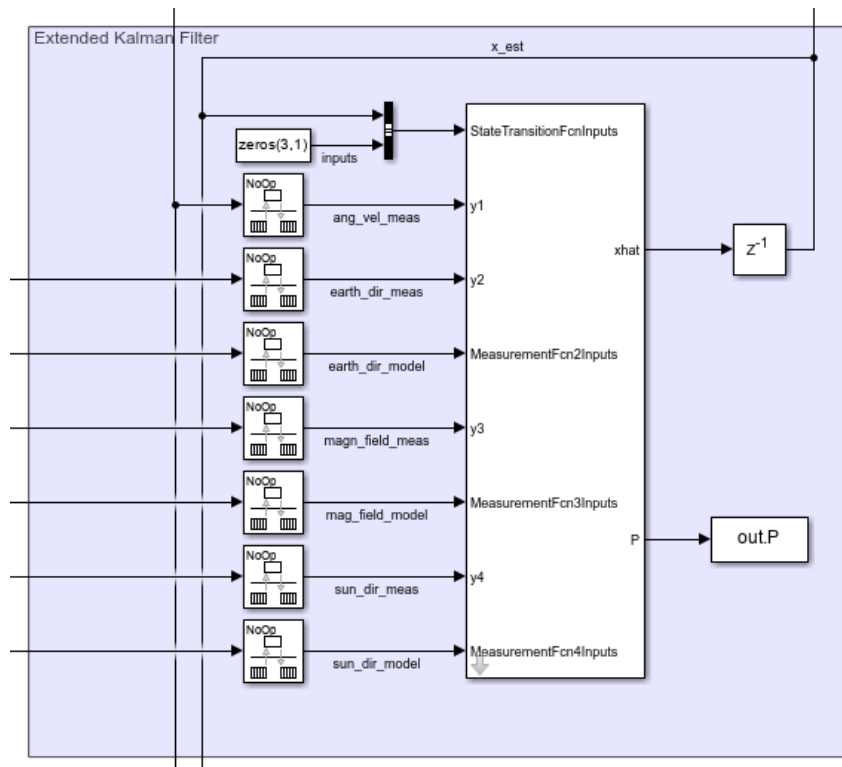


Figure 187: Extended Kalman Filter Implementation (Block configuration given below)

Extended Kalman Filter

Discrete-time extended Kalman filter. Estimate states of a nonlinear plant model. Use Simulink Function blocks or .m MATLAB Functions to specify state transition and measurement functions.

See block help for function syntaxes, which depend on if noise is additive or nonadditive.

System Model: Multirate

State Transition

Function: StateTransitionFcn ☐ Jacobian

Process noise: Additive ☐ Time-varying

Covariance: $1e-1 \cdot \pi / 180 \cdot \text{eye}(6)$

Initialization

Initial state: $x_0(7:12)$ $<6 \times 1 \text{ double}>$ Initial covariance: $1e-2 \cdot \pi / 180 \cdot \text{eye}(6)$ $<6 \times 6 \text{ double}>$

Measurement 1

Function: AngVelMeas ☐ Jacobian ☐ Add Enable port

Measurement noise: Additive ☐ Time-varying

Covariance: $2.63e-4 \cdot \pi / 180 \cdot \text{eye}(3)$

☐ Has measurement wrapping

Measurement 2

Function: AttMeas ☐ Jacobian ☐ Add Enable port

Measurement noise: Additive ☐ Time-varying

Covariance: $0.33 \cdot \pi / 180 \cdot \text{eye}(3)$

☐ Has measurement wrapping

Measurement 3

Function: AttMeas ☐ Jacobian ☐ Add Enable port

Measurement noise: Additive ☐ Time-varying

Covariance: $3.0 \cdot \pi / 180 \cdot \text{eye}(3)$

☐ Has measurement wrapping

Measurement 4

Function: AttMeas ☐ Jacobian ☐ Add Enable port

Measurement noise: Additive ☐ Time-varying

Covariance: $0.1 \cdot \pi / 180 \cdot \text{eye}(3)$

☐ Has measurement wrapping

Add Measurement Remove Measurement

Figure 188: Extended Kalman Filter Configuration (Matlab functions given below)

```

1 function x_next = StateTransitionFcn(x_est, inputs)
2 %STATETRANSITIONFCN Nonlinear state transition function
3     dt = 0.1;
4     Ip = [1.0e+03*1.4392 1.0e+03*1.8933 1.0e+03*2.7702];
5
6     phi = x_est(1); theta = x_est(2); psi = x_est(3);
7     om_x = x_est(4); om_y = x_est(5); om_z = x_est(6);
8     M_1 = inputs(1); M_2 = inputs(2); M_3 = inputs(3);
9     Ip_x = Ip(1); Ip_y = Ip(2); Ip_z = Ip(3);
10
11     % Time-derivative of attitude angles
12     detadt = [1 sin(phi)*tan(theta) cos(phi)*tan(theta); ...
13              0 cos(phi) -sin(phi); ...
14              0 sin(phi)*sec(theta) cos(phi)*sec(theta)] * x_est(4:6);
15
16     % Time-derivative of angular velocity
17     dwdt = [((Ip_y - Ip_z)*om_y*om_z + M_1)/Ip_x;
18            ((Ip_z - Ip_x)*om_z*om_x + M_2)/Ip_y;
19            ((Ip_x - Ip_y)*om_x*om_y + M_3)/Ip_z];
20
21     x_next = x_est + dt * [detadt; dwdt];
22 end

```

```

1 function ang_vel = AngVelMeas(x_est)
2 %ANGVELMEAS Measurement model for angular velocity
3     ang_vel = x_est(4:6);
4 end

```

```

1 function att_pred = AttMeas(x_est, inputs)
2 %EARTHMEAS Measurement model for Earth sensor
3     Rbp = [-0.0121 0.9984 -0.0556;
4            0.0207 0.0558 0.9982;
5            0.9997 0.0109 -0.0213];
6     att_pred = Rbp * euler2rotm(x_est(1:3)) * inputs; % rotated into ...
7                 body frame
7 end

```

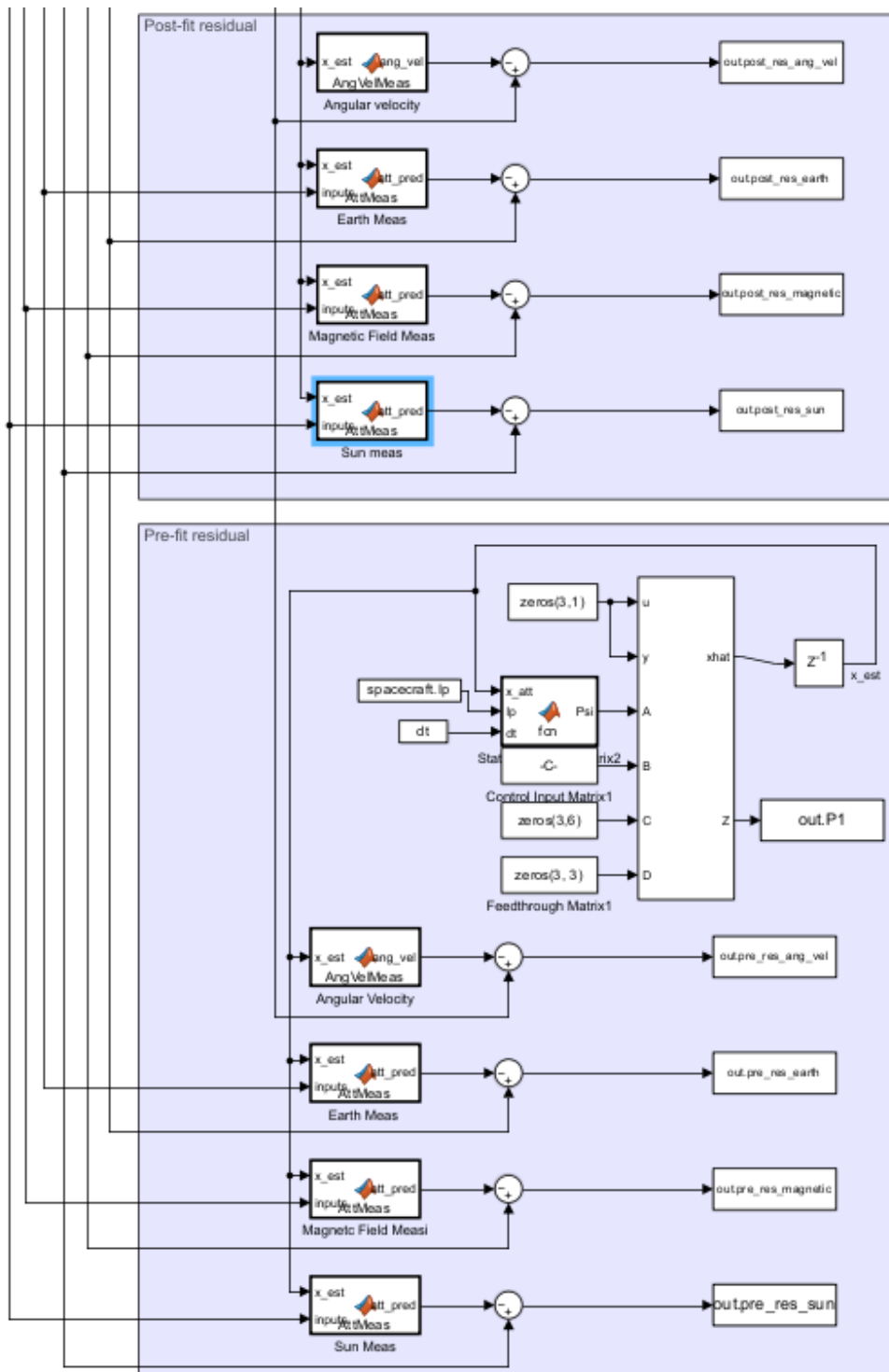


Figure 189: Determination of the measurement residuals (pre- and post)

13.14.7 Attitude Control

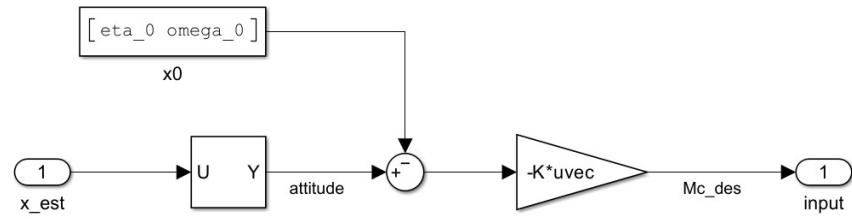


Figure 190: Attitude control implementation through LQR.

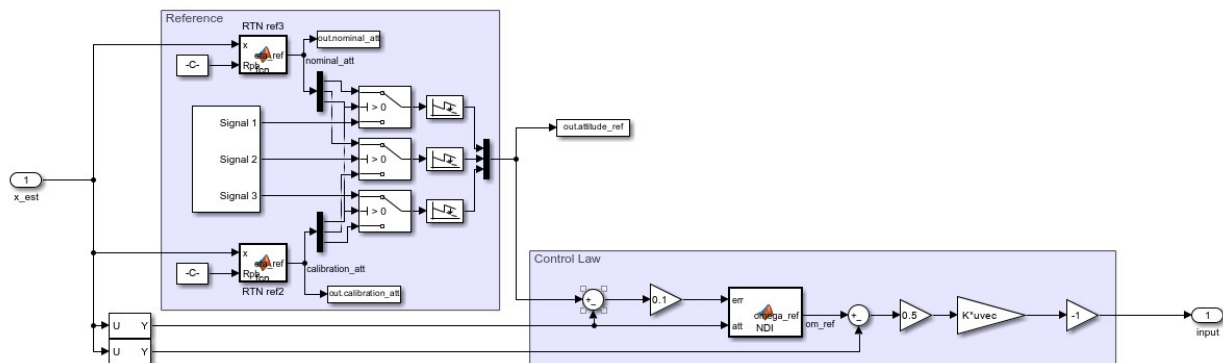


Figure 191: Nonlinear attitude tracking controller.